

Notes concernant le ML pour l'énergétique du bâtiment.

William Maréchal

Update on October 25, 2025

Contents

I Introduction 7

1 Introduction 9

- 1.1 Contexte et Enjeux 9
- 1.2 L'IA au Service de l'Énergétique des Bâtiments 9
- 1.3 Avantages et Défis 10
- 1.4 Acteurs de l'énergétique du bâtiment utilisant l'IA en France. 11

2 Notions generales 13

- 2.1 Notions générales 13
 - 2.1.1 IA / ML / DL 13
 - 2.1.2 Les différents types de Machine Learning 14
 - 2.1.3 Les différents types de données 14
 - 2.1.4 Feature engineering 16
 - 2.1.5 Quelques résultats mathématiques utiles 21
 - 2.1.6 Évaluation de la performance d'un algorithme 23
 - 2.1.7 Train, validation, test / Cross validation 27
 - 2.1.8 Cas des séries temporelles 28
 - 2.1.9 Underfitting et overfitting 29
 - 2.1.10 Compromis biais / variance 29
 - 2.1.11 Régularisation 32
 - 2.1.12 interprétabilité 32
 - 2.1.13 AutoML 36
- 2.2 Cycle de vie d'un projet de Machine Learning - matériels sources 37

3 Algorithmes classiques 47

- 3.1 Classification et régression 47
- 3.2 Régression linéaire 48
- 3.3 Régression logistique 54
- 3.4 Les k plus proches voisins (k-NN) 56

- 3.5 Les k-moyennes (k-means) 59
- 3.6 Analyse en composantes principales (PCA) 69
- 3.7 Décomposition en Valeurs Singulières (SVD) 75
- 3.8 GAMS 76
- 3.9 Factorisation non-négative de matrice 79
- 3.10 Random Forests (RF) 81
- 3.11 Machines à vecteurs de support (SVM) 87
- 3.12 Algorithmes de règles 92

4 Reseaux de neurones 97

- 4.1 Généralités 97
- 4.2 MLP 97
- 4.3 CNN 99
- 4.4 RNN 100
- 4.5 RNN, GRU & LSTM 101
- 4.6 Autoencoder 102
- 4.7 Les hyperparamètres 104
- 4.8 vanishing gradient 107
- 4.9 Rex: utilisation de modèle d'IA 109
- 4.10 Reinforcement Learning 111
- 4.11 Les Réseaux de Kolmogorov-Arnold (KAN) 120

5 Outils de Buisness Intelligence 123

- 5.1 NoteBooks : Jupyter, Google Colab... 123
- 5.2 Github & IDE: Pycharm, VS code, Spyder... 124
- 5.3 Power BI 125
- 5.4 Tableau 129

6 PiML 133

- 6.1 Physics informed Machine Leaning (PiML) 133

7 Environement ML 135

- 7.1 Les tests logicielles 135
 - 7.1.1 Les tests unitaires : fondations et outils 135
 - 7.1.2 Tests d'intégration et fonctionnels : valider les interactions 135
 - 7.1.3 Les tests end-to-end et l'équilibre de la stratégie de test 135
- 7.2 IA générative 136
 - 7.2.1 Variational Autoencoder (VAE) 136

7.2.2	Generative Adversarial Network (GAN)	136	7.3	Des menaces nouvelles.	141
7.2.3	Diffusion models	137	7.4	L'IA et l'éthique dans le contexte de l'Europe	143
7.2.4	LLM	138	7.5	Des outils commerciaux.	147
7.2.5	Les transformeurs :	139	7.6	Des voraces énergétiques.	150
7.2.6	Les hallucinations :	140			
7.2.7	L'Intelligence Artificielle Générative au Service de l'Éducation	140	8 Conclusions		153
7.2.8	L'Intelligence Artificielle Générative au Service du développement informatique. Cas de Github Copilot	141	Appendices		155
			Appendix A Notions de Machine Learning		155

List of Figures

2.1	L'IA et ses sous-ensembles	14
2.2	Les différents types de Machine Learning. (Source : www.cognub.com)	14
2.3	Exemple de données tabulaires. (Source : UCI Machine Learning Repository)	15
2.4	Exemple d'une image matricielle en niveaux de gris (valeurs des pixels entre 0 et 255 [codage sous 8 bits]). (Source : base de données MNIST)	15
2.5	Conversion d'un mot en code binaire. (Source : www.computerhope.com)	16
2.6	Exemple d'une série temporelle (cours de l'action <i>Tesla</i>).	16
2.7	Rendre des grandeurs comparables. Source saedsayad.com	17
2.8	Espace latent. (source TDS)	17
2.9	Matrice de confusion d'un classificateur binaire. (Source : openclassrooms.com)	24
2.10	Une courbe ROC. La performance d'un modèle naïf (aléatoire) est indiquée en pointillés. Plus la courbe tend à épouser les axes supérieurs gauches, plus le modèle est performant	25
2.11	Illustration du critère AUC (Source : stanford.edu).	26
2.12	Séparation d'un jeu de données en 3 sous-ensembles. Source TDS	27
2.13	Source article	28
2.14	Cas des séries temporelles. Source TDS	29
2.15	Sous-interprétation et sur-interprétation. (Source : Towards Data Science)	30
2.16	Compromis biais / variance. (Source wikimedia.org)	30
2.17	Courbes d'apprentissages typiques dans un cas de <i>high variance</i> (gauche) et <i>high bias</i> (droite) Source dataquest.io	31
2.18	Principales méthodes de régularisation.	32
2.19	Exemple de PDP appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour les features RM et LSTAT.	34
2.20	Exemple de ALE appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour les features RM et LSTAT.	34
2.21	Exemple de ICE appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour les features RM et LSTAT.	35
2.22	Exemple de LIME appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour la 42ème prédiction.	35
2.23	Exemple de LIME appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour la 42ème prédiction.	36
2.24	Cycle de vie d'un modèle de ML.	37
2.25	Effet de la dérive de modèle.	43
2.26	Model Drift Types with Input and Output. - Source medium	45
3.1	Source : scikit-learn	47
3.2	Source : scikit-learn	48
3.3	Source : scikit-learn	48

3.4	Source : scikit-learn	49
3.5	Exemple : taille de t-shirt en fonction du poids et de la taille d'un individu.	57
3.6	Utilisation de KNN en régression. Source bookdown.org	58
3.7	Exemple de silhouette.	62
3.8	Impact de la silhouette.	62
3.9	Illustrations de métriques. source TDS	63
3.10	Illustrations de métriques. source TDS	64
3.11	source medium	70
3.12	Source stackoverflow.com	71
3.13	Image moyenne et les 15 premières <i>eigenfaces</i> . (Source : N. Morizet, PhD Thesis, 2009.)	74
3.14	Exercice de classification (deux classes). Ici, le problème est linéairement séparable	88
3.15	Un hyperplan de $\mathbb{R}^2$ est une droite. Un hyperplan de $\mathbb{R}^3$ est un plan.	89
3.16	Illustration 3D de la méthode des multiplicateurs de Lagrange. Sur cette figure, il s'agit de trouver le point le plus élevé possible, tout en satisfaisant la contrainte d'être sur la ligne rouge.	91
3.17	Exercice de classification (deux classes). Ici, le problème est non linéairement séparable	92
3.18	Exemple d'arbre de régression. Source article	94
3.19	Exemple de liste de règles. (extrait de l'article <i>Scalable Bayesian Rule Lists</i>)	94
4.1	Les différents d'apprentissages du DeepLearning. Sources article	97
4.2	Quelques structures de réseaux. Source article	98
4.3	Génération d'images. (source: https://arxiv.org/pdf/1712.08838)	102
4.4	Quelques fonctions d'activation. Source CERN	105
4.5	Les branches du Machine Learning (Source : www.mathworks.com).	113
4.6	Exploitation vs. Exploration. (Source : ai-ml-analytics.com).	114
4.7	Schéma générique du RL appliqué au trading CID avec une batterie.	116
4.8	MLP VS KAN (https://arxiv.org/pdf/2404.19756).	121
6.1	Physic informed Machine Learning - Source article	134
7.1	Principe de diffusion.	137
7.2	Hub France IA - source	145
7.3	afnor	146
7.4	CO_2 VS type d'IA (https://arxiv.org/pdf/2311.16863).	150
7.5	Des modèles de plus en plus gros. (https://arxiv.org/pdf/2311.16863).	151

Part I

Introduction

Chapter 1

Introduction

L'intelligence artificielle (IA) a transformé de nombreux secteurs de notre société, de la médecine à la finance en passant par les transports. L'un des domaines les plus prometteurs où l'IA apporte des innovations significatives est celui de l'énergétique des bâtiments. L'efficacité énergétique est devenue une préoccupation majeure dans un monde confronté à des défis environnementaux croissants et à la nécessité de réduire les émissions de gaz à effet de serre. Les bâtiments, qu'ils soient résidentiels ou commerciaux, représentent une part importante de la consommation d'énergie globale, et l'application de l'IA dans ce domaine offre des opportunités considérables pour optimiser la consommation d'énergie, réduire les coûts et minimiser l'impact environnemental.

1.1 Contexte et Enjeux

Les bâtiments sont responsables d'une part significative de la consommation mondiale d'énergie. Selon l'Agence internationale de l'énergie (AIE), le secteur des bâtiments représente près de 30 % de la consommation énergétique mondiale et environ 28 % des émissions de dioxyde de carbone (CO_2) en 2022. Dans un contexte de changement climatique et de réchauffement planétaire, réduire cette empreinte énergétique est devenu une priorité essentielle.

L'inefficacité énergétique des bâtiments peut être attribuée à de nombreux facteurs, notamment une isolation inadéquate, une mauvaise gestion des systèmes de chauffage, de ventilation et de climatisation (CVC), ainsi que des comportements inefficaces des occupants. Pour résoudre ces problèmes et optimiser la consommation d'énergie des bâtiments, l'IA se révèle être une solution puissante.

1.2 L'IA au Service de l'Énergétique des Bâtiments

L'IA est un domaine de l'informatique qui vise à créer des systèmes capables d'apprendre, de raisonner et de prendre des décisions de manière autonome, en imitant les processus cognitifs humains. Dans le contexte de l'efficacité énergétique des bâtiments, l'IA offre une gamme d'applications et de possibilités innovantes :

- Analyse de Données

L'un des aspects fondamentaux de l'efficacité énergétique des bâtiments est la collecte et l'analyse de données. Les capteurs installés dans les bâtiments peuvent recueillir une grande quantité d'informations sur la consommation d'énergie, la température, l'humidité et d'autres paramètres. L'IA peut être utilisée pour analyser ces données en temps réel et détecter des schémas, des anomalies ou des opportunités d'optimisation. Par exemple, l'IA peut identifier des heures creuses pour réduire la consommation d'énergie ou repérer des équipements défectueux nécessitant des réparations.

- Gestion de l'Énergie

L'IA peut jouer un rôle central dans la gestion intelligente de l'énergie dans les bâtiments. Les systèmes de CVC, l'éclairage et d'autres équipements peuvent être contrôlés de manière dynamique en fonction des besoins réels. Les algorithmes d'apprentissage automatique peuvent apprendre les préférences des occupants, les habitudes de travail et d'autres facteurs pour ajuster automatiquement les paramètres et maximiser l'efficacité énergétique.

- Prévision et Planification

L'IA peut être utilisée pour anticiper la demande énergétique future en fonction de données météorologiques, de modèles de comportement des occupants et d'autres facteurs. Cela permet une planification plus efficace de la production et de la distribution d'énergie, notamment en intégrant des sources d'énergie renouvelable comme l'énergie solaire et éolienne.

- Amélioration de l'Isolation

L'IA peut aider à concevoir des bâtiments plus efficaces sur le plan énergétique en optimisant la conception de l'isolation, des fenêtres et des matériaux de construction. Les modèles de simulation basés sur l'IA peuvent prédire la performance énergétique d'un bâtiment avant même sa construction.

- Sensibilisation des Occupants

L'IA peut contribuer à sensibiliser les occupants à leur consommation d'énergie en fournissant des informations en temps réel sur la consommation et en suggérant des comportements plus économes en énergie. Des tableaux de bord interactifs et des applications mobiles peuvent encourager la réduction de la consommation d'énergie.

- Maintenance Prédictive

L'IA peut anticiper les besoins de maintenance des équipements et des systèmes dans les bâtiments en analysant les données de performance. Cela permet de planifier les interventions de manière proactive, d'éviter les pannes coûteuses et de prolonger la durée de vie des équipements.

1.3 Avantages et Défis

L'utilisation de l'IA dans l'énergétique des bâtiments offre de nombreux avantages. Elle permet une réduction significative de la consommation d'énergie, une meilleure gestion des ressources, des coûts réduits et une empreinte environnementale moindre. Cependant, il existe également des défis à relever, notamment :

- Coûts initiaux élevés :

La mise en place de systèmes d'IA et de capteurs peut être coûteuse.

- Sécurité des données :

La collecte de données sensibles sur les occupants et les systèmes de sécurité nécessite une protection rigoureuse des données.

- Acceptation des occupants :

Les occupants des bâtiments doivent s'adapter à des systèmes de gestion automatisés, ce qui peut susciter des préoccupations.

- Formation et expertise :

Il faut former le personnel pour comprendre et utiliser les systèmes d'IA de manière efficace.

L'IA a le potentiel de transformer radicalement l'énergétique des bâtiments en rendant les structures plus efficaces, économiques et respectueuses de l'environnement. Les avantages de réduire la consommation d'énergie et les émissions de carbone sont cruciaux dans un monde confronté à des enjeux environnementaux critiques. Bien que des défis subsistent, le développement et l'adoption croissants de l'IA

1.4 Acteurs de l'énergétique du bâtiment utilisant l'IA en France.

- **Towards Data Science** : entreprise canadienne qui permet à des utilisateurs de publier sur la plateforme web de blog des articles traitant de la science des données.
- **huggingface** : entreprise française qui met à disposition de ses utilisateurs une infrastructure cloud, des algorithmes et des formations en IA.
- **Schneider Electric** : solutions de gestion de l'énergie pour bâtiments résidentiels, tertiaires et industriels. L'IA est utilisée pour améliorer l'efficacité énergétique, la maintenance prédictive et la sécurité.
- **Vinci Energies** : services d'ingénierie, de construction et de maintenance pour bâtiments. L'IA est utilisée pour la conception de bâtiments plus efficaces, la maintenance prédictive et la gestion des données.
- **Enel X** : solutions de gestion de l'énergie et d'efficacité énergétique pour bâtiments. L'IA est utilisée pour l'optimisation de la consommation d'énergie, la maintenance prédictive et la gestion des énergies renouvelables.
- **Groupe EDF** : solutions d'énergie et d'efficacité énergétique pour bâtiments. L'IA est utilisée pour l'optimisation de la consommation d'énergie, la maintenance prédictive et la gestion des énergies renouvelables.
- **Delta Dore** : solutions de domotique pour bâtiments. L'IA est utilisée pour l'automatisation des tâches, la personnalisation des expériences et la sécurité.
- **Atos** : services numériques pour bâtiments. L'IA est utilisée pour la modélisation énergétique, la gestion des données et la sécurité des systèmes.
- **Dalkia** : services énergétiques pour bâtiments. L'IA est utilisée pour la gestion des contrats d'énergie, la maintenance prédictive et la gestion des données.
- **Greenyellow** : solutions d'énergies renouvelables pour bâtiments. L'IA est utilisée pour la gestion des installations photovoltaïques et des batteries.
- **Accenta.ia** : solution de gestion de l'énergie pour bâtiments résidentiels. L'IA est utilisée pour l'optimisation de la consommation d'énergie, la maintenance prédictive et la sécurité.
- **deepki** : solution de diagnostic énergétique pour bâtiments. L'IA est utilisée pour l'analyse des données de consommation d'énergie afin d'identifier les opportunités d'économies d'énergie.
- **Enerlis** : solution de gestion des énergies renouvelables pour bâtiments. L'IA est utilisée pour la gestion des installations photovoltaïques et des batteries.
- **Enercoop** : fournisseur d'énergie verte pour bâtiments. L'IA est utilisée pour la gestion des contrats d'énergie et la personnalisation des offres.
- **GreenFlex** : conseil en efficacité énergétique pour bâtiments. L'IA est utilisée pour l'audit énergétique et la sélection des solutions.
- **Legrand** : fabricant de solutions électriques et d'automatisation pour bâtiments. L'IA est utilisée pour la conception de produits plus efficaces et pour la maintenance prédictive.
- **LANCEY Energy Storage** : propose un système innovant qui optimise l'autoconsommation photovoltaïque pour réduire la facture d'électricité et améliorer le confort thermique.

Chapter 2

Notions generales

2.1 Notions générales

2.1.1 IA / ML / DL

Definition de Machine Learning ou apprentissage automatique :

1998, (American computer scientist) Tom Mitchell:

“On dit qu’un programme informatique apprend de l’expérience E en ce qui concerne une classe de tâches T et une mesure de performance P , si sa performance dans les tâches T , telle que mesurée par P , s’améliore avec l’expérience E .”

Exemple : jouer aux dames.

E = l’expérience de nombreuses parties de dames.

T = la tâche de jouer aux dames.

P = la probabilité que le programme gagne la prochaine partie.

Par la suite on a définie l’apprentissage supervisé comme étant un type d’apprentissage automatique dans lequel un modèle est formé sur un ensemble de données étiqueté. Les données étiquetées sont des données qui contiennent à la fois les entrées et les sorties souhaitées. L’objectif de l’apprentissage supervisé est d’entraîner un modèle qui apprend à prédire les sorties souhaitées à partir des entrées.

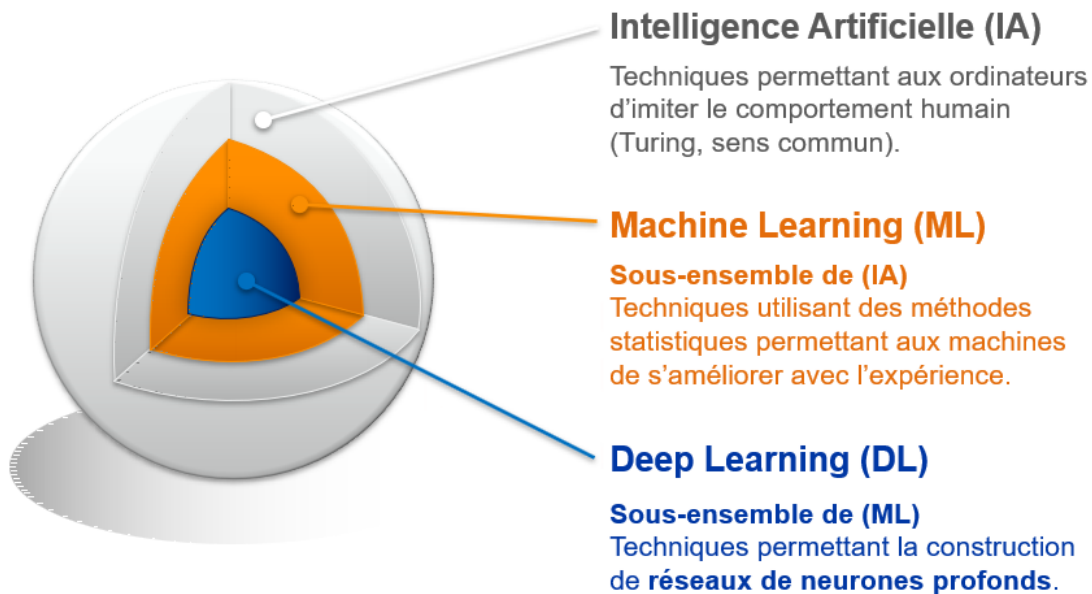


Figure 2.1: L'IA et ses sous-ensembles

2.1.2 Les différents types de Machine Learning

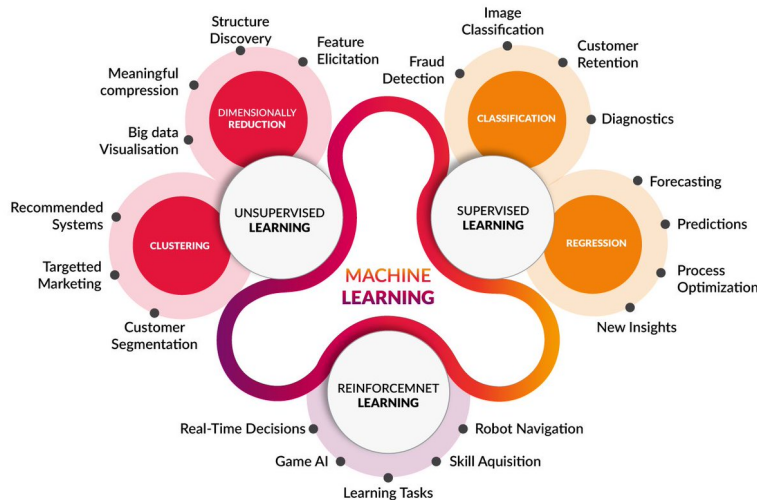


Figure 2.2: Les différents types de Machine Learning.
(Source : www.cognub.com)

2.1.3 Les différents types de données

- Les données **tabulaires** : ce sont des **tableaux** où toutes les lignes sont appelées des *enregistrements* et toutes les colonnes des *champs*. Chaque champ permet de stocker un *type* spécifique de variable (texte, nombre, date, etc.). Ce format est particulièrement présent dans les bases de données relationnelles.

	age (n)	job (c)	marital (c)	education (c)	balance (n)	housing (c)
0	30	unemployed	married	primary	1787	no
1	33	services	married	secondary	4789	yes
2	35	management	single	tertiary	1350	yes
3	30	management	married	tertiary	1476	yes
4	59	blue-collar	married	secondary	0	yes
5	35	management	single	tertiary	747	no

Figure 2.3: Exemple de données tabulaires.
(Source : UCI Machine Learning Repository)

- Les **images** : elles sont représentées sous la forme de **matrices** dont les éléments sont des *entiers* appelés *pixels*. Plus le nombre de pixels dans une image est élevé, plus sa *résolution* est grande.

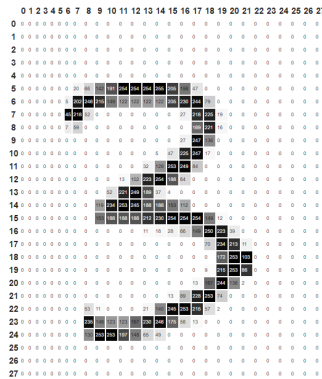


Figure 2.4: Exemple d'une image matricielle en niveaux de gris (valeurs des pixels entre 0 et 255 [codage sous 8 bits]).

(Source : base de données MNIST)

- Les données **textuelles** : elles sont représentées sous la forme de **chaînes de caractères**. Un code où chaque chiffre représente un caractère peut être utilisé pour convertir un texte en binaire (*ASCII* ou *Unicode*) de façon à pouvoir être traité par ordinateur.
- Les **séries temporelles** : ce sont des **séries** de données indexées par le **temps**, on parle également de séries *chronologiques*. Elles représentent l'évolution d'une *quantité spécifique* ou d'un *phénomène physique* (à l'aide de *capteurs*) au cours du temps.

Converting the text "hope" into binary

Characters:	h	o	p	e
ASCII Values:	104	111	112	101
Binary Values:	01101000	01101111	01110000	01100101
Bits:	8	8	8	8

Figure 2.5: Conversion d'un mot en code binaire.

(Source : www.computerhope.com)

Figure 2.6: Exemple d'une série temporelle (cours de l'action Tesla).

2.1.4 Feature engineering

A process that involves the following steps:

Determining which features might be useful in training a model. Converting raw data from the dataset into efficient versions of those features. For example, you might determine that temperature might be a useful feature. Then, you might experiment with bucketing to optimize what the model can learn from different temperature ranges.

Feature engineering is sometimes called feature extraction.

Rebaser, Standardiser et Normaliser

- Rebaser ou redimensionner un vecteur signifie ajouter ou soustraire une constante, puis multiplier ou diviser par une constante, comme vous le feriez pour changer les unités de mesure des données, par exemple, pour convertir une température de Celsius en Fahrenheit.
- Standardiser un vecteur signifie soustraire une mesure de localisation et la diviser par une mesure d'échelle. Ce process suppose implicitement que les données ont une distribution gaussienne et donne de meilleurs résultats dans ce cas.
- Normaliser un vecteur signifie le plus souvent diviser par une norme du vecteur et ramener l'ordonnée à l'origine. C'est important de le faire lorsque nous comparons des mesures qui ont différentes unités. Les variables qui sont mesurées à différentes échelles ne contribuent pas également à l'analyse et pourraient finir par créer un biais. Ce processus a pour effet de borner les variables, ce qui permet de comparer les évolutions des variables, de leur donner un sens.

Exemple : cas si les composante sont de différents ordres

Age	Loan	House Price Index	Distance
0.125	0.11	135	0.7652
0.375	0.21	256	0.5200
0.625	0.31	231	0.3160
0	0.01	267	0.9245
0.375	0.50	139	0.3428
0.8	0.00	150	0.6220
0.075	0.38	127	0.6669
0.5	0.22	216	0.4437
1	0.41	139	0.3650
0.7	1.00	250	0.3861
0.325	0.65	264	0.3771
0.7	0.61	?	

Figure 2.7: Rendre des grandeurs comparables. Source saedsayad.com

On peut standardiser si la méthode s'y prête :

$$x_{0,standard} = \frac{x_0 - x_{min}}{\sigma} \text{ avec } \sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - x_{mean})^2} \quad (2.1)$$

ou, si l'on souhaite des variables bornées, on normalise :

$$x_{0,norm} = \frac{x_0 - x_{min}}{x_{max} - x_{min}} \quad (2.2)$$

Reduction de dimension

La réduction des dimensions nous aide à réduire le nombre de caractéristiques en convertissant les données de l'espace des caractéristiques en un espace latent. Cet espace plus petit est censé représenter les caractéristiques apprises d'une manière plus comprimée. Si nous choisissons correctement la dimension de l'espace latent, nous pouvons même réduire le bruit et supprimer les informations non pertinentes. Le grand inconvénient de cette approche est sa capacité d'explication. En passant de l'espace des caractéristiques à l'espace latent, nous ne regardons plus directement les caractéristiques, mais seulement leurs représentations, qui n'ont généralement pas d'explication intuitive ou de logique comme les caractéristiques. Cette approche devient pratique lorsque l'on travaille sur des données non structurées, telles que des images ou des signaux.

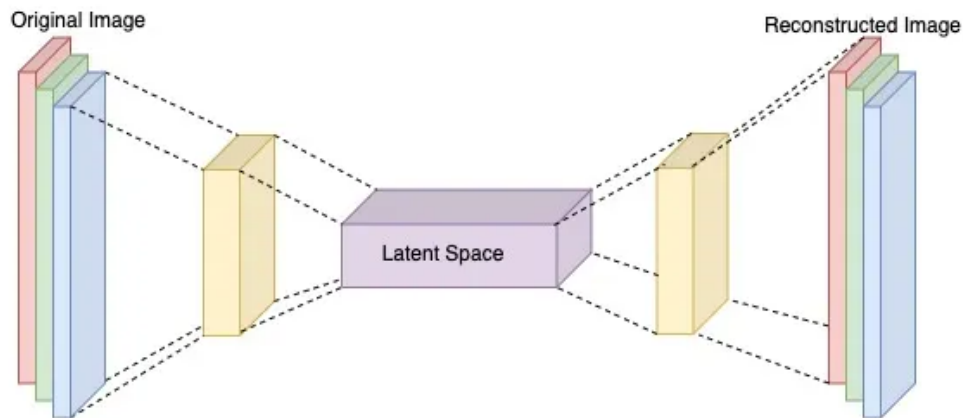


Figure 2.8: Espace latent.
(source TDS)

Feature selection

Les méthodes de sélection des caractéristiques vous permettent de choisir les meilleures caractéristiques dans votre ensemble de données, en fonction de certains critères. Bien que cela paraisse, il existe BEAUCOUP de méthodes différentes, chacune définissant la qualité d'une caractéristique d'une manière légèrement différente, et pour sélectionner correctement la meilleure caractéristique, vous aurez probablement besoin de plus d'une méthode.

Tout d'abord, un peu de terminologie : Les méthodes de sélection des caractéristiques peuvent être divisées en trois familles :

- Méthodes de filtrage : La sélection des caractéristiques est effectuée dans le cadre du prétraitement, c'est-à-dire avant la formation d'un modèle. Nous filtrons les caractéristiques dont les performances sont médiocres en fonction de certains critères. Un critère naïf serait une simple corrélation. Cette approche est particulièrement efficace lorsque nous disposons d'ensembles de données volumineux.
- Méthodes dites enveloppantes : Choisir de manière itérative un sous-ensemble de caractéristiques, entraîner votre modèle et choisir la meilleure combinaison. Comme vous pouvez déjà le supposer, cette approche est très coûteuse et presque irréalisable.
- Les méthodes intégrées : Elles tirent parti des estimations de l'importance des caractéristiques qui sont intégrées dans l'algorithme. Par exemple, Random Forest choisit un sous-ensemble différent de caractéristiques afin d'éviter un surajustement. Une fois le modèle formé, nous pouvons voir combien de nos caractéristiques ont amélioré les performances de prédiction. Néanmoins, comme pour les tâches de classification, en se basant uniquement sur des méthodes intégrées, nous pourrions finir par suradapter la sélection des caractéristiques à un algorithme donné, ce qui nous empêcherait d'utiliser différents algorithmes et différents ensembles de données, en particulier lors de l'utilisation de méthodes à haute variance comme les arbres de décision.

Les distances

“Si vous ne pouvez pas le mesurer, vous ne pouvez pas l'améliorer.”

Il s'agit d'une des citations que vous verrez assez souvent attribuées “à tort” au Dr William Edwards Deming. Ce serait lord Kelvin qui aurait tenue des propos de la sorte.

Demings tenait d'ailleurs un propos contraire : “Il est faux de supposer que si l'on ne peut pas mesurer, on ne peut pas gérer - un mythe coûteux.”

Si vous lisez attentivement la citation originale, elle dit le contraire de ce que la plupart des gens pensent lorsqu'ils lisent la citation abrégée. C'est l'un des dangers de l'utilisation de citations sans contexte.

Il savait également qu'il ne suffisait pas de mesurer les choses et d'examiner les données. Il y a beaucoup de choses qui ne peuvent pas être mesurées et qui doivent quand même être gérées. Et il y a beaucoup de choses qui ne peuvent pas être mesurées et sur lesquelles les managers doivent quand même prendre des décisions. La data n'est pas tout, mais elle peut faire beaucoup.

Pourquoi la citation la plus courte est-elle si souvent utilisée et (mal) attribuée au Dr Deming ? Il a insisté sur l'importance d'utiliser des données pour confirmer les convictions concernant les stratégies et pratiques de gestion qui fonctionnent et celles qui ne fonctionnent pas. Et pour les utiliser, il faut les mesurer.

Quelques métriques de proximités :

Afin de définir les k plus proches voisins, il faut pouvoir mesurer la distance entre deux points.

Métriques de proximités	type des données
Tableau de contingence (ratio de Odd, coefficient de Phi, V de Cramer), Coefficient de Jaccard, distance	Binaire
Z-score, Min-max, Normalisation, distance	numérique
produit scalaire, produit vectoriel	vecteur
Dynamic time wrapping (DTW), Shapelets	série temporelle

Les principales distances utilisés:

- Distance de Manhattan / norme 1 :

Il s'agit de la distance entre deux points parcourue par un taxi lorsqu'il se déplace dans une ville où les rues sont agencées selon un réseau ou quadrillage, comme de Manhattan.

$$d_1(x_0) = \sum_{i \in D} |x_0 - x_i| \quad (2.3)$$

- Distance euclidienne / norme 2 :

La plus connues et la plus familière. Elle est sensible aux valeurs extrêmes.

$$d_2(x_0) = \sqrt{\sum_{i \in D} (x_0 - x_i)^2} \quad (2.4)$$

- Norme p ou distance de Minkowski:

Il s'agit d'une généralisation des deux précédentes à une dimension quelconque.

$$d_p(x_0) = \left(\sum_{i \in D} (x_0 - x_i)^p \right)^{1/p} \quad (2.5)$$

- Norme infinie ou distance de Minkowski:

C'est la limite à l'infini de la distance précédente.

$$d_\infty(x_0) = \max_{i \in D} (|x_0 - x_i|) \quad (2.6)$$

- Distance pondérée :

$$d_\omega(x_0) = \sqrt{\sum_{i \in D} \omega_i \times (x_0 - x_i)^2} \text{ avec } \sum_i \omega_i = 1 \quad (2.7)$$

On utilise les distances précédentes avec des attributs numériques.

- Distance cosinus :

Cette mesure est approprié pour comparer des vecteurs représentant des données, par exemples des mots ou des textes.

$$d_{cos}(x_0) = \frac{x_i^T \cdot x_j}{\sqrt{(d(x_i))} \times \sqrt{(d(x_j))}} \quad (2.8)$$

- Corrélation de Pearson :

$$d_{Pearson}(x_0) = 1 - \frac{Cov(x_i, x_j)}{\sqrt{d(x_i)} \times \sqrt{d(x_j)}} \quad (2.9)$$

- Distance de Mahalanobi :

Cette mesure est utilisé pour comparer des ensembles, des distributions.

$$d_{Mahalanobi}(x_0) = \sqrt{(x_i - x_j)^T \cdot S^{-1} \cdot (x_i - x_j)} \quad (2.10)$$

S est la matrice de covariance entre les deux ensembles. Si celle-ci est égale à la matrice identité, la distance de Mahalanobi devient distance euclidienne normalisée ou pondéré par son écart-type :

$$d_{Mahalanobi}(x_0) = \sqrt{\sum_i \frac{(x_i - y_i)^2}{\sigma_i^2}} \quad (2.11)$$

Table 2.1: Rex: mesures de distance et de similarité

Mesure	Description	Application
Euclidienne	Distance en ligne droite entre deux points dans un espace multidimensionnel.	Clustering, algorithmes des plus proches voisins.
Cosinus	Mesure la similarité entre deux vecteurs via le cosinus de l'angle qui les sépare.	Recherche textuelle, systèmes de recommandation.
Hamming	Compte le nombre de positions différentes entre deux chaînes de même longueur.	Détection d'erreurs, similarité de documents.
Manhattan	Somme des différences absolues entre les coordonnées de deux points.	Analyse de réseaux, traitement d'images.
Minkowski	Généralisation des distances euclidienne et de Manhattan.	Flexibilité selon le paramètre choisi.
Chebyshev	Distance maximale entre les coordonnées de deux points.	Algorithmes de jeu, analyse de scénarios pessimistes.
Jaccard	Mesure la dissimilarité entre deux ensembles.	Fouille de données, recommandation.
Haversine	Distance entre deux points sur une sphère.	Analyse géospatiale, navigation.
Sorensen-Dice	Mesure la dissimilarité entre deux ensembles.	Recherche textuelle, analyse de séquences biologiques.

2.1.5 Quelques résultats mathématiques utiles

Le machine learning est fondamentalement des statistiques. Donc pour comprendre ce qu'il se passe et aller plus loin dans l'utilisation des algorithmes, il faut comprendre la modélisation mathématique et ce qu'elle représente. Voici des résultats de statistique avancée qui servent.

Table 2.2: Rex : Lois de probabilité et applications

Loi	Définition	Applications en risque de crédit
Bernoulli	Variable binaire (0 ou 1) : succès ou échec.	Modélise l'événement de défaut individuel (défaut ou non-défaut). Base des modèles de scoring (logit, probit). Exemple : Si un emprunteur a une PD de 2%, on tire une Bernoulli(0,02) pour simuler un défaut.
Binomiale	Somme de n lois de Bernoulli (n essais indépendants).	Modélise le nombre de défauts dans un portefeuille homogène (même PD). Exemple : Pour 1000 prêts avec PD = 1%, la variable binomiale $B(1000, 0.01)$ donne la distribution des défauts.
Poisson	Modélise le nombre d'événements rares sur un intervalle de temps donné. Pratique pour modéliser des événements rares agrégés.	Utilisée pour modéliser les fréquences de défauts sur des portefeuilles de grande taille. Utilisée aussi dans les modèles de durée/intensité (Jarrow-Turnbull).
Exponentielle	Loi des temps d'attente entre deux événements dans un processus de Poisson. Utile pour modéliser des temps d'attente cumulés.	Modélise le temps jusqu'au défaut. Utilisée dans les modèles de durée.
Normale	Distribution en cloche. Base du théorème central limite.	Dans le modèle de Merton, la valeur des actifs suit un mouvement brownien (distribution normale). La distance au défaut est calculée à partir de cette hypothèse.
Log-normale	Exponentielle d'une normale.	Modélise des taux de recouvrement (LGD) positifs.
Beta	Distribution sur $[0,1]$, paramétrable pour des variables continues bornées.	Modélise souvent les taux de recouvrement (LGD), car elle s'adapte bien à des distributions asymétriques.
Gamma	Distribution de durée, flexible, non symétrique.	Très utilisée en modèles de survie (analyse du temps jusqu'au défaut ou à un événement).
Weibull	Distribution de durée, flexible.	Modélise des LGD dynamiques ou temps de recouvrement.
Copules	Outils pour modéliser la dépendance entre variables aléatoires.	Essentielles en modélisation de portefeuilles de crédit (ex. : modèle de Vasicek, copule de Li). Permet de capter le risque systémique (ex. défauts simultanés).

Les copules en modélisation probabiliste

En statistiques et en modélisation probabiliste, une **copule** est une fonction qui permet de modéliser la structure de dépendance entre plusieurs variables aléatoires, **indépendamment de leurs distributions marginales**. Lorsqu'on parle d'un **paramètre pour une copule**, on fait généralement référence à un ou plusieurs paramètres qui définissent la forme et l'intensité de la dépendance entre les variables.

Comme de nombreux processus physiques n'ont pas pu être modélisé de manière déterministe. Une approche statistique est donc utilisée pour se donner une représentation. Tout comme les modèles déterministes ont des plages de validités, les modèles statistiques peuvent être évalué selon divers méthodes, comme les mesures d'association. Dans la plupart des modèles, plusieurs variables sont à prendre en compte, et ces variables interagissent, sont corrélées. Modéliser et ajuster cette interdépendance est difficile. Mais, d'après le théorème de Sklar, une statistique multivariée peut être considéré comme l'association des statistiques, ou lois marginales, de chacune des variables et d'un outil appelé copule qui modélise les interactions, et que la distribution des variables n'est changée de manière non-monotone. Les copules sont d'ailleurs bien meilleurs pour capturer les relations entre des variables que les corrélations ([Neural vs Classical Copulas](#))

Exemples de paramètres selon le type de copule

- **Copule de Gauss (ou normale) :**

- Le paramètre est la **matrice de corrélation** (par exemple, le coefficient de corrélation linéaire ρ pour deux variables).
- Ce paramètre détermine comment les variables sont corrélées.

- **Copule de Student (ou t-copule) :**

- Deux paramètres principaux :
 1. Le degré de liberté (ν), qui influence l'épaisseur des queues de la distribution.
 2. La matrice de corrélation (ρ), similaire à la copule de Gauss.

- **Copule de Clayton :**

- Un seul paramètre (θ), qui contrôle la dépendance dans les **queues inférieures** (plus θ est grand, plus la dépendance est forte).

- **Copule de Gumbel :**

- Un paramètre (θ), qui contrôle la dépendance dans les **queues supérieures** (plus θ est grand, plus la dépendance est forte).

- **Copule de Frank :**

- Un paramètre (θ), qui peut modéliser à la fois des dépendances symétriques **positives** et **négatives**.

Le paramètre d'une copule permet de **calibrer le modèle** pour qu'il reflète au mieux la dépendance observée dans les données. Par exemple, dans le cas de la copule de Clayton, un θ élevé signifie que les variables ont tendance à prendre des **valeurs extrêmes simultanément** (dépendance en queue inférieure).

Table 2.3: Rex : mesures d'information et de divergence

Mesure	Définition	Cas d'usage
Entropie	Mesure de l'incertitude ou du désordre dans une distribution.	Calcul du gain d'information dans les arbres de décision.
Impureté de Gini	Alternative à l'entropie pour accélérer les calculs.	Accélération des algorithmes basés sur les arbres.
Divergence de Kullback-Leibler	Mesure de la différence entre deux distributions de probabilité.	Inférence variationnelle.
Inégalité de Jensen	Utilisée pour dériver la borne inférieure de l'évidence (ELBO).	Inférence variationnelle, statistiques inférentielles.
Divergence de Jensen-Shannon	Version symétrique et lissée de la divergence de KL.	Comparaison de distributions.
Entropie conditionnelle	Mesure de l'incertitude restante dans X sachant Y.	Calcul du gain d'information.
Information mutuelle	Mesure de l'information partagée entre deux variables aléatoires.	Sélection de variables pertinentes.
Information mutuelle normalisée	Information mutuelle normalisée entre 0 et 1.	Évaluation de la dépendance entre variables.
Entropie croisée	Mesure de la différence entre la distribution prédite et la vraie distribution.	Fonction de perte en classification, évaluation des modèles de langage (perplexité).
Information de Fisher	Mesure de l'information qu'une variable aléatoire apporte sur un paramètre de sa distribution.	Estimation de paramètres, optimisation de modèles.

2.1.6 Évaluation de la performance d'un algorithme

- Une fois qu'un algorithme est entraîné et que l'on est capable de l'éprouver avec un jeu de données de test, la première question que l'on peut se poser est la suivante : *“Comment savoir si mon système est performant ?”*.
- Selon qu'il s'agisse d'un exercice de classification ou de régression, l'évaluation et les métriques impliquées sont différentes.
- Une évaluation rigoureuse des performances d'un algorithme est une étape indispensable à son déploiement.
- “Ce qui ne peut pas être mesuré ne peut pas être amélioré” - Peter Drucker

Évaluation d'un modèle de classification

- Dans ce type de modèle, les données utilisées sont labellisées et le système doit pouvoir attribuer une classe à chaque observation.
- Dans ce qui suit, nous nous intéresserons au cas particulier d'un *classificateur binaire*.

- Considérons un algorithme capable de prédire la présence d'un incendie dans une pièce donnée. Plusieurs cas sont alors possibles et sont rassemblés dans ce que l'on appelle une **matrice de confusion** (Fig. 2.9).

		Classe réelle	
		-	+
Classe prédite	-	True Negatives (vrais négatifs)	False Negatives (faux négatifs)
	+	False Positives (faux positifs)	True Positives (vrais positifs)

Figure 2.9: Matrice de confusion d'un classificateur binaire.
(Source : openclassrooms.com)

En apprentissage supervisé, une matrice de confusion permet de qualifier rapidement un système de classification.

Appelons *positive* la classe correspondant à un incendie et *négative* l'autre (pas d'incendie). Voici les différents cas qui peuvent se présenter :

- **Vrai négatif** (TN) : le système ne prédit pas d'incendie et il n'y en a effectivement pas.
- **Faux négatif** (FN) : le système ne prédit pas d'incendie alors qu'il y en a bien un.
- **Faux positif** (FP) : le système prédit la présence d'un incendie mais il n'y en a pas.
- **Vrai positif** (TP) : le système prédit la présence d'un incendie et il y en a bien un.

A partir de la matrice de confusion, nous pouvons dériver **plusieurs critères de performance** :

- **Justesse** (*Accuracy*) – Il s'agit de la proportion d'éléments correctement prédits. Ce critère reflète la **performance globale** de notre modèle :

$$\text{Accuracy} = \frac{TN + TP}{TN + FN + FP + TP} \quad (2.12)$$

- **Rappel** (*Recall*) – Il s'agit du **taux de vrais positifs**, c'est à dire la proportion de positifs correctement identifiés. Ce critère reflète la capacité de notre modèle à **détecter tous les incendies** :

$$\text{Rappel} = \frac{TP}{TP + FN} \quad (2.13)$$

Remarque : Il est facile d'obtenir un excellent score de rappel en prédisant tout le temps *positif*. Aucun incendie ne nous échappera mais il s'agit de tout sauf d'apprentissage automatique !

- **Spécificité** (*Specificity*) – Il s'agit du **taux de vrais négatifs**. Ce critère reflète la capacité de notre modèle à **détecter toutes les situations où il n'y a pas d'incendie** :

$$\text{Spécificité} = \frac{TN}{FP + TN} \quad (2.14)$$

- **Précision** (*Precision*) – Il s'agit de la proportion de prédictions correctes parmi les éléments qui ont été prédits positifs. Ce critère reflète la capacité de notre modèle à **ne déclencher une alarme qu'en présence d'un incendie** :

$$\text{Précision} = \frac{TP}{TP + FP} \quad (2.15)$$

Remarque : Là encore, il est possible d'avoir une très bonne précision en ne prédisant que très peu de fois *positif*. On risque moins de se tromper mais c'est très dangereux !

Une bonne pratique consiste à utiliser un critère de **compromis** entre le *rappel* et la *précision* que l'on appelle le F_1 -score. Il s'agit simplement de leur *moyenne harmonique* :

$$F_1\text{-Score} = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} = \frac{2TP}{2TP + FP + FN} \quad (2.16)$$

La **courbe ROC** (de l'anglais *Receiver Operating Characteristic*), est une courbe représentant le taux de vrais positifs (*rappel*) en fonction du taux de faux positifs ($1 - \text{spécificité}$). Elle est obtenue en faisant varier un *seuil de discrimination*.

Par exemple, $Y = 1$ si $h(X) \geq \text{seuil de discrimination}$, h étant le prédicteur de X .

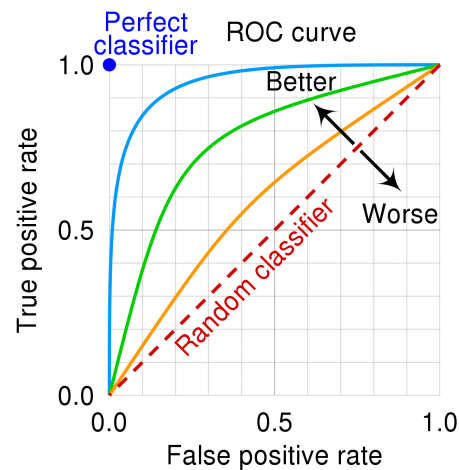


Figure 2.10: Une courbe ROC. La performance d'un modèle naïf (aléatoire) est indiquée en pointillés. Plus la courbe tend à épouser les axes supérieurs gauches, plus le modèle est performant

Alors que la courbe ROC est une courbe de probabilité, l'aire sous la courbe ROC, notée **AUC** (de l'anglais *Area Under the Curve*) représente le degré ou la **mesure de séparabilité**. Plus le critère AUC est élevé, meilleur est le modèle à distinguer les deux classes. Le critère est compris entre 0 et 0.5.

Évaluation d'un modèle de régression

Contrairement à la classification, nous ne pouvons plus utiliser la métrique de *précision*. Nous devons utiliser des mesures d'erreur spécifiquement conçues pour évaluer les prédictions faites sur des problèmes de régression.

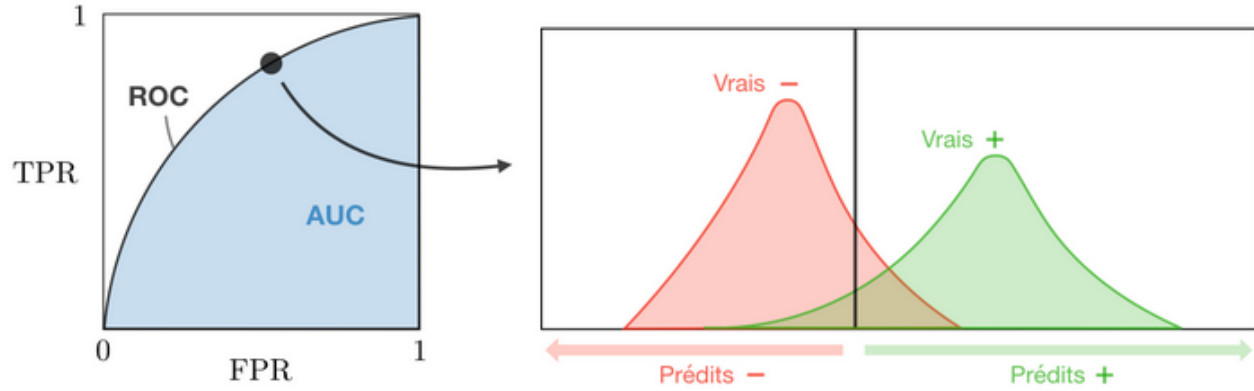


Figure 2.11: Illustration du critère AUC (Source : stanford.edu).

Soit x_i un point d'un jeu de N données de test ayant y_i pour *vrai label* et $\hat{y}_i$ pour *label prédit* par un modèle de régression.

- **MAE (Mean Absolute Error)** – Elle représente la moyenne de la différence absolue entre les valeurs réelles et les valeurs prédites. Elle mesure la **moyenne des résidus** du jeu de données :

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (2.17)$$

- **MSE (Mean Squared Error)** – Elle représente la moyenne de la différence au carré entre les valeurs originales et les valeurs prédites. Elle mesure la **variance des résidus** du jeu de données :

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2.18)$$

- **RMSE (Root Mean Squared Error)** – Il s'agit de la racine carrée de la MSE. Elle mesure **l'écart-type des résidus** du jeu de données :

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (2.19)$$

- **R^2 (Coefficient de détermination)** – Il représente la **proportion de la variance de la variable dépendante** qui est expliquée par le modèle de régression linéaire. Il s'agit d'un **score sans échelle**, c'est-à-dire que peu importe le fait que les valeurs soient petites ou grandes, la valeur de R^2 sera **inférieure à un** :

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (2.20)$$

Remarque : On note ce coefficient de détermination R^2 car il s'agit du **carré de la corrélation de Pearson** entre les vraies valeurs et les valeurs prédites.

Table 2.4: Rex : critères d'évaluation de modèles

Critère	Description	Utilité
AIC / BIC	Critères pour comparer des modèles en pénalisant la complexité.	Sélection de modèles.
Cp de Mallows	Équilibre entre biais et variance.	Sélection de modèles en régression.
R² et R² ajusté	Mesure de la qualité d'ajustement d'un modèle.	Évaluation de la performance prédictive.
RMSE / MAE	Mesure de l'erreur de prédiction.	Quantification de l'erreur moyenne.
PRESS	Somme des carrés des résidus prédits.	Validation croisée.
Log-vraisemblance	Mesure de la performance d'un modèle probabiliste.	Évaluation de la qualité d'ajustement.

2.1.7 Train, validation, test / Cross validation

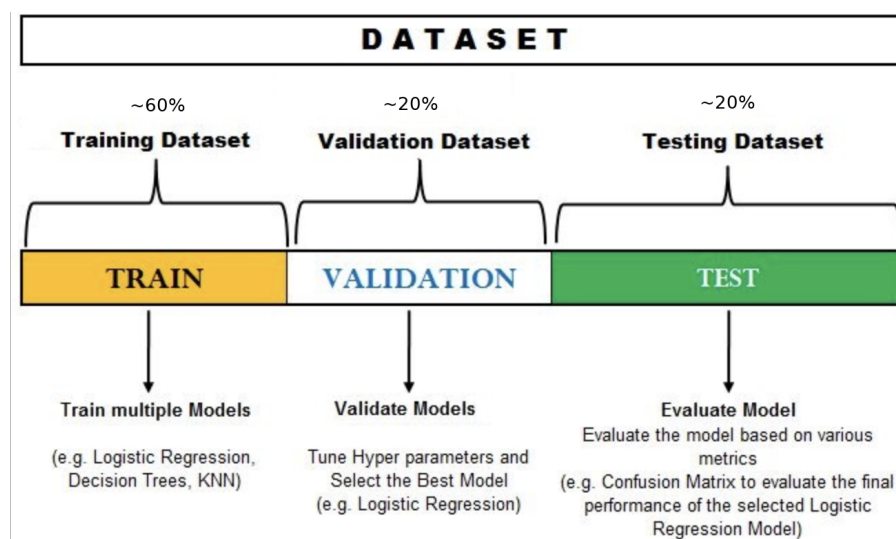


Figure 2.12: Séparation d'un jeu de données en 3 sous-ensembles. Source TDS

Cross-validation

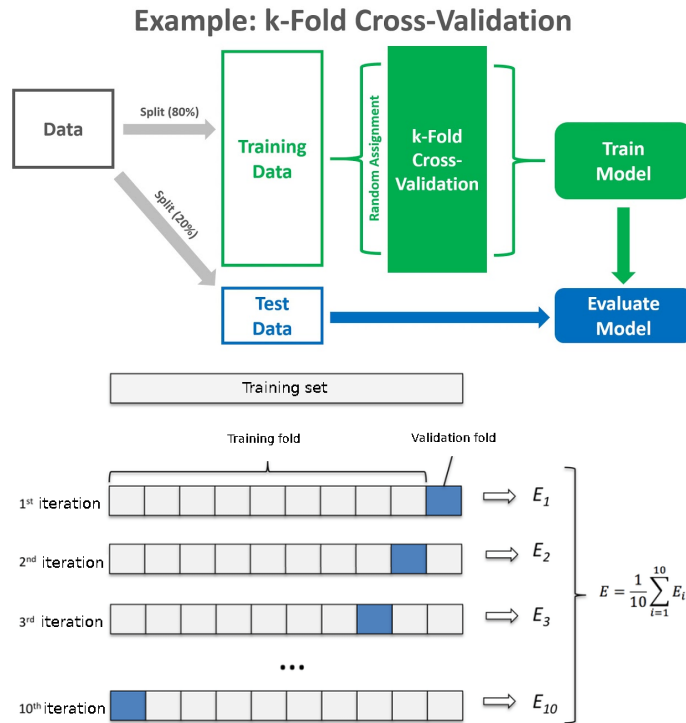


Figure 2.13: Source article

2.1.8 Cas des séries temporelles

Attention à ne pas utiliser le futur pour prédire le présent, voir le passé...

Bien incorporer les "lags" (retard dans la publication des événements) par rapport au système considéré.

Alertes saisonnières et non-stationnaires Une série temporelle est dite stationnaire lorsque ses propriétés statistiques (telles que la moyenne et la variance) ne changent pas au cours de la période d'observation de la série. Inversement, une série temporelle est non stationnaire lorsque ses propriétés statistiques dépendent du temps. Par exemple, les séries temporelles présentant des tendances et une saisonnalité (voir plus loin) ne sont pas stationnaires, car ces phénomènes peuvent affecter la valeur de la série temporelle à différents moments.

Les processus stationnaires sont comparativement plus faciles à analyser, car il existe une relation statique entre le temps et les variables. En fait, la stationnarité est devenue une hypothèse courante pour la plupart des analyses de séries temporelles.

Bien qu'il existe des modèles pour les séries temporelles non stationnaires, la plupart des algorithmes ML s'attendent à une relation statique entre les caractéristiques d'entrée et la sortie. Lorsque la série temporelle n'est pas stationnaire, la précision d'un modèle modélisé à partir des données varie en différents points. Cela signifie que les choix de modélisation sont affectés par la nature stationnaire/non stationnaire de la série temporelle, et que différentes étapes de préparation des données s'appliquent lorsque vous souhaitez convertir la

Sliding window vs. forward chaining cross validation

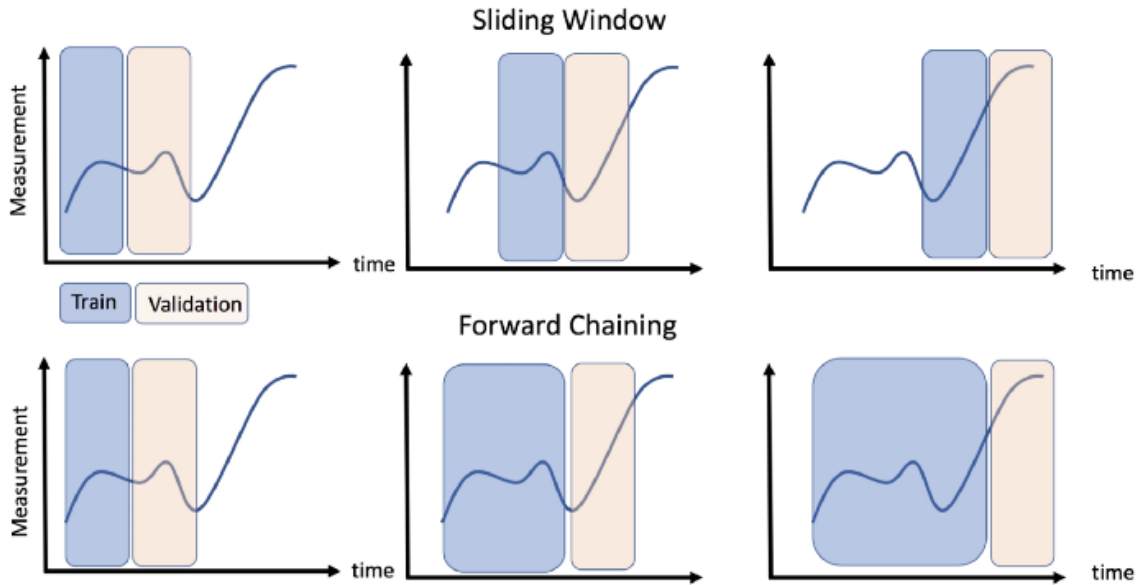


Figure 2.14: Cas des séries temporelles. Source TDS

série temporelle en une série stationnaire.

La saisonnalité dans les séries temporelles est un scénario dans lequel les données subissent des changements réguliers et prévisibles qui se répètent au cours d'un cycle défini. Cette saisonnalité peut masquer le signal que nous souhaitons modéliser lors de la modélisation des séries temporelles et, pire encore, elle peut fournir un signal fort aux modèles.

Les informations recueillies grâce au profilage des données, la nature des séries temporelles et les alertes telles que la non-stationnarité et la saisonnalité vous donnent une longueur d'avance pour comprendre les données de séries temporelles que vous avez sous la main. Cela ne signifie pas que l'analyse exploratoire des données est terminée - l'objectif est d'utiliser ces informations comme point de départ et de travailler à une analyse plus approfondie des données et à d'autres étapes de préparation des données.

2.1.9 Underfitting et overfitting

2.1.10 Compromis biais / variance

Les **trois ingrédients** d'un algorithme d'apprentissage supervisé sont :

- l'espace des hypothèses, le modèle,
- la fonction de coût ou loss,
- l'algorithme d'optimisation qui permet de trouver l'hypothèse optimale au sens de la fonction de coût sur les données (minimisation du risque empirique).

Le compromis biais-variance traduit le compromis entre l'erreur d'approximation, correspondant au biais de l'algorithme d'apprentissage, et l'erreur d'estimation, correspondant à sa variance.

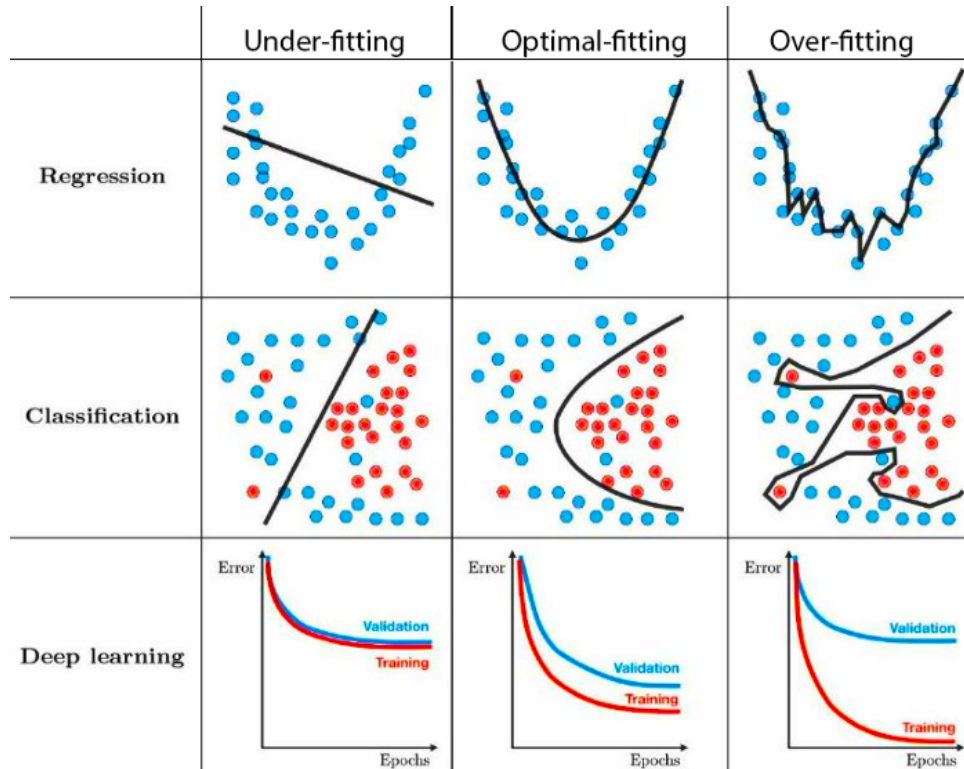


Figure 2.15: Sous-interprétation et sur-interprétation.
(Source : Towards Data Science)

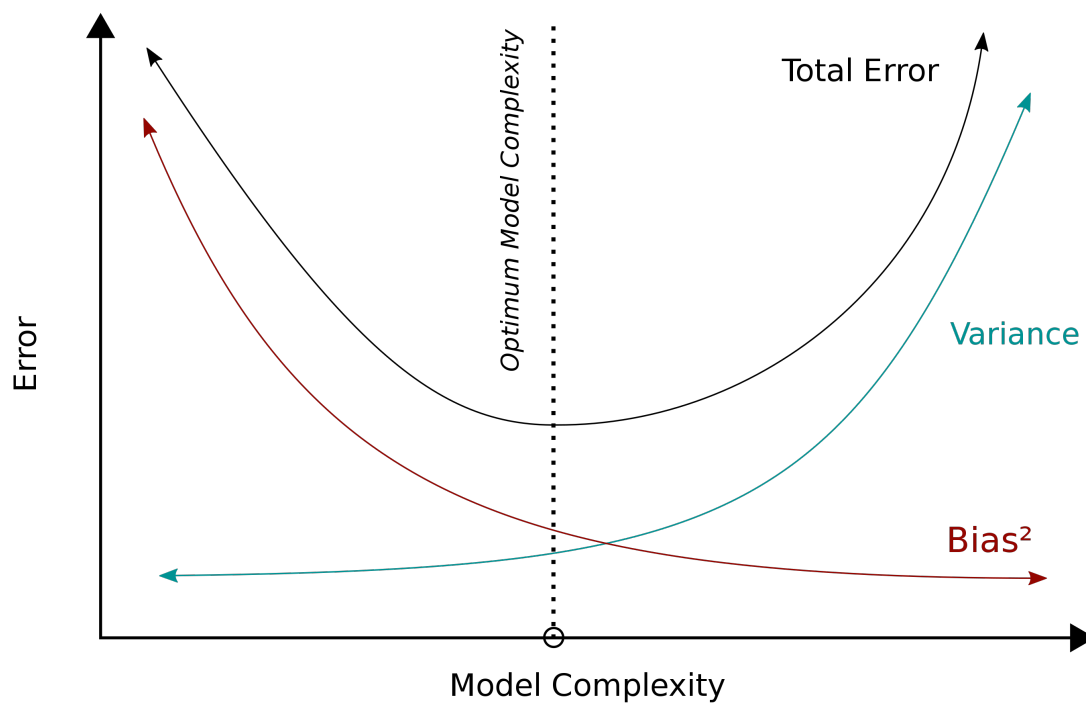


Figure 2.16: Compromis biais / variance. (Source [wikimedia.org](https://commons.wikimedia.org/wiki/File:Bias-variance-tradeoff.png))



Figure 2.17: Courbes d'apprentissages typiques dans un cas de *high variance* (gauche) et *high bias* (droite)

Source dataquest.io

La **généralisation** et le **sur-apprentissage** sont des **préoccupations majeures** en machine learning : comment s'assurer que des modèles entraînés pour minimiser leur erreur de prédiction sur les données observées seront généralisables aux données pour lesquelles il nous intéresse de faire des prédictions.

Courbes d'apprentissages

Donnée une tâche, l'erreur de *Bayes* σ_{Bys} est l'erreur minimale. Il s'agit d'une bonne approximation de l'erreur d'un expert humain. On peut donc avoir les cas suivants:

- $\sigma_{\text{train}} \sim \sigma_{\text{Bys}}$ et $\sigma_{\text{test}} \gg \sigma_{\text{train}}$ alors : *High Variance*
- $\sigma_{\text{train}} \gg \sigma_{\text{Bys}}$ et $\sigma_{\text{test}} \sim \sigma_{\text{train}}$ alors : *High Bias*
- $\sigma_{\text{train}} \gg \sigma_{\text{Bys}}$ et $\sigma_{\text{test}} \gg \sigma_{\text{train}}$ alors : *High Variance and High Bias*
- $\sigma_{\text{train}} \sim \sigma_{\text{Bys}}$ et $\sigma_{\text{test}} \sim \sigma_{\text{train}}$ alors : **Low Variance and Low Bias**

Pour arriver au compromis optimal, on peut prendre les décisions suivantes :

- Avoir plus des données d'apprentissage : réduit la Variance
- Choisir un sous-ensemble des *features* : réduit la Variance
- Augmenter les *features* : réduit le Biais
- Complexifier le modèle : réduit le Biais
- Réduire la régularisation des paramètres : réduit le Biais
- Augmenter la régularisation des paramètres : réduit la Variance

Points clefs

- Pour éviter le sur-apprentissage, il est essentiel lors de l'étape de sélection du modèle de valider les différents modèles testés sur un jeu de données différent de celui utilisé pour l'entraînement.
- Pour estimer la performance en généralisation d'un modèle, il est essentiel de l'évaluer sur des données qui n'ont été utilisées ni pour l'entraînement, ni pour la sélection de ce modèle.

- De nombreux critères permettent d'évaluer la performance prédictive d'un modèle. On les choisira en fonction de l'application.
- Pour interpréter la performance d'un modèle, il peut être utile de le comparer à une approche naïve.

2.1.11 Régularisation

La régularisation est une technique utilisée en apprentissage automatique pour améliorer la performance d'un modèle en empêchant le surapprentissage. Elle consiste à ajouter un terme de pénalité à la fonction de coût du modèle.

Lasso, Tikhonov et Elastic Net sont des méthodes de régularisation courantes. Lasso ajoute une pénalité basée sur la norme L1 des coefficients du modèle, ce qui favorise des modèles parcimonieux (avec peu de coefficients non nuls). Tikhonov (ou ridge) utilise la norme L2, ce qui réduit la magnitude des coefficients. Elastic Net combine les deux, offrant un compromis entre les deux approches.

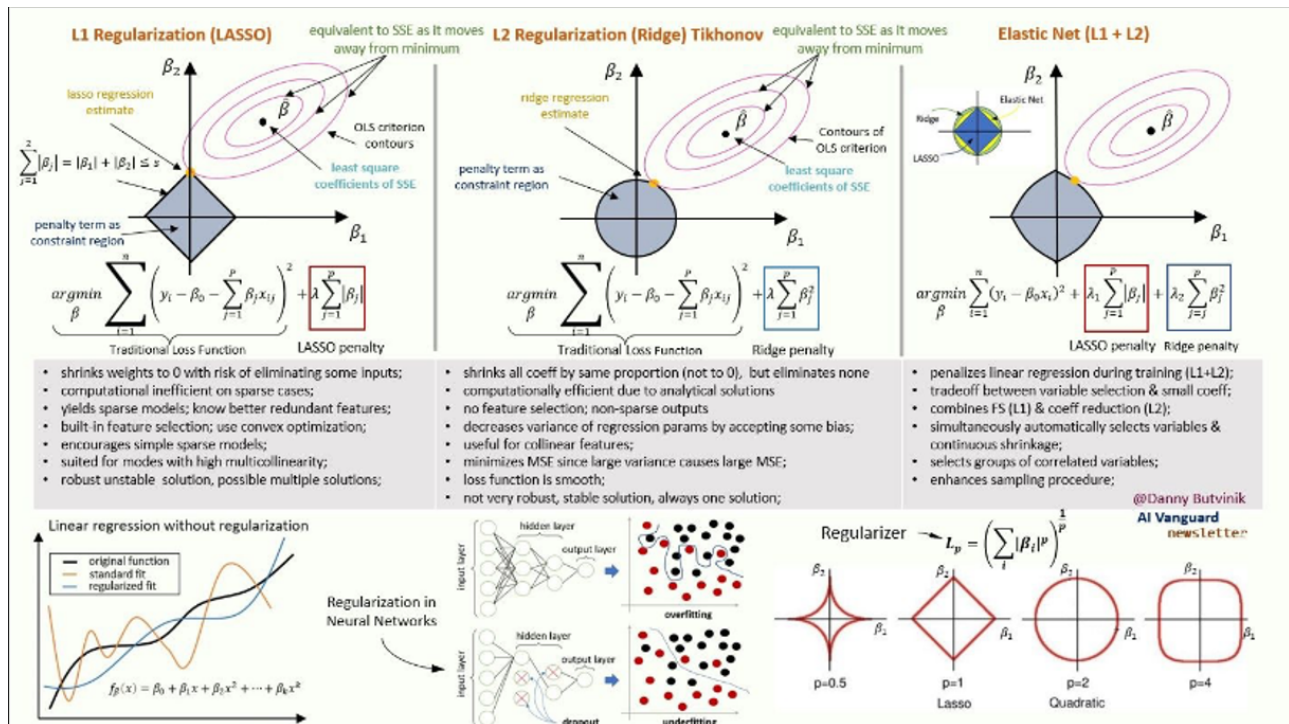


Figure 2.18: Principales méthodes de régularisation.

2.1.12 interprétabilité

Pourquoi l'interprétabilité ?

Il y a une légende urbaine dans le milieu du machine du learning qui illustre très bien l'importance de l'interprétabilité. Le Pentagon demande un logiciel pour identifier des chars camouflés.

- 100 photos de chars camouflés dans les arbres et 100 photos d'arbres sans char.
- Entraînement d'un réseau de neurone profond sur un ensemble d'apprentissage de 100 photos.

- Excellents résultats sur l'ensemble de test (les 100 photos restantes).
- Sur le terrain les résultats sont catastrophiques. Pourquoi ?

Les photos de chars camouflés avaient été prises avec un ciel nuageux alors que celles de forêts avaient été prises par beau temps.

→ Le NN avait appris à savoir s'il faisait beau ou non.

C'est quoi l'interprétabilité ?

Doshi-Velez and Kim [2017]

"Interpret means to explain or to present in understandable terms. In the context of ML systems, interpretability is the ability to explain or to present in understandable terms to a human."

Murdoch et al. [2019]

"We define interpretable machine learning as the use of machine-learning models for the extraction of relevant knowledge about domain relationships contained in data."

→ Il n'existe pas de définition arrêtée et mesurable de l'interprétabilité.

Quelques outils pour l'interprétabilité ?

L'interprétabilité des modèles prédictifs est divisible en deux familles :

- L'interprétabilité dite **ad-hoc** consiste à utiliser des algorithmes qui sont intrinsèquement interprétables (e.g. Les algorithmes à bases de règles ou à bases d'arbres).
- L'interprétabilité dite **post-hoc** consiste à utiliser un algorithme quelconque (type black-box) et d'ajouter une couche pour interpréter les prédictions du modèle.

Dans l'interprétabilité **post-hoc** on distingue l'interprétabilité **globale** et **locale**.

- L'interprétabilité dite **post-hoc** globale essaye d'expliquer tout le modèle généré par la black-box (e.g. les Partial Dependent Plot PDP ou les Accumulated Local Effect ALE).
- L'interprétabilité dite **post-hoc** locale essaye d'expliquer une prédiction du modèle black-box (e.g. Local Interpretable Model-agnostic Explanations LIME ou SHapley Additive exPlanations SHAP).

Partial Dependent Plot (PDP)

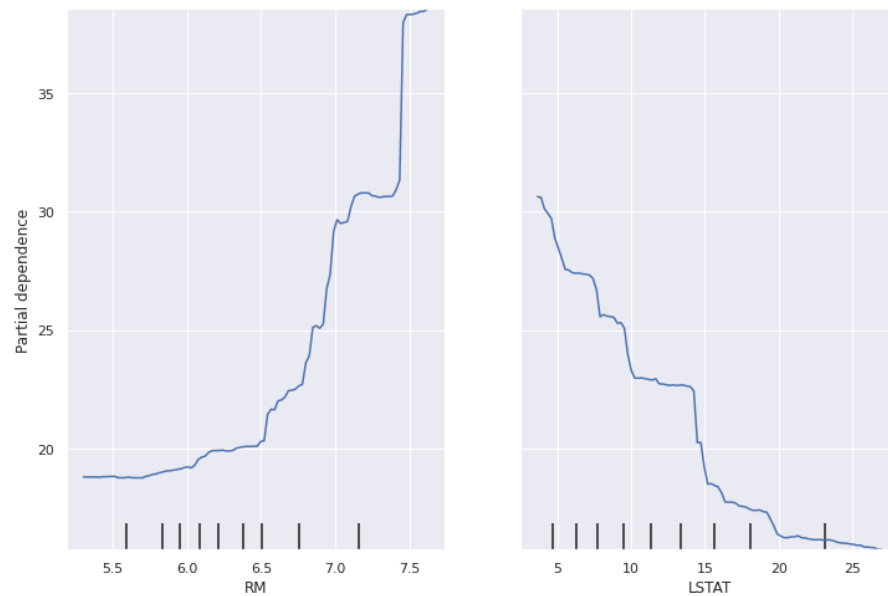


Figure 2.19: Exemple de PDP appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour les features RM et LSTAT.

Accumulated Local Effect (ALE)

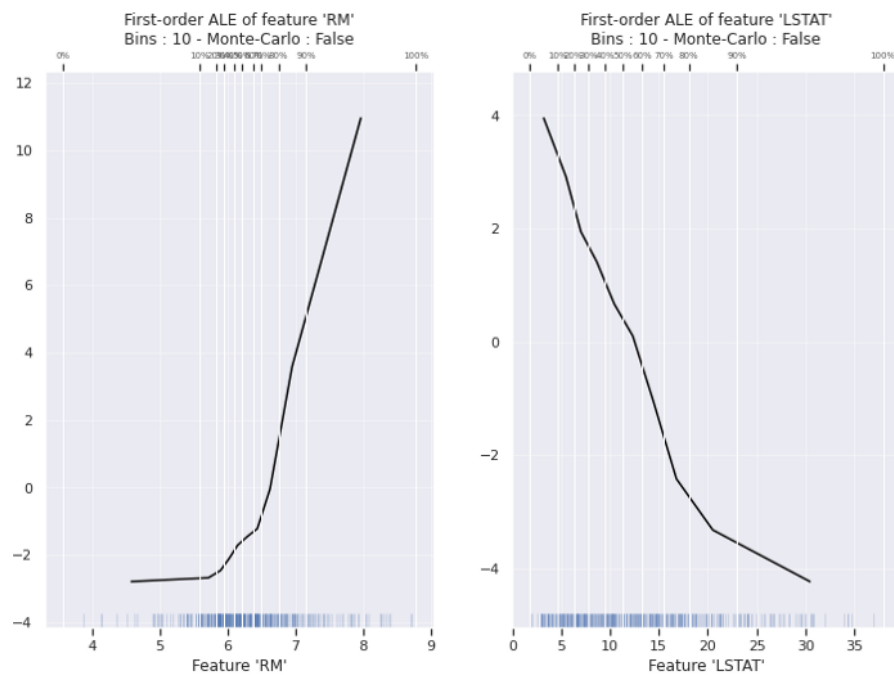


Figure 2.20: Exemple de ALE appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour les features RM et LSTAT.

Individual Conditional Expectation (ICE).

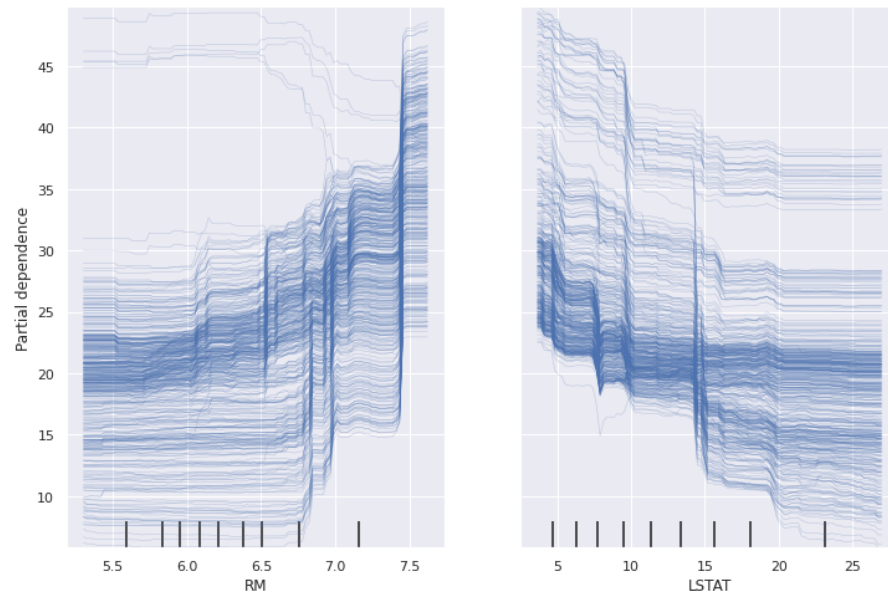


Figure 2.21: Exemple de ICE appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour les features RM et LSTAT.

Local Interpretable Model-agnostic Explanations (LIME).

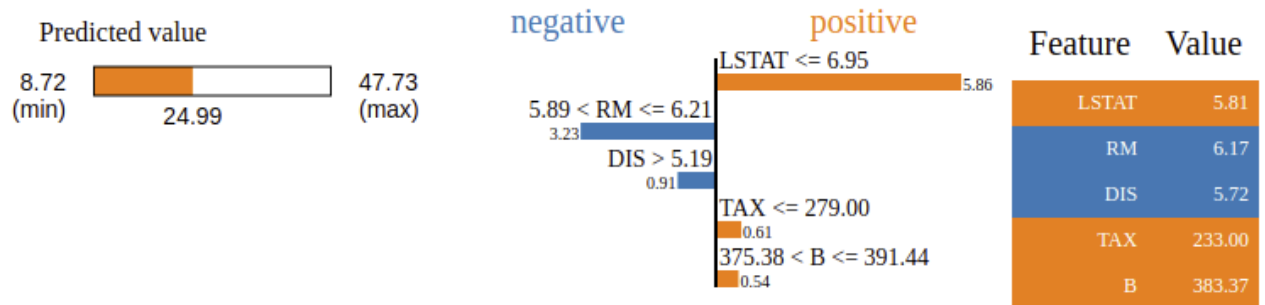


Figure 2.22: Exemple de LIME appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour la 42ème prédiction.

SHapley Additive exPlanations (SHAP).

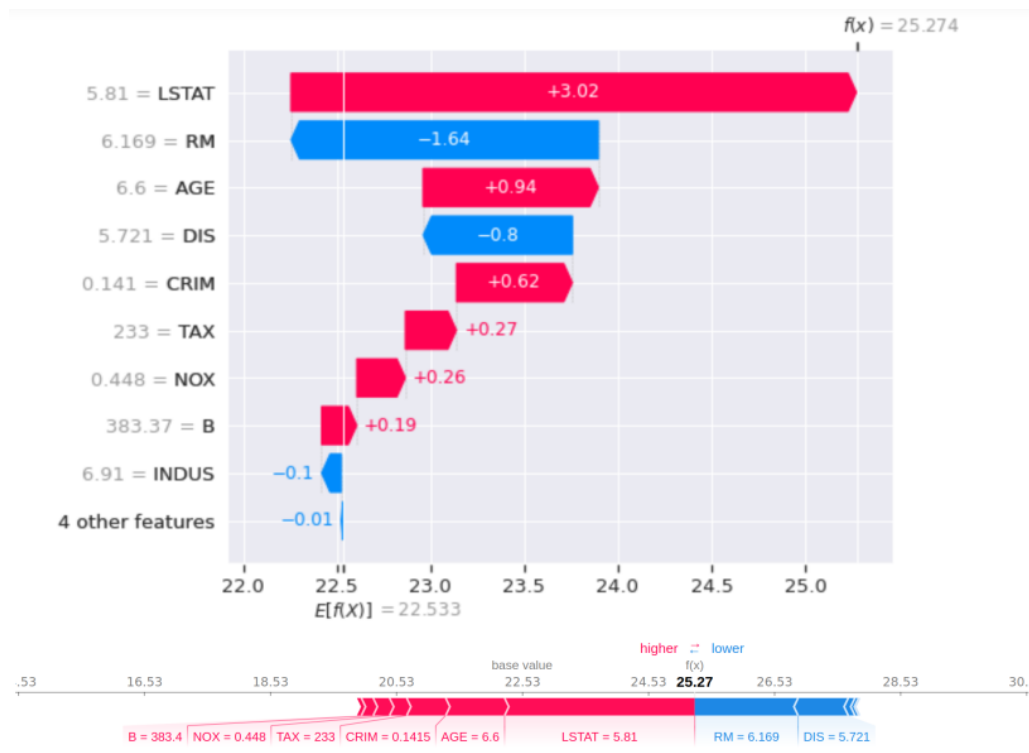


Figure 2.23: Exemple de LIME appliqué à une forêt aléatoire entraînée sur les données Housing Boston data pour la 42ème prédiction.

2.1.13 AutoML

L'autoML, ou apprentissage automatique automatique, est un ensemble de techniques et d'outils qui automatisent le processus de construction d'un modèle d'apprentissage automatique. L'autoML vise à rendre l'apprentissage automatique plus accessible et plus facile à utiliser pour les non-experts.

Il existe de nombreux packages et outils d'autoML disponibles. Certains des principaux services d'autoML sont :

- **Google AutoML** : Un ensemble d'outils d'autoML de Google Cloud Platform.
- **Amazon SageMaker Autopilot** : Un service d'autoML de Amazon Web Services.
- **Azure Machine Learning Service** : Un service d'autoML de Microsoft Azure.
- **TPOT** : Un des premiers packages open-source d'auto ML. Il utilise principalement des arbres.
- **AutoKeras** : un packages open-source complet qui dispose d'une large communauté.

Alors pourquoi continuer le cours ? D'abord c'est très récent, les premiers déploiements commerciaux dates de 2020, visant les data scientists. Ensuite, cela revient à utiliser une voiture autonome sans connaissances de son fonctionnement, des règles qu'elle suit. Comment savoir alors si elle réagit mal ou si elle court à la catastrophe ?

L'AutoML est plein de promesse, mais il faut :

- **Comprendre les limites** : L'AutoML ne peut pas résoudre tous les problèmes d'apprentissage automatique. Il est important de comprendre les limites de l'AutoML pour pouvoir identifier les cas où il ne convient pas.
- **Pouvoir personnaliser les modèles** : L'AutoML peut générer des modèles de base qui sont souvent efficaces. Cependant, il est souvent possible d'améliorer les performances des modèles en les personnalisant en fonction des besoins spécifiques de l'application.
- **Pouvoir créer de nouveaux modèles** : L'AutoML est un outil puissant, mais il ne peut pas remplacer la créativité humaine. Il est important de comprendre les concepts et les principes fondamentaux de la data science et du ML pour pouvoir créer de nouveaux modèles qui répondent aux besoins spécifiques de l'application.

2.2 Cycle de vie d'un projet de Machine Learning - matériels sources

- Compréhension du problème métier
- Collection , acquisition des données
- Annotation des données
- Transformation des données
- Conception, developpement d'un modèle dans un environnement de développement
- Entraînement et validation du modèle dans un environnement de test
- Déploiement et maintenance du modèle dans l'environnement de production

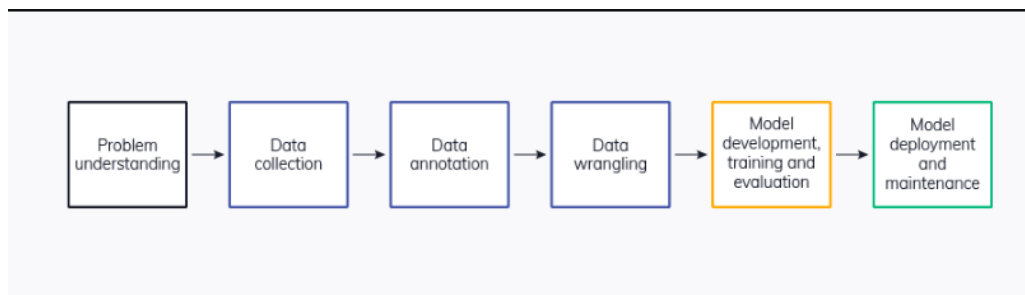


Figure 2.24: Cycle de vie d'un modèle de ML.

Étape 1 : Compréhension du problème

Chaque projet commence par un problème à résoudre. Idéalement, une définition claire du problème devrait être décrite numériquement. Les chiffres permettent non seulement de savoir quel est votre point de départ, mais aussi de suivre l'effet des changements ultérieurs.

Par exemple, l'entreprise dans laquelle je travaille a effectué des calculs qui montrent combien chaque opération manuelle coûte à l'entreprise. Cette approche nous aide à stratifier nos opérations et à les classer par ordre de priorité en fonction de ce que nous devons dépenser.

Notre direction a récemment lancé un nouveau projet d'apprentissage automatique, visant à automatiser une opération manuelle particulière qui figure actuellement en tête de notre liste de dépenses. L'équipe a également effectué des recherches, comparant les coûts de cette opération à ceux de nos concurrents.

Le résultat s'est avéré décevant : des opérations manuelles similaires sont jusqu'à 20 % moins chères pour d'autres entreprises de notre secteur. Pour être compétitifs sur le marché, nous devons réduire nos coûts. C'est pourquoi nous avons lancé le projet d'automatisation.

Notre problème se limite-t-il à cela ? Pas tout à fait. Connaître les coûts ne signifie pas que nous pouvons confier ce problème à notre équipe d'apprentissage automatique et attendre d'elle qu'elle le résolve.

Jusqu'à présent, nous n'avons défini le problème qu'en termes commerciaux. Avant tout apprentissage automatique, nous devons abandonner les unités monétaires et passer à d'autres indicateurs de performance clés, KPIs, que notre équipe d'apprentissage automatique peut comprendre.

Pour ce faire, notre direction a calculé que si nous voulons réduire les coûts d'une opération manuelle donnée de 20 %, nous devons diminuer le nombre de traitements manuels de 100 % à au moins 70 %. Cela signifie que 30 % de toutes les opérations doivent être traitées automatiquement. Savoir cela peut nous aider à réduire la portée d'un projet, en nous faisant comprendre que nous ne devons cibler qu'une partie d'un problème, et non l'ensemble du problème.

Ensuite, l'opération manuelle que nous voulions cibler a été décomposée en plusieurs parties. Connaissant le coût de chaque élément en termes de temps (et d'argent), l'équipe a pu dresser une liste de propositions pour les tâches susceptibles d'être automatisées.

En discutant de cette liste avec l'équipe chargée de l'apprentissage automatique, ils ont choisi quelques tâches qui peuvent être résolues par des algorithmes d'apprentissage automatique supervisé si les données appropriées sont disponibles.

Enfin, la compréhension du problème est complète : chaque équipe de l'entreprise sait ce qu'elle vise et pourquoi. Le projet peut commencer.

Étape 2 : Collecte de données

Les données, c'est le pouvoir. Lorsque le problème est clair et qu'une approche d'apprentissage automatique appropriée est établie, il est temps de collecter des données.

Les données peuvent provenir de plusieurs sources. Vous disposez peut-être d'une base de données interne qui peut être interrogée pour trouver des données pertinentes. Vous pouvez demander à des ingénieurs de données d'extraire les données pour vous ou utiliser des services existants tels qu'Amazon Mturk, ou encore le faire vous-même.

D'autres reçoivent des données de leurs clients. C'est généralement le cas lorsque vous travaillez côte à côte sur le problème d'un client. Le client est intéressé par le résultat final et est prêt à partager ses données.

Il existe également des ensembles de données en libre accès. Ils sont particulièrement utiles si vous travaillez sur un problème général que de nombreuses entreprises et industries pourraient également rencontrer. Il y a de fortes chances que l'ensemble de données dont vous avez besoin se trouve déjà quelque part sur le web. Certains ensembles de données proviennent d'organisations gouvernementales, d'autres d'entreprises publiques et d'universités.

Votre objectif est de collecter autant de données pertinentes que possible. Cela implique généralement d'obtenir des données sur une large période si l'on parle de données tabulaires. Rappelez-vous : plus vous avez d'échantillons pertinent, meilleur sera votre futur modèle.

Plus tard dans le cycle de vie, vous passerez par l'étape de la préparation des données, qui peut réduire considérablement le nombre d'échantillons dans votre ensemble de données. C'est pourquoi il est crucial, au tout début du cycle de vie du projet, d'accumuler autant de données que possible.

Si vous n'avez pas collecté suffisamment de données, vous avez le choix entre deux options :

Data augmentation

L'augmentation des données introduit des variations supplémentaires dans l'ensemble de données existant, ce qui permet au modèle de mieux se généraliser. Elle n'ajoute pas vraiment d'échantillons supplémentaires, elle manipule simplement les données actuelles pour en tirer le meilleur parti.

Vous devez examiner attentivement les types d'augmentation des données que vous appliquez. Il suffit de rechercher une augmentation qui reflète l'environnement de production réel dans lequel le modèle sera utilisé. Il n'est pas nécessaire d'"apprendre" au modèle à se préparer à des cas dont vous savez pertinemment qu'ils ne se produiront jamais dans la vie réelle. Plus loin dans cet article, nous aborderons l'analyse exploratoire des données (AED), qui peut révéler le type de données avec lesquelles vous travaillez et le type d'extension approprié.

Synthetic data generation

Les ensembles de données synthétiques, en revanche, sont de nouveaux échantillons qui peuvent être utilisés comme entrées dans votre modèle. Il s'agit de données entièrement nouvelles que vous pouvez générer artificiellement à l'aide de l'apprentissage profond non supervisé (par exemple, Generative Adversarial Networks) ou de bibliothèques qui travaillent avec des images (par exemple, en Python, vous pouvez penser à OpenCV ou PIL).

Les réseaux antagonistes génératifs ou Generative adversarial network (GAN), génèrent de nouveaux exemples à partir d'exemples existants. Les ingénieurs utilisent ce type d'architecture pour créer de nouvelles images uniques à partir d'ensembles de données existants, généralement de petite taille. Personnellement, je peux dire que les images générées par les GAN sont de très bonne qualité et qu'elles sont très utiles pour l'annotation (étape 4 du cycle de vie du projet ML) et pour l'entraînement des réseaux neuronaux (étape 5 du cycle de vie).

Étape 3 : Préparation des données

Les données collectées sont dites "sales". Les ingénieurs en machine learning sont confrontés à de nombreux problèmes lorsqu'ils traitent des données brutes. Voici les problèmes les plus courants :

Les données pertinentes doivent être filtrées. Les données non pertinentes doivent être nettoyées ; Les échantillons bruyants, illusoires et erronés doivent être identifiés et supprimés ; Les valeurs aberrantes doivent être reconnues et éliminées ; Les valeurs manquantes doivent être repérées, puis supprimées et imputées à l'aide de méthodes appropriées ; Les données doivent être converties dans des formats appropriés. Comme vous pouvez le constater, un ingénieur en apprentissage automatique peut être confronté à de multiples problèmes lorsqu'il traite des données brutes. Chaque ensemble de données est unique en termes de problèmes. Il n'existe pas de règle empirique sur la manière d'aborder le prétraitement des données. Il s'agit d'un processus créatif et à multiples facettes.

Considérons les valeurs manquantes. Il s'agit d'un problème assez courant dans la plupart des ensembles de données. Les ingénieurs ML peuvent simplement supprimer ces valeurs et ne travailler qu'avec les enregistrements valides de l'ensemble de données.

Vous pouvez également opter pour l'imputation et remplir les enregistrements avec des NaN. Vous cherchez à nouveau une règle empirique ? Malheureusement, il n'y en a pas non plus. L'imputation peut se faire de plusieurs manières, en fonction des différents critères que vous avez sélectionnés. Les algorithmes mathématiques pour l'imputation diffèrent également, et là encore, vous avez plusieurs options à considérer.

La création de nouvelles features à partir de features existantes est une autre option que les ingénieurs en machine learning doivent envisager. Ce processus s'appelle l'ingénierie des données. Un bon exemple d'ingénierie des données, que je pratique personnellement assez souvent, est la réduction de la dimensionnalité via l'analyse en composantes principales (ACP ou PCA). L'ACP réduit le nombre de caractéristiques dans l'ensemble de données, en ne conservant que celles qui sont les plus utiles pour la prise de décision future.

Dès que vous avez terminé la partie centrale de la préparation des données, vous pouvez passer au traitement des données. Le prétraitement des données est une étape qui rend vos données digestes pour le réseau neuronal ou l'algorithme que vous entraînez. Il implique généralement la normalisation, la standardisation et la mise à l'échelle des données.

Généralement, une étape de préparation des données s'accompagne d'une analyse exploratoire des données (AED ou EDA) qui complète le processus de préparation global. L'AED aide les ingénieurs à se familiariser avec les données avec lesquelles ils travaillent. Elle implique généralement la création de graphiques qui permettent d'examiner les données sous différents angles. L'intuition que les ingénieurs tirent d'une telle analyse les aide par la suite à trouver les bonnes méthodes et les bons outils pour la préparation des données, l'architecture du modèle / la sélection des algorithmes (étape 5.2 du cycle de vie du projet ml) et, bien sûr, le choix des mesures appropriées (étape 5.4 du cycle de vie du projet).

La préparation des données (ou "data wrangling") est l'une des étapes les plus longues, mais aussi l'une des plus importantes, car elle a une incidence directe sur la qualité des données qui seront envoyées sur le réseau.

On termine généralement par une étape de prétraitement des données en divisant les données traitées en trois sous-ensembles distincts : les données pour la formation, la validation et le test. Pour les petits ensembles de données, je n'alloue pas plus de 30 % à la validation et au test, le reste des données étant réservé à la formation. Pour les grands ensembles de données (plus de 10 000), la pratique conduit au ratio suivant :

10-20% pour les tests, 10-20 % pour la validation de l'entraînement, 60-80% pour l'entraînement. Une stratégie de fractionnement recommandé est le fractionnement stratifié. Il permet de maintenir une proportion égale de classes dans chaque ensemble de données. C'est important pour une estimation correcte des performances.

Étape 4 : Annotation des données

Si vous travaillez dans le domaine de l'apprentissage supervisé, vous aurez besoin d'une étiquette pour chaque échantillon de votre ensemble de données. Le processus d'attribution d'étiquettes aux échantillons de données s'appelle l'annotation ou l'étiquetage des données.

L'annotation des données est une opération manuelle, qui prend beaucoup de temps et qui est souvent réalisée par des tiers. Il existe de rares cas où les ingénieurs en apprentissage automatique travaillent eux-mêmes à l'étiquetage. Étant donné que vous et votre équipe ne passerez probablement pas en revue le processus d'annotation par vous-même, votre principal objectif à cette étape est de concevoir une ligne directrice complète en matière d'annotation.

Les lignes directrices permettent aux annotateurs de savoir ce qu'ils doivent faire. C'est pourquoi il est crucial d'élaborer un guide complet qui couvre les aspects les plus essentiels du travail d'annotation. N'oubliez pas les cas extrêmes qui peuvent survenir au cours de l'étiquetage. Votre équipe d'annotation doit être préparée à tous les scénarios possibles auxquels elle peut être confrontée. Il n'y a pas de place pour les suppositions dans le travail d'annotation. Tout doit être clair et transparent. Vous devez également désigner une personne qui assistera l'équipe d'annotation. Si vous ne pouvez pas traiter un exemple particulier, les annotateurs doivent savoir qui contacter pour répondre à leurs questions.

Étape 5 : Modélisation

À ce stade, vous devriez disposer d'un ensemble de données complet prêt à être introduit dans le modèle. Quelle est la prochaine étape ? Il est temps de prendre une décision concernant le futur modèle et de l'assembler.

5.1. Essayez de résoudre votre problème par transfer learning

Les ingénieurs en machine learning ne créent pas de modèles à partir de zéro. Ils ont tendance à réutiliser les modèles qui ont déjà montré des performances décentes sur de grands ensembles de données publiques. Ces modèles pré-entraînés peuvent être utilisés pour un réglage fin. Cette approche a été largement établie dans l'apprentissage profond. Dans le domaine de la vision par ordinateur, par exemple, le réglage fin fonctionne bien parce que les caractéristiques de bas niveau que les CNN extraient sont unifiées pour un large éventail de tâches.

Les endroits où vous pouvez trouver des modèles pré-entraînés publics sont appelés ZOO de modèles. Github est une excellente source de modèles pré-entraînés, ainsi que Hugging Face, avec des centaines d'options possibles. Il vous suffit de rechercher le modèle correspondant à l'architecture et au cadre avec lesquels vous travaillez.

Par exemple, TensorFlow est un cadre d'apprentissage automatique qui permet d'importer des modèles pré-entraînés. Voici un exemple de zoo avec des modèles de détection, créés par TensorFlow. Ces modèles peuvent être utilisés pour l'apprentissage par transfert en vision artificielle.

Vous devriez toujours rechercher un modèle pré-entraîné pour votre projet afin de commencer à travailler avec. Cela vous permettra de gagner du temps, d'économiser des ressources informatiques et même d'améliorer la qualité du résultat final.

5.2. Adapter l'architecture du modèle en conséquence

Il est important de noter qu'un modèle pré-entraîné que nous importons doit être modifié pour refléter la tâche spécifique que nous effectuons.

Si vous travaillez dans le domaine de la vision par ordinateur, vous vous souvenez probablement que le nombre de classes qu'un modèle de classification peut identifier dépend de la partie supérieure de l'architecture du modèle. La dernière couche dense doit comporter un nombre d'unités égal au nombre de classes que vous souhaitez distinguer. Votre tâche consiste à préparer une conception finale de l'architecture du modèle qui

soit adaptée à vos objectifs.

5.3. Expérimenter beaucoup

Les ingénieurs en machine learning ont tendance à expérimenter beaucoup. Nous aimons jouer avec de multiples configurations de modèles, architectures et paramètres. Vous n'accepterez probablement pas le résultat de base que vous avez obtenu et le passerez en production. Ce résultat devient rarement le meilleur possible. Un processus de formation itératif pour trouver la meilleure configuration de modèle est une pratique courante chez les ingénieurs en apprentissage automatique.

À ce stade, vous devez essayer plusieurs hypothèses alternatives susceptibles de fonctionner pour une tâche donnée. Pour réduire la liste des options possibles, vous pouvez envisager d'utiliser les méthodes de réglage des hyperparamètres proposées par la plupart des frameworks de ML. Ces méthodes estiment les performances de plusieurs configurations, les comparent et vous indiquent celles qui sont les plus performantes. Il vous suffit de spécifier les valeurs et les paramètres à échantillonner.

Les expériences sont dans la nature de ce que nous faisons. Si vos ressources informatiques ne sont pas limitées, vous devriez en profiter. Les résultats que vous pouvez obtenir peuvent être tout à fait inattendus. Qui sait, peut-être qu'une nouvelle configuration de modèle de pointe résultera de l'une de vos expériences.

5.4. Évaluer correctement

L'évaluation va toujours de pair avec l'expérimentation. Vous devez savoir comment chaque modèle se comporte afin de sélectionner celui qui est le plus performant. Pour comparer les modèles, il faut définir un ensemble de paramètres.

Selon le problème sur lequel vous travaillez, votre ensemble de mesures sera différent. Pour les problèmes de régression, par exemple, nous considérons généralement MSE ou MAE. Pour évaluer un modèle de classification, en revanche, la précision peut être un bon choix pour un ensemble de données équilibré. Les ensembles déséquilibrés nécessitent des mesures plus sophistiquées. Le F1-score est une bonne mesure pour ces cas.

L'évaluation pendant la formation est effectuée sur un ensemble de données de validation distinct. Elle permet d'évaluer la capacité de notre modèle à se généraliser, en évitant les biais et l'overfitting.

C'est toujours une bonne pratique de visualiser la progression du modèle pendant le travail de formation. Tensorboard est la première et la plus basique des options à considérer. Alternativement, neptune.ai est un outil plus avancé qui visualise la performance du modèle dans le temps, et permet également le suivi des expériences. Il est essentiel de disposer du bon outil. Prenez le temps de trouver un outil de suivi des expériences qui réponde à vos besoins particuliers. Vous gagnerez beaucoup de temps et améliorerez votre flux de travail global.

Étape 6 : Déploiement du modèle

C'est excellent ! Vous avez un modèle brillant, prêt à être mis en production. Maintenant, les ingénieurs déploient un modèle de train et le rendent disponible pour les demandes d'inférence externes.

C'est la dernière étape du cycle de vie de l'apprentissage automatique. Mais le travail est loin d'être terminé, nous ne pouvons pas nous contenter de nous détendre et d'attendre un nouveau projet.

Les modèles déployés doivent être contrôlés. Vous devez suivre les performances des modèles déployés, afin de vous assurer qu'ils continuent à faire leur travail avec la qualité requise par l'entreprise. Nous connaissons tous des effets négatifs qui peuvent se produire au fil du temps : la dégradation du modèle est l'un

des plus courants.

Une autre bonne pratique consisterait à collecter des échantillons qui ont été traités de manière incorrecte par le modèle afin d'en déterminer les causes profondes et de les utiliser pour entraîner à nouveau le modèle afin qu'il soit plus robuste face à de tels échantillons. Cette petite recherche continue vous aidera à mieux comprendre les éventuels cas limites et autres événements inattendus auxquels votre modèle actuel n'est pas préparé.

model drift - Matériel source

La dérive du modèle ou model drift est le terme utilisé pour décrire la façon dont la précision d'un modèle se dégrade en raison de l'évolution des données et des relations entre ses variables d'entrée et de sortie.

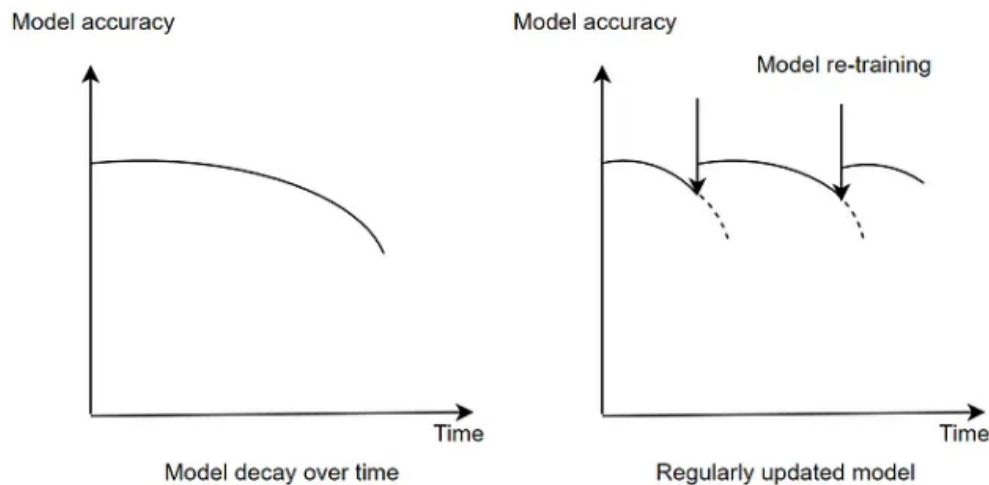


Figure 2.25: Effet de la dérive de modèle.

Causes:

- Sélection biaisée de l'échantillon - Cela se produit lorsqu'une stratégie biaisée ou inefficace a été utilisée pour collecter ou préparer les données d'entraînement. L'environnement opérationnel dans lequel le modèle sera utilisé n'est pas représenté avec précision.
- Modifications des variables cachées - Bien que les variables cachées ne puissent pas être mesurées directement, elles ont un impact significatif sur certaines des variables observées. En substance, si les facteurs cachés changent, les données que nous pouvons observer changeront également. Par exemple, nous pourrions considérer la valeur de la voiture comme une variable cachée : Nous pourrions prendre en compte la valeur de la voiture, les revenus de l'emprunteur, ses antécédents en matière de prêts automobiles, le montant de l'acompte, etc. Cependant, la situation générale d'une personne peut avoir un impact encore plus important, par exemple si un membre de sa famille est atteint d'une maladie et a besoin d'un fonds d'urgence pour son traitement. Bien que ces facteurs ne puissent être mesurés directement, ils ont un impact crucial sur le nombre de mauvais payeurs (notre cible).
- Environnement dynamique - Il s'agit du cas d'instabilité le plus simple et le plus évident, lorsque l'évolution des données et des relations échappe à notre contrôle. Par exemple, les règles et réglementations peuvent changer, les intérêts des utilisateurs peuvent évoluer, de meilleurs concurrents peuvent émerger, etc.

- Problèmes techniques - Un mauvais fonctionnement du pipeline de données, une modification de la valeur de l'un des paramètres de la fonctionnalité, voire un bogue, peuvent être à l'origine de ces problèmes.
- Changement du domaine - Il s'agit des modifications de la signification des concepts ou des valeurs. Par exemple, comme l'argent perd de sa valeur en raison de l'inflation, le prix d'un article ou le salaire d'une personne aura des conséquences variables au fil du temps.

Typologie de changement :

- Graduelle - Au fil du temps, lorsque de nouvelles idées sont mises en œuvre, une transformation graduelle s'opère. Prenons l'exemple d'un modèle de prédiction du prix des voitures qui a été développé en 2015 et dont le niveau de précision était élevé à l'époque. En raison de l'augmentation des prix des voitures au fil du temps, la validité et la précision des prédictions commencent à diminuer au bout de quelques années. Cela est logique car, comme nous le savons tous, les prix des voitures augmentent généralement avec le temps. Toutefois, si ce facteur n'est pas pris en compte, il peut avoir une incidence négative sur la précision d'un modèle.
- Soudain - Une dérive peut se produire soudainement. Par exemple : Ajustements brusques des habitudes d'achat et du comportement des consommateurs pendant et après une pandémie.
- Récurrent - Dans ce cas, les changements se répètent après l'observation initiale, ou nous pourrions dire qu'ils se produisent périodiquement. Prenons l'exemple des achats de vêtements d'hiver.
- Spike - Il s'agit d'événements exceptionnels susceptibles d'avoir un impact sur le modèle. Par exemple : Changements provoqués par une guerre, une pandémie, une récession, etc.

3 familles de drift : Covariate/feature drift

La dérive des covariables se produit lorsque les données utilisées pour former un algorithme divergent des données utilisées pour l'utiliser. Cela indique que si la distribution de la caractéristique n'a pas changé, la relation entre la caractéristique et la variable cible a elle-même changé. Lorsque les propriétés statistiques de ces données d'entrée changent, le même modèle qui a été construit auparavant ne fournira pas de résultats impartiaux et conduira donc à des prédictions inexactes. Par exemple, un fabricant de dispositifs médicaux peut utiliser des informations provenant d'hôpitaux urbains importants pour créer son système basé sur l'apprentissage automatique. Cependant, une fois le produit commercialisé, les données médicales saisies dans le système par les professionnels de la santé dans les zones rurales peuvent ne pas ressembler aux données de développement. Dans certains groupes sociodémographiques, les patients des hôpitaux urbains peuvent être plus susceptibles d'avoir des problèmes médicaux sous-jacents que ceux des hôpitaux ruraux. Ces divergences ne seront remarquées qu'une fois que le produit aura été mis sur le marché et qu'il commencera à fonctionner plus fréquemment qu'il ne l'a fait pendant les tests.

Label drift

Ce type de dérive se produit lorsque la distribution de la variable de classe (y), la sortie du modèle ou la distribution des étiquettes change. Par exemple, la pandémie a entraîné une augmentation considérable des coûts de l'automobile, ce qui a fait basculer la distribution des prix des voitures vers une valeur par défaut plus élevée. Le modèle ML développé pour prédire le prix des voitures avant la pandémie ne sera pas en mesure de prédire les valeurs d'entrée avec une précision suffisante après la pandémie.

Concept drift

Lorsque $p(y|X)$ change mais que $p(X)$ reste inchangé, il y a dérive conceptuelle. Ici, $p(X)$ et $p(y)$ représentent la probabilité d'observer les caractéristiques automobiles X et les prix des voitures y , respectivement, et $p(y|X)$

représente la distribution conditionnelle des prix des voitures en fonction des caractéristiques des voitures. La probabilité conditionnelle du prix de la voiture en fonction des attributs de la voiture $p(y|X)$ peut fluctuer dans l'exemple de la prédiction du prix de la voiture. Considérons que la distribution du nombre de places assises de la voiture ne change pas. De nos jours, les clients veulent des voitures plus grandes, et leur prix a donc augmenté. La probabilité conditionnelle du prix de la voiture en fonction du nombre de places assises peut changer, en particulier pour les voitures plus grandes.

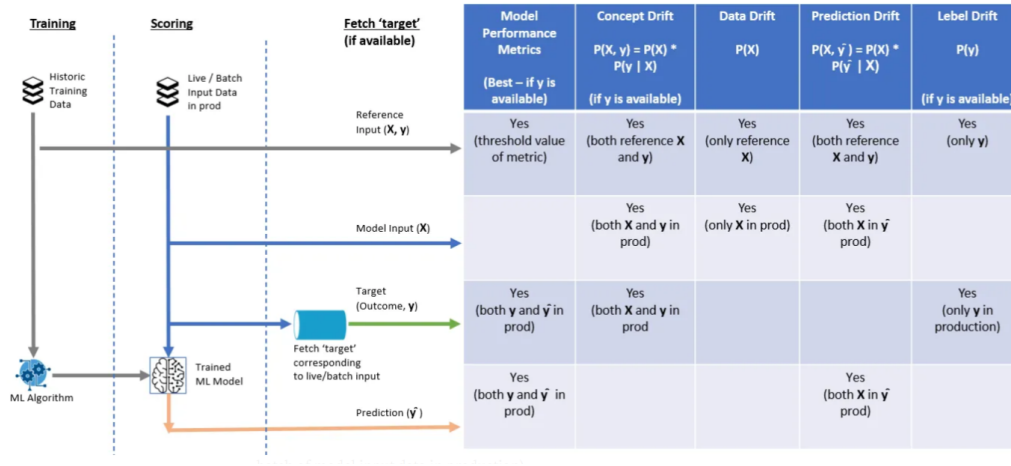


Figure 2.26: Model Drift Types with Input and Output. - Source medium

Chapter 3

Algorithmes classiques

3.1 Classification et régression

- La *classification* et la *régression* sont deux sous-ensembles de l'apprentissage **supervisé**.
- Dans un exercice de **classification**, on cherche à construire un modèle capable de prédire une **variable discrète qualitative** (*classe* ou *label*). L'objectif est de pouvoir assigner à chaque observation d'un jeu de données une catégorie spécifique.
→ *Exemple* : classer un *email* dans la catégorie *spam* ou non.
- Dans une tâche de **régression**, on cherche à construire un modèle capable de prédire une **variable continue quantitative**. Pour cela, plusieurs méthodes d'analyse statistique peuvent être utilisées, permettant d'approcher une variable à partir d'autres qui lui sont corrélées.
→ *Exemple* : prédire le prix d'une maison à partir de son emplacement, nombre de pièces, proximité des commerces et des transports, etc.

Visual illustration: classification

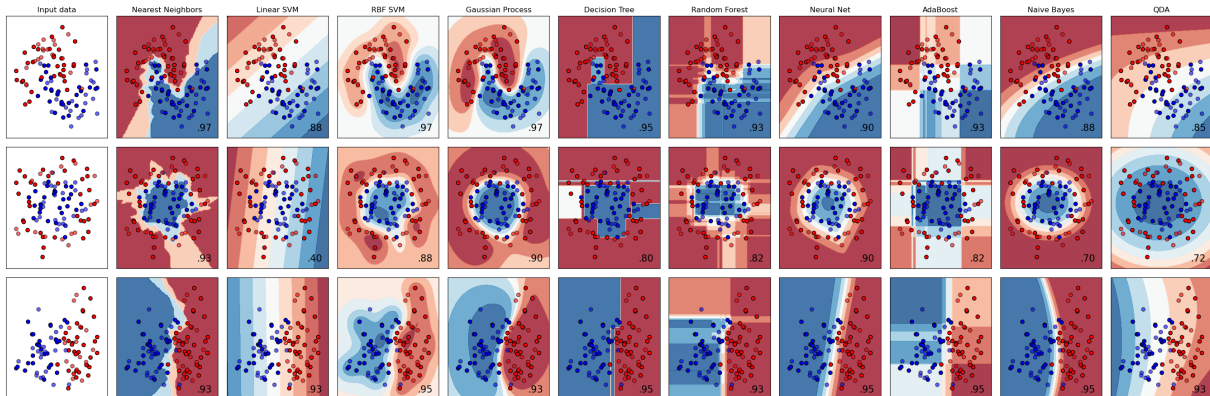


Figure 3.1: Source : scikit-learn

Visual illustration: regression (Decision Tree)

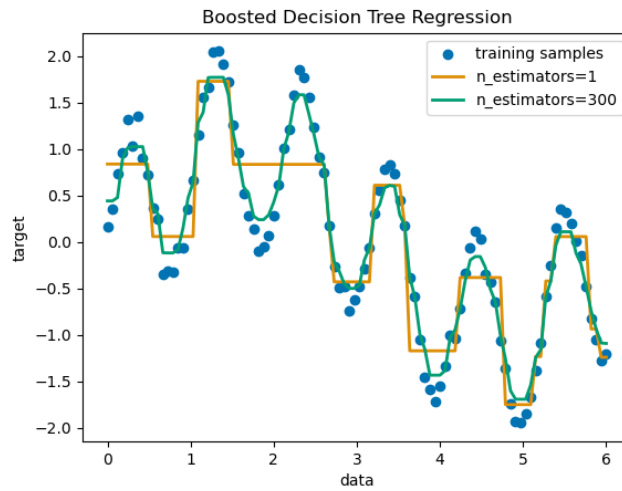


Figure 3.2: Source : scikit-learn

Visual illustration: regression (Gradient Boosting)

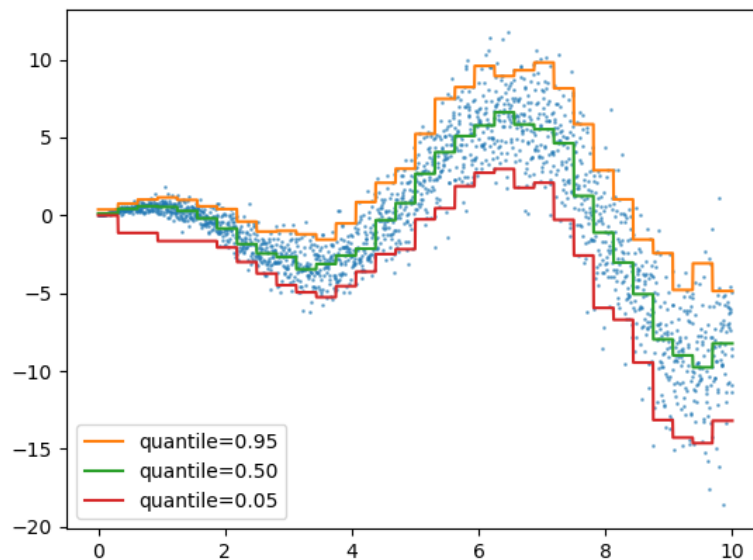


Figure 3.3: Source : scikit-learn

Visual illustration: clustering

3.2 Régression linéaire

- **Définition** : algorithme **supervisé** dont le modèle cherche à trouver une relation linéaire entre une variable dépendante et une ou plusieurs variables indépendantes (ou explicatives).

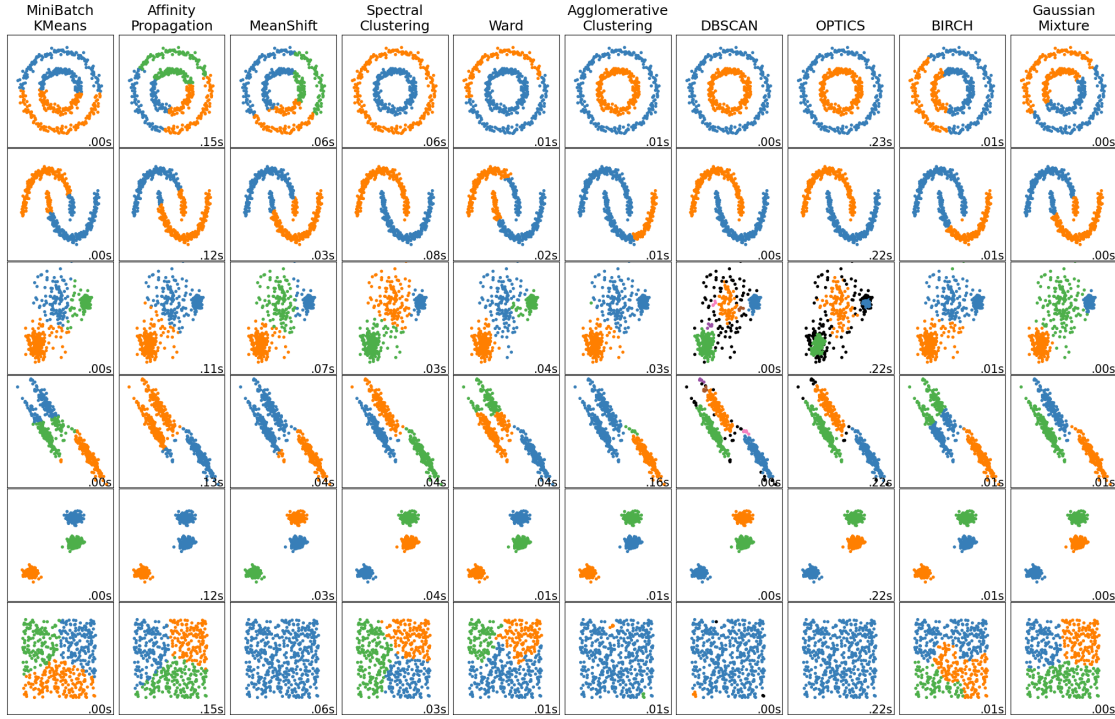


Figure 3.4: Source : scikit-learn

- **Naissance** : le terme originel de régression provient des expériences statistiques de l'anthropologue *Francis Galton* qui, dans un article de 1886, constate un phénomène de “*régression vers la moyenne*” de la taille des fils en fonction de la taille des pères.
- **Applications** : Toute application où le modèle linéaire est pertinent, les erreurs sur les variables indépendantes sont décorréliées et issues d'une distribution Gaussienne à variance constante.

Données

- La Régression (multi-)linéaire est le plus simple algorithme de Machine Learning supervisé. Pour la suite, nous introduisons la notation suivante :

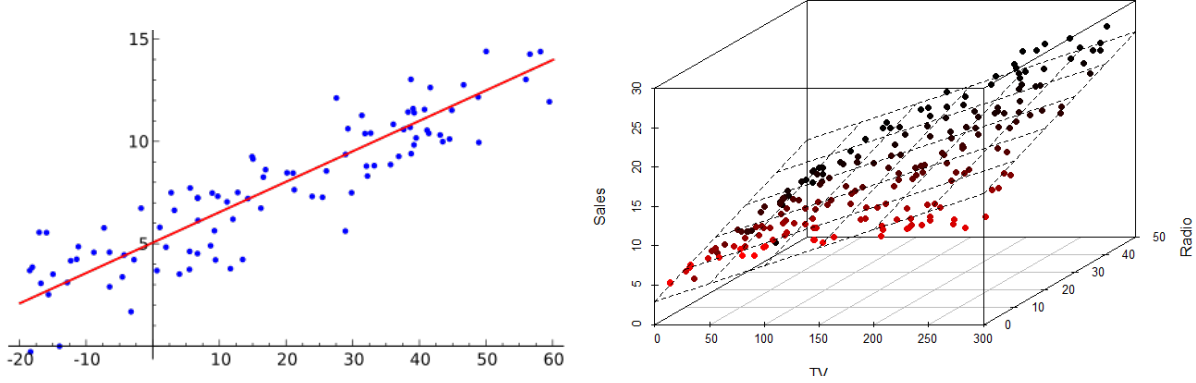
$$\{\mathbf{x}_i, y_i\}_{i=1, \dots, n}, \quad \mathbf{x}_i \in \mathbf{R}^d.$$

Pour chaque observation i , nous appelons $\{\mathbf{x}_i\}$ *explanatory* ou *independent variables* et y_i *dependent variable*. Le but de la régression (multi-)linéaire est d'apprendre un modèle qui relie $\{\mathbf{x}_i\}$ à y_i à travers un combinaison linéaire.

Paramétrisation du modèle

- Le modèle suppose une dépendance linéaire entre la sortie et ses *paramètres*

$$\begin{aligned} \mathcal{Y}(\mathbf{c}, \mathbf{x}_i) &= c^{(0)} + c^{(1)} x_i \quad \text{if } x_i \in \mathbb{R}^1, \quad [\text{reg. linéaire}] \\ &= c^{(0)} + c^{(1)} x_i^{(1)} + c^{(2)} x_i^{(2)} \quad \text{if } \mathbf{x}_i \in \mathbb{R}^2, \\ &\dots, \\ &= c^{(0)} + \sum_{k=1}^{d-1} c^{(k)} x_i^{(k)}, \quad \text{if } \mathbf{x}_i \in \mathbb{R}^d \end{aligned}$$



- En général, pour n observations d –dimensionnelles, on a :

$$\mathcal{Y}(\mathbf{c}) \equiv \{\mathcal{Y}(\mathbf{c}, \mathbf{x}_1), \dots, \mathcal{Y}(\mathbf{c}, \mathbf{x}_d)\} = \mathbf{X}\mathbf{c},$$

$$X_{i,j+1} = x_i^{(j)}, \quad X_{i,0} = 1, \quad \mathbf{X} \in \mathbf{R}^{n,d+1}, \quad \mathbf{c} \in \mathbf{R}^{d+1}$$

- **Loss function:** étant donné n observations $\{\mathbf{x}_i, y_i\}_{i=1,\dots,n}$, la *loss* est

$$\mathcal{L}_{\mathbf{c}} = \frac{1}{2n} \sum_{i=1}^n \phi(y_i, \mathcal{Y}(\mathbf{c}, \mathbf{x}_i)) \quad (3.1)$$

- **Cost Function**

$$\phi(y_i, y_i^{\text{pred}}) = \omega_i |y_i - \mathcal{Y}(\mathbf{c}, \mathbf{x}_i)|^2 \quad \omega_i = 1/\sigma_{y_i}^2, \quad (3.2)$$

- **Objective:** trouver $\bar{\mathbf{c}}$ tel que :

$$\mathcal{L}_{\bar{\mathbf{c}}} = \min [\mathcal{L}_{\mathbf{c}}] \quad (3.3)$$

- **Solution analytique**

$$\bar{\mathbf{c}} = (\mathbf{X}^t \mathbf{W}^{-1} \mathbf{X})^{-1} \mathbf{X}^t \mathbf{W}^{-1} \mathbf{y}, \quad W_{ij} = \delta_{ij} \omega_i \quad (3.4)$$

Optimisation des paramètres du modèle

- Écrire la *loss* sous une forme vectorielle :

$$\mathcal{L}_{\mathbf{c}} = \frac{1}{2n} [\mathbf{y} - \mathcal{Y}(\mathbf{c})]^t \mathbf{W}^{-1} [\mathbf{y} - \mathcal{Y}(\mathbf{c})], \quad (3.5)$$

avec $\mathcal{Y}(\mathbf{c}) = \mathbf{X}\mathbf{c}$

- Trouver la solution tel que le gradient vaut zéro :

$$\nabla_{\mathbf{c}} \mathcal{L}_{\mathbf{c}} = \left\{ \frac{\partial \mathcal{L}_{\mathbf{c}}}{\partial c^{(1)}}, \dots, \frac{\partial \mathcal{L}_{\mathbf{c}}}{\partial c^{(d)}} \right\} = \mathbf{0} \quad (3.6)$$

- Rappel: $\mathbf{y}$ ne dépend pas des paramètres $\mathbf{c}$ et que $a^t = a$ si $a \in \mathbb{R}^1$ (scalaire)

Que l'on peut ré-écrire comme :

- À partir des suggestions :

$$\begin{aligned} \mathbf{0} &= \nabla_{\mathbf{c}} \mathcal{L}_{\mathbf{c}} \propto \nabla_{\mathbf{c}} [\mathbf{y}^t \mathbf{W}^{-1} \mathbf{y} - 2\mathbf{c}^t \mathbf{X}^t \mathbf{W}^{-1} \mathbf{y} + \mathbf{c}^t \mathbf{X}^t \mathbf{W}^{-1} \mathbf{X} \mathbf{c}] \\ &= \nabla_{\mathbf{c}} [-2\mathbf{c}^t \mathbf{X}^t \mathbf{W}^{-1} \mathbf{y} + \mathbf{c}^t \mathbf{X}^t \mathbf{W}^{-1} \mathbf{X} \mathbf{c}] \\ &= -2\mathbf{X}^t \mathbf{W}^{-1} \mathbf{y} + 2\mathbf{X}^t \mathbf{W}^{-1} \mathbf{X} \mathbf{c} \propto -\mathbf{X}^t \mathbf{W}^{-1} \mathbf{y} + \mathbf{X}^t \mathbf{W}^{-1} \mathbf{X} \mathbf{c} \end{aligned}$$

- En fin, trouver $\mathbf{c}$:

$$\begin{aligned} \mathbf{0} &= -\mathbf{X}^t \mathbf{W}^{-1} \mathbf{y} + \mathbf{X}^t \mathbf{W}^{-1} \mathbf{X} \mathbf{c}, \\ \mathbf{X}^t \mathbf{W}^{-1} \mathbf{X} \mathbf{c} &= \mathbf{X}^t \mathbf{W}^{-1} \mathbf{y}, \\ \mathbf{c} &= (\mathbf{X}^t \mathbf{W}^{-1} \mathbf{X})^{-1} \mathbf{X}^t \mathbf{W}^{-1} \mathbf{y} \end{aligned}$$

Ce qui devient :

• **Exemple unidimensionnel**

$$\mathcal{Y}(\mathbf{c}, x_i) = a x_i + b \quad W_{ij} = \delta_{ij} 1, \quad x_i \in \mathbf{R}^1, \quad (3.7)$$

et on retrouve finalement¹ :

$$a = \frac{\mathbf{y}^t \mathbf{x} - \frac{1}{n} \sum_i x_i \sum_j y_j}{\mathbf{x}^t \mathbf{x} - \frac{1}{n} (\sum_i x_i)^2}, \quad b = \frac{\sum_i y_i}{n} - a \frac{\sum_i x_i}{n} \quad (3.8)$$

- **Solution numérique** : tous les algorithmes dédiés à la minimisation d'une fonctionne numérique. Les méthodes basées sur la **descente du gradient** sont les plus connues.

Petit rappel : Toute fonction différentiable $f(\mathbf{x})$ peut être minimisée à travers une *descente de gradient*. Étant donné un point de départ $\mathbf{x}^0$, nous pouvons suivre la direction de pente maximale jusqu'à arriver à un point où le gradient est proche de zéro (condition d'arrêt).

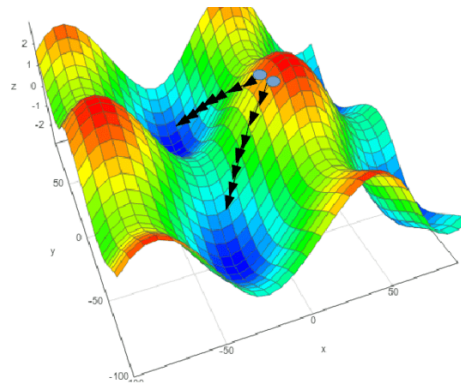
Dans le voisinage d'un point $\tilde{\mathbf{x}}$, la direction ci-dessus est donnée par la direction du gradient calculé sur $\tilde{\mathbf{x}}$. La méthode est *itérative*:

$$\tilde{\mathbf{x}} \leftarrow \mathbf{x}^0$$

while not condition :

$$\tilde{\mathbf{x}} \leftarrow \tilde{\mathbf{x}} - \alpha * [\nabla f](\tilde{\mathbf{x}})$$

solution: $\tilde{\mathbf{x}}$



- **Laquelle choisir ?** ça dépend du nombre de *features* d .

Solution analytique

La solution analytique peut être sujet au mauvais conditionnement de la matrice $\mathbf{X}$. Des techniques de régularisation sont donc nécessaires.

¹https://www.amherst.edu/system/files/media/1287/SLR_Leastsquares.pdf

Solution analytique	Solution numérique
No hyperparameters tuning	Hyperparameters tuning
No iterations needed	Iterations needed
Cost for inverting $(X^t X)^{-1} \sim \mathcal{O}(d^3)$	Bonne performance pour large d

Table 3.1: **Pros.** and **Cons.** de l'optimisation numérique par rapport à l'optimisation analytique pour le modèle de régression (multi-)linéaire

- **Quand dit-on qu'une matrice est mal conditionnée ?** Dans le domaine du *scientific computing*, une matrice A est mal conditionnée si son *condition number* κ_A (le rapport entre la valeur propre la plus grande et la valeur propre la plus petite) est trop grand.
- **Que se passe-t-il lorsque κ est trop grand ?** Étant donné une variation $\delta \mathbf{y}$ de la variable dépendante, une fluctuation de la solution $\mathbf{c}$ se produit par conséquent selon :

$$\max \left[\frac{|\delta \mathbf{c}(\mathbf{y})|}{|\delta \mathbf{y}|} \right] = \kappa_X \quad (3.9)$$

Donc, plus le *condition number* est grand, plus l'incertitude sur les données d'entrée s'amplifiera au niveau de la solution : **le résultat sera donc instable**. *Ill-conditioning*, en général, c'est un problème pour tout algorithme, pas seulement pour la régression linéaire.

- **Quand dit-on qu'une matrice est mal conditionnée ?** $\max \left[\frac{|\delta \mathbf{c}(\mathbf{y})|}{|\delta \mathbf{y}|} \right] = \kappa_X$
- **Propriétés algébriques dont on a besoin**

Norme d'une matrice: $\|B\|_2 = \max \left[\frac{\|B\mathbf{x}\|}{\|\mathbf{x}\|} \right]$, où $\mathbf{x}$ est un vecteur.

$$\|B\|_2 = \sigma_{\max}(B) \text{ et } \|B^t B\| = \|B\|_2^2$$

- **Solution complète**

$$\bar{\mathbf{c}} = (X^t W^{-1} X)^{-1} W^{-1} X \mathbf{y} = A^{-1} \mathbf{b}$$

Considérons $W = \mathbf{1}$

$$\frac{|\delta \mathbf{c}(\mathbf{y})|}{|\delta \mathbf{y}|} \sim \frac{|\delta \mathbf{c}|}{|\mathbf{c}|} \frac{|\mathbf{b}|}{|\delta \mathbf{b}|} = \frac{|A^{-1} \delta \mathbf{b}|}{|\delta \mathbf{b}|} \frac{|\mathbf{b}|}{|A^{-1} \mathbf{b}|}$$

- **Que se passe-t-il lorsque κ est trop grand ?** $\max \left[\frac{|\delta \mathbf{c}(\mathbf{y})|}{|\delta \mathbf{y}|} \right] = \kappa_X$
- **Propriétés algébriques dont on a besoin :**

Norme d'une matrice : $\|B\|_2 = \max \left[\frac{\|B\mathbf{x}\|}{\|\mathbf{x}\|} \right]$, où $\mathbf{x}$ est un vecteur

$$\|B\|_2 = \sigma_{\max}(B) \text{ et } \|B^t B\| = \|B\|_2^2$$

- ce qui devient :

$$\begin{aligned} \max \left[\frac{|\delta c(\mathbf{y})|}{|\delta \mathbf{y}|} \right] &= \max \left[\frac{|\mathbf{A}^{-1} \delta \mathbf{b}|}{|\delta \mathbf{b}|} \right] \max \left[\frac{|\mathbf{b}|}{|\mathbf{A}^{-1} \mathbf{b}|} \right] = \\ &= \max \left[\frac{|\mathbf{A}^{-1} \delta \mathbf{b}|}{|\delta \mathbf{b}|} \right] \max \left[\frac{|\mathbf{Ac}|}{|c|} \right] = \|\mathbf{A}^{-1}\|_2 \|\mathbf{A}\|_2 \end{aligned}$$

Étant donné que $\mathbf{A} = \mathbf{X}^t \mathbf{X}$, alors :

$$\|\mathbf{A}\|_2 = \sqrt{\|\mathbf{X}^t \mathbf{X}\|_2} = \sigma_{\max}(\mathbf{X}), \text{ et } \|\mathbf{A}^{-1}\|_2 = \sigma_{\min}^{-1}(\mathbf{X})$$

$$\text{Nous concluons : } \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2 = \sigma_{\max}(\mathbf{X}) / \sigma_{\min}(\mathbf{X}) = \kappa_{\mathbf{X}}$$

La solution analytique peut être sujet au mauvais conditionnement de la matrice $\mathbf{X}$. Des techniques de régularisation sont donc nécessaires.

- **Feature importance selection** : utile à cause d'un mauvais conditionnement et que $n < d$
- **Singular values truncation** : plutôt qu'inverser la matrice $\mathbf{X}_W$

$$\mathbf{X}_W \equiv \mathbf{X}^t \mathbf{W}^{-1} \mathbf{X}, \quad (3.10)$$

nous utilisons la *Pseudo-inverse*, ou l'inverse de *Moore-Penrose* $\mathbf{X}_W^\dagger$

$$\mathbf{X}_W \stackrel{\text{svd}}{=} [\mathbf{U}_1 | \mathbf{U}_2] \text{diag}[\lambda_1, \dots, \lambda_s | \lambda_{s+1}, \dots, \lambda_p] [\mathbf{V}_1 | \mathbf{V}_2]^t, \quad (3.11)$$

$$\mathbf{X}_W^\dagger = \mathbf{V}_1 \text{diag}[1/\lambda_1, \dots, 1/\lambda_s] \mathbf{U}_1^t, \quad \text{avec } \lambda_i < \epsilon \text{ for } i = s+1, \dots, p \quad (3.12)$$

...Des techniques de régularisation sont donc nécessaires.

- **Régression Régularisée** aussi connue comme régularisation de *Tikhonov* ou *régularisation crête* (*ridge regression*).

$$\mathcal{L}_c = \|\mathbf{y} - \mathbf{X} \cdot \mathbf{c}\|_W^2 + \lambda^2 \|\mathbf{Lc}\|, \quad \hat{\mathbf{c}} = (\mathbf{X}^t \mathbf{W} \mathbf{X} + \lambda^2 \mathbf{L}^t \mathbf{L})^{-1} \mathbf{X} \mathbf{y} \quad (3.13)$$

Comment choisir l'hyper-paramètre λ ?

- **L-curve** le λ optimal correspond au coude dans le plot paramétrique $\{x(\lambda), y(\lambda)\}$ avec :

$$x(\lambda) = \|\mathbf{y} - \mathbf{X} \cdot \mathbf{c}\|_W, \quad y(\lambda) = \|\mathbf{Lc}\| \quad (3.14)$$

- **Generalized Cross Validation** le λ optimal est celui qui minimise la fonction :

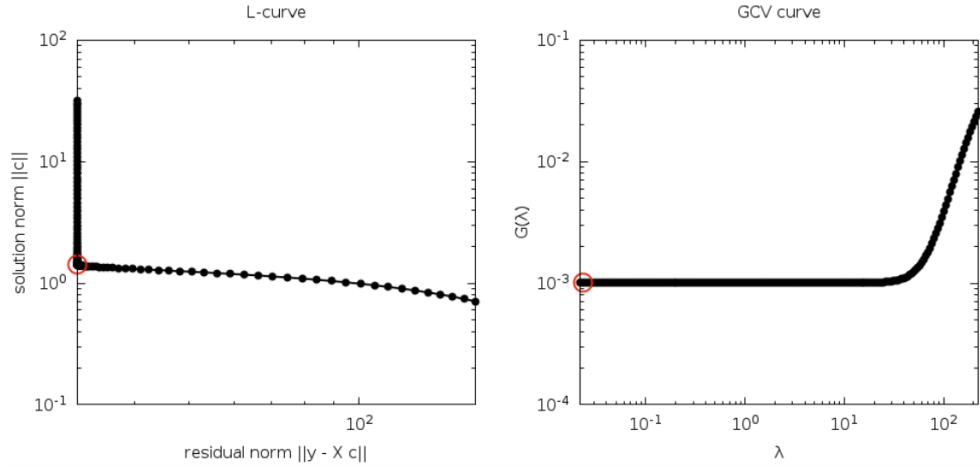
$$\mathcal{G}(\lambda) = \frac{\|\mathbf{y} - \mathbf{X} \cdot \mathbf{X}_\lambda^{-1} \mathbf{y}\|}{\text{Tr}[\mathbf{1}_n - \mathbf{X} \cdot \mathbf{X}_\lambda^{-1}]}, \quad \text{avec } c_\lambda = \mathbf{X}_\lambda^{-1} \mathbf{y} \quad (3.15)$$

- **L-curve** le λ optimal correspond au coude dans le plot paramétrique $\{x(\lambda), y(\lambda)\}$ avec

$$x(\lambda) = \|\mathbf{y} - \mathbf{X} \cdot \mathbf{c}\|_W, \quad y(\lambda) = \|\mathbf{Lc}\| \quad (3.16)$$

- **Generalized Cross Validation** le λ optimal c'est ce qui minimise la fonction :

$$\mathcal{G}(\lambda) = \frac{\|\mathbf{y} - \mathbf{X} \cdot \mathbf{X}_\lambda^{-1} \mathbf{y}\|}{\text{Tr}[\mathbf{1}_n - \mathbf{X} \cdot \mathbf{X}_\lambda^{-1}]}, \quad \text{avec } c_\lambda = \mathbf{X}_\lambda^{-1} \mathbf{y} \quad (3.17)$$



- **Comment juger la qualité du résultat final ?**

R^2 **score** : est-ce que notre modèle est meilleur que la simple moyenne de y ?

$$SS_{\text{res}} = \sum_{i=0}^{n-1} (y_i - \mathcal{Y}(\mathbf{c}, \mathbf{x}_i))^2, \quad (3.18)$$

$$SS_{\text{tot}} = \sum_{i=0}^{n-1} \left(y_i - \frac{1}{n} \sum_i y_i \right)^2, \quad (3.19)$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \quad (3.20)$$

A noter : la notation "carré" est trompeuse : R^2 peut être inférieure à 0 si $\frac{SS_{\text{res}}}{SS_{\text{tot}}} > 1$. Dans ce cas, le modèle est *particulièrement* mauvais : sa capacité de prédiction est *moins bonne* que simplement prendre la moyenne de y .

3.3 Régression logistique

- **Définition** : algorithme **supervisé** de classification *binaire* ou *multi-classe*.
- **Naissance** : le terme *logistique* dérive de la fonction mathématique que la méthode utilise : la fonction logistique. Cette fonction a été introduite pour la première fois par le mathématicien Pierre François Verhulst pendant les années 1830 et 1840 pour ses études sur la croissance des populations. L'utilisation moderne pour des tâches de classification trouve ses racines dans les travaux de Chester Ittner Bliss, John Gaddum et Ronald A. Fisher pendant les années 1930.
- **Applications** : Santé, score de crédit, caractérisation des images, reconnaissance d'écriture manuscrite

Données

- **Classification binaire** : n observations $\{\mathbf{x}_i, y_i\}_{i=1, \dots, n}$, $\mathbf{x}_i \in \mathbb{R}^d$ qui appartiennent à la première classe (classe *positive*), ($y_i = 1$) ou à la seconde classe (classe *negative*) ($y_i = 0$).
- **Classification Multi-classes**: le numéro des classes $k > 2$, $y_i = 0, \dots, k - 1$.

Modèle

Dépendance non-linéaire sur $\mathbf{c}$. Étant donné un seuil h :

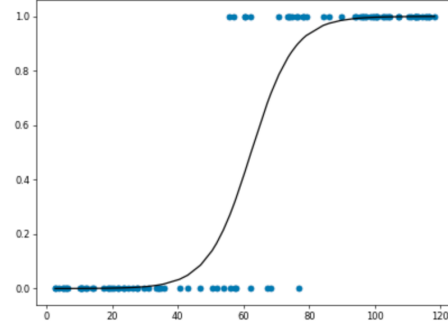
- *Classification binaire* :

$$g(z) = \frac{1}{1+e^{-z}} \quad \text{Sigmoid function,}$$

$$\mathcal{Y}(\mathbf{c}, \mathbf{x}_i) = \begin{cases} 1, & \text{if } g(\mathbf{c}^t \mathbf{x}_i) \geq h, \\ 0, & \text{if } g(\mathbf{c}^t \mathbf{x}_i) < h, \end{cases}$$

$g(\mathbf{c}^t \mathbf{x}_i) \in [0, 1]$, peut être interprété comme

$$h_{\mathbf{c}}(\mathbf{x}_i) = \mathcal{P}(y_i = 1 | \mathbf{c}, \mathbf{x}_i) = 1 - \mathcal{P}(y_i = 0 | \mathbf{c}, \mathbf{x}_i)$$



- *Classification Multi-classes: classification one-vs-all*. Pour chaque classe j ($j = 0, \dots, k-1$),

$$\{\mathbf{x}_i, y_i\}_{i=1, \dots, n}, \quad y_i = 0, \dots, k-1 \rightarrow \{\mathbf{x}_i, y_i^{[j]}\}_{i=1, \dots, n}, \quad y_i^{[j]} = 0, 1$$

nous définissons la classe *positive* ($y_i^{[j]} = 1$) et la classe *négative* ($y_i^{[j]} = 0$) pour l'échantillon i selon le fait de faire, ou pas, partie de la classe j . Nous apprenons un classificateur binaire $h_{\mathbf{c}}^{[j]}$ sur un jeu de données construit de cette façon. Finalement, on aura :

$$\mathbf{x}_i \rightarrow h_{\mathbf{c}}^{[j]}(\mathbf{x}_i) = \mathcal{P}(y_i^{[j]} = 1 | \mathbf{c}, \mathbf{x}_i) \quad \text{for } j = 0, \dots, k-1 \quad (3.21)$$

et

$$\mathcal{Y}(\mathbf{c}, \mathbf{x}_i) = \underset{j}{\operatorname{argmax}} \{h_{\mathbf{c}}^{[j]}(\mathbf{x}_i)\} \quad (3.22)$$

Optimisation des paramètres du modèle

- **Loss function**: étant donné n observations $\{\mathbf{x}_i, y_i\}_{i=1, \dots, n}$, la *loss* est

$$\mathcal{L}_{\mathbf{c}}^2 = \frac{1}{n} \sum_{i=1}^n \phi(y_i, \mathcal{Y}(\mathbf{c}, \mathbf{x}_i)) \quad (3.23)$$

- **Cost Function**: Pour la régression logistique, il est possible de démontrer mathématiquement que le *cost* défini ci-dessous permet d'avoir une *loss function* qui est convexe par rapport aux paramètres $\mathbf{c}$

$$\phi(\mathbf{c}, \mathbf{x}_i) = \begin{cases} -\log[h_{\mathbf{c}}(\mathbf{x}_i)], & \text{if } y_i = 0, \\ -\log[h_{\mathbf{c}}(1 - \mathbf{x}_i)], & \text{if } y_i = 1, \end{cases} \quad (3.24)$$

autrement dit :

$$\phi(\mathbf{c}, \mathbf{x}_i) = -y_i \log[h_{\mathbf{c}}(\mathbf{x}_i)] - (1 - y_i) \log[1 - h_{\mathbf{c}}(\mathbf{x}_i)] \quad (3.25)$$

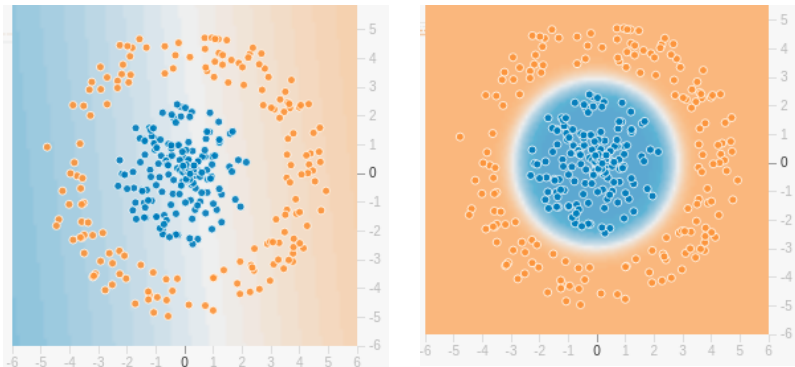
- **Objective**: trouver $\hat{\mathbf{c}}$ tel que :

$$\mathcal{L}_{\hat{\mathbf{c}}}^2 = \min [\mathcal{L}_{\mathbf{c}}^2] \quad (3.26)$$

Feature Engineering

- Tant que la fonction $g(z)$ est nourrie avec une combinaison linéaire des *features* originales $z = \mathbf{c}^t \mathbf{x}$, nous ne pouvons pas faire mieux que de classifier selon une séparation linéaire (*hyper-plane*) dans l'espace des données. La plupart du temps, cela ne suffit pas.
- Un *Feature engineering* est requis et il faut enrichir les *features* originelles $\mathbf{x}_i$ à travers des fonctions non-linéaire.

$$\mathbf{z} = \mathbf{c}^t \tilde{\mathbf{x}}_i, \quad \tilde{\mathbf{x}}_i = \{\mathbf{x}_i, \mathbf{f}(\mathbf{x}_i)\}$$



left) Input features :

$$\mathbf{x} = \{x_1, x_2\};$$

right) Input features :

$$\mathbf{x} = \{x_1, x_2, x_1^2, x_2^2\}$$

Figures faites avec playground.tensorflow.org.

- On peut apprendre les coefficients d'un modèle de régression paramétrique par maximisation de vraisemblance, ce qui équivaut à minimiser le risque empirique en utilisant le coût quadratique comme fonction de perte, et revient à la méthode des moindres carrés.
- Pour un problème de classification binaire, utiliser le coût logistique comme fonction de perte dans la minimisation du risque empirique revient à maximiser la vraisemblance d'un modèle dans lequel la probabilité d'appartenir à la classe positive est la transformée logit d'une combinaison linéaire des variables.
- La régression logistique n'a pas de solution analytique, mais on peut utiliser un algorithme à directions de descente pour apprendre le modèle.

3.4 Les k plus proches voisins (k-NN)

- **Définition** : Algorithme **supervisé** de classification. L'algorithme des k plus proches voisins, k-NN (k-Nearest Neighbor) classe les données en se basant sur leur similarité avec leurs voisins. k est le nombre de points *voisins* à considérer pour la classification.
Très simple, cette méthode non paramétrique ne cherche pas à apprendre une relation entrée/sortie mais à répondre à la question suivante : “dis-moi qui sont tes voisins, je te dirais qui tu es”
- **Naissance** : Les k-NN ont été inventés par **Evelyn Fix** et **Joseph Hodges** en 1951 puis approfondis par **Thomas Cover**.

- **Applications** : Biologie, thermique, recherche économique et sociale, algorithmes de recommandation...

Influence de k

- Le choix de k , le nombre de voisins à utiliser pour catégoriser une nouvelle donnée, va influencer la qualité de l'apprentissage. Sa valeur optimale dépend du problème traité, et correspond au point où l'erreur sur le test set commence à croître.
- La bonne pratique est de choisir un k impair, afin d'éviter les égalités.

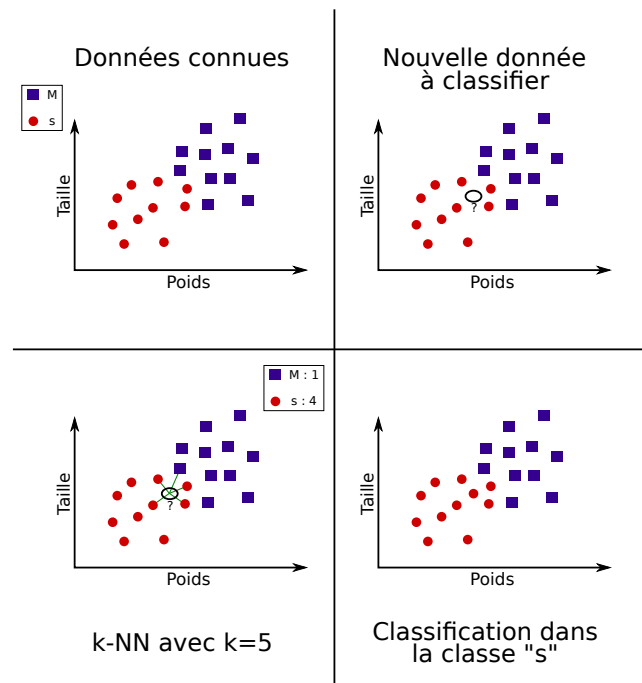
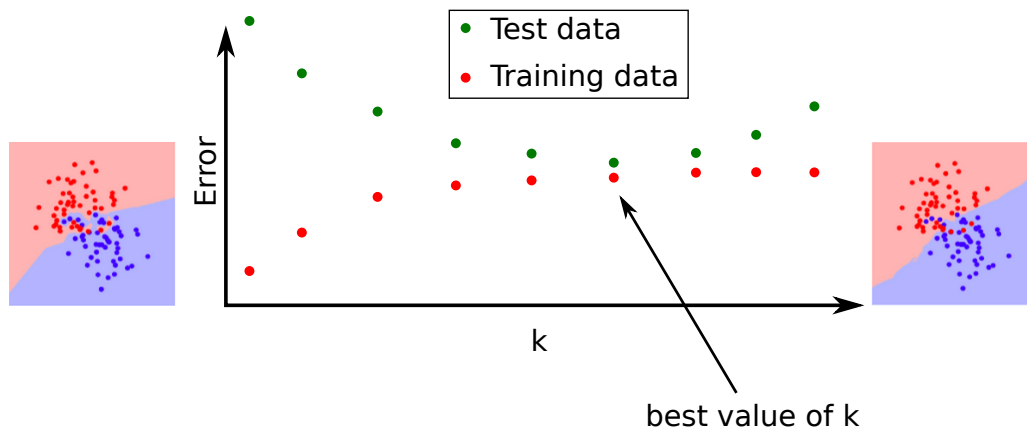


Figure 3.5: Exemple : taille de t-shirt en fonction du poids et de la taille d'un individu.

L'algorithme

Concrètement, Connaissant les attributs $A = X_1, X_2 \dots X_D$, où D est la dimension de l'espace de données, nous devons prédire la classification correspondante $G = Y_1, Y_2 \dots Y_n$ en se servant d'une métrique de proximité pour déterminer le degré de voisinage.

C'est-à-dire, qu'il faut :

- Charger les données.
- Initialiser la valeur de k
- Puis, sur l'ensemble d'apprentissage il faut :
 - évaluer la proximité ou distance
 - ordonner selon la métrique
 - sélectionner les k premiers voisins
 - déterminer la classe la plus fréquente
 - affecter cette classe au point courant

k-NN pour régression non-linéaire

Plutôt que de chercher à déterminer l'expression de la fonction affine représentant le mieux un nuage de points (x_i, y_i) . On peut dire que chaque nouveau point est la moyenne des k plus proches voisins de son abscisse, $D_k(x_i)$. Attention à son utilisation (ex: informations sur internet, marketing)...

$$\hat{f}(x_0) = \frac{1}{k} \sum_{D_k(x_0)} y_i \quad (3.27)$$

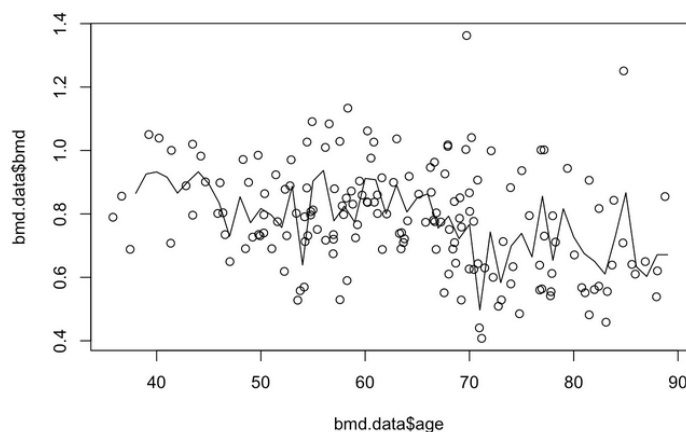


Figure 3.6: Utilisation de KNN en régression. Source bookdown.org

Applications en énergétique du bâtiment

- Classification du climat, gestion des ressources
- Estimation de l'efficacité énergétique des bâtiments
- Confort thermique
 - Régulation par rapport à des utilisateurs à préférence connues
 - Classifier des préférences utilisateurs par rapport à des mesures systèmes par capteurs
 - Prédiction de la sensation thermique à partir de la température de peau, de la température ambiante et de mesure infrarouge.
- Consommation des bâtiments par similitude
- Tentative de prédiction de charge froide, demande, production.
- Compréhension du modèle de consommation d'énergie de différents actifs (réfrigérations, déshumidificateurs, éclairage...) par DTW. Traitement des changements soudains (givre, météo...).

Points clefs

- L'algorithme des k plus proches voisins a un entraînement paresseux ; pour compenser, le coût algorithmique d'une prédiction peut être élevé si la base de données d'entraînement est grande.
- La qualité des prédictions d'un algorithme des k plus proches voisins dépend principalement du choix d'une bonne distance ou similarité.
- L'algorithme des k plus proches voisins fonctionne mieux en faible dimension:
 - son exécution est plus rapide;
 - il est moins susceptible d'être biaisé par des variables qui ne sont pas pertinentes;
 - il est moins susceptible de souffrir du fléau de la dimension.

3.5 Les k-moyennes (k-means)

- **Définition** : Algorithme **non-supervisé** de regroupement (clustering) de données sans labels. k est le nombre de cluster que l'on souhaite obtenir. *"Qui se ressemble, s'assemble"*
- **Naissance** : Le k-means a été inventé par **Stuard Lloyd** à des fins de modulation d'impulsion codée. Après plusieurs implémentations, une version stable a été publiée en 1975 par **Hartigan** et **Wong** en Fortran.
- **Applications** : Logistique, traitement du langage (bag of words), recherche économique et sociale, détection d'anomalie, compression d'image, etc.

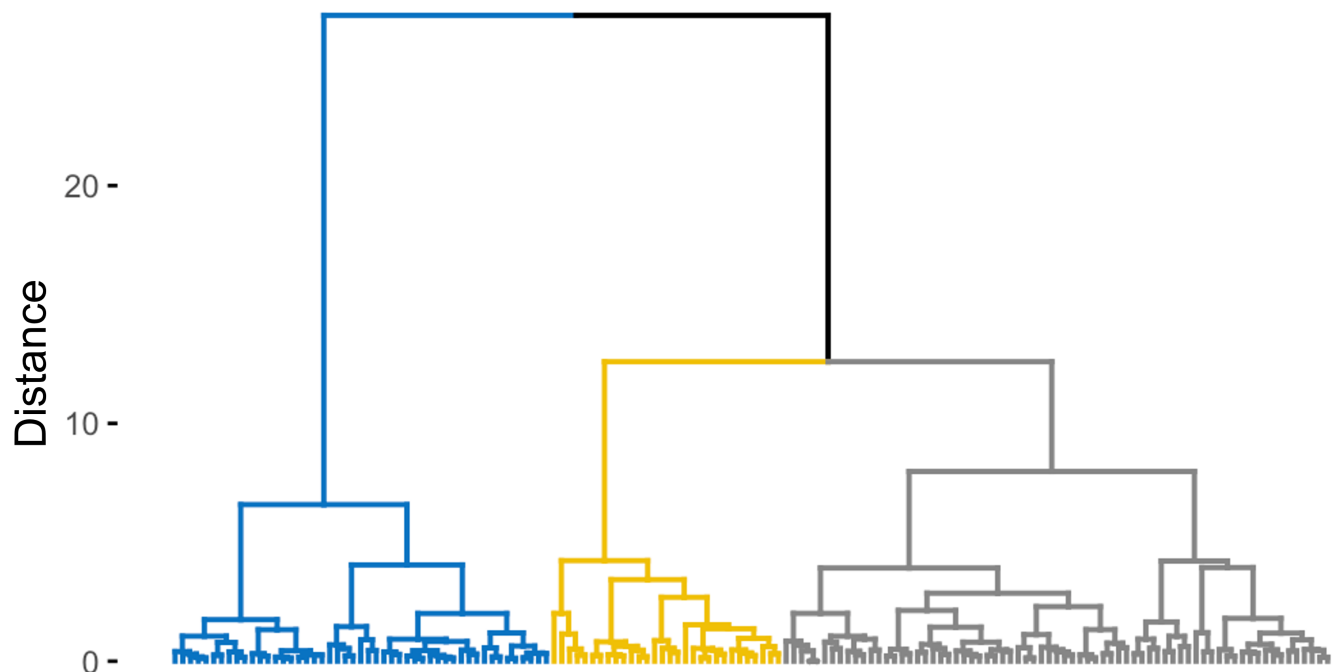
Les k-moyennes (k-means) - Un algorithme itératif

Idée de base : Les points de l'ensemble d'apprentissage sont regroupés dans des *clusters* définis selon leurs distances aux barycentres des *clusters*. Ces barycentres sont nommés *centroïdes*.

- Choisir la valeur de k .
- Initialiser les k centroïdes.
- Tant qu'un critère de convergence n'est pas atteint à chaque itération :
 - 1 Calculer les distances entre chaque point et chacun des centroïdes.
 - 2 Assigner à chaque point le centroïde le plus proche selon une distance $\mathcal{D}$. Cela définit les clusters $\mathcal{C}(\{\tilde{\mathbf{x}}_i\}_{i=1,\dots,k})$ pour cette itération de l'algorithme.
 - 3 Recalculer les centroïdes, ex: en prenant les points moyens de chaque *cluster* défini précédemment.

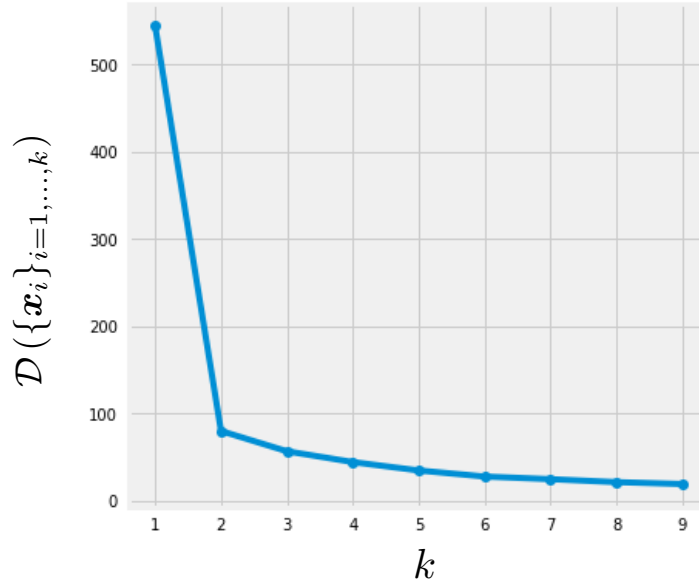
Exemple de dendrogramme

Dendrogramme de Clusterisation



Les k-moyennes (k-means) - Quelles valeurs pour k ?

- Connaissance métier : analyse des résultats
- Méthode du **coude** :



- Méthode l'analyse de **silhouette** - [source - scikit-learn.org](https://scikit-learn.org)

Coefficient de silhouette : reflète l'homogénéité des clusters

$$\begin{aligned}
 S_{sil} &= \frac{1}{K} \sum_{k=1}^K s_k \\
 s_k &= \frac{1}{n_{I_k}} \sum_{i \in I_k} s_{sil}(i) \\
 s_{sil}(i) &= \frac{b(i) - a(i)}{\max(a(i), b(i))}
 \end{aligned} \tag{3.28}$$

I_k : Cluster k

$a(i)$: distance moyenne à son groupe

$b(i)$: plus petite distance moyenne du point à son groupe voisin

$s_{sil}(i)$: largeur de silhouette du point

s_k : silhouette du cluster k

S_{sil} : coefficient de silhouette $\in [-1, 1]$

S_{sil} : -1 : très mauvais, 0 : il y a des intersections, 1 : excellent

$$K = 2, S_{sil} = 0.705$$

$$K = 3, S_{sil} = 0.588$$

$$K = 4, S_{sil} = 0.651$$

$$K = 5, S_{sil} = 0.564$$

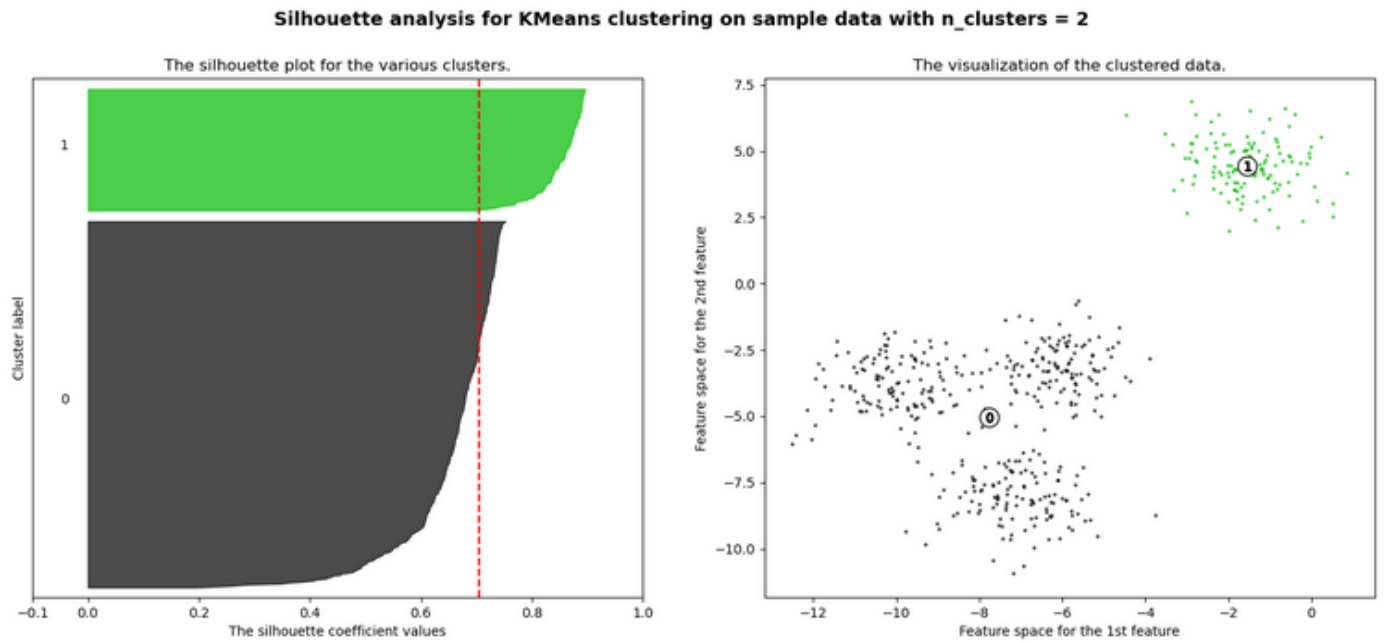


Figure 3.7: Exemple de silhouette.

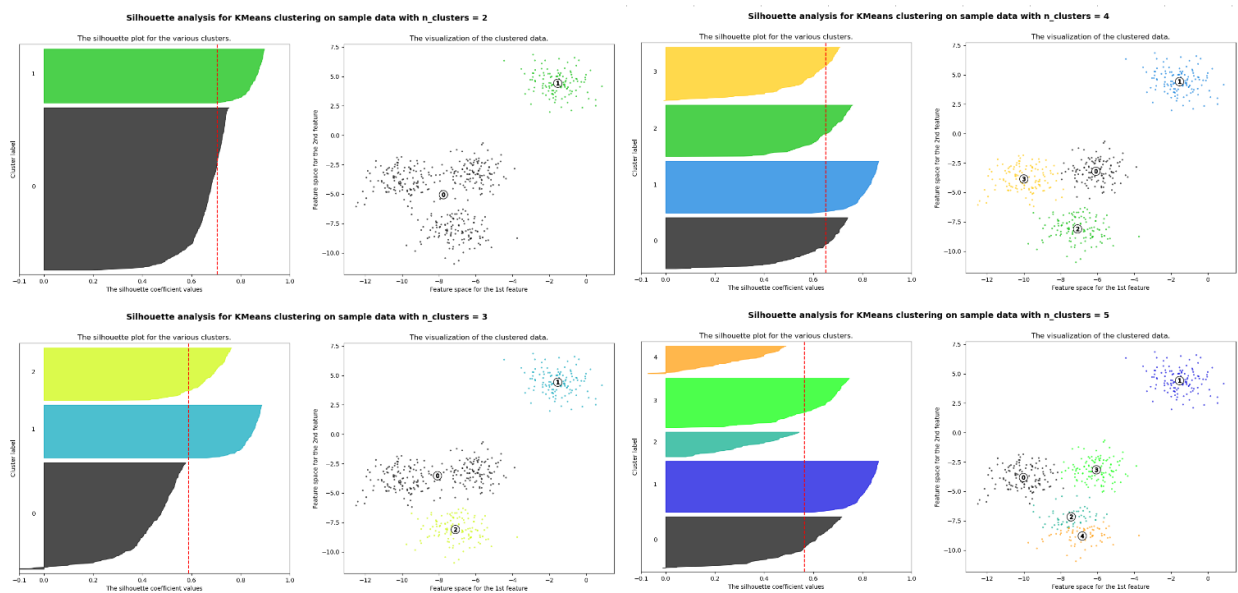


Figure 3.8: Impact de la silhouette.

D'autres métriques - source [cran.r-project](https://cran.r-project.org/) :

- Indices internes ou de qualité des clusters
Elbow/coude, Silhouette, PBM
- Indices basé sur les barycentres
Ball-Hall, Banfeld-Raftery, Davies-Bouldin, Ray-Turi, Wemmert-Gançarki.
- Indices basé sur des comparaisons de distances
Dunn, C index ou concordance d'Harrel, Basker-Hubert ou index Gamma.
- Mixte barycentre & distance
Index SD et S_Dbw.
- Indices sur les paires discordantes
G_plus et tau.
- Indices d'éparpillements
Trace_WiB ou Trace critère sur la trace de Hotelling, Ksq_DetW, Scott-Symons
- Indices de comparaison entre l'intra et l'inter groupe
Calinski-Harabasz, McClain-Rao, Point-Biserial, Ratkowsky-Lance
- Indices floue
Index Xie-Beni.
- Indices externes ou de similarité des clusters
Czekanowski-Dice, Folkes-Mallows, Hubert Γ , Russel-Rao, Jaccard, Kulczynski, McNemar, Phi, Rand, Rogers-Tanimoto

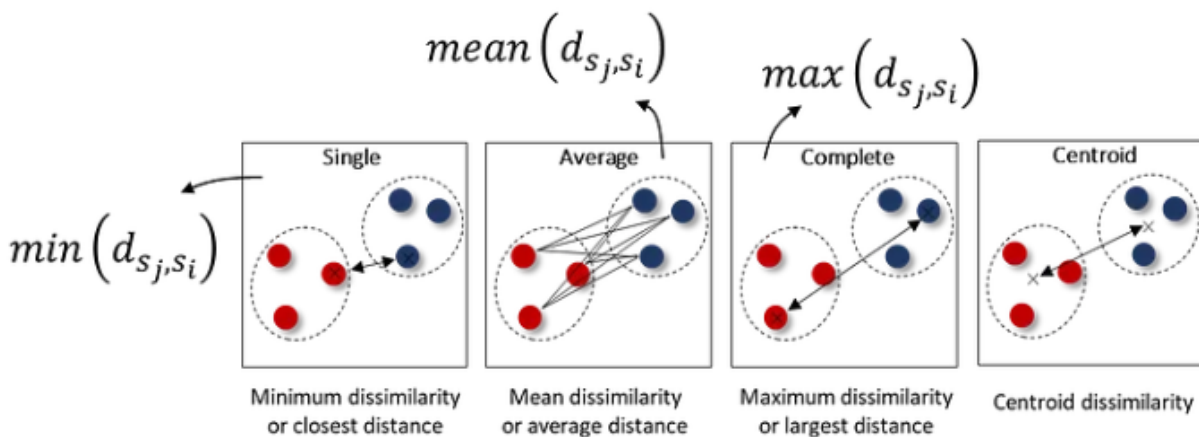


Figure 3.9: Illustrations de métriques. source TDS

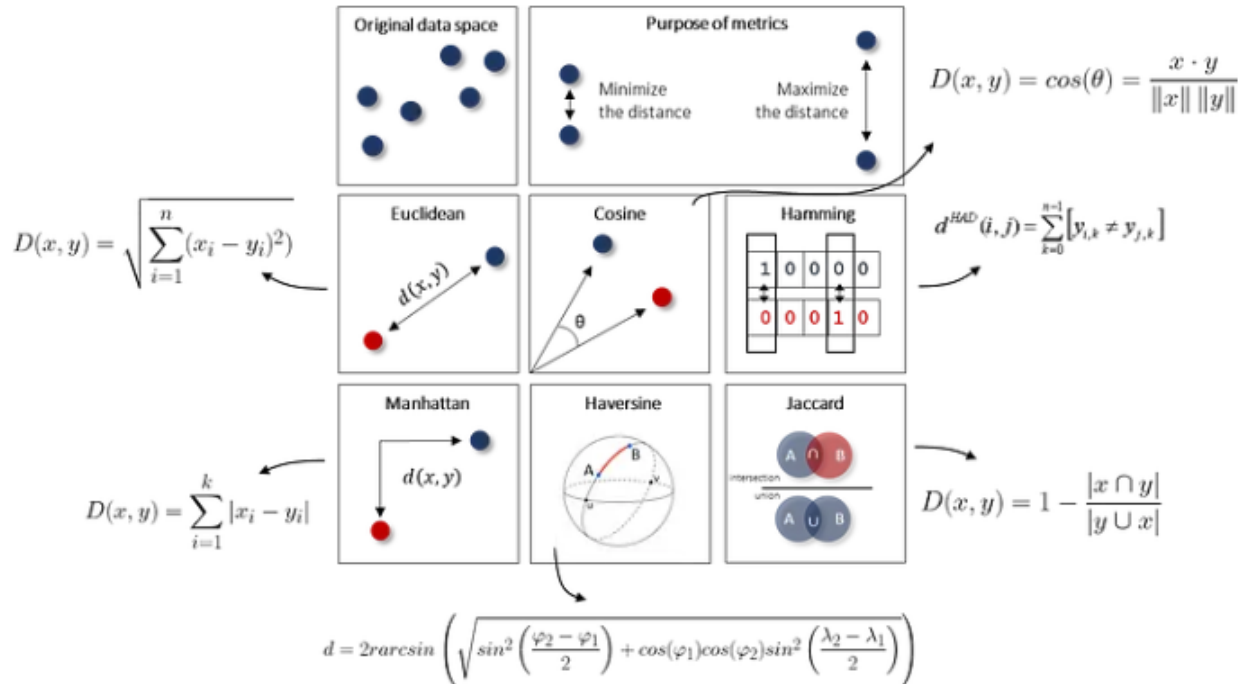


Figure 3.10: Illustrations de métriques. source TDS

Applications à une estimation d'occupations - article source

Bâtiment résidentiel à Milan

-Emprise au sol 4500 m²

-70 appartement

-200 occupants

-3 scénario :

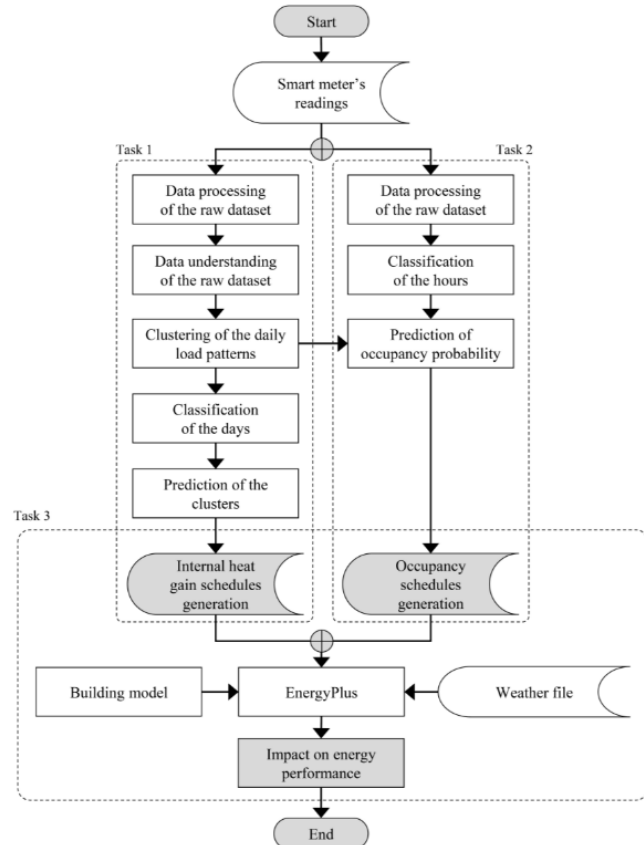
Q1 : 1er quartile, familles qui passent leur temps de-hors

Q2 : 2e quartile, scénario intermédiaire

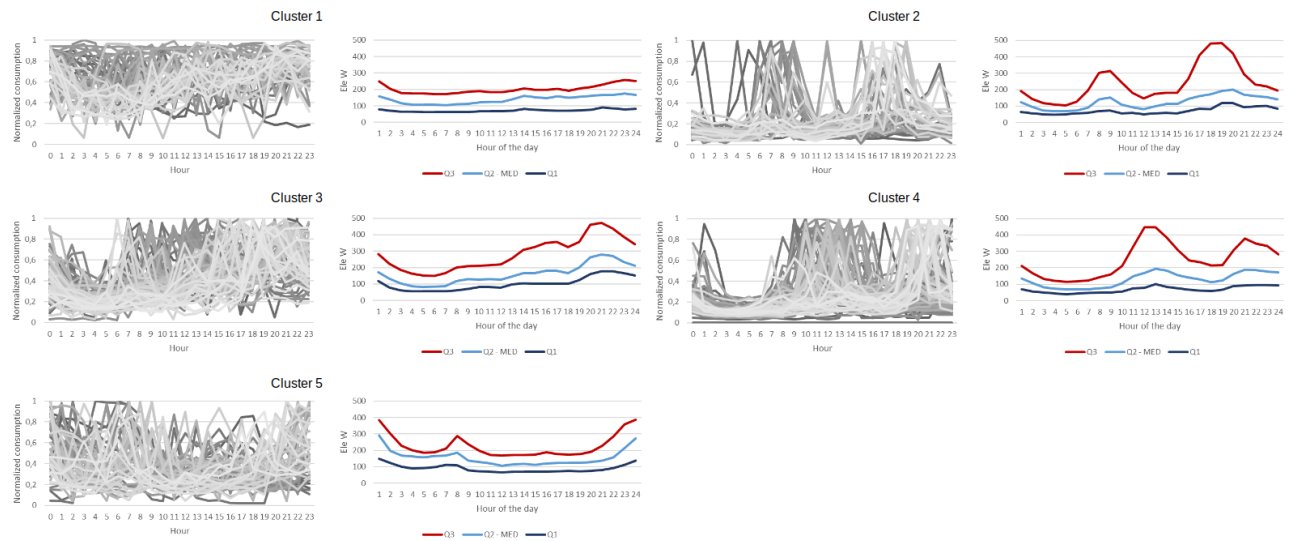
Q3 : 3e quartile, familles qui passent leurs temps chez eux

-métrique pour choisir K: Davies-Bouldin Index (rapport entre la dispersion intra-cluster et la distance inter-cluster).

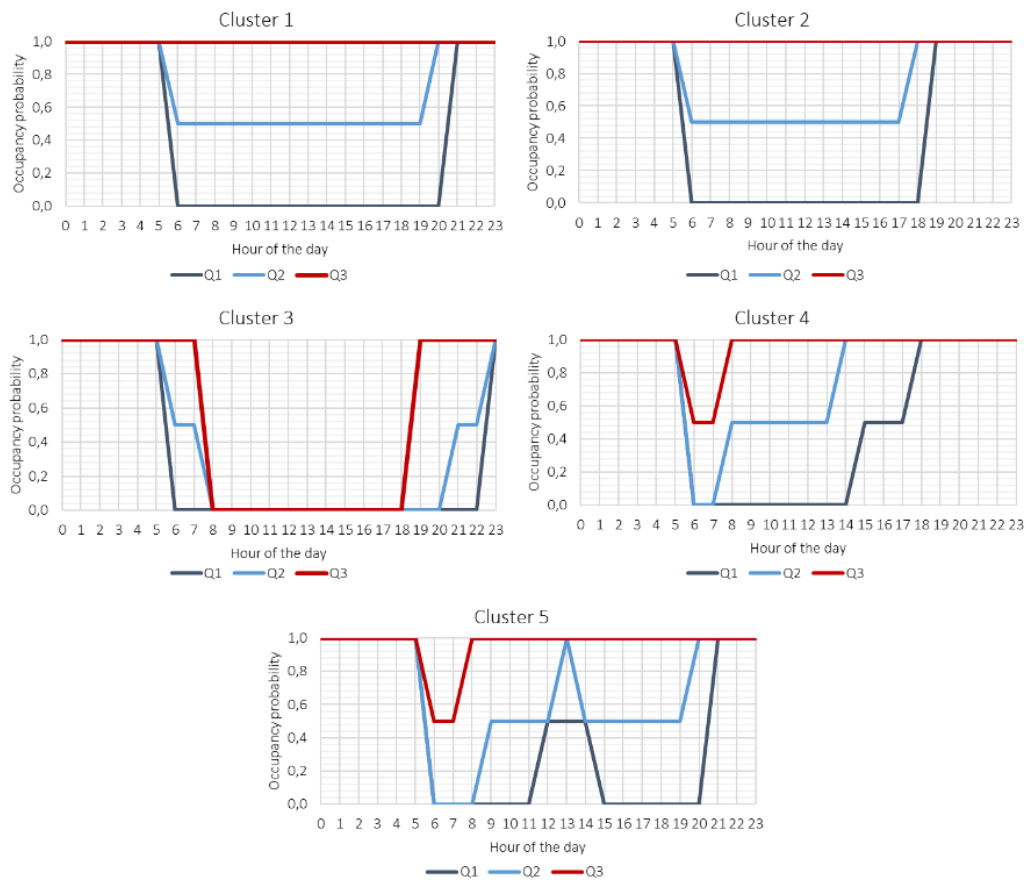
-Classification avec un KNN



Estimation de la consommation



Détermination des profils d'occupations



Les techniques de clustering sont nombreuses, éparpillées et souvent obscure, voici des classifications de ce que l'on peut trouver:

Techniques de Clustering :

- **K-Means :**

- Métrique : Distance Euclidienne
- Applications :
 - * Segmentations de marché
 - * Analyse de la clientèle
 - * Regroupement d'images

- **Hierarchical Clustering :**

- Métrique : Différentes distances (Euclidienne, Manhattan, etc.)
- Applications :
 - * Taxonomie biologique
 - * Analyse de similarité génétique
 - * Classification de documents

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise) :**

- Métrique : Distance (epsilon) et densité (minPts)
- Applications :
 - * Détection d'anomalies
 - * Regroupement de points géographiques
 - * Séparation de clusters de formes irrégulières

- **Mean Shift :**

- Métrique : Distance
- Applications :
 - * Suivi d'objets dans des vidéos
 - * Détection de visages

- **Gaussian Mixture Model (GMM) :**

- Métrique : Probabilités et paramètres de distribution
- Applications :
 - * Modélisation de données complexes
 - * Détection d'anomalies

Métriques de Clustering :**• Inertie (K-Means) :**

- Mesure la somme des distances entre les points et leurs centres de cluster.
- Plus basse est l'inertie, mieux c'est.

• Coefficient de Silhouette :

- Mesure la distance entre les points d'un cluster et les autres clusters.
- Plus proche de +1, meilleure est la séparation des clusters.

• Indice de Davies-Bouldin :

- Mesure la similarité moyenne entre chaque cluster et son cluster le plus similaire.
- Plus bas est l'indice, meilleure est la séparation des clusters.

• Indice de Dunn :

- Mesure la distance minimale entre les centres de cluster divisée par la plus grande distance intra-cluster.
- Plus grand est l'indice, meilleure est la séparation des clusters.

Techniques de Clustering et Métriques de Distances :**• Marketing et Segmentations de Marché :**

- Techniques : K-Means, GMM
- Métriques : Inertie (K-Means), Probabilités et paramètres de distribution (GMM)
- Objectif : Identifier des groupes de clients avec des comportements similaires.

• Traitement de Données Géospatiales :

- Techniques : DBSCAN, Mean Shift
- Métriques : Distance (epsilon pour DBSCAN, Distance pour Mean Shift), Densité (minPts pour DBSCAN)
- Objectif : Regrouper des points géographiques similaires.

• Biologie et Génétique :

- Techniques : Hierarchical Clustering
- Métriques : Différentes distances (Euclidienne, Manhattan, etc.)
- Objectif : Classer des espèces en fonction de leurs caractéristiques génétiques.

• Suivi d'Objets dans des Vidéos :

- Techniques : Mean Shift
- Métriques : Distance
- Objectif : Suivre des objets en mouvement dans une séquence vidéo.

- **Modélisation de Données Complexes :**

- Techniques : Gaussian Mixture Model (GMM)
- Métriques : Probabilités et paramètres de distribution
- Objectif : Modéliser des données avec des distributions mixtes.

Distances utilisées en Clustering et Applications Métiers :

- **Applications en Marketing et Segmentations de Marché :**

- Distance Euclidienne :
 - * Mesure la distance directe entre les points dans l'espace euclidien.
 - * Peut être utilisée avec K-Means pour segmenter les clients en fonction de leurs comportements d'achat.
- Distance de Manhattan (Taxicab) :
 - * Mesure la somme des distances horizontales et verticales entre les points.
 - * Utile pour segmenter les clients en fonction de caractéristiques similaires, comme le revenu et l'âge.

- **Applications en Traitement de Données Géospatiales :**

- Distance Euclidienne :
 - * Peut être utilisée pour mesurer la distance directe entre les points géospatiaux.
 - * Utile pour regrouper des points géographiques similaires, tels que des emplacements commerciaux.
- Distance de Haversine :
 - * Mesure la distance entre deux points sur une sphère (terre) en utilisant des coordonnées géographiques (latitude et longitude).
 - * Utilisée pour le clustering de données géospatiales, comme la regroupement de lieux d'intérêt touristique.

- **Applications en Biologie et Génétique :**

- Distance de Jaccard :
 - * Mesure la similarité entre deux ensembles en considérant la proportion d'éléments partagés par rapport à l'union totale des éléments.
 - * Utilisée pour regrouper des espèces ou des gènes en fonction de leurs caractéristiques communes.
- Distance de Corrélation :
 - * Mesure la similarité entre les profils d'expression génique.
 - * Utilisée pour regrouper des gènes ou des échantillons en fonction de leurs profils d'expression similaires.

- **Applications en Suivi d'Objets dans des Vidéos :**

- Distance Euclidienne :

- Utilisée pour mesurer la distance spatiale entre les objets en mouvement dans différentes images d'une séquence vidéo.
- Utile pour suivre des objets dans des vidéos.

- **Applications en Modélisation de Données Complexes :**

- Distance de Mahalanobis :

- Mesure la distance entre un point et la moyenne des points d'un ensemble, en tenant compte de la covariance entre les dimensions.
- Utilisée pour modéliser des données avec des distributions complexes.

Pour aller plus loin : [An entry point to clustering methods](#)

Points clefs

- Le clustering, ou partitionnement de données, cherche à identifier des classes sans utiliser d'étiquettes.
- En l'absence d'étiquette, la qualité d'une partition peut s'évaluer sur des critères de séparabilité et d'homogénéité.
- Le clustering hiérarchique partitionne les données de manière itérative. Son résultat peut être visualisé sur un dendrogramme.

3.6 Analyse en composantes principales (PCA)

- **Définition** : Algorithme **non-supervisé** combinant une approche géométrique et statistique. La PCA (aussi connue sous le nom de transformée de *Karhunen-Loève*) permet d'effectuer une **réduction de dimension** d'un jeu de données en le projetant dans un espace où les axes sont *orthogonaux* et où la **variance est maximale**.
- **Naissance** : La PCA a été inventée par **Karl Pearson** en **1901**, avec le souhait de prolonger ses travaux de régression et de corrélation multivariées.
- **Applications** : Biologie, recherche économique et sociale, traitement d'image (reconnaissance faciale), astrophysique, etc.

La PCA peut être appliquée dans le domaine de l'énergie comme outil de **reconnaissance faciale** à travers deux exemples :

- Pour retrouver un profil utilisateur dans une base de données.
- Pour améliorer le confort de l'environnement de l'utilisateur.

Sources :

- *A. Aryal et al.*, Building and Environment, 2019
- *H. Metzmacher et al.*, Building Simulation, 2017.
- *A. Aryal et al.*, Proc. of BuildSys'19, 2019

- L'algorithme PCA appliqué à la **reconnaissance faciale** est né des travaux de *M. A. Turk* et *A. P. Pentland* au MIT Media Lab, en 1991. Il est aussi connu sous le nom de *Eigenfaces* car il utilise des vecteurs propres et des valeurs propres (respectivement *Eigenvectors* et *Eigenvalues* en anglais).
- L'idée principale consiste à exprimer M images de départ selon une base de vecteurs orthogonaux particuliers - les vecteurs propres - contenant des informations indépendantes d'un vecteur à l'autre.
- Les données originales seront donc exprimées d'une manière plus appropriée à la reconnaissance du visage.

PCA : illustration en reconnaissance faciale **Idee de base**

Compresser l'information en maximisant la variance retenue.

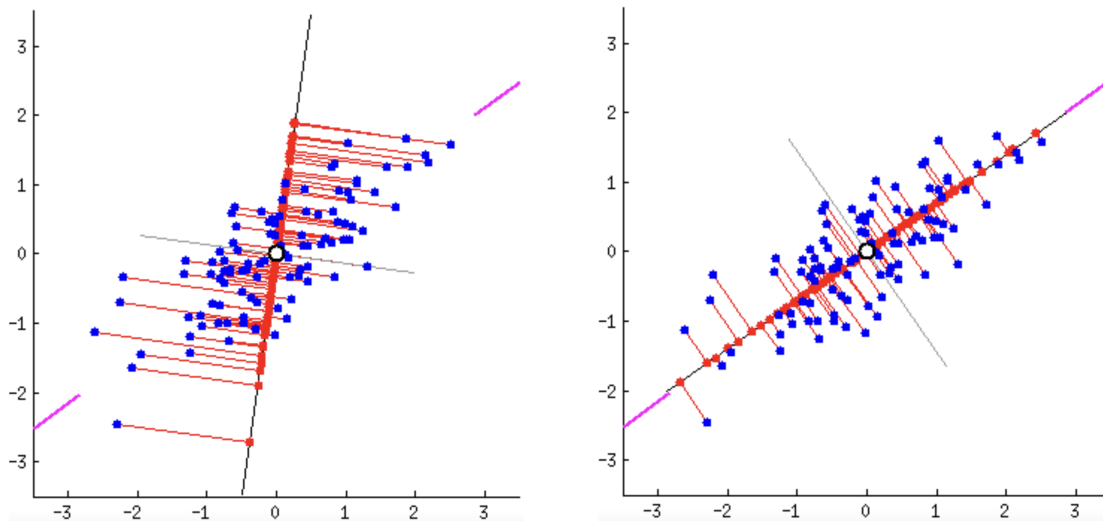


Figure 3.11: source medium

Rappelle de Algèbre linéaire

- La covariance peut être écrite sous forme matricielle

$$[\text{Cov}_S]_{ij} = \frac{1}{n-1} \sum_{k=1}^n (x_k^i - \bar{x}^i)(x_k^j - \bar{x}^j) = \frac{1}{n-1} \mathbf{X}^t \mathbf{X},$$

$$\text{où } \mathbf{X} = \{\mathbf{x}_0, \dots, \mathbf{x}_{n-1}\} - \bar{\mathbf{x}}$$

- Décomposition en valeurs singulières d'une matrice (réelle) $\mathbf{A} \in \mathbf{R}^{m \times n}$, $n \geq m$

$$\mathbf{A} = \mathbf{V} \begin{pmatrix} \sigma_1 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \dots & \vdots \\ 0 & \dots & \sigma_{m-1} & \dots & 0 \end{pmatrix} \mathbf{U}^t, \quad \text{où } \begin{cases} \mathbf{V} \in \mathbf{R}^{m \times m}, & \mathbf{V}^t \mathbf{V} = \mathbf{1}_m, \\ \mathbf{U} \in \mathbf{R}^{n \times n}, & \mathbf{U}^t \mathbf{U} = \mathbf{1}_n, \\ \sigma_k \geq 0 \end{cases}$$

- Si $A \in \mathbf{R}^{m \times m}$ est définie positive, alors la SVD est aussi une décomposition en valeurs propres (*eigenvalue decomposition*)

$$A = V \text{diag} [\sigma_0, \dots, \sigma_{m-1}] V^t \quad (3.29)$$

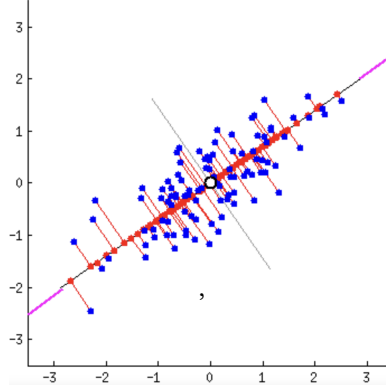
Idée de base

Compresser l'information en maximisant la variance retenue.

$$S = \{\mathbf{x}_i\}_{i=1,\dots,n}, \quad \mathbf{x}_i = \{x_i^0, \dots, x_i^d\}$$

$$[\text{Cov}_S]_{ij} = \frac{1}{n-1} \sum_{k=1}^n (x_k^i - \bar{x}^i)(x_k^j - \bar{x}^j)$$

$$\text{Cov}_S \xrightarrow{\text{SVD}} \begin{cases} \mathbf{v}_1, \dots, \mathbf{v}_n, & \text{vecteurs propres} \\ \sigma_1, \dots, \sigma_n, & \text{valeurs propres} \end{cases}$$



composantes principales $\leftrightarrow$ vecteurs propres

Attention:

La première composante principale, ce n'est pas celle qui est issue d'une régression linéaire

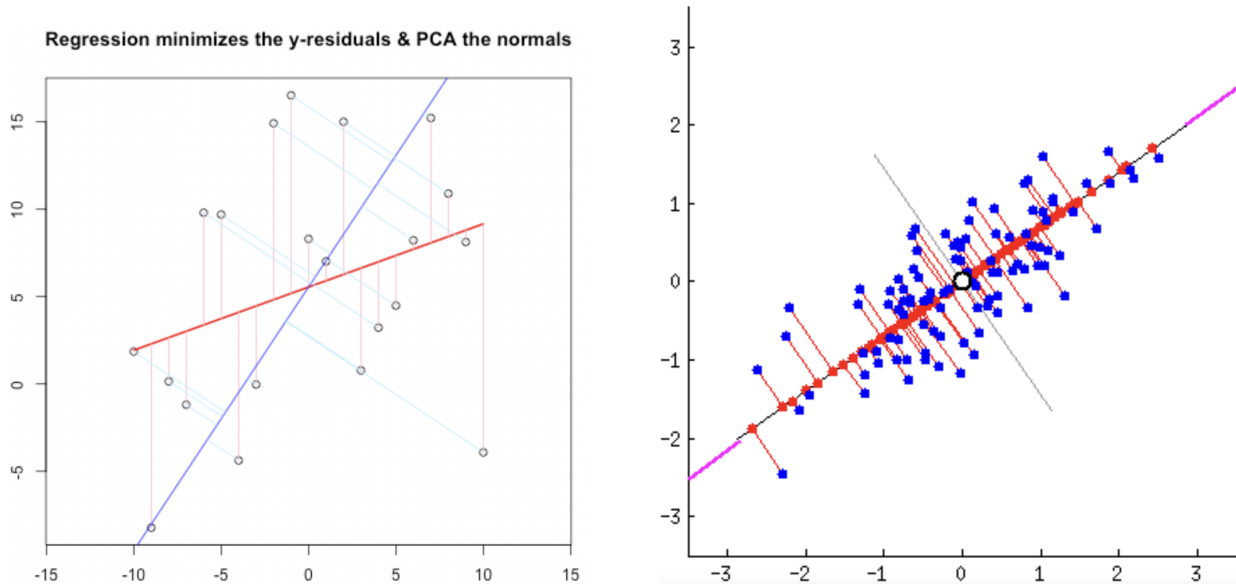


Figure 3.12: Source stackoverflow.com

Une image $\mathbf{I}_{i(m,n)}$ est traitée comme un vecteur $\Gamma_{i(m \times n, 1)}$ dans un espace vectoriel de grande dimension

($N = m \times n$), par concaténation des colonnes :

$$\mathbf{I}_1 = \begin{pmatrix} a_{1,1} & \dots & a_{1,m} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \dots & a_{n,m} \end{pmatrix} \Rightarrow \mathbf{\Gamma}_1 = \begin{pmatrix} a_{1,1} \\ \vdots \\ a_{n,1} \\ \vdots \\ a_{1,m} \\ \vdots \\ a_{n,m} \end{pmatrix} \quad (3.30)$$

Passage d'une image vers un vecteur dans un espace vectoriel de grande dimension. Les coefficients $a_{i,j}$ représentent les valeurs des pixels en niveau de gris, codés de 0 à 255.

Après avoir rassemblé les M images dans une unique matrice, nous obtenons une *matrice d'images* $\mathbf{\Gamma}$, où chaque colonne représente une image $\mathbf{\Gamma}_i$:

$$\mathbf{\Gamma} = \begin{pmatrix} a_{1,1} & b_{1,1} & \dots & z_{1,1} \\ \vdots & \vdots & \dots & \vdots \\ a_{n,1} & b_{n,1} & \dots & z_{n,1} \\ \vdots & \vdots & \dots & \vdots \\ a_{1,m} & b_{1,m} & \dots & z_{1,m} \\ \vdots & \vdots & \dots & \vdots \\ a_{n,m} & b_{n,m} & \dots & z_{n,m} \end{pmatrix} \quad (3.31)$$

On calcule ensuite l'image moyenne Ψ de toutes les images collectées. Cette image peut être vue comme le *centre de gravité* du jeu d'images :

$$\Psi = \frac{1}{M} \sum_{i=1}^M \mathbf{\Gamma}_i \quad (3.32)$$

On ajuste ensuite les données par rapport à la moyenne. L'image moyenne est alors soustraite de chaque image :

$$\Phi_i = \mathbf{\Gamma}_i - \Psi, i = 1 \dots M \quad (3.33)$$

On calcule ensuite la matrice de covariance du jeu de données :

$$\mathbf{C} = \sum_{i=1}^M \Phi_i \Phi_i^T = \mathbf{A} \mathbf{A}^T, \mathbf{A} = [\Phi_1 \Phi_2 \dots \Phi_M] \quad (3.34)$$

- La prochaine étape consiste à calculer les vecteurs propres et les valeurs de cette matrice de covariance $\mathbf{C}$ de taille $(N \times N)$, c'est-à-dire de l'ordre de la résolution d'une image.

- Le problème est que cela peut parfois être très difficile et très long ! En effet, si $N > M$ (si la résolution est supérieure au nombre d'images), il y aura seulement M vecteurs propres qui contiendront de l'information (les vecteurs propres restants auront des valeurs propres associées nulles).
- Par exemple, pour 100 images de résolution 320×240 , nous pourrions résoudre une matrice $\mathbf{L}$ de 100×100 au lieu d'une matrice de 76800×76800 pour ensuite prendre les combinaisons linéaires appropriées des images Φ_i .
- Le gain de temps de calcul serait considérable ! Typiquement, nous passerions d'une complexité de l'ordre du nombre de pixels dans une image à une complexité de l'ordre du nombre d'images.

Voici comment procéder pour accélérer les calculs :

Considérons les vecteurs propres e_i de $\mathbf{C} = \mathbf{A}\mathbf{A}^T$, associés aux valeurs propres λ_i :

$$\mathbf{C}e_i = \lambda_i e_i \quad (3.35)$$

Les vecteurs propres v_i de $\mathbf{L} = \mathbf{A}^T \mathbf{A}$, associés aux valeurs propres μ_i sont tels que :

$$\mathbf{L}v_i = \mu_i v_i \quad (3.36)$$

Soit :

$$\mathbf{A}^T \mathbf{A}v_i = \mu_i v_i \quad (3.37)$$

On multiplie à gauche par $\mathbf{A}$ des deux côtés de l'égalité :

$$\mathbf{A}\mathbf{A}^T \mathbf{A}v_i = \mathbf{A}\mu_i v_i \quad (3.38)$$

Puisque $\mathbf{C} = \mathbf{A}\mathbf{A}^T$, nous pouvons simplifier :

$$\mathbf{C}(Av_i) = \mu_i (Av_i) \quad (3.39)$$

Les équations (3.35) et (3.39) montrent que Av_i et μ_i sont respectivement les vecteurs propres et les valeurs propres de $\mathbf{C}$, soit :

$$\begin{cases} e_i = Av_i \\ \lambda_i = \mu_i \end{cases} \quad (3.40)$$

- Nous pouvons donc trouver les valeurs propres de cette énorme matrice $\mathbf{C}$ en trouvant les valeurs propres d'une matrice $\mathbf{L}$ beaucoup plus petite.
- Pour trouver les vecteurs propres de $\mathbf{C}$, il suffit juste de pré-multiplier les vecteurs propres de $\mathbf{L}$ par la matrice $\mathbf{A}$.
- Les vecteurs propres trouvés sont ensuite ordonnés selon leurs valeurs propres correspondantes, de manière décroissante. Plus une valeur propre est grande, plus la variance capturée par le vecteur propre est importante.
- Cela implique que la majeure partie des informations est contenue dans les premiers vecteurs propres.
- Une part de la grande efficacité de l'algorithme PCA vient de l'étape suivante qui consiste à ne sélectionner que les k meilleurs vecteurs propres (ceux avec les k plus grandes valeurs propres). Par exemple, on peut décider de garder un nombre de vecteurs propres tel que la *variance cumulée expliquée* soit $\geq 95\%$.

- On définit alors un espace vectoriel engendré par ces k vecteurs propres, que l'on appelle l'espace des visages E_v (*Face Space*).
- Les images originales peuvent être reconstituées par combinaison linéaire de ces vecteurs propres.
- Les représentations graphiques de ces vecteurs rappellent un peu des *images fantômes*, chacune mettant en avant une partie du visage, on les appelle *eigenfaces* (Fig. 3.13).

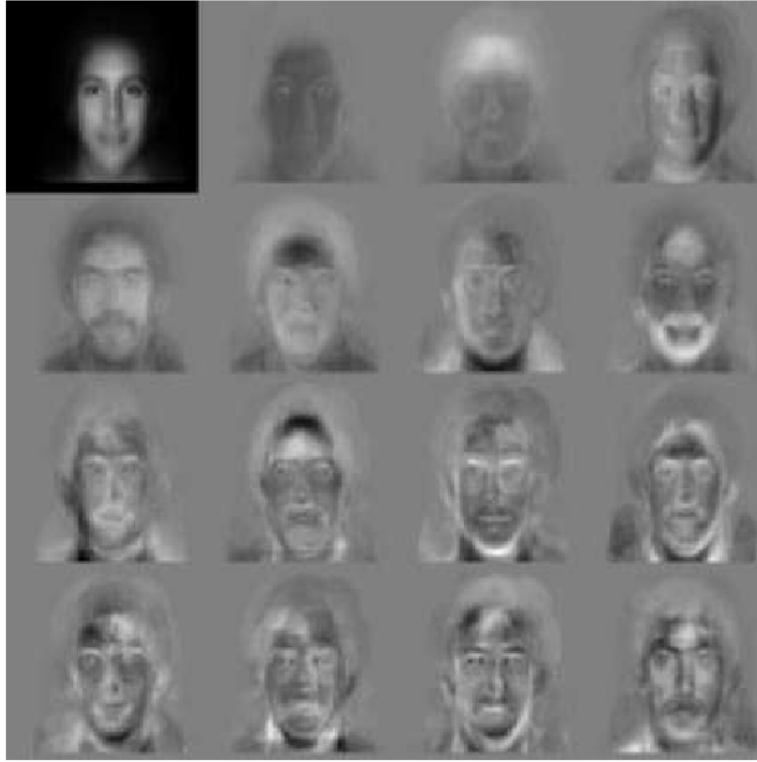


Figure 3.13: Image moyenne et les 15 premières *eigenfaces*.
(Source : N. Morizet, PhD Thesis, 2009.)

Nous allons maintenant projeter nos images de départ sur E_v . Une image Γ_i est alors transformée en ses composantes *eigenfaces* par une simple opération de projection vectorielle :

$$\omega_i^j = e_j^T (\Gamma_i - \Psi), j = 1, \dots, k \quad (3.41)$$

Les vecteurs ω_i^j (pour une image i) sont appelés *poids* et forment une matrice $\Omega_i^T = [\omega_i^1, \omega_i^2, \dots, \omega_i^k]$ qui décrit la contribution de chaque eigenface dans la représentation de l'image d'entrée.

La matrice Ω_i^T est alors utilisée pour trouver quelle est, parmi un nombre pré-défini des individus (classes possibles), celle qui décrit le mieux une image d'entrée.

La méthode la plus simple pour déterminer quelle classe de visage (quel visage dans notre base des données) fournit la meilleure description d'une image d'entrée est de trouver le visage $\hat{k}$ qui minimise la distance euclidienne :

$$\varepsilon_{\hat{k}}^2 = \|\Omega_i - \Omega_{\hat{k}}\|^2 \quad (3.42)$$

où $\Omega_{\hat{k}}$ est une matrice qui décrit la $\hat{k}$ ème classe de visage.

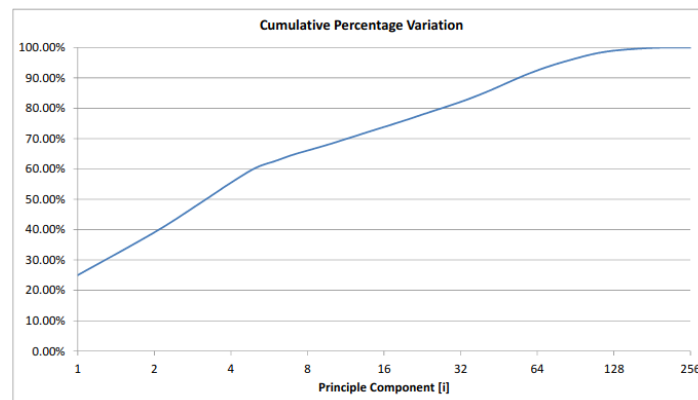
Un visage appartient à une classe k quand le minimum ε_k est en dessous d'un certain seuil θ_ε .

Dans le cas contraire, le visage est classé comme étant inconnu et peut éventuellement être utilisé pour créer une nouvelle classe de visage.

- En résumé, l'algorithme PCA est une méthode globale utilisant en premier lieu les niveaux de gris des pixels d'une image.
- Le principe selon lequel on peut construire un sous-espace vectoriel en ne retenant que les *meilleurs* vecteurs propres, tout en conservant beaucoup d'information utile, fait du PCA un algorithme efficace et couramment utilisé en réduction de dimensionnalité.
- Ce fut *un des tous premiers* algorithmes de reconnaissance faciale (1991) qui a maintenant laissé sa place à la robustesse et à la puissance des *réseaux de neurones profonds*.

PCA : applications en énergétique

- **Pré-classification** de données (météo, consommation, etc.) par *réduction de dimensionnalité*. Mais attention à l'effet diurne qui a tendance à donner au temps une plus grande importance.
- **Analyse de l'importance relative** des différentes variables.



Source manchester.ac.uk

3.7 Décomposition en Valeurs Singulières (SVD)

La SVD (Décomposition en Valeurs Singulières) est une technique fondamentale en algèbre linéaire et en traitement de données qui permet de décomposer une matrice en trois autres matrices qui capturent les informations essentielles de cette matrice d'origine. Elle est utilisée dans de nombreux domaines, tels que l'analyse de données, la compression d'images, la réduction de dimension, le traitement du signal et bien d'autres. La SVD a des applications diverses grâce à sa capacité à extraire des structures et des motifs sous-jacents des données.

Voici comment fonctionne la SVD en détail :

Supposons que nous ayons une matrice réelle A de dimensions $m \times n$. La SVD décompose cette matrice en trois matrices :

- Matrice U ($m \times m$) : Les colonnes de cette matrice sont les vecteurs singuliers à gauche. Ces vecteurs représentent les directions principales dans lesquelles la matrice d'origine A se dilate lorsqu'elle est transformée.
- Matrice Σ ($m \times n$) : Cette matrice diagonale contient les valeurs singulières de la matrice A , triées de la plus grande à la plus petite. Les valeurs singulières mesurent l'importance de chaque composant principal dans la décomposition.
- Matrice V^T ($n \times n$) : Les lignes de cette matrice sont les vecteurs singuliers à droite. Ces vecteurs représentent les directions principales dans lesquelles la matrice résultante $U\Sigma$ (la "reconstruction" de A) se dilate.

La décomposition SVD s'exprime mathématiquement comme : $A = U\Sigma V^T$

Les étapes de la SVD sont les suivantes :

- Calcul des Valeurs et Vecteurs Propres :
Calculer les valeurs et vecteurs propres de la matrice $A^T * A$ (ou $A * A^T$ en fonction des dimensions). Les valeurs propres sont les carrés des valeurs singulières et les vecteurs propres correspondent aux vecteurs singuliers de la matrice A .
- Calcul des Vecteurs Singuliers à Gauche (U) :
Normaliser les vecteurs propres obtenus pour obtenir les vecteurs singuliers à gauche (colonnes de la matrice U).
- Calcul des Vecteurs Singuliers à Droite (V) :
Normaliser les vecteurs propres obtenus pour obtenir les vecteurs singuliers à droite (lignes de la matrice V^T).
- Construction de la Matrice Σ :
Placer les racines carrées des valeurs propres (valeurs singulières) sur la diagonale de la matrice Σ .

La SVD est utilisée pour diverses applications, telles que la compression d'images, où une image peut être décomposée en ses composantes principales (valeurs singulières) et reconstruite à partir de celles-ci. Elle est également utilisée pour réduire la dimension des données, où les valeurs singulières moins significatives peuvent être négligées tout en conservant l'essentiel de l'information.

En résumé, la SVD est une décomposition matricielle puissante qui permet de décomposer une matrice en trois autres matrices, capturant ainsi les structures et les relations sous-jacentes des données.

3.8 GAMS

En bref

Les modèles additifs généralisés (GAM) sont une extension de la théorie des modèles linéaires généralisés (GLM). Par conséquent, comme pour les GLM, seules les distributions appartenant à la famille exponentielle linéaire peuvent être utilisées comme distribution de la variable de réponse d'un GAM. Les GAM relâchent l'hypothèse de linéarité, et les fonctions de lissage des covariables peuvent être incluses dans le prédicteur. Pour un GLM, la moyenne est donnée par une expression linéaire à travers une fonction de lien. Les GAM permettent une modélisation plus flexible de la moyenne en introduisant des fonctions de lissage dans le prédicteur linéaire.

1. Définition et principe

Les GAMS relâchent l'hypothèse de **linéarité** des GLMs en introduisant des **fonctions de lissage** pour les covariables. La structure générale d'un GAM est :

$$g(\mathbb{E}[Y]) = \beta_0 + \sum_{j=1}^p f_j(X_j)$$

où :

- g est une **fonction de lien** (comme pour les GLMs),
- f_j sont des **fonctions de lissage** (splines, loess, etc.) appliquées aux covariables X_j ,
- β_0 est l'intercept.

2. Comparaison avec les GLMs

Caractéristique	GLM	GAM
Forme du prédicteur	Linéaire : $\beta_0 + \sum \beta_j X_j$	Additif : $\beta_0 + \sum f_j(X_j)$
Flexibilité	Linéaire	Non linéaire
Interprétabilité	Élevée	Modérée à élevée
Famille de distributions	Famille exponentielle	Famille exponentielle

3. Avantages des GAMS

- **Flexibilité** : Capture des relations non linéaires complexes.
- **Modélisation des effets non linéaires** : Sans hypothèse restrictive.
- **Interprétabilité** : Chaque $f_j(X_j)$ peut être visualisée.
- **Extension naturelle des GLMs** : Même cadre théorique.

4. Fonctions de lissage

Les fonctions f_j sont estimées à l'aide de méthodes de lissage, comme :

- **Splines** (cubiques, B-splines, P-splines),
- **Loess** (régression locale),
- **Kernel smoothers**.

Exemple de fonction de lissage

Un exemple courant est l'utilisation de **splines cubiques** pour modéliser une relation non linéaire entre une covariable X et la réponse Y :

$$f(X) = \sum_{k=1}^K \beta_k b_k(X)$$

où $b_k(X)$ sont des **bases de splines**.

5. Applications

Les GAMS sont utilisés dans divers domaines :

- **Écologie** : Modélisation de la biodiversité,
- **Épidémiologie** : Étude des facteurs de risque,
- **Économie** : Analyse des séries temporelles,
- **Météorologie** : Prévission des phénomènes climatiques.

En Python :

```
from pygam import LinearGAM
model = LinearGAM().fit(X, y)
```

6. Limites

- **Complexité computationnelle** : Plus élevée que les GLMs,
- **Risque de surapprentissage** : Nécessite une validation croisée,
- **Choix des paramètres de lissage** : Peut être délicat.

7. Extensions

Les GAMS peuvent être étendus pour inclure :

- **Interactions** entre covariables,
- **Effets aléatoires** (GAMMs),
- **Variables catégorielles**.

Conclusion

Les GAMS offrent un **compromis idéal** entre flexibilité et interprétabilité, ce qui les rend très populaires pour l'analyse de données complexes. Ils sont particulièrement utiles lorsque les relations entre les variables ne sont pas linéaires mais que l'on souhaite conserver une certaine transparence dans le modèle.

3.9 Factorisation non-négative de matrice

La NMF (Non-Negative Matrix Factorization) est une technique d'analyse de données qui vise à décomposer une matrice en deux matrices non négatives de plus petite dimension. Cette technique est souvent utilisée pour extraire des caractéristiques ou des composantes sous-jacentes à partir de données brutes. La NMF est particulièrement utile pour la réduction de dimension, la détection de motifs cachés et la représentation des données dans des domaines tels que le traitement de texte, la vision par ordinateur et la bioinformatique.

Fonctionnement de la NMF :

La NMF prend une matrice non négative V et la décompose en deux matrices non négatives, W (matrice de caractéristiques) et H (matrice de poids), telles que $V \approx WHV \approx WH$. La NMF repose sur l'hypothèse que les données et les caractéristiques sont des valeurs non négatives, ce qui permet une interprétation plus intuitive des résultats.

Applications de la NMF :

- Détection de thèmes dans des ensembles de documents
- Analyse de séquences temporelles
- Compression d'images et traitement du signal
- Identification de motifs génétiques dans la bioinformatique
- Décomposition de mélange de signaux audio

Maintenant, voici une liste des méthodes principales de NMF, ainsi que leurs avantages et inconvénients :

- **DNMF Classique :**

- Avantages :
 - * Intuitive et facile à comprendre.
 - * Peut être adaptée pour des domaines spécifiques.
- Inconvénients :
 - * Sensible aux valeurs aberrantes.
 - * Peut converger vers des minima locaux.

- **NMF Multiplicative :**

- Avantages :
 - * Converge généralement rapidement.
 - * Donne des résultats stables.
- Inconvénients :
 - * Possibilité de divergence.
 - * Sensible aux valeurs initiales.

- **NMF à Divergence de Kullback-Leibler (KL-NMF) :**

- Avantages :
 - * Basée sur la divergence de Kullback-Leibler, ce qui peut être approprié pour certaines distributions.
 - * Peut être plus robuste pour les données sparses.
- Inconvénients :
 - * Plus complexe à optimiser que la NMF classique.
 - * Sensible aux valeurs initiales.

- **NMF basée sur la Divergence de Poulsen :**

- Avantages :
 - * Peut traiter des données de Poisson, comme les données comptées.
 - * Adaptée aux données non gaussiennes.
- Inconvénients :
 - * Moins largement utilisée et moins d'outils disponibles.

- **NMF avec Régularisation :**

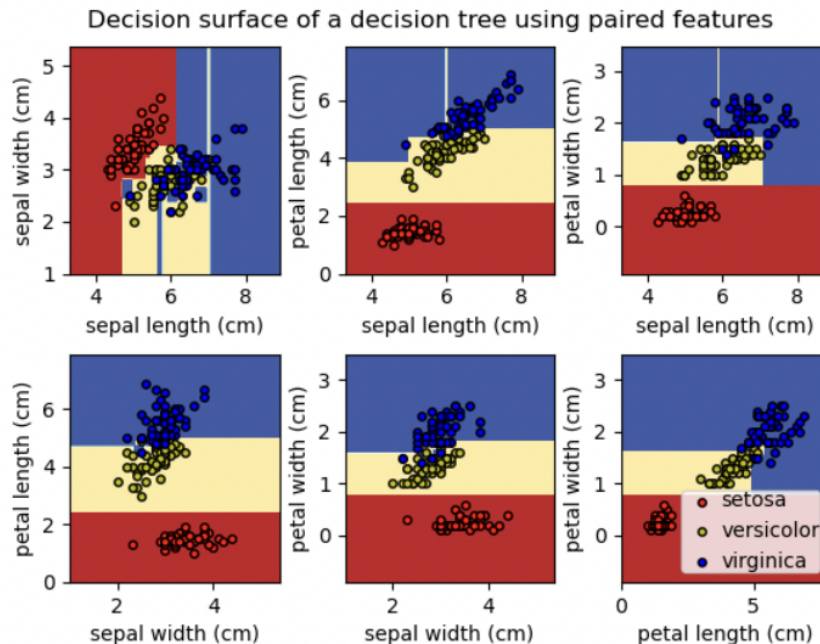
- Avantages :
 - * Peut améliorer la stabilité et la généralisation du modèle.
 - * Contrôle la complexité du modèle et peut éviter le surajustement.
- Inconvénients :
 - * Besoin de choisir des paramètres de régularisation.

3.10 Random Forests (RF)

- **Définition :** Algorithme **supervisé** qui permet de faire de la *classification* ou de la *régression*. Il fait partie de la famille des **apprentissage ensemblistes** et ne nécessite que très peu de paramètres. Il est fondé sur le principe de *vote majoritaire* en agréant des classifieurs faibles.
- **Naissance :** Les *Random Forests* ont été inventées en 2001 par *Leo Breiman & Adele Cutler* (Université de Princeton). Le nom de l'algorithme est une marque déposée depuis 2006.
- **Applications :** Industries financières et bancaires, commerce électronique, santé et médecine, etc.

Dans une forêt il y a des arbres ...

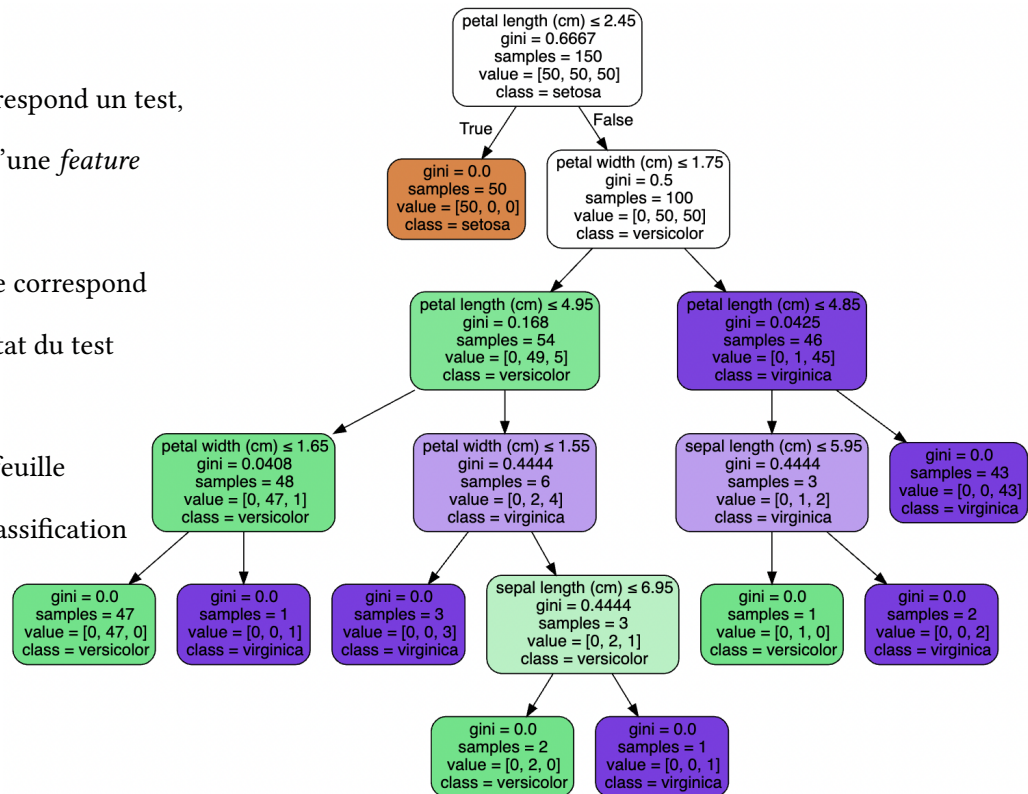
Decision tree



Source archive.ics.uci.edu

Points cibles:

- A chaque noeud correspond un test, basé sur la valeur d'une *feature*
- A chaque branche correspond un possible résultat du test
- A chaque feuille correspond une classification

**Data**

Applications: *regression* ou *classification*. Nous allons présenter la **classification**.

Data set: $\{\mathbf{x}_i, y_i\}_{i=1, \dots, n}$, $\mathbf{x}_i \in \mathbb{R}^d$, avec $y = 0, \dots, k-1$,

Comment “faire grandir” un arbre ?

A chaque noeud, il faut choisir la *feature* et le seuil de décision tel que des “coupures propres” sont faites sur le jeu de données.

Que signifie l’expression “coupures propres”? Cela veut dire augmenter le niveau de *pureté* des données dans chaque noeud après classification. Comment pouvons-nous la mesurer ?

En utilisant l’**entropie**

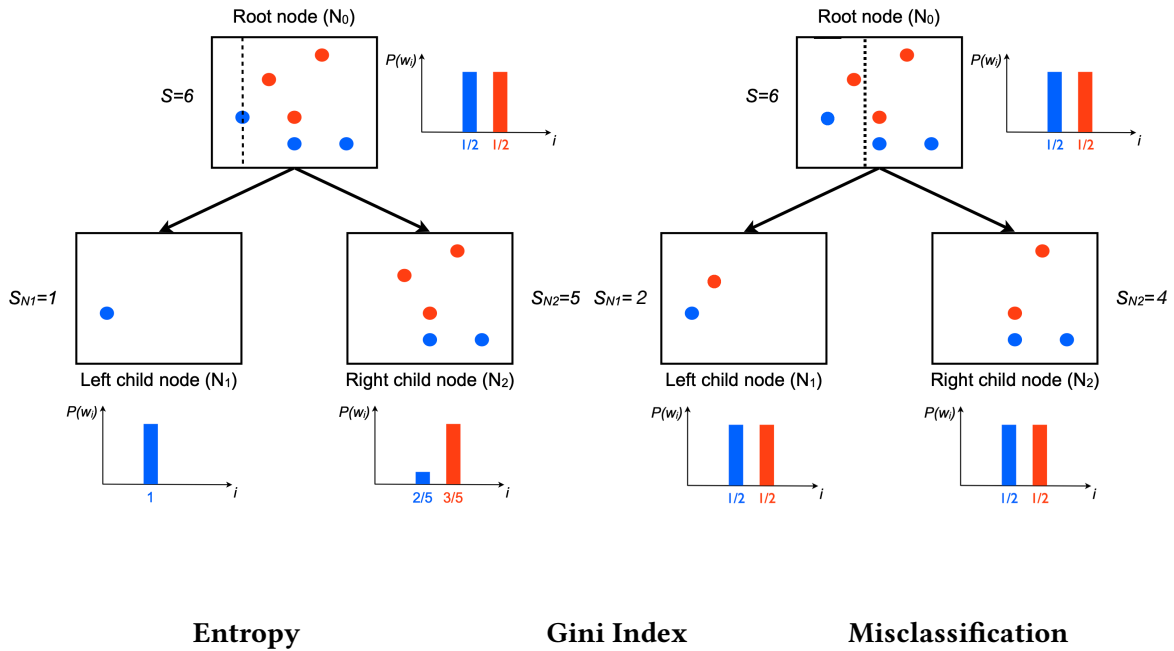
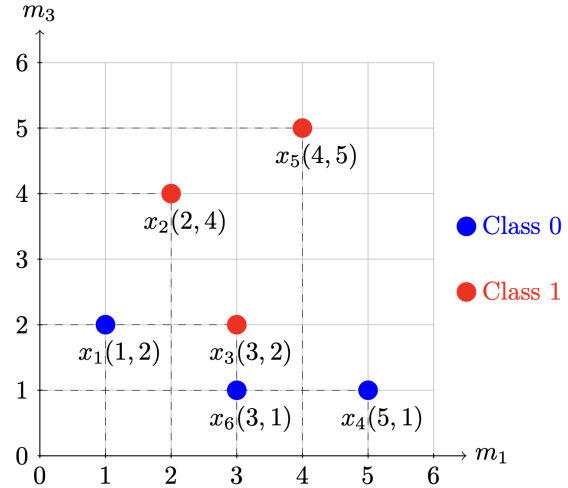
Gain d’Information

$$\Delta \mathcal{I}_{N_i \rightarrow N} = \mathcal{S}(N_i) - \sum_{N_j \in N} w(N_i, N_j) \mathcal{S}(N_j), \quad w(N_i, N_j) = \frac{\text{elements in } N_j}{\text{elements in } N_i}$$

La coupure sur la gauche est retenue. Nous pouvons itérer jusqu’à ce que²:

²<https://scikit-learn.org/stable/modules/tree.html>

Data Vector	Features		True Class
	m_1	m_3	
x_1	1	2	0
x_3	3	2	1
x_6	3	1	0
x_2	2	4	1
x_4	5	1	0
x_5	4	5	1



$$\mathcal{S}(N_j) = -\sum_{i=1}^k f_i^j \log_2(f_i^j) \quad \mathcal{G}(N_j) = 1 - \sum_{i=1}^k (f_i^j)^2 \quad \mathcal{M}(N_j) = 1 - \max_i \{f_i^j\}$$

- La profondeur du noeud est plus grande qu'une certaine valeur
- Le nombre des données dans chaque feuille est plus petit qu'une certaine valeur
- La pureté dans un nœud dépasse un certain seuil

Les conditions ci-dessus sont des conditions d'arrêt de l'apprentissage et on peut les utiliser pour calibrer le compromis **biais / variance**. Plus les arbres sont profonds, plus grand est le risque d'*overfitting*.

Dans le cas d'une forêt aléatoire (random forest), le split ratio, ou taux de division, représente la proportion d'exemples d'apprentissage utilisée pour créer chaque arbre décisionnel dans la forêt. Ce paramètre

	Split on the left	Split on the right
Node N_0	$\frac{3}{6} \log_2 \left(\frac{3}{6} \right) + \frac{3}{6} \log_2 \left(\frac{3}{6} \right) = 1$	$\frac{3}{6} \log_2 \left(\frac{3}{6} \right) + \frac{3}{6} \log_2 \left(\frac{3}{6} \right) = 1$
Node N_1	$\frac{1}{1} \log_2 \left(\frac{1}{1} \right) = 0$	$\frac{1}{2} \log_2 \left(\frac{1}{2} \right) + \frac{1}{2} \log_2 \left(\frac{1}{2} \right) = 1$
Node N_2	$\frac{3}{5} \log_2 \left(\frac{3}{5} \right) + \frac{2}{5} \log_2 \left(\frac{2}{5} \right) \sim 0.03$	$\frac{1}{2} \log_2 \left(\frac{1}{2} \right) + \frac{1}{2} \log_2 \left(\frac{1}{2} \right) = 1$

Split on the left

Split on the right

$$1 - \frac{1}{6} * 0 - \frac{5}{6} * 0.03 \sim 0.975$$

$$1 - \frac{2}{6} * 1 - \frac{4}{6} * 1 = 0$$

Split on the left

Split on the right

$$1 - \frac{1}{6} * 0 - \frac{5}{6} * 0.03 \sim \mathbf{0.975}$$

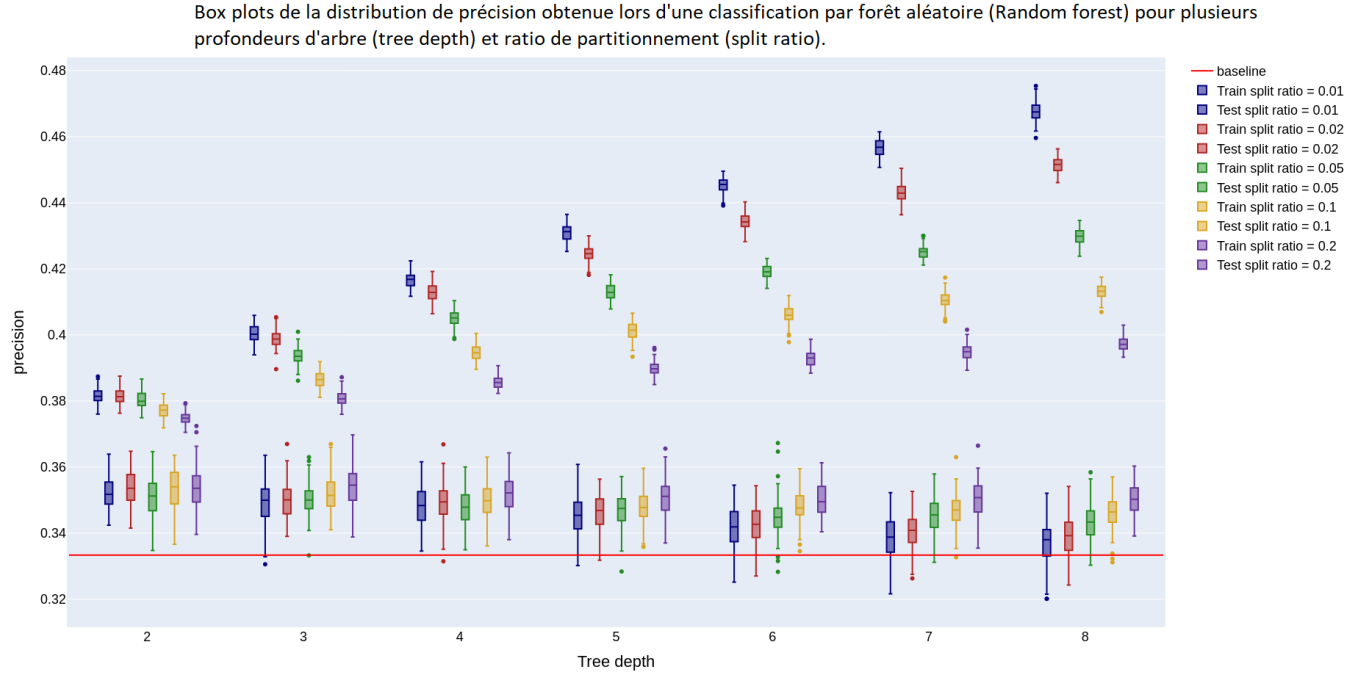
$$1 - \frac{2}{6} * 1 - \frac{4}{6} * 1 = 0$$

détermine la diversité des arbres décisionnels et influence la performance globale de la forêt aléatoire.

Un split ratio élevé conduit à des arbres décisionnels plus profonds et plus complexes, tandis qu'un split ratio faible entraîne des arbres décisionnels plus petits et plus simples. En général, un split ratio compris entre 0,5 et 0,75 est considéré comme efficace pour les forêts aléatoires.

Le choix du split ratio optimal dépend de la nature des données d'apprentissage et de la tâche de prédiction. Si les données d'apprentissage sont bruyantes ou contiennent de nombreuses caractéristiques redondantes, un split ratio plus faible peut être bénéfique pour réduire la sur-adaptation (overfitting). En revanche, si les données d'apprentissage sont de bonne qualité et contiennent des caractéristiques informatives, un split ratio plus élevé peut permettre à la forêt aléatoire de capturer des relations plus complexes entre les caractéristiques et la variable cible.

... Plus les arbres sont profonds, plus grand est le risque d'*overfitting*.



A partir d'un arbre de décision, on peut construire une **Random Forests**. Les *Random Forests* sont un exemple de **Bootstrap Aggregation**.

- Méthode **Bootstrap** : À partir du dataset original $S = \{x_i, y_i\}_{i=1, \dots, n}$, nous désirons construire T arbres de décision. Pour chaque arbre nous avons :

Training set S_{tr}^i : n échantillons sont choisis de façon aléatoire à partir de S . Dès qu'un échantillon est retenu, il est encore disponible pour des échantillonnages futurs et, par conséquent, il peut apparaître plusieurs fois dans l'ensemble S_{tr}^i .

In-Bag set [IB] S_{IB}^i : il est composé par les échantillons uniques de S_{tr}^i

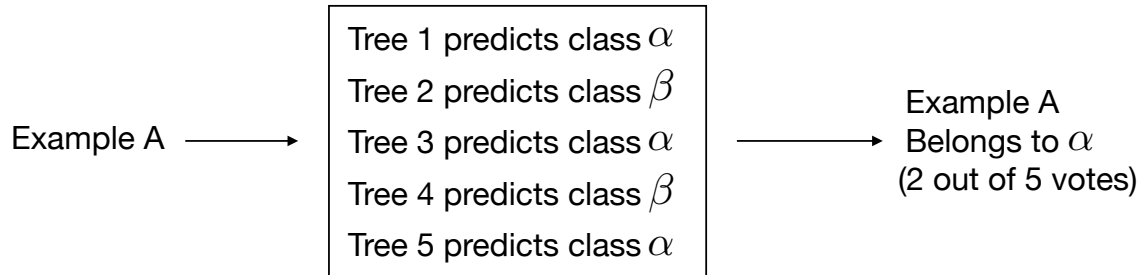
Out-of-Bag set [OOB] S_{OOB}^i : il est composé des échantillons uniques de S qui ne sont pas présents dans S_{IB}^i , i.e $S_{OOB}^i = S - S_{IB}^i$.

- *OOB-error-estimation*. Il s'agit d'une estimation de l'erreur de la prédiction de la forêt.
- Pour chaque échantillon dans l'union des tous les ensembles *OOB*, nous faisons une prédiction en utilisant une sous-partie de la forêt avec seulement les arbres qui n'ont pas l'échantillon ci-dessus dans ses *IBs*.
- L'erreur d'estimation est donnée par la fraction des échantillons dont le *label* est mal prédit.

A partir d'un arbre de décision, on peut construire une **Random Forests**. Les *Random Forests* sont un exemple de **Bootstrap Aggregation**.

- **Bagging** : les prédictions des différents arbres sont agrégés dans une sortie unique.

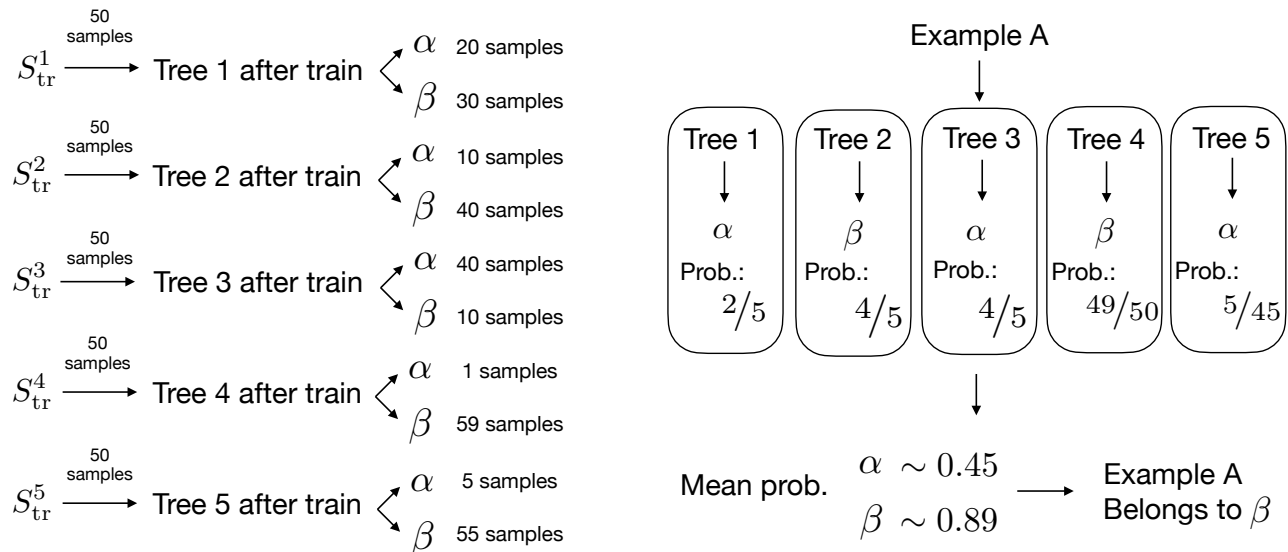
Hard voting : la prédiction finale est la prédiction la plus votée.



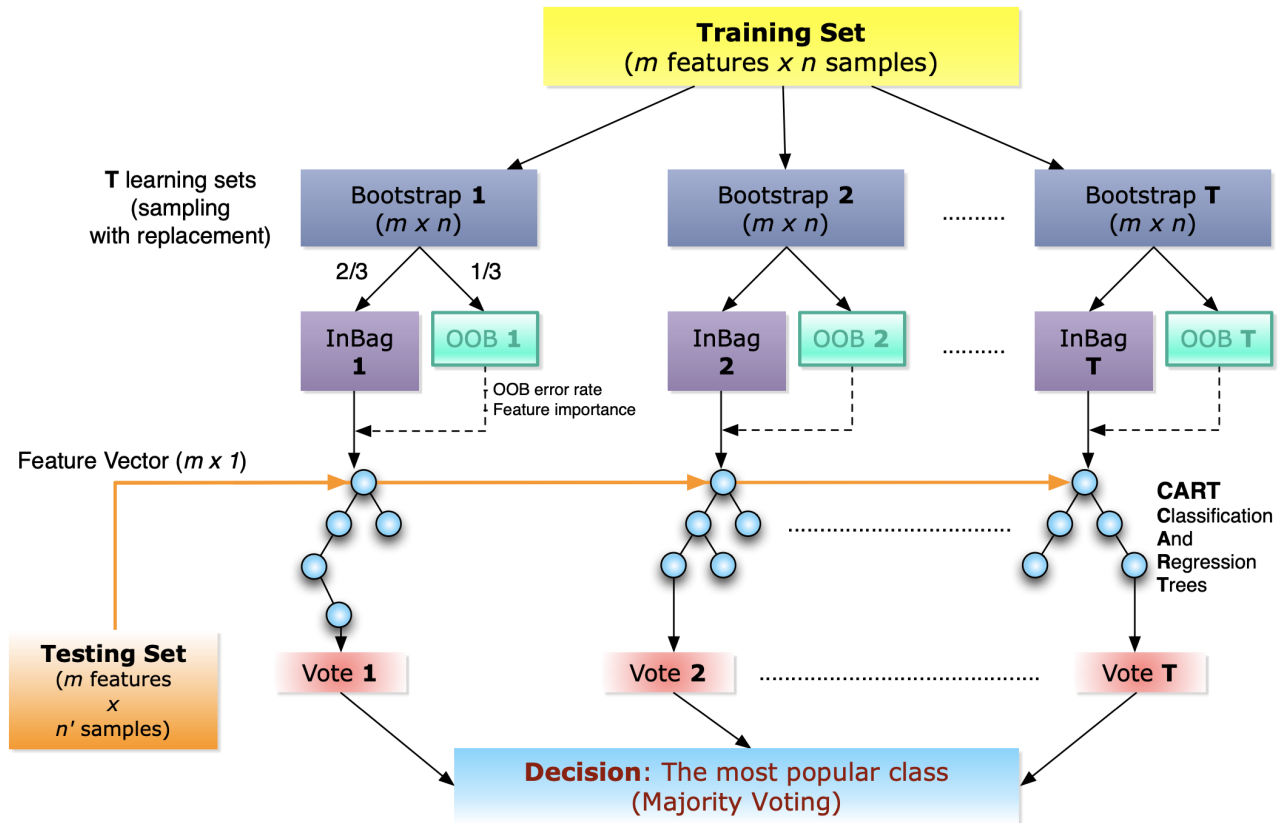
A partir d'un arbre de décision, on peut construire une **Random Forests**. Les *Random Forests* sont un exemple de **Bootstrap Aggregation**.

- **Bagging** : les prédictions des différents arbres sont agrégés dans une sortie unique.

Soft voting : la prédiction finale est la prédiction avec la probabilité moyenne la plus élevée.



Random Forests : Majority Voting - Source article



3.11 Machines à vecteurs de support (SVM)

- **Définition** : Les **machines à vecteurs de support** ou **séparateurs à vaste marge** (pour garder l'acronyme anglais **SVM**) font partie de la famille des algorithmes d'**apprentissage supervisé**. Les SVM permettent de résoudre des problèmes de **classification** ou de **régression**³ et sont une généralisation des **classifieurs linéaires**.
- **Naissance** : Les SVM ont été inventés en 1992 (Boser, Guyon et Vapnik)⁴ et sont issus du développement d'une théorie statistique de l'apprentissage (théorie de Vapnik-Chervonenkis ou **théorie VC**).
- **Applications** : bio-informatique, data mining, computer vision, finance, etc.
- Les SVM reposent sur 2 **idées principales** : les **hyperplans à marge maximale** (Vapnik et Lerner, 1963) et la **fonction noyau** (cf. *Théorème de Mercer*, 1909).
- Selon les données, la performance des SVM peut être équivalente voire supérieure à celle d'un réseau de neurones ou d'un modèle de mélanges gaussiens.
- Un **brevet américain** sur les SVM a été déposé⁵ en 1997 par les inventeurs originaux.

³Harris Drucker, Chris J. C. Burges, Linda Kaufman, Alex Smola and Vladimir Vapnik, "Support Vector Regression Machines" [Lien]. Advances in Neural Information Processing Systems 9, NIPS 1996, 155-161, MIT Press., 1997.

⁴Bernhard E. Boser, Isabelle M. Guyon, Vladimir N. Vapnik, "A Training Algorithm for Optimal Margin Classifiers" [Lien]. In Fifth Annual Workshop on Computational Learning Theory, pages 144-152, Pittsburgh, ACM, 1992.

⁵Pattern recognition system using support vectors. B. Boser, I. Guyon, and V. Vapnik, US Patent 5,649,068 1997.

Approche intuitive

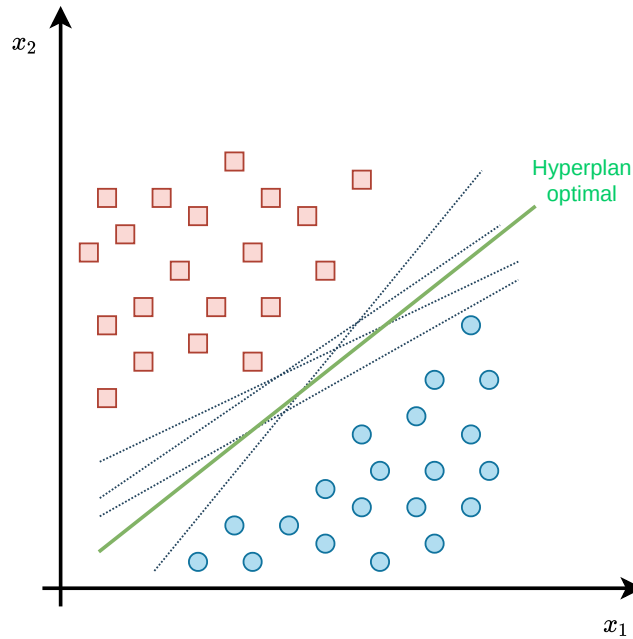


Figure 3.14: Exercice de classification (deux classes). Ici, le problème est **linéairement séparable**.

- Il existe une infinité d'hyperplans séparateurs, dont les performances en apprentissage sont identiques mais dont les performances durant la phase d'inférence peuvent être très différentes. **Comment trouver l'hyperplan optimal ?**
- L'objectif d'un SVM est de trouver l'hyperplan qui maximise la **marge**, *i.e.* la distance minimale entre les points du jeu d'entraînement et l'hyperplan.
- Les points du jeu d'entraînement les plus proches de la frontière sont appelés **vecteurs support**.

Formalisme

Définition : Soit E un espace vectoriel de dimension finie n non nulle. Un **hyperplan** de E est un sous-espace vectoriel H de dimension $n - 1$.

Formalisme

Définition : Soit E un espace vectoriel de dimension finie n non nulle. L'équation cartésienne d'un hyperplan affine $\mathcal{H}$ de E est de la forme suivante :

$$h(x) = \sum_{k=1}^n \omega_k \cdot x_k + b = \omega^T \cdot x + b \quad (3.43)$$

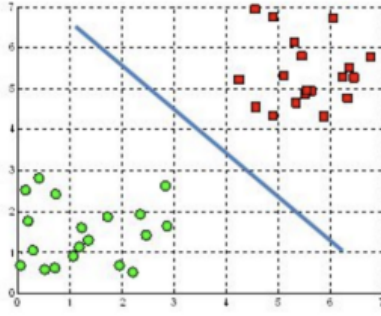
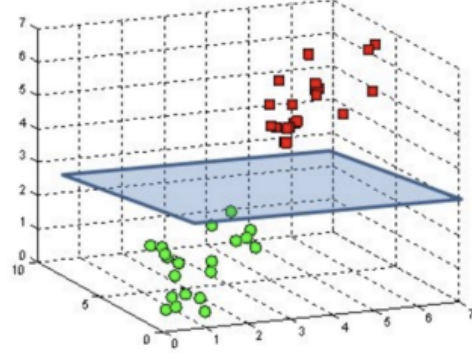
A hyperplane in $\mathbb{R}^2$ is a lineA hyperplane in $\mathbb{R}^3$ is a plane

Figure 3.15: Un hyperplan de $\mathbb{R}^2$ est une droite.
Un hyperplan de $\mathbb{R}^3$ est un plan.

$$\text{où } x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n \text{ est un \textbf{vecteur d'entrée},}$$

$$\omega = \begin{pmatrix} \omega_1 \\ \vdots \\ \omega_n \end{pmatrix} \in \mathbb{R}^n \text{ le \textbf{vecteur de poids}, et } b \in \mathbb{R} \text{ le \textbf{biais}.}$$

Formalisme

On décide alors que x est de classe 1 si $h(x) \geq 0$ et de classe -1 sinon.

Le but d'un SVM est d'apprendre la fonction h via un ensemble d'apprentissage :

$$\{(x_1, l_1), \dots, (x_k, l_k), \dots, (x_p, l_p)\} \subset \mathbb{R}^n \times \{-1, 1\} \quad (3.44)$$

où l_k sont les labels, p la taille de l'ensemble d'apprentissage et n la dimension des vecteurs d'entrée.

Si le problème est linéairement séparable, on doit toujours avoir :

$$\forall k, \quad l_k h(x_k) \geq 0 \quad \text{soit} \quad l_k (\omega^T \cdot x_k + b) \geq 0 \quad (3.45)$$

Lors de la phase d'entraînement, un SVM doit donc trouver un vecteur de poids ω et un biais b tels que la condition de séparabilité (eq. 3.45) soit satisfaite.

Calcul de la marge

Proposition : Soit E un espace euclidien⁶ et $\mathcal{H}$ un hyperplan affine de E d'équation caractéristique $\omega^T \cdot x + b = 0$. La **distance** de tout point $x_k \in \mathbb{R}^n$ à l'hyperplan $\mathcal{H}$ est donnée par sa **projection orthogonale sur le vecteur de poids ω** :

$$d_k = \frac{l_k(\omega^T \cdot x_k + b)}{\|\omega\|} \quad (3.46)$$

La **marge** étant la **distance minimale** entre l'hyperplan $\mathcal{H}$ et les points du jeu d'entraînement, elle est donc définie par :

$$m = \min_k d_k = \min_k \left(\frac{l_k(\omega^T \cdot x_k + b)}{\|\omega\|} \right) \quad (3.47)$$

Maximisation de la marge

Objectif : On souhaite trouver l'**unique hyperplan** de paramètres (ω, b) qui **maximise la marge** (eq. 3.47).

Cet hyperplan est donné par la formule suivante :

$$\operatorname{argmax}_{\omega, b} \left\{ \min_k \left(\frac{l_k(\omega^T \cdot x_k + b)}{\|\omega\|} \right) \right\} \quad (3.48)$$

Remarque : Cela doit permettre d'être plus robuste face aux faibles variations. Cette intuition est justifiée par Vapnik qui a prouvé (1995) que cette maximisation produit un **hyperplan optimal**, qui minimise les erreurs possibles (on parle aussi de **capacité de généralisation maximale**).

Normalisation

- L'hyperplan optimal est **unique** mais il existe *plusieurs couples* (ω, b) qui décrivent ce *même* hyperplan.⁷
- Afin de **faciliter l'optimisation**, on choisit de **normaliser** ω et b de telle sorte que les vecteurs support x_s vérifient $l_s(\omega^T \cdot x_s + b) = 1$. Autrement dit, $\forall k, l_k(\omega^T \cdot x_k + b) \geq 1$, l'égalité étant atteinte si x_k est un vecteur support.
- Cette normalisation est appelée **hyperplan canonique** et la marge devient alors égale à $\frac{1}{\|\omega\|}$.

Résolution

- Après cette *mise à l'échelle*, on aboutit à ce que l'on appelle la **formulation primale des SVM**, à savoir :

$$\text{Minimiser } \frac{\|\omega\|^2}{2} \text{ sous les contraintes } \forall k, l_k(\omega^T \cdot x_k + b) \geq 1$$

- Ce type de problème est appelé **problème d'optimisation quadratique** et peut-être résolu grâce à la méthode des **multiplicateurs de Lagrange**⁸.

⁶Un espace euclidien est un espace vectoriel de dimension finie, sur le corps des réels $\mathbb{R}$, muni du produit scalaire.

⁷On peut vérifier que si un hyperplan $\mathcal{H}$ est décrit par les paramètres (ω, b) , alors il est également décrit par les paramètres $(\alpha\omega, \alpha b)$, $\forall \alpha \in \mathbb{R}^{*+}$.

⁸En analyse, la méthode des multiplicateurs de Lagrange permet de trouver les points stationnaires (extremums) d'une fonction dérivable univariée ou multivariée, sous contraintes.

Multiplicateurs de Lagrange

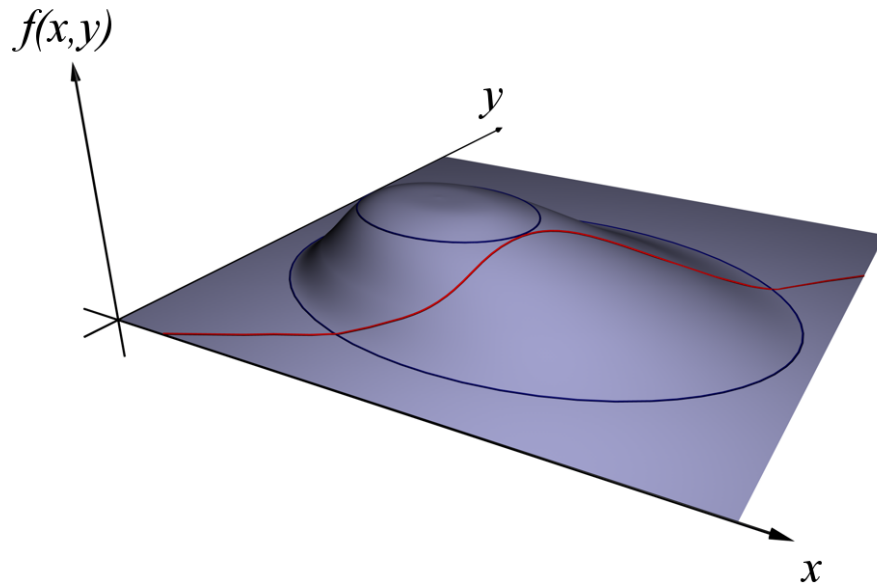


Figure 3.16: Illustration 3D de la méthode des multiplicateurs de Lagrange.

Sur cette figure, il s'agit de trouver le point le plus élevé possible, tout en satisfaisant la contrainte d'être sur la ligne rouge.

Kernel Trick

- Dans un **cas réel** concret, le problème est très souvent **non linéairement séparable**. Il faut alors utiliser une astuce...
- Il s'agit de l'**astuce du noyau** (*kernel trick*) qui consiste à reconsidérer le problème dans un **espace de dimension supérieure** où la probabilité d'existence d'un *hyperplan séparateur* est **plus grande** (cf. *Théorème de Cover*).
- Pour cela, on applique aux vecteurs X de l'espace d'entrée (aussi appelé **espace de description**) une transformation non-linéaire ϕ .
- Dans l'espace d'arrivée $\phi(X)$ (aussi appelé **espace de redescription**), on cherche alors l'hyperplan d'équation $h(x) = \omega^T \cdot \phi(x) + b$ qui vérifie $\forall k, l_k h(x_k) \geq 0$.

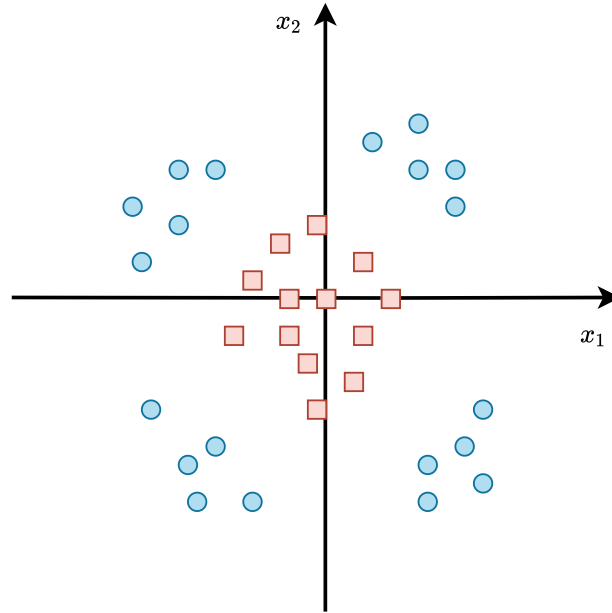


Figure 3.17: Exercice de classification (deux classes). Ici, le problème est **non linéairement séparable**.

Vidéo Youtube : “Performing nonlinear classification via linear separation in higher dimensional space”.

[Lien](#)

Références

- Site Internet *Zeste de Savoir*, *Un peu de Machine Learning avec les SVM*, 2020.
- Page FR Wikipédia sur les SVM, 2021.

SVM : des applications en énergétique

- Classification de situation par rapport à des cas de référence :
 - Sensation thermique des individus
 - Consommation énergétique
- Sélection de paramètres / Feature selection

Exemple d’application pour l’estimation de la température corporelle par caméra thermique

- Le SVM est un algorithme linéaire de classification binaire qui cherche à maximiser la séparation entre les classes, formalisée par le concept de marge, tout en contrôlant le nombre d’erreurs de classification sur le jeu d’entraînement.
- L’astuce du noyau permet d’appliquer efficacement cet algorithme, ainsi que d’autres algorithmes linéaires, dans un espace de redescription, sans calculer explicitement l’image des observations dans cet espace.

3.12 Algorithmes de règles

- **Définition** : Famille d’algorithmes **supervisés** et **non supervisés** dont le but est l’extraction de règles à partir d’un jeu de données. Des **règles d’associations** dans le cadre non supervisé et des **règles prédictives** dans le cas supervisé.

- **Naissance** : La recherche débute dans les années 60. Ces algorithmes ont été les premiers à être utilisés dans un contexte industriel (*GASOIL* et *BTM* en 1986 et 1990). L'algorithme le plus emblématique de recherche de règles d'associations est *APriori*.
- **Applications** : Le domaine d'application est très varié. Ces algorithmes ont l'avantage d'avoir été pensés de façon générale. La contrepartie est qu'ils sont souvent moins performants dans un contexte supervisé.

Algorithmes de règles - c'est quoi une règle ? Une règle est une assertion du type *If... Then ...* :

- La partie *If* de la règle, appelée *condition de la règle* ou simplement règle, est composée de la conjonction de k tests vérifiant si une feature (une coordonnée de X) vérifie une certaine propriété. Le nombre de tests k est appelé *longueur* d'une règle.
- La partie *Then* de la règle, appelée *conclusion* quand la règle est *active*, c'est-à-dire lorsque sa condition est satisfaite.

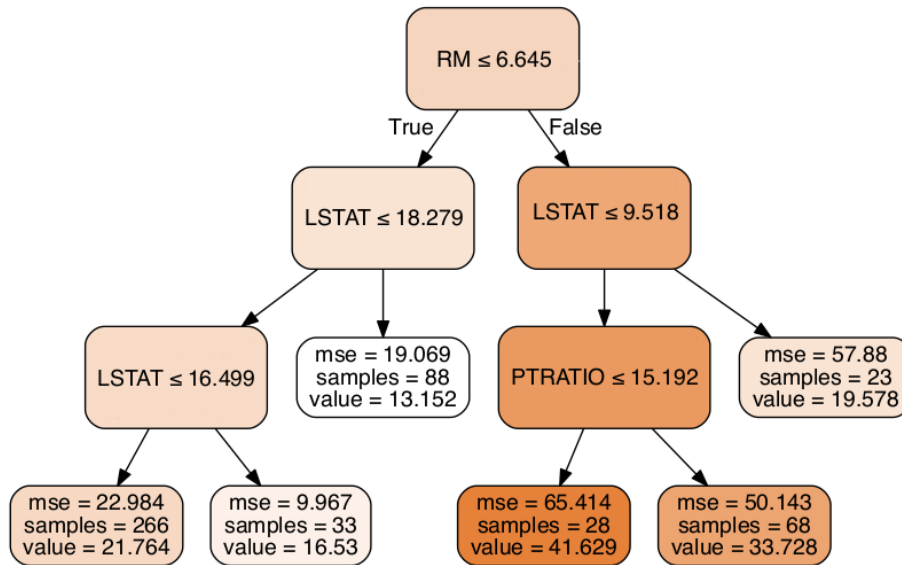
Algorithmes de règles - non-supervisé Ils sont souvent utilisés dans les systèmes de recommandation ou la prédiction séquentielle d'événements.

- **Exemple** : Si A aime "Squid Game" Alors proposer "Battle Royale".
- **Exemple** : Si B aime "How I met your mother" Alors proposer "Friends".
- **Exemple** : Si C achète "lait, œufs, farine, sucre" Alors proposer "pâte à tartiner".
- **Exemple** : Si D ["rentre dans sa voiture", "il est 18:00"] Alors proposer "itinéraire pour rentrer".

Le but de ces algorithmes est d'identifier ce type de règle et de les sélectionner selon des critères statistiques.

Algorithmes de règles - supervisé Il y a **trois types d'algorithmes** :

- **Arbres de décision** : Classification and Regression Tree *CART* (1984), Iterative Dichotomizer 3 *ID3* (1993), *C4.5*, Repeated Incremental Pruning to Produce Error Reduction *RIPPER* (1995).
- **Ensemble de règles** : *GASOIL* (1986), *BMT* (1990), *RuleFit* (2008), *NodeHarvest* (2010), *Sirus* (2021), *CoveringAlgorithm* (2021).
- **Liste de règles** : Bayesian Rule List *BRL* (2015), Scalable Bayesian Rule Lists *SBRL* (2017).

Algorithmes de règles - exemple de règlesFigure 3.18: Exemple d'arbre de régression. [Source article](#)

if	bruises=no, odor=not-in-(none,foul)	then probability that the mushroom is edible = 0.00112
else if	odor=foul, gill-attachment=free,	then probability that the mushroom is edible = 0.0007
else if	gill-size=broad, ring-number=one,	then probability that the mushroom is edible = 0.999
else if	stalk-root=unknown,	then probability that the mushroom is edible = 0.996
else if	stalk-surface-above-ring=smooth,	then probability that the mushroom is edible = 0.0385
else if	stalk-root=unknown, ring-number=one,	then probability that the mushroom is edible = 0.995
else if	bruises=foul, veil-color=white,	then probability that the mushroom is edible = 0.986
else if	stalk-shape=tapering,	then probability that the mushroom is edible = 0.958
else if	ring-number=one,	then probability that the mushroom is edible = 0.001
else	habitat=paths,	
	(default rule)	

Figure 3.19: Exemple de liste de règles.
(extrait de l'article *Scalable Bayesian Rule Lists*)

Rule	Conditions	Samples	Value
<i>R1</i>	$LSTAT \in [9.6, 37.9]$	293	17.7
<i>R2</i>	$LSTAT \in [1.7, 7.8]$	157	32.3
<i>R3</i>	$RM \in [3.8, 6.1]$ $LSTAT \in [7.7, 37.9]$ $DIS \in [1.2, 12.1]$	187	17.6
<i>R4</i>	$RM \in [6.4, 8.7]$ $LSTAT \in [1.7, 14.4]$ $DIS \in [1.2, 12.1]$	167	31.8
<i>R5</i>	$RM \in [6.1, 8.7]$ $LSTAT \in [1.73, 9.9]$ $CRIM \in [0.05, 88.98]$	136	32.1
<i>R6</i>	$LSTAT \in [5.1, 37.9]$ $PTRATIO \in [13.9, 22]$ $INDUS \in [15, 27.7]$	182	16.3

Table 3.2: Exemple d'ensemble de règles.

Chapter 4

Reseaux de neurones

4.1 Généralités

Table 1: Different kinds of machine learning techniques often resemble differentiating through some iterative system. f is the iteratively applied function, s is an input, θ stands for parameters and l is the optimization objective

Domain	f	s	θ	l
Neural Network Training	A layer transformation of a neural network	The inputs to that layer	The weight matrix and bias vector for that layer	cross entropy mean squared error, l2 regularization, etc.
Reinforcement Learning	The step function of an environment	The state data of the environment and agent	The parameters of a policy	The reward function for the environment
Learned Optimization	The application of an optimizer	The parameters in a network being optimized	The tunable parameters for the optimizer, e.g., learning rate	The performance of the network being optimized on a task after some number of steps of optimization

Figure 4.1: Les différents d'apprentissages du DeepLearning. [Sources article](#)

4.2 MLP

Un réseau de neurones multicouche (MLP, acronyme de "Multilayer Perceptron") est un type de modèle d'apprentissage automatique appartenant à la catégorie des réseaux de neurones artificiels. Il s'agit d'une architecture fondamentale utilisée pour résoudre une variété de tâches d'apprentissage automatique, telles que la classification, la régression et d'autres types de problèmes d'analyse de données.

Les éléments constitutifs d'un réseau MLP sont :

- Neurones et Couches :

Un MLP est composé de plusieurs couches de neurones artificiels. Chaque neurone dans une couche est connecté à tous les neurones de la couche précédente et de la couche suivante (dans le cas de couches non adjacentes, il n'y a pas de connexion directe). Les couches dans un MLP sont généralement organisées en trois types principaux :

- **Couche d'Entrée :** Cette couche reçoit les données brutes en tant que caractéristiques d'entrée. Chaque neurone dans cette couche correspond à une caractéristique ou à une variable d'entrée.
- **Couches Cachées :** Ces couches intermédiaires entre la couche d'entrée et la couche de sortie sont responsables de l'extraction et de la transformation des informations contenues dans les données d'entrée.

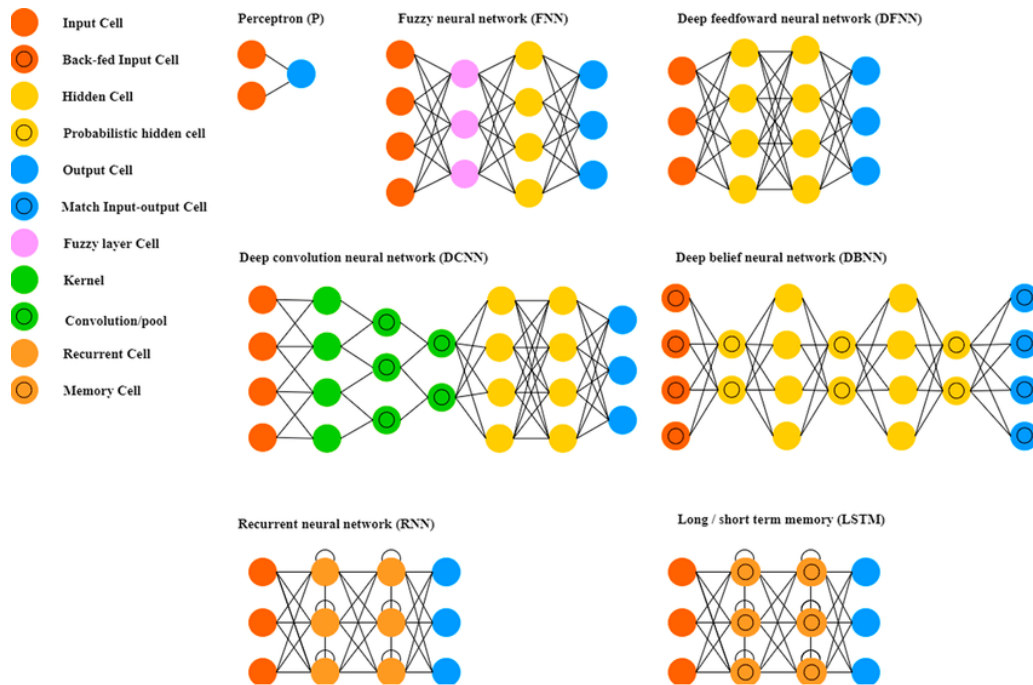


Figure 4.2: Quelques structures de réseaux. Source article

Chaque neurone dans une couche cachée est connecté à tous les neurones de la couche précédente et de la couche suivante.

- **Couche de Sortie** : Cette couche génère les prédictions ou les sorties du modèle en fonction des transformations effectuées dans les couches cachées. Le nombre de neurones dans cette couche dépend du type de tâche que le MLP doit accomplir (par exemple, classification binaire, classification multiclasse, régression, etc.).
- Fonctions d'Activation :
Chaque neurone dans un MLP applique une fonction d'activation à la somme pondérée des valeurs des neurones de la couche précédente, également appelée "activation pondérée". Les fonctions d'activation introduisent une non-linéarité dans le modèle, permettant ainsi au MLP de capturer des motifs et des relations complexes dans les données. Quelques exemples de fonctions d'activation couramment utilisées sont la fonction sigmoïde, la fonction ReLU (Rectified Linear Unit) et la fonction tangente hyperbolique.
- Propagation Avant (Forward Propagation) :
Lorsqu'une observation est introduite dans le réseau, les valeurs des neurones de la couche d'entrée sont propagées à travers les couches cachées jusqu'à atteindre la couche de sortie. Chaque neurone calcule son activation pondérée en utilisant les poids associés aux connexions et les valeurs des neurones précédents, puis applique la fonction d'activation pour produire sa sortie.
- Entraînement :
L'entraînement d'un MLP implique de mettre à jour les poids des connexions de manière à minimiser la différence entre les prédictions du modèle et les valeurs réelles (c'est-à-dire l'erreur). Cela se fait généralement à l'aide d'un algorithme d'optimisation tel que la descente de gradient. La rétropropagation de l'erreur (backpropagation) est une technique clé dans laquelle l'erreur est propagée de la couche de sortie vers la couche d'entrée, en ajustant les poids à chaque étape pour minimiser l'erreur.
- Fonctions Coût :
Les fonctions coût, également appelées fonctions d'objectif ou de perte, évaluent la différence entre les prédictions du modèle et les valeurs réelles. L'objectif de l'entraînement est de minimiser cette fonction coût.
- Architecture et Hyperparamètres :
L'architecture d'un MLP est définie par le nombre de couches, le nombre de neurones dans chaque couche,

les fonctions d'activation et d'autres hyperparamètres tels que le taux d'apprentissage, le nombre d'itérations d'entraînement, etc. Le choix de l'architecture et des hyperparamètres peut avoir un impact significatif sur les performances du modèle.

En résumé, un réseau MLP est une architecture de réseau de neurones multicouche utilisée pour l'apprentissage automatique, caractérisée par ses couches d'entrée, ses couches cachées et sa couche de sortie, ainsi que par l'utilisation de fonctions d'activation et d'algorithmes d'optimisation pour l'entraînement. Il est capable de modéliser des relations complexes dans les données en introduisant des non-linéarités à travers ses fonctions d'activation.

4.3 CNN

Un réseau de neurones convolutif (CNN, acronyme de "Convolutional Neural Network") est un type spécialisé de réseau de neurones artificiels conçu spécifiquement pour le traitement efficace et la reconnaissance de motifs dans les données structurées, en particulier les données bidimensionnelles telles que les images et les vidéos. Les CNN sont largement utilisés pour des tâches de vision par ordinateur telles que la classification d'images, la détection d'objets, la segmentation sémantique, etc.

Les éléments constitutifs d'un réseau CNN sont :

- Convolution et Filtrage :

L'une des caractéristiques clés des CNN est l'utilisation de convolutions. Une convolution consiste à appliquer un filtre (ou noyau) à une partie de l'image, puis à déplacer ce filtre sur l'ensemble de l'image pour extraire des caractéristiques locales. Ces filtres sont conçus pour capturer des motifs spécifiques tels que des bords, des textures et d'autres caractéristiques importantes de l'image. Ces filtres permettent l'exploitation des propriétés intrinsèques des données, comme les invariances, pour optimiser les performances et réduire les ressources nécessaires.

- Couches Convolutives :

Les couches convolutives constituent la partie fondamentale d'un CNN. Chaque couche convolutive est composée de plusieurs filtres, et chaque filtre effectue une convolution avec l'image en entrée pour produire des cartes d'activation (également appelées "feature maps"). Ces cartes d'activation mettent en évidence les caractéristiques importantes de l'image telles que les bords, les coins, etc.

- Fonctions d'Activation :

Après chaque opération de convolution, une fonction d'activation est appliquée à chaque élément de la carte d'activation. La fonction d'activation la plus couramment utilisée dans les CNN est la fonction ReLU (Rectified Linear Unit), qui élimine les valeurs négatives et maintient les valeurs positives inchangées.

- Pooling (Sous-échantillonnage) :

Les couches de pooling sont souvent utilisées pour réduire la dimension spatiale des cartes d'activation générées par les couches convolutives. Le sous-échantillonnage (ou pooling) permet de réduire le nombre de paramètres dans le réseau, ce qui peut rendre l'entraînement plus rapide et réduire le risque de surajustement (overfitting). La méthode la plus courante de pooling est le max pooling, qui consiste à sélectionner la valeur maximale dans une région de la carte d'activation.

- Couches Entièrement Connectées :

Après plusieurs couches convolutives et de pooling, les informations sont généralement aplaties en un vecteur et passent ensuite par des couches entièrement connectées (comme dans les réseaux MLP). Ces couches sont responsables de la prise de décision finale et de la génération des prédictions. La sortie de ces couches peut être alimentée dans des fonctions d'activation appropriées en fonction de la tâche, telles que les fonctions softmax pour la classification.

- Entraînement et Rétropropagation :

Comme dans les autres types de réseaux de neurones, les CNN sont entraînés en utilisant des méthodes de rétropropagation de l'erreur (backpropagation) pour ajuster les poids des filtres et des couches entièrement connectées. L'objectif est de minimiser la fonction de coût, qui mesure la différence entre les prédictions du modèle et les valeurs réelles.

- Transfert de Connaissances :

Les CNN peuvent bénéficier du transfert de connaissances en utilisant des modèles préentraînés sur de grandes bases de données d'images, comme ImageNet. Cela permet de tirer parti des caractéristiques apprises par ces modèles pour résoudre des tâches spécifiques avec moins de données d'entraînement.

En résumé, un réseau de neurones convolutif (CNN) est une architecture spécifiquement conçue pour la vision par ordinateur et le traitement d'images. Il tire parti de convolutions, de fonctions d'activation et de couches de pooling pour extraire et hiérarchiser les caractéristiques des données d'entrée, ce qui le rend particulièrement adapté à la détection de motifs dans des données bidimensionnelles.

4.4 RNN

Un réseau de neurones récurrents (RNN) est un type d'architecture de réseau de neurones artificiels conçu pour traiter et modéliser des données séquentielles. Contrairement aux réseaux de neurones traditionnels, les RNN introduisent des boucles récurrentes qui leur permettent de maintenir une mémoire interne des états précédents, ce qui les rend particulièrement adaptés pour traiter des séquences temporelles, du texte et d'autres types de données séquentielles.

Les éléments constitutifs d'un réseau CNN sont :

- Boucle Récurrente :

La caractéristique clé d'un RNN est sa boucle récurrente, qui permet au réseau de prendre en compte les informations de l'étape de temps précédente en plus des données d'entrée actuelles. Cette boucle crée un lien entre les étapes de temps successives, ce qui permet au RNN de conserver une forme de mémoire de l'évolution temporelle des données.

- Propagation Avant (Forward Propagation) :

Lors de la propagation avant dans un RNN, à chaque étape de temps, l'entrée actuelle (par exemple, un vecteur représentant un mot dans une séquence de mots) est combinée avec l'état caché précédent à l'aide de poids appris (matrices de poids). Cette combinaison génère un nouvel état caché qui est utilisé pour générer une sortie et est également transmis à l'étape suivante comme état caché.

- État Caché :

L'état caché est un vecteur qui représente la mémoire interne du réseau à un moment donné. Il capture l'information contextuelle des étapes de temps précédentes et la propage à travers le réseau. Cet état caché peut être considéré comme une forme de résumé de l'histoire passée du RNN jusqu'à l'étape de temps actuelle.

- Fonction d'Activation :

Les RNN utilisent généralement une fonction d'activation non linéaire, comme la fonction tanh (tangente hyperbolique) ou la fonction ReLU (Rectified Linear Unit), pour transformer la combinaison de l'entrée et de l'état caché en une sortie ou un nouvel état caché. Cette fonction d'activation introduit de la non-linéarité dans le modèle et lui permet de capturer des relations complexes dans les données.

- Entraînement et Rétropropagation :

L'entraînement d'un RNN se fait généralement en utilisant la rétropropagation du gradient, tout comme dans les autres types de réseaux de neurones. L'objectif est de minimiser la fonction de coût, qui mesure la différence entre les prédictions du modèle et les valeurs réelles, en ajustant les poids et les biais des connexions.

- Problème du Gradient qui Disparaît ou Explode :

Les RNN traditionnels peuvent rencontrer des problèmes de gradient qui disparaît ou explose lors de l'entraînement sur de longues séquences. Les dérivées partielles peuvent devenir très petites (problème du gradient qui disparaît) ou très grandes (problème du gradient qui explose), ce qui rend l'entraînement difficile. Les architectures telles que les LSTM et les GRU ont été conçues pour résoudre ces problèmes.

- Applications :

Les RNN sont utilisés dans une variété de domaines, notamment la modélisation de séquences temporelles, la génération de texte, la traduction automatique, la reconnaissance d'écriture manuscrite, l'analyse de sentiments et bien d'autres tâches impliquant des données séquentielles.

En résumé, un réseau de neurones récurrents (RNN) est une architecture conçue pour traiter des données séquentielles en introduisant des boucles récurrentes et en maintenant une mémoire interne de l'évolution temporelle des données. Malgré leur capacité à capturer des dépendances temporelles, les RNN traditionnels peuvent rencontrer des problèmes de gradient, ce qui a conduit au développement d'architectures telles que les LSTM et les GRU pour mieux gérer ces problèmes.

4.5 RNN, GRU & LSTM

Les réseaux de neurones récurrents (RNN), les réseaux LSTM (Long Short-Term Memory) et les réseaux GRU (Gated Recurrent Unit) sont tous des architectures de réseaux de neurones utilisées pour le traitement de séquences de données, telles que des séquences temporelles, du langage naturel ou d'autres données séquentielles. Ils sont conçus pour capturer les dépendances à long terme dans les séquences, ce qui les rend particulièrement adaptés à des tâches telles que la prédiction de séquences, la traduction automatique et la génération de texte.

- Réseau de Neurones Récurrents (RNN) :

Les réseaux de neurones récurrents (RNN) sont une classe d'architectures de réseaux de neurones conçue pour travailler avec des données séquentielles en introduisant des boucles récurrentes dans le réseau. Chaque neurone dans une couche RNN reçoit une entrée non seulement de la couche précédente, mais aussi de sa propre sortie à l'étape de temps précédente. Cela leur permet de maintenir une mémoire interne qui peut être utilisée pour capturer les dépendances temporelles dans les séquences. Cependant, les RNN standard peuvent avoir du mal à capturer des dépendances à long terme en raison du phénomène de disparition du gradient.

- Réseau LSTM (Long Short-Term Memory) :

Les réseaux LSTM sont une amélioration des RNN traditionnels pour traiter les problèmes de disparition du gradient et pour capturer des dépendances à long terme. Les LSTMs introduisent des portes qui contrôlent le flux d'informations à travers les cellules de mémoire. Chaque cellule de mémoire stocke et régule l'information en fonction de trois types de portes : la porte d'oubli (forget gate) contrôle les informations à oublier, la porte d'entrée (input gate) contrôle les nouvelles informations à ajouter et la porte de sortie (output gate) contrôle les informations à envoyer en tant que sortie. Les LSTMs permettent donc aux réseaux de conserver et d'utiliser des informations sur des périodes de temps plus longues.

- Réseau GRU (Gated Recurrent Unit) :

Les réseaux GRU sont une variante des LSTMs qui simplifie leur architecture tout en conservant une capacité de modélisation similaire des dépendances temporelles à long terme. Les GRUs ont deux portes : une porte de réinitialisation (reset gate) qui détermine quelles informations de l'état précédent doivent être oubliées, et une porte de mise à jour (update gate) qui détermine quelles informations de l'entrée courante doivent être mises à jour. Les GRUs sont conçus pour être plus faciles à entraîner que les LSTMs et nécessitent souvent moins de paramètres.

Relations entre les trois :

- Les réseaux LSTM et GRU sont tous les deux des extensions des RNN conçues pour résoudre les problèmes liés aux dépendances temporelles à long terme et au gradient qui disparaît ou explose.
- Les LSTMs introduisent un mécanisme plus complexe de portes de contrôle de l'information dans les cellules de mémoire, tandis que les GRUs simplifient cette architecture en utilisant moins de portes.
- Les GRUs sont souvent utilisés lorsque la complexité des LSTMs n'est pas nécessaire, ce qui peut les rendre plus efficaces à l'entraînement et à l'inférence dans certains cas.
- Les trois architectures (RNN, LSTM et GRU) sont conçues pour traiter des séquences, mais les LSTMs et les GRUs sont généralement préférés lorsque des dépendances à long terme sont attendues ou nécessaires.

En résumé, les RNN, les LSTMs et les GRUs sont des architectures de réseaux de neurones conçues pour travailler avec des données séquentielles. Les LSTMs et les GRUs sont des extensions des RNN qui introduisent des mécanismes de contrôle de l'information pour capturer des dépendances à long terme et résoudre les problèmes de gradient. Les GRUs sont une variante simplifiée des LSTMs, souvent utilisée lorsque la complexité des LSTMs n'est pas nécessaire.

4.6 Autoencoder

Un auto-encodeur est une architecture de réseau de neurones artificiels utilisée pour la réduction de dimension, la compression et la reconstruction de données. Il appartient à la catégorie des modèles non supervisés, ce qui signifie qu'il n'a pas besoin d'étiquettes de sortie pour l'entraînement. L'objectif principal d'un auto-encodeur est d'apprendre une représentation compressée d'une donnée d'entrée, appelée "code latent", et de pouvoir reconstruire cette entrée à partir du code latent.

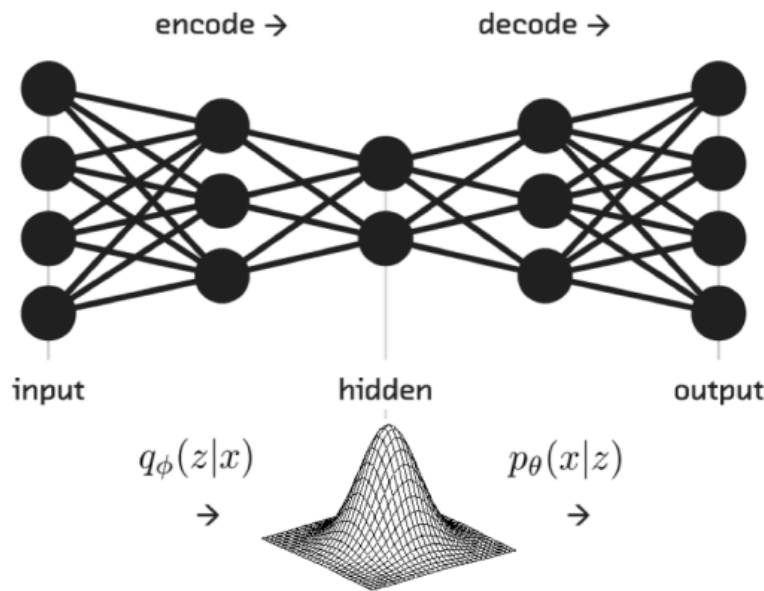


Figure 4.3: Génération d'images.
(source: <https://arxiv.org/pdf/1712.08838>)

Les éléments constitutifs d'un réseau CNN sont :

- Structure de Base :
Un auto-encodeur se compose de deux parties principales : l'encodeur et le décodeur. L'encodeur transforme la donnée d'entrée en une représentation de dimension réduite appelée "code latent". Le décodeur prend ensuite ce code latent et tente de le reconstruire en une sortie qui devrait être aussi proche que possible de la donnée d'entrée d'origine.
- Fonctionnement de l'Encodeur :
L'encodeur est généralement composé de plusieurs couches de neurones. Ces couches successives transforment progressivement la donnée d'entrée en une représentation de dimension plus basse (le code latent). Les activations de la dernière couche de l'encodeur sont considérées comme le code latent qui résume les caractéristiques les plus importantes de la donnée.
- Fonctionnement du Décodeur :
Le décodeur est également composé de plusieurs couches de neurones. Il prend le code latent comme entrée et tente de reconstruire la donnée d'entrée d'origine. Les poids du décodeur sont généralement ajustés pendant l'entraînement pour minimiser l'erreur de reconstruction entre la sortie du décodeur et la donnée d'entrée réelle.
- Fonction de Coût :
La fonction de coût (ou de perte) utilisée pour entraîner un auto-encodeur est généralement une mesure de la différence entre l'entrée d'origine et la sortie du décodeur (la reconstruction). L'objectif est de minimiser cette différence, ce qui encourage l'auto-encodeur à apprendre une représentation compacte du signal d'entrée.

- Applications :

Les auto-encodeurs ont diverses applications, notamment la réduction de dimension pour la visualisation de données complexes, la compression de données, la détection d'anomalies (en utilisant la capacité de l'auto-encodeur à bien reconstruire des données normales, mais pas des anomalies), la génération de données similaires, et la débruitage de données (en apprenant à reconstruire des données corrompues).

- Variétés d'Auto-encodeurs :

Il existe plusieurs variantes d'auto-encodeurs, dont les auto-encodeurs variationnels (VAE) qui introduisent des concepts probabilistes dans le modèle, et les auto-encodeurs contractifs qui ajoutent une régularisation pour rendre le code latent plus robuste.

En résumé, un auto-encodeur est une architecture de réseau de neurones qui apprend à compresser et à reconstruire les données d'entrée en utilisant une représentation interne de dimension réduite appelée code latent. Il est souvent utilisé pour la réduction de dimension, la compression de données et d'autres tâches de traitement non supervisées.

Comparaison à d'autres méthode de réduction de dimension: PCA & SVD

Les auto-encodeurs, la PCA (Analyse en Composantes Principales) et la SVD (Décomposition en Valeurs Singulières) sont tous des techniques utilisées pour la réduction de dimension et la compression de données. Bien qu'ils aient des approches différentes, ils partagent des concepts sous-jacents et des objectifs similaires. Voici comment ils sont liés :

- Analyse en Composantes Principales (PCA) :

La PCA est une technique mathématique qui vise à réduire la dimension d'un ensemble de données tout en préservant autant d'informations que possible. Elle consiste à transformer les données d'origine en un nouvel ensemble de dimensions réduites, appelé "composantes principales", qui capturent les directions dans lesquelles les données varient le plus. Les composantes principales sont calculées en diagonalisant la matrice de covariance des données. La PCA est une méthode linéaire et statistique.

- Décomposition en Valeurs Singulières (SVD) :

La SVD est une décomposition matricielle qui peut être appliquée à diverses tâches, notamment la réduction de dimension et la compression de données. En réduisant les valeurs singulières moins significatives d'une matrice, on peut obtenir une approximation de la matrice d'origine avec une dimension réduite. Cela peut être utilisé pour compresser des données en préservant une grande partie de leur structure. La SVD peut être utilisée pour des données non seulement numériques, mais aussi textuelles et autres.

- Auto-encodeurs et Relation avec PCA et SVD :

Les auto-encodeurs sont des modèles de réseau de neurones qui apprennent des représentations de données de manière non supervisée. Ils ont une structure similaire à la PCA et à la SVD en ce sens qu'ils apprennent également des représentations de données de dimension réduite. Cependant, les auto-encodeurs sont des modèles plus flexibles, car ils ne sont pas limités à des transformations linéaires comme la PCA et la SVD. Les auto-encodeurs peuvent apprendre des transformations non linéaires complexes, ce qui les rend plus adaptés à la capture de relations complexes entre les variables.

En effet, les auto-encodeurs peuvent être utilisés pour simuler la PCA et la SVD dans le contexte des réseaux de neurones. Un auto-encodeur avec une fonction d'activation linéaire dans la couche de code latent et la couche de sortie peut se comporter de manière similaire à une PCA. L'auto-encodeur apprendrait ainsi à projeter les données dans l'espace des composantes principales.

Cependant, les auto-encodeurs sont plus puissants et flexibles car ils peuvent apprendre des représentations non linéaires, ce qui les rend adaptés à des tâches plus complexes de réduction de dimension et de génération de données similaires.

En résumé, les auto-encodeurs, la PCA et la SVD sont toutes des techniques de réduction de dimension et de compression de données. Les auto-encodeurs sont plus flexibles car ils peuvent apprendre des transformations non linéaires, tandis que la PCA et la SVD sont des méthodes linéaires plus classiques. Cependant, un auto-encodeur avec une fonction d'activation linéaire peut se comporter de manière similaire à une PCA.

Pour aller plus loin : [Autoencode - PCA & SVD](#)

4.7 Les hyperparamètres

Les hyperparamètres sont des paramètres qui définissent la configuration d'un modèle de réseau de neurones, mais qui ne sont pas appris directement à partir des données lors de l'entraînement. Ils sont décidés par l'utilisateur avant le processus d'entraînement et affectent la façon dont le modèle apprend et généralise. Le choix judicieux des hyperparamètres peut avoir un impact significatif sur les performances du modèle. Voici quelques-uns des principaux hyperparamètres d'un réseau de neurones :

- **Nombre de Couches et de Neurones par Couche :**

Le nombre de couches et de neurones dans chaque couche détermine la complexité du modèle. Un réseau avec plus de couches et de neurones a une capacité plus grande à apprendre des motifs complexes dans les données, mais il peut également être plus enclin au surapprentissage (overfitting) si les données d'entraînement sont limitées.

- **Fonctions d'Activation :**

Les fonctions d'activation déterminent comment les neurones transmettent l'information aux couches suivantes. Des fonctions comme la fonction ReLU, la tangente hyperbolique (tanh) et la sigmoïde sont couramment utilisées. Le choix des fonctions d'activation peut affecter la capacité du réseau à modéliser des relations non linéaires. Elles introduisent de la non-linéarité dans le modèle, ce qui lui permet de capturer des relations complexes et de résoudre des problèmes plus diversifiés. Voici une liste et une description des fonctions d'activation les plus couramment utilisées, ainsi qu'une comparaison de leurs caractéristiques :

- **Fonction Sigmoïde :**

La fonction sigmoïde a une courbe en forme de S et transforme les valeurs d'entrée en un intervalle de 0 à 1. Elle était largement utilisée dans le passé, mais elle a perdu en popularité en raison de certaines limitations, notamment le problème de disparition du gradient pour des valeurs d'entrée très grandes ou très petites.

- **Fonction Tanh (Tangente Hyperbolique) :**

La fonction tangente hyperbolique est similaire à la sigmoïde, mais elle transforme les valeurs d'entrée en un intervalle de -1 à 1. Elle peut atténuer le problème de disparition du gradient mieux que la sigmoïde car elle est centrée sur zéro.

- **Fonction ReLU (Rectified Linear Unit) :**

La fonction ReLU est définie comme $f(x) = \max(0, x)$. Elle renvoie simplement l'entrée si elle est positive et zéro sinon. La fonction ReLU est très populaire en raison de sa simplicité et de sa capacité à atténuer le problème de disparition du gradient. Cependant, elle peut causer des neurones "morts" (neurones qui ne sont jamais activés) si elles deviennent négatives pendant l'entraînement.

- **Fonction Leaky ReLU :**

Leaky ReLU est une variante de ReLU qui permet à une petite pente positive pour les valeurs négatives (p. ex. $f(x) = x$ si $x \geq 0$, sinon $f(x) = 0.01x$). Cela aide à atténuer le problème des neurones "morts" en permettant un petit gradient pour les valeurs négatives.

- **Fonction ELU (Exponential Linear Unit) :**

ELU est une fonction d'activation qui combine les avantages de ReLU et de Leaky ReLU. Elle maintient la non-linéarité tout en évitant les problèmes de neurones "morts". Pour les valeurs négatives, elle a une courbe exponentielle qui évite le plateau de zéro.

- **Fonction SeLU (Scaled Exponential Linear Unit) :**

SeLU est une extension de ELU qui tend à avoir des propriétés de normalisation intégrées. Elle est conçue pour maintenir la moyenne et la variance des activations proches de 0 et 1 respectivement.

- **Fonction Softmax :**

La fonction softmax est couramment utilisée dans la couche de sortie d'un réseau pour la classification multiclasse. Elle transforme les scores bruts en probabilités normalisées. La somme des probabilités de sortie est toujours égale à 1.

Comparaison :

- **Sigmoïde et Tanh :** Bien qu'elles soient efficaces dans certaines situations, elles peuvent souffrir du problème de disparition du gradient et ne sont pas souvent utilisées dans les couches cachées des réseaux profonds.

- **ReLU** : Très populaire, ReLU fonctionne bien dans de nombreux cas, mais il peut causer des problèmes de neurones "morts" lorsqu'ils deviennent négatifs.
- **Leaky ReLU** : Améliore le problème de neurones "morts" de ReLU en introduisant une pente pour les valeurs négatives.
- **ELU** : Similaire à Leaky ReLU, mais a une courbe exponentielle qui peut éviter le plateau de zéro et permettre des valeurs négatives plus faibles.
- **SeLU** : Conçu pour la normalisation des activations et peut donner de bonnes performances, mais nécessite une initialisation spéciale des poids.
- **Softmax** : Utilisé principalement dans la couche de sortie pour la classification multiclasse, où il normalise les scores en probabilités.

Le choix de la fonction d'activation dépend de la tâche, de l'architecture du réseau et des défis spécifiques à la résolution. Il est souvent recommandé d'expérimenter plusieurs fonctions d'activation et de choisir celle qui donne les meilleures performances sur les données de validation.

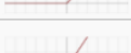
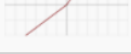
Nom	Plot	Equation	Derivative
Identité		$f(x) = x$	$f'(x) = 1$
Créneau		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Sigmoïde ou Logistique		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
Tangente hyperbolique		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
Arctangente		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified linear unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric Rectified Linear Unit (PReLU)		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU)		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

Figure 4.4: Quelques fonctions d'activation. Source CERN

• Taux d'Apprentissage :

Le taux d'apprentissage contrôle la taille des ajustements faits aux poids des connexions pendant l'entraînement. Un taux d'apprentissage élevé peut entraîner une convergence rapide, mais peut également provoquer des sauts à travers le minimum global. Un taux d'apprentissage trop faible peut ralentir l'entraînement et potentiellement aboutir à des minima locaux.

• Algorithme d'Optimisation :

L'algorithme d'optimisation détermine comment les poids des connexions sont mis à jour pendant l'entraînement. Des méthodes telles que la descente de gradient stochastique (SGD), Adam, RMSProp et d'autres sont largement utilisées. Chaque algorithme a ses avantages et inconvénients en termes de vitesse de convergence et de stabilité. Les principaux:

- Descente de gradient

C'est la méthode de base pour entraîner un modèle d'apprentissage automatique. Imaginez une balle roulant vers le bas d'une colline : la balle suit le chemin de plus forte pente. En apprentissage profond, on ajuste les paramètres du modèle (les poids) en suivant la direction opposée au gradient de la fonction de coût (l'erreur). C'est comme si on poussait la balle dans la direction qui fait le plus baisser la colline.

- Momentum

Le momentum ajoute une sorte d'inertie à la descente de gradient. Au lieu de suivre uniquement le gradient actuel, on prend en compte une moyenne exponentiellement pondérée des gradients précédents. Cela permet d'accélérer la convergence dans les directions où le gradient pointe constamment dans la même direction, et de réduire les oscillations dans les directions où le gradient change fréquemment.

- AdaGrad (Adaptive Gradient Algorithm)

AdaGrad adapte le taux d'apprentissage pour chaque paramètre en fonction de l'historique des gradients. Les paramètres qui ont déjà subi de fortes mises à jour auront un taux d'apprentissage plus faible, tandis que ceux qui ont peu changé auront un taux d'apprentissage plus élevé. Cela permet d'accélérer la convergence pour les paramètres qui ont besoin de plus de mises à jour.

- RMSProp (Root Mean Square Propagation)

RMSProp est une variante d'AdaGrad qui résout un problème d'AdaGrad : le taux d'apprentissage peut devenir trop petit trop rapidement. RMSProp utilise une moyenne exponentiellement pondérée des carrés des gradients pour normaliser les mises à jour. Cela permet d'avoir un taux d'apprentissage qui décroît plus lentement qu'avec AdaGrad.

- Adam (Adaptive Moment Estimation)

Adam combine les avantages de Momentum et RMSProp. Il utilise à la fois une moyenne exponentiellement pondérée des gradients (comme Momentum) et une moyenne exponentiellement pondérée des carrés des gradients (comme RMSProp). Adam est souvent considéré comme l'algorithme d'optimisation par défaut pour l'apprentissage profond car il converge rapidement et efficacement sur une grande variété de problèmes.

- **Fonction de Coût :**

La fonction de coût (ou de perte) mesure l'écart entre les prédictions du modèle et les valeurs réelles. Le choix de la fonction de coût dépend de la tâche. Par exemple, pour la classification binaire, on pourrait utiliser l'entropie croisée binaire, et pour la régression, on pourrait utiliser l'erreur quadratique moyenne.

- **Batch Size :**

Le batch size détermine le nombre d'exemples d'entraînement utilisés pour calculer le gradient et mettre à jour les poids à chaque étape d'entraînement. Un batch size plus grand peut accélérer l'entraînement, mais peut nécessiter plus de mémoire. Un batch size plus petit peut rendre l'entraînement plus stable, plus général, mais prend plus de temps.

- **Régularisation :**

Les techniques de régularisation telles que le dropout et la pénalisation L1 ou L2 des poids peuvent être utilisées pour prévenir le surapprentissage en réduisant la complexité du modèle. Le Dropout, est une approche de régularisation qui consiste à désactiver aléatoirement des neurones pendant l'entraînement, pausant leur évolution.

- **Architecture Spécifique :**

Certains modèles de réseau de neurones, comme les réseaux convolutifs (CNN) ou les réseaux récurrents (RNN), ont leurs propres hyperparamètres spécifiques liés à leur architecture.

Il est important de noter que le choix des hyperparamètres dépend du contexte, de la nature des données et de la tâche que le modèle doit accomplir. Une pratique courante est de faire des essais avec différentes combinaisons d'hyperparamètres et d'utiliser des méthodes comme la validation croisée pour sélectionner ceux qui donnent les meilleures performances sur des données de validation distinctes.

Pour aller plus loin :

- Optimiseurs
- fonctions de loss

4.8 vanishing gradient

Le concept de "vanishing gradient" (gradient qui disparaît) est un phénomène qui se produit lors de l'entraînement de réseaux de neurones profonds, en particulier avec des architectures telles que les réseaux de neurones récurrents (RNN) ou les réseaux de neurones avec de nombreuses couches. Ce phénomène a un impact négatif sur la capacité du réseau à apprendre efficacement à partir de données d'entraînement, en particulier sur les séquences temporelles longues ou les architectures profondes.

Lors de l'entraînement d'un réseau de neurones par la méthode de rétropropagation du gradient, le gradient (la dérivée partielle de la fonction de perte par rapport aux poids du réseau) est calculé à chaque couche du réseau. Ce gradient est utilisé pour ajuster les poids du réseau afin de minimiser la fonction de perte et d'améliorer les performances.

Le problème du vanishing gradient survient lorsque les gradients deviennent extrêmement petits à mesure qu'ils sont propagés vers les couches profondes du réseau lors de la rétropropagation. En d'autres termes, les dérivées partielles des couches profondes par rapport aux poids deviennent très proches de zéro. Lorsque cela se produit, les poids de ces couches ont des mises à jour minimales, ce qui rend leur apprentissage difficile.

Plusieurs facteurs contribuent au phénomène de vanishing gradient :

- **Fonctions d'Activation** : Certaines fonctions d'activation, comme la sigmoïde et la tangente hyperbolique, ont des dérivées qui sont plus petites lorsque les valeurs d'entrée sont grandes ou petites. Cela peut entraîner la diminution rapide des gradients à mesure qu'ils sont propagés dans les couches profondes.
- **Réseaux Profonds** : Plus le réseau est profond, plus le nombre de couches à travers lesquelles les gradients doivent se propager est élevé. À chaque propagation, les gradients peuvent se diluer, conduisant au vanishing gradient.
- **Initialisation des Poids** : Des poids initialisés de manière inappropriée peuvent amplifier le phénomène de vanishing gradient. Si les poids sont trop petits, les sorties de chaque couche peuvent rapidement converger vers des valeurs très faibles.

Le vanishing gradient peut conduire à des problèmes d'entraînement tels que :

- L'apprentissage lent ou inefficace.
- Le risque d'atteindre un minimum local rapidement, laissant le modèle coincé dans un état sous-optimal.
- La difficulté d'apprentissage de relations complexes entre les caractéristiques des données.

Pour surmonter le problème du vanishing gradient, plusieurs techniques ont été proposées, notamment l'utilisation de fonctions d'activation qui atténuent le phénomène (comme ReLU), l'initialisation adaptative des poids, la normalisation de lot (batch normalization), et l'utilisation d'architectures spéciales comme les réseaux LSTM et GRU, qui ont été conçus pour mieux gérer les dépendances temporelles à long terme.

Suggestion d'hyperparamètres pour des réseaux simples, peu profonds :

- cas général :
 - **Taux d'apprentissage (Learning Rate)** :
Pour la plupart des cas : 0.001 à 0.01
Pour les optimiseurs adaptatifs comme Adam : 0.001
 - **Nombre d'époques (Epochs)** :
Commencez par un petit nombre (qui dépend de la profondeur du réseau) et augmentez progressivement en surveillant les performances sur un ensemble de validation.

- **Taille du Batch (Batch Size) :**
32, 64, 128 sont des valeurs courantes.
Expérimentez pour trouver la taille optimale en fonction de la mémoire disponible et des performances.
 - **Fonction d'Activation :**
Commencez par ReLU ou ses variantes (Leaky ReLU, ELU) pour les couches cachées.
Softmax pour la couche de sortie de classification multiclasse.
 - **Initialisation des Poids :**
Initialisation normale ou Glorot (Xavier) pour les couches cachées.
Initialisation uniforme pour la couche de sortie.
 - **Régularisation :**
Dropout : Commencez avec une valeur de 0.2 à 0.5.
Régularisation L2 : Ajoutez un terme de pénalité sur les poids avec un facteur lambda approprié.
 - **Architecture du Réseau :**
Commencez par quelques couches cachées et augmentez progressivement en complexité si nécessaire.
 - **Taille des Couches Cachées :**
Varie en fonction de la complexité de la tâche et de la quantité de données disponibles.
Commencez par quelques dizaines à quelques centaines de neurones.
 - **Optimiseur :**
Adam ou RMSProp sont souvent de bons choix de départ.
SGD (Descente de Gradient Stochastique) avec momentum peut également être efficace.
 - **Fonction de Coût (Loss Function) :**
Entropie croisée pour la classification.
Erreur quadratique moyenne pour la régression.
 - **Validation Croisée (Cross-Validation) :**
Divisez vos données en ensembles d'entraînement, de validation et de test pour évaluer les performances.
 - **Taille du Code Latent (Auto-encodeurs) :**
Dépend de la complexité de la tâche et de la quantité d'informations que vous voulez extraire.
- **Classification d'Images (Réseaux Convolutifs - CNN) :**
 - Taux d'apprentissage : 0.001 à 0.01
 - Nombre d'époques : 20 à 2000
 - Taille du Batch : 32, 64, 128
 - Fonction d'Activation : ReLU pour les couches cachées, Softmax pour la couche de sortie
 - Initialisation des Poids : Initialisation Glorot (Xavier)
 - Régularisation : Dropout (0.2 à 0.5) Architecture du Réseau : Convolution suivie de couches de pooling, couches fully connected
 - Taille des Couches Cachées : Varie en fonction de la complexité de la tâche
 - Optimiseur : Adam ou RMSProp
 - Fonction de Coût : Entropie croisée catégorique
 - **Traduction Automatique (Réseaux Récurrents - RNN ou LSTM) :**
 - Taux d'apprentissage : 0.001 à 0.01
 - Nombre d'époques : 50 à 3000
 - Taille du Batch : 32, 64
 - Fonction d'Activation : Tanh ou LSTM pour les couches cachées
 - Initialisation des Poids : Initialisation Glorot (Xavier)

- Régularisation : Dropout (0.2 à 0.5)
- Architecture du Réseau : Couches LSTM empilées ou réseaux Seq2Seq
- Taille des Couches Cachées : Varie en fonction de la complexité de la tâche
- Optimiseur : Adam ou RMSProp
- Fonction de Coût : Entropie croisée catégorique

• **Détection d'Anomalies (Réseaux Auto-encodeurs) :**

- Taux d'apprentissage : 0.001 à 0.01
- Nombre d'époques : 50 à 3000
- Taille du Batch : 32, 64
- Fonction d'Activation : ReLU pour les couches cachées, Sigmoid pour la couche de sortie (pour la reconstruction)
- Initialisation des Poids : Initialisation Glorot (Xavier)
- Régularisation : Dropout (0.2 à 0.5)
- Architecture du Réseau : Auto-encodeur classique (encoder et décodeur)
- Taille du Code Latent : Petit (8 à 32)
- Optimiseur : Adam ou RMSProp
- Fonction de Coût : Erreur quadratique moyenne

• **Régression (Réseaux Neurones Profonds) :**

- Taux d'apprentissage : 0.001 à 0.01
- Nombre d'époques : 50 à 3000
- Taille du Batch : 32, 64, 128
- Fonction d'Activation : ReLU pour les couches cachées, Linéaire pour la couche de sortie
- Initialisation des Poids : Initialisation Glorot (Xavier)
- Régularisation : Régularisation L2
- Architecture du Réseau : Plusieurs couches cachées empilées
- Taille des Couches Cachées : Varie en fonction de la complexité de la tâche
- Optimiseur : Adam ou RMSProp
- Fonction de Coût : Erreur quadratique moyenne

Pour des réseaux complexes ou profonds, le nombre d'epoch peut atteindre les 100 000.

4.9 Rex: utilisation de modèle d'IA

Table 4.1: Modèles d'IA en finance

Modèle/Acronyme	Description	Application
BERT	Transformers pour comprendre le sentiment de marché à partir de données textuelles.	Analyse de l'impact des actualités financières sur les marchés.
LSTM	Réseaux de neurones récurrents spécialisés dans la reconnaissance de motifs temporels.	Détection de tendances dans les séries temporelles de prix d'actions.

Table 4.2: Prédiction de la consommation et de la production énergétique

Application	Description	Réseaux utilisés
Prédiction de la consommation	Estimation de la consommation énergétique (électricité, chauffage, climatisation) à court, moyen ou long terme, en fonction des conditions météo, de l'occupation et des caractéristiques du bâtiment.	LSTM, CNN, MLP, RNN.
Prédiction de la production solaire	Prévision de la production photovoltaïque en fonction de l'ensoleillement, de la température et des données historiques.	LSTM, CNN, Réseaux hybrides (CNN-LSTM).
Prédiction de la demande de chauffage/climatisation	Anticipation des besoins thermiques pour optimiser le pilotage des systèmes CVC (Chauffage, Ventilation, Climatisation).	RNN, LSTM, Réseaux bayésiens.

Table 4.3: Contrôle et optimisation des systèmes énergétiques

Application	Description	Réseaux utilisés
Contrôle intelligent des systèmes CVC	Optimisation en temps réel des paramètres de chauffage, ventilation et climatisation.	Deep Reinforcement Learning, LSTM
Gestion des batteries et stockage	Optimisation de la charge/décharge des batteries pour maximiser l'autoconsommation.	DNN, Q-Learning
Pilotage des micro-réseaux	Gestion intelligente des flux d'énergie entre sources renouvelables, stockage et consommation.	RNN, Algorithmes génétiques + ANN

Table 4.4: Diagnostic et maintenance des équipements

Application	Description	Réseaux utilisés
Détection de défauts	Identification précoce des pannes dans les équipements (chaudières, pompes à chaleur).	Autoencodeurs, CNN
Maintenance prédictive	Prédiction des défaillances pour planifier les interventions.	LSTM, RNN
Analyse des performances	Évaluation en continu des performances énergétiques des bâtiments.	DNN, Régression non linéaire

4.10 Reinforcement Learning

L'apprentissage par renforcement ou *Reinforcement Learning* (RL) est une branche du *Machine Learning*, aux côtés de l'apprentissage supervisé et non-supervisé.

Définition: Le RL consiste à mettre au point des algorithmes (*agents*) qui apprennent par l'expérience en interagissant avec leur *environnement* et qui reçoivent des récompenses (*rewards*) en fonctions de leurs *actions*. Ils sont

Table 4.5: Optimisation de la conception et de la rénovation

Application	Description	Réseaux utilisés
Optimisation des enveloppes	Sélection des matériaux pour minimiser les déperditions thermiques.	ANN, Algorithmes génétiques
Simulation énergétique	Prédiction des performances énergétiques en phase de conception ou de rénovation.	DNN, ANN + simulation dynamique
Optimisation des systèmes hybrides	Combinaison optimale de sources d'énergie pour réduire l'empreinte carbone.	DRL, Modèles multi-agents

Table 4.6: Amélioration du confort et interaction avec les occupants

Application	Description	Réseaux utilisés
Adaptation aux préférences des occupants	Apprentissage des habitudes pour ajuster automatiquement les paramètres de confort.	RNN, Clustering + ANN
Détection des comportements	Analyse des comportements énergétiques pour proposer des actions d'économie.	DNN, Classification
Interfaces intelligentes	Systèmes de recommandation pour guider les occupants vers des comportements sobres.	Réseaux bayésiens, Reinforcement Learning

capables d'apprendre une manière d'interagir avec leur environnement (*policy*) pour **maximiser l'espérance d'un signal aléatoire** (ex. somme attendue des récompenses).

Le RL est différent des autres paradigmes de ML car :

- Il n'y a pas de superviseur, seulement un signal de récompense.
- Le retour d'information est différé et non instantané.

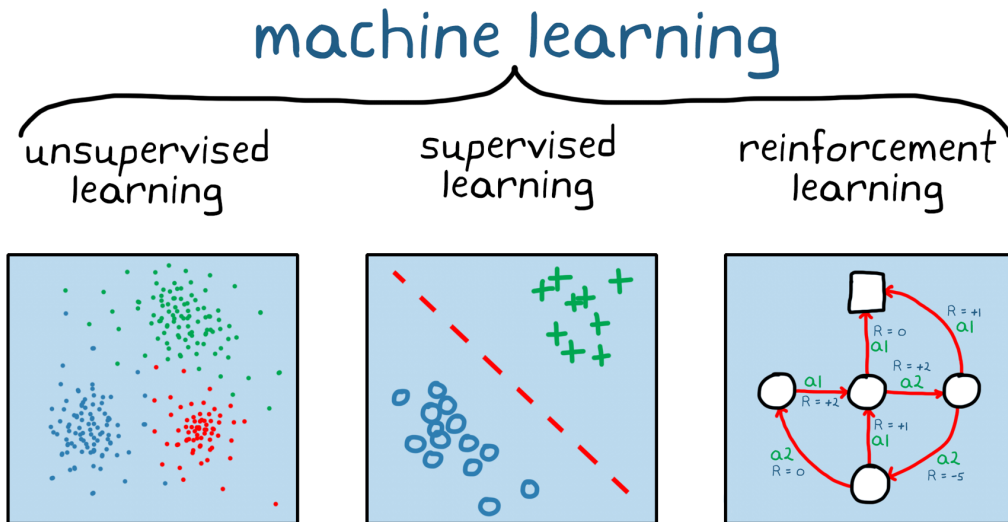
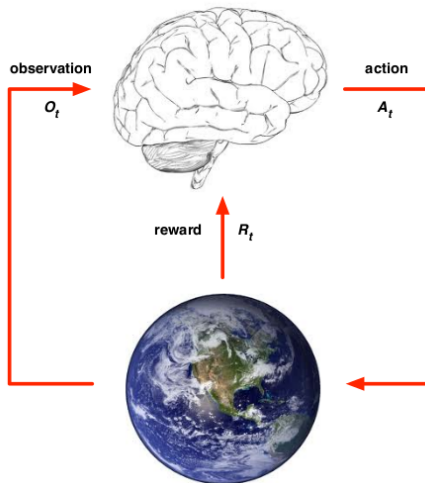


Figure 4.5: Les branches du Machine Learning
(Source : www.mathworks.com).

- L'aspect temporel est important (données séquentielles).
- Les actions de l'agent affectent les données qu'il reçoit par la suite.



- A chaque étape t , l'**agent** :
 - Exécute une action A_t
 - Reçoit une observation O_t
 - Reçoit une récompense R_t
- L'**environnement** :
 - Reçoit une action A_t
 - Émet une observation O_{t+1}
 - Émet une récompense R_{t+1}
- t est incrémenté à l'étape d'environnement

Il s'agit d'un *dilemme* classique pour l'agent :

- **Exploitation** : **Prendre la meilleure décision**, compte tenu des informations disponibles. La meilleure stratégie à long terme peut impliquer des sacrifices à court terme.
- **Exploration** : **Recueillir davantage d'informations**, afin de prendre les meilleures décisions globales.
- **Historique**: c'est la *séquence* des observations, actions et récompenses. Il s'agit de toutes les variables observables à l'instant t :

$$H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t \quad (4.1)$$

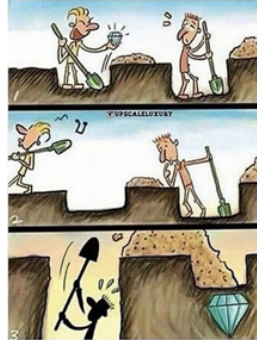


Figure 4.6: Exploitation vs. Exploration.
(Source : ai-ml-analytics.com).

- **L'état** : c'est l'*information* utilisée pour déterminer ce qui va se passer ensuite. Formellement, il s'agit d'une fonction de l'historique :

$$S_t = f(H_t) \quad (4.2)$$

- **Politique** : c'est une *stratégie* qui définit le comportement de l'agent. Il s'agit d'une fonction qui relie un état S à une action A :

$$\pi(S) = A \quad (4.3)$$

- **Définition** : un processus S_t est dit **Markovien** si et seulement si:

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1, \dots, S_t] \quad (4.4)$$

“Le futur est indépendant du passé compte tenu du présent.”

- **État des informations** : c'est un état *Markovien* et il contient toutes les informations utiles de l'Histoire (slide précédente).
- **Fonction de valeur** : c'est une prédiction de la récompense future. Elle est utilisée pour évaluer les états afin de sélectionner les actions à entreprendre ($\gamma = \text{discount factor}$) :

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \quad (4.5)$$

- **State-Action-Reward-State-Action (SARSA)** :

La politique apprise est la même que celle utilisée pour choisir les actions à exécuter. Dans cet algorithme, l'agent saisit la politique optimale et l'utilise pour agir.
L'algorithme est dit **on-policy**.

- **Q-Learning**¹ :

Le choix des actions à effectuer ne dépend que de l'état courant et des politiques déjà apprises. Dans cet algorithme, l'agent apprend la politique optimale à l'aide d'une politique avide (*greedy*).
L'algorithme est dit **off-policy**.

D. Ernst, “Apprentissage par renforcement pour les marchés de l'électricité et la transition énergétique”, Institut Henri Poincaré, Novembre 2021

¹Le Q-Learning est un algo de RL qui ne nécessite pas de modèle initial de l'environnement. La lettre 'Q' désigne la fonction qui mesure la qualité d'une action exécutée dans un état donné du système.

Vidéo

D. Ernst, “Apprentissage par renforcement pour les marchés de l’électricité et la transition énergétique”

Structure des marchés de l’électricité :

- Circulation autour d’un réseau de **transport** de l’électricité (ex. RTE en France).
- Consommateurs connectés au réseau de **distribution** de l’électricité (ex. ENEDIS en France).
- **Complexification** avec les modèles d’échange en réseau, ou chaque entité est à la fois client et fournisseur (*peer-to-peer*).

Comment fonctionne un marché de l’électricité ? :

- A chaque fois que l’on consomme une unité d’énergie d’électricité, elle est achetée ou produite avec un coût marginal par le *fournisseur*.
- Sur les marchés de l’électricité, on achète toujours une **quantité d’électricité** associée à une **période de temps** (ex. 1 MWh à soutirer entre 16h15 et 16h30 – *donc pour une période de marché de 15mn*).

3 types de marchés principaux :

- Marché “**Forward / Future**” : contrats de type “*calendar*” (Y, Q ou M) – Achat d’une quantité d’électricité en amont de la consommation / production. Il s’arrête juste quelques jours (heures) avant la période de livraison d’électricité.
- Marché “**Day-ahead**” : marché **centralisé** qui intervient une fois les marchés “*forward*” clôturés. Marché très important où tous les acteurs vont envoyer leurs requêtes par créneau horaire (prix min de vente, prix max d’achat). Optimisation réalisée au niveau européen. Il se clôture vers 12h.
- Marché “**Intraday**” : marché **centralisé** avec trading complètement automatisé.

3 raisons qui montrent que le RL est un outil prometteur pour interagir avec les marchés de l’électricité :

- Le RL est adapté à la **maximisation de l’espérance des profits**.
- Le RL est adapté à la **nature séquentielle** des problèmes rencontrés sur les marchés de l’électricité. Le développement à grande vitesse du stockage d’énergie provoque des couplages entre les décisions à prendre (inertie *via* le niveau de charge de la batterie) : les décisions à prendre à un instant t dépendent donc des précédentes. On sort donc du cadre des processus de *Markov* simples. Il faut y associer un effet mémoire (ex. cellules LSTM) ou des politiques.
- Le RL est naturellement fait pour gérer des **problèmes stochastiques** (incertitudes météo, charge). Les changements dans le temps ne peuvent être déterminés à partir d’un état initial et des équations ou modèles (**cas des marchés de l’électricité**).

RL pour attaquer le marché continu “*intraday*” (CID : Current Interrupt Device) :

- **Article #1** : travaux en collaboration avec le groupe de trading de chez ENGIE, en Belgique.
- Illustration du CID (“*gate closure*” 15mn avant la période où les quantités sont consommées/produites).
- **Contrainte d’équilibre des positions ouvertes**. En cas de déséquilibre, ajustement avec une batterie. Analogie avec contrainte de neutralité *overnight* pour un portefeuille.
- Design d’un agent capable de contrôler la batterie (charge / décharge) et de décider quelle offre d’achat / de vente saisir (*plusieurs milliers*).

RL scheme applied to CID trading with a battery

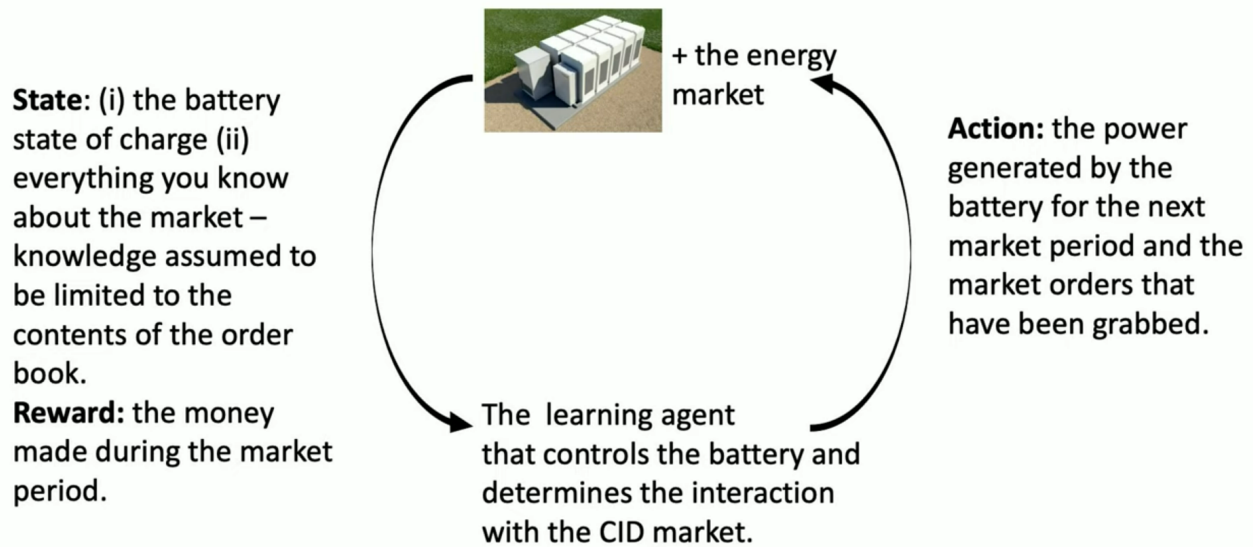


Figure 4.7: Schéma générique du RL appliqué au trading CID avec une batterie.

Réduction de la complexité du problème :

- **Espace d'état** : grande dimension. Nécessité d'utiliser des techniques de feature extraction pour en réduire la taille.
- **Espace d'action** : grande dimension. Le nombre d'actions à un instant dépend du carnet d'ordres. Réduction à 2 variables de haut niveau : *idle* (ne fait rien) ou *trade* (opération de trading).
- **Contexte adverse** : conception d'un algorithme de RL avec l'hypothèse (forte) que l'agent n'est pas un "gros trader" et donc que son comportement n'influence pas le comportement des autres acteurs du marché (cas *mono-agent*).

Quelques précisions techniques sur l'algorithme de RL utilisé :

- Génération de *trajectoires* par un agent RL interagissant avec un jeu de données d'entraînement du carnet d'ordres et la batterie. Utilisation d'une politique de type *Epsilon-greedy* (équilibre entre l'exploration et l'exploitation, choisie au hasard).
- Utilisation de l'algorithme "*Fitted Q Iteration*" (FIQ²) pour l'extraction de politiques haute performance à partir des trajectoires.
- La structure du réseau de neurones utilisée pour approximer la fonction Q est composée d'une couche LSTM et de 5 couches "*fully connected feed-forward*".
- $4 \times 24 = 96$ réseaux de neurones à entraîner (1 modèle par créneau de 15mn pour une journée de 24h).

Communautés de l'énergie renouvelable (CER) :

²Dans l'algorithme FIQ, la fonction Q est approximée linéairement.

- Législation européenne qui autorise les CERs.
- Communautés où il est possible d'échanger de l'énergie avec ses voisins (et bien d'autres choses).
- Question : "Quels investissement dois-je faire en matériel (PVs, batterie, etc.) pour maximiser mes gains ?"
→ Energy Management System (EMS).

Dimensionnement de systèmes énergétiques complexes :

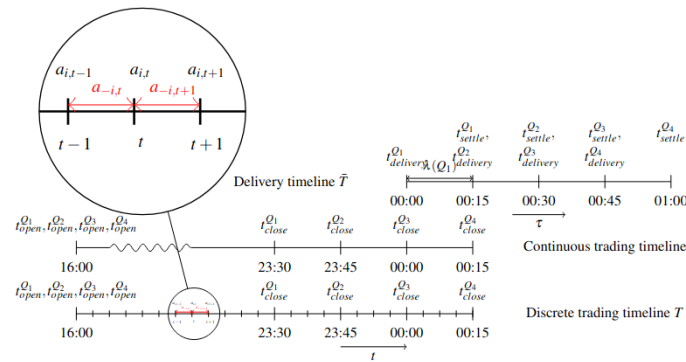
- Le dimensionnement initial des équipements doit être optimisé (*capex*).
- Optimisation de l'*opex* qui revient à maximiser une fonction représentant l'espérance de la somme des rewards.

Application sur les marchés de l'électricité.

I. Boukas et al., "A Deep Reinforcement Learning Framework for Continuous Intraday Market Bidding"

- RL appliqué au cas d'application présenté en détails dans la vidéo.
- Certains aspects semblent spécifiques au marché de l'électricité, notamment l'importance du moment du trading des quantités.
- Mais plusieurs aspects paraissent adaptables :
 - La modélisation de l'order book en temps réel et la préparation des données pour le RL.
 - La modélisation du comportement des différents participants.
 - La gestion du problème du stock de la batterie, qui a pour effet de coupler les prises de décision entre elles.

Cas d'usage :



- Récompense : gain de chacune des politiques pour chaque jour.
- Les données de tests couvrent 110 jours et celles d'entraînement couvrent 252 jours. Le découpage résulte d'un tirage aléatoire d'un historique de 362 jours.
- Le stockage introduit des limites opérationnelles sur les actions.

Amélioration de 1.5% en moyenne dans le backtest par rapport à une stratégie aveugle (*i.e* sans anticipation).

Hypothèses utilisées sur le système / marché :

- *Behaviour of the other agents* : les actions ne dépendent que de l'historique. Un seul agent ne peut pas influencer le marché.
- *Exogenous information*: Elles ne dépendent que du passé.
- *Strategy for storage control* : les charges / décharges d'un système de stockage sont basées sur la stratégie dite *greedy* ou *myope*.

- *Aggressor* : Un agent ne peut soumettre que des ordres qui ne provoquent pas de décalage des prix.
- *No imbalances permitted* : Les agents ne peuvent traiter que pour équilibrer un appel ou une production de puissance.
- *Optimization decoupling* : Les valeurs finales et initiales du stockage sont fixées pour chaque période (journée).
- Les suppositions précédentes impliquent que certaines variables du système sont superflues car conditionnées.

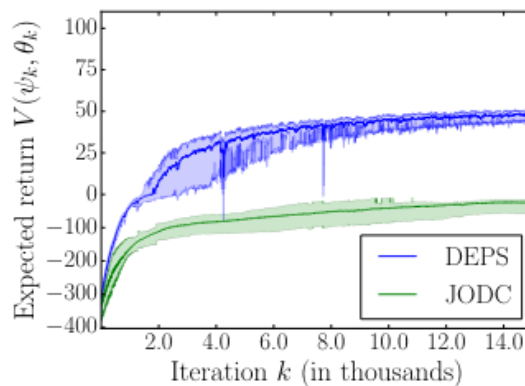
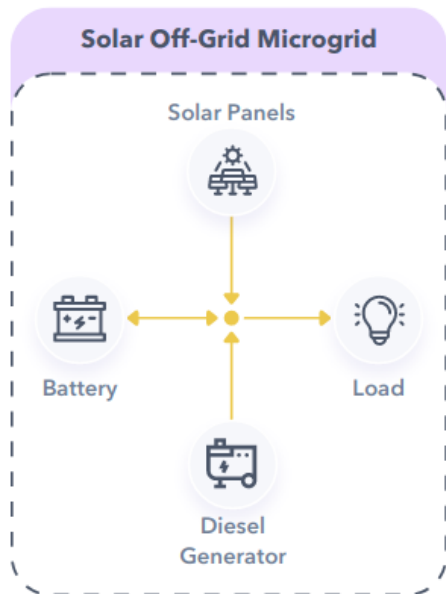
Application à la gestion d'un îlot renouvelable.

S. Aittahar et al., "Optimal Control of Renewable Energy Communities with Controllable Assets"

- L'article présente un algorithme de RL développé pour résoudre le problème joint du dimensionnement et du pilotage pour le comparer ensuite à un autre algorithme de RL utilisé pour ce type d'application et à des pilotages déterministes.
- Direct Environment and Policy Search (DEPS).
- Joint Optimization of Design and Control (JODC).
- La principale différence entre les deux algorithmes réside dans l'approximation du gradient.

Cas d'usage : Sizing and Operation of an Off-Grid Microgrid

Objectif : Minimiser le coût opérationnel du projet.



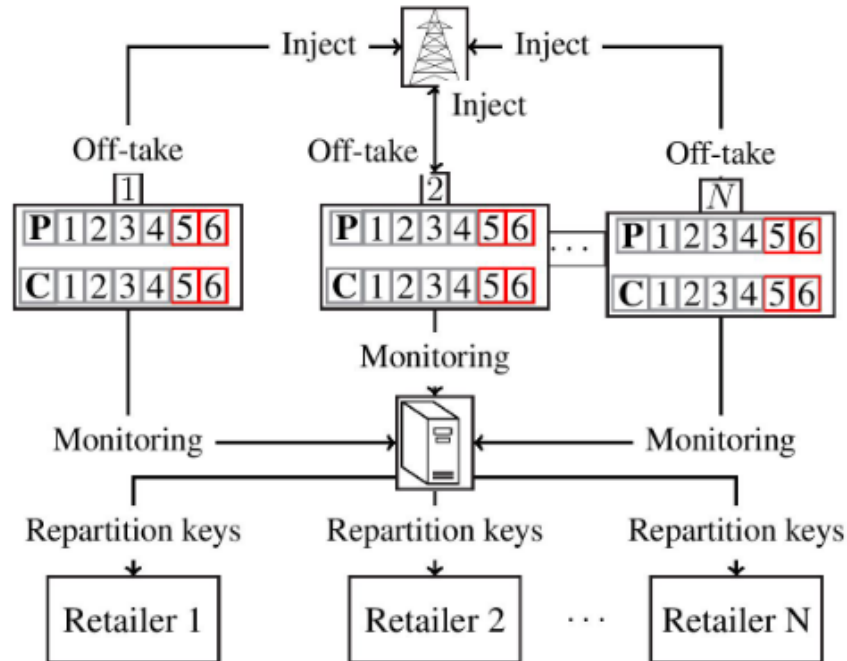
Application à la coordination d'une communauté renouvelable.

A. Bolland et al., "Jointly Learning Environments and Control Policies with Projected Stochastic Gradient Ascent"

Cette article cherche à optimiser la répartition de l'énergie entre les différents producteurs d'une communauté à énergies renouvelables et trouve que l'optimisation est d'autant meilleure que le point de vue est holistique ou global.

Cas d'usage : Réduction de la facture électrique

Optimisation du trading et de l'assistance entre les membres d'une communauté productrice d'énergie renouvelable par le contrôle de la production et des clés de répartition afin de diminuer les factures, tout en respectant les contraintes opérationnelles.



Application à l'équilibrage d'un réseau de distribution électrique.

R. Henry et D. Ernst, "Gym-ANM: RL Environments for ANM Tasks in Electricity Distribution Systems"

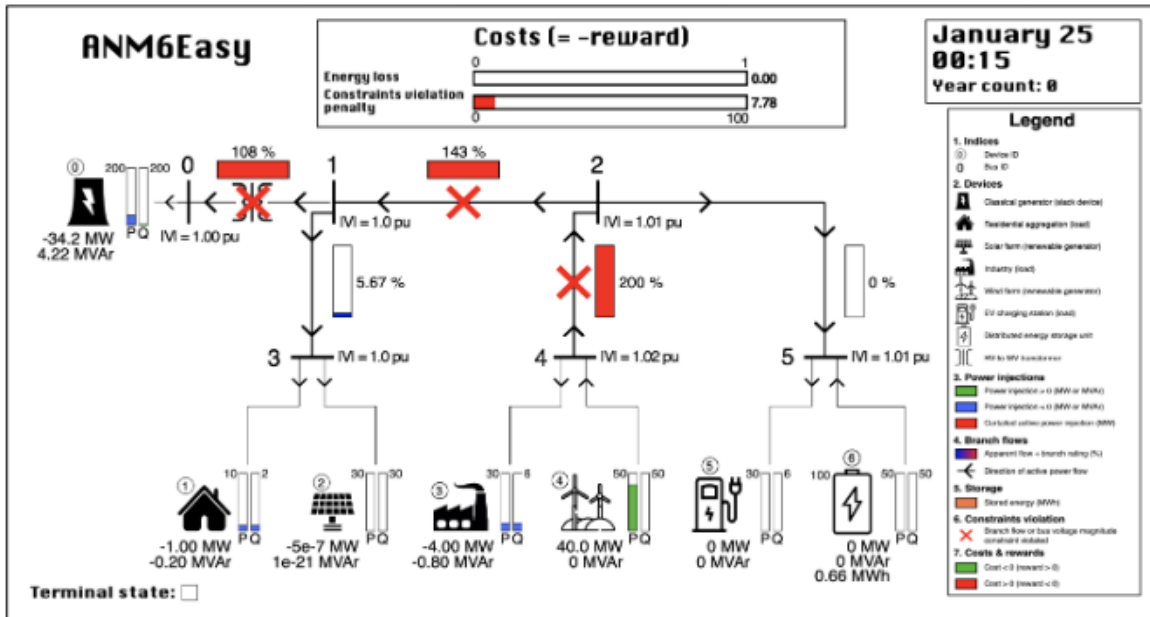
- Les auteurs proposent un framework intitulé **Gym-ANM** (*Active Network Management*) pour concevoir et développer des algorithmes de gestion active de réseau de distribution d'électricité, qui utilisent l'apprentissage par renforcement.

Note : **Gym-ANM** est construit sur l'outil **Open AI Gym**, notamment connu pour avoir battu tous les jeux vidéo ATARI 2600.

Cas d'usage : Réseau de distribution

Verrous techniques

- Capacité exploratoire cruciale pour la performance du RL.
- Besoin de nombreuses trajectoires à chaque pas de temps, d'où un coût élevé en calcul et en RAM. Nécessité d'une architecture asynchrone, implémentable au moyen de calcul parallèle, d'une modélisation et d'une conception appropriées des réseaux de neurones.
- Plus une politique "neuronale" est complexe, plus le nombre de paramètres à explorer augmente et pénalise la convergence du système. Une réduction de dimension (NMF par ex.) permet de diminuer le nombre de paramètres d'entrée et améliore la convergence.
- Le feature engineering en collaboration avec l'expertise métier (systèmes énergétiques & marchés financiers) est encore peu développé et constitue une réserve importante d'optimisations.
- Besoins en données pouvant excéder les stocks historiques. Recours possibles à des méthodes génératives (GANs par ex.).



Références

Références

- **Cours** ([lien](#)) – David Silver, “*Introduction to Reinforcement Learning*”, DeepMind x UCL, 2015.
- **Vidéo** ([lien](#)) – Damien Ernst, “*Apprentissage par renforcement pour les marchés de l’électricité et la transition énergétique*”, Institut Henri Poincaré, Novembre 2021.
- **Article #1** ([lien](#)) – I. Boukas et al., “*A Deep Reinforcement Learning Framework for Continuous Intraday Market Bidding*”, 2020.
- **Article #2** ([lien](#)) – S. Aittahar et al., “*Optimal Control of Renewable Energy Communities with Controllable Assets*”, 2022.
- **Article #3** ([lien](#)) – A. Bolland et al., “*Jointly Learning Environments and Control Policies with Projected Stochastic Gradient Ascent*”, 2022.
- **Article #4** ([lien](#)) – R. Henry et D. Ernst, “*Gym-ANM: Reinforcement Learning Environments for Active Network Management Tasks in Electricity Distribution Systems*”, 2021.
- **Bonus** ([lien](#)) – Damien Ernst, “*Gym-ANM: example of a distribution network controlled by a reinforcement learning agent*”, 2021.

4.11 Les Réseaux de Kolmogorov-Arnold (KAN)

Les Réseaux de Kolmogorov-Arnold (KAN) représentent une avancée significative dans le domaine de l’intelligence artificielle appliquée à la science, connue sous le nom d’AI + Science. Inspirés par le théorème de représentation de Kolmogorov-Arnold, les KANs sont une nouvelle classe de réseaux de neurones qui promettent une amélioration de la précision et de l’interprétabilité par rapport aux perceptrons multicouches (MLP) pour des tâches scientifiques à petite échelle.

Les KANs sont conçus pour capturer la structure compositionnelle des données, ce qui leur permet de surmonter la malédiction de la dimensionnalité et d’atteindre de meilleures lois d’échelle que les MLPs. Ils sont particulièrement efficaces pour l’ajustement de données et la résolution d’équations aux dérivées partielles (PDE), comme illustré par leur application à l’équation de Poisson.

L’architecture des KANs est fondée sur des techniques de simplification de réseau pour améliorer leur interprétabilité et une technique d’extension de grille pour augmenter leur précision. Ces réseaux sont capables de découvrir des relations non triviales dans des domaines tels que la théorie des nœuds et la physique de la matière condensée, agissant

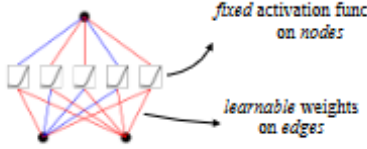
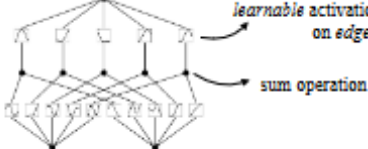
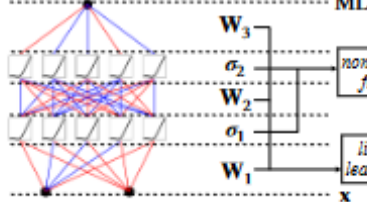
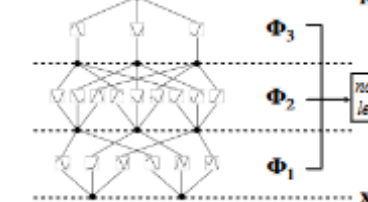
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(c)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a)  fixed activation functions on nodes learnable weights on edges	(b)  learnable activation functions on edges sum operation on nodes
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)	(c)  MLP(x) $\mathbf{W}_3$ σ_2 $\mathbf{W}_2$ σ_1 $\mathbf{W}_1$ $\mathbf{x}$ nonlinear, fixed linear, learnable	(d)  KAN(x) Φ_3 Φ_2 Φ_1 $\mathbf{x}$ nonlinear, learnable

Figure 4.8: MLP VS KAN
(<https://arxiv.org/pdf/2404.19756>).

comme des collaborateurs pour les scientifiques dans la (re)découverte de lois mathématiques et physiques.

Les KANs sont également plus efficaces en termes de paramètres que les MLPs, ce qui est démontré par leur capacité à obtenir une meilleure précision de test avec beaucoup moins de paramètres. Par exemple, un KAN avec environ 200 paramètres a surpassé un MLP de Deepmind avec environ 300 000 paramètres en termes de précision de test. Cette efficacité paramétrique est accompagnée d'une interprétabilité accrue, car il est possible de visualiser directement les variables d'entrée importantes sans recourir à des méthodes d'attribution de caractéristiques.

Les KANs peuvent être entraînés pour représenter des formules symboliques, offrant une frontière de Pareto de formules qui équilibrent la simplicité et la précision. Cependant, des biais inductifs supplémentaires sont nécessaires pour simplifier davantage ces équations. Les KANs ont également montré leur potentiel en tant que modèle de réduction pour les PDEs, bien qu'ils soient généralement plus lents à entraîner que les MLPs.

En termes d'applications pratiques, les KANs ont montré des preuves préliminaires qu'ils sont plus efficaces que les MLPs pour des tâches liées à la science, comme l'ajustement d'équations physiques et la résolution de PDEs. Les chercheurs envisagent d'appliquer les KANs à des équations plus complexes comme les équations de Navier-Stokes et la théorie de la fonctionnelle de la densité.

Les KANs sont composés de fonctions interprétables, ce qui permet une communication intuitive avec les utilisateurs humains, utilisant le langage des fonctions. Cette caractéristique promeut le paradigme de collaboration entre l'IA et les scientifiques, où les KANs pourraient devenir l'un des premiers "langages" permettant une telle communication.

En conclusion, les KANs représentent une avancée prometteuse pour l'AI + Science, offrant une précision, une interprétabilité et une efficacité paramétrique supérieures. Ils ouvrent la voie à de nouvelles méthodes de collaboration entre l'IA et les scientifiques pour la découverte de connaissances et la résolution de problèmes complexes.

Illustration pratique :

- <https://daniel-bethell.co.uk/posts/kan/>

- <https://medium.com/accredian/kolmogorov-arnold-networks-kans-vs-multi-layer-perceptrons-mlps-a-comparison-9e33359564c2>

Chapter 5

Outils de Business Intelligence

5.1 NoteBooks : Jupyter, Google Colab...

Le Jupyter Notebook est un environnement interactif de développement, d'exécution et de partage de code. Il permet de créer et de partager des documents qui contiennent du code, des visualisations, des explications textuelles et des résultats d'exécution. Jupyter Notebook est largement utilisé par les scientifiques, les chercheurs, les ingénieurs et les analystes de données pour explorer, analyser et présenter des données de manière interactive.

Caractéristiques :

- **Support Multi-Langage**

Jupyter Notebook prend en charge plusieurs langages de programmation, mais est le plus couramment utilisé avec Python. Cependant, il permet également d'écrire et d'exécuter du code dans des langages tels que R et Julia. Cela en fait un outil polyvalent pour différents domaines et tâches.

- **Combinaison de Code, Texte et Visualisations**

L'une des principales forces de Jupyter Notebook est sa capacité à combiner du code exécutable, des explications textuelles et des visualisations dans un seul document. Cela facilite la création de rapports interactifs et exploitables, où le code peut être exécuté en temps réel pour produire des résultats visuels et textuels.

- **Environnement Interactif**

Jupyter Notebook offre un environnement interactif où vous pouvez exécuter des cellules de code individuellement. Cela permet de tester et de déboguer du code étape par étape, en observant les résultats à chaque étape.

- **Visualisations Intégrées**

L'utilisation de bibliothèques de visualisation telles que Matplotlib, Seaborn et Plotly dans les cellules de code permet de créer des graphiques, des diagrammes et des visualisations interactives directement dans le notebook.

- **Exécution Non-Linéaire**

L'avantage majeur de Jupyter Notebook réside dans sa capacité à exécuter du code de manière non linéaire. Vous pouvez exécuter les cellules dans n'importe quel ordre, ce qui facilite l'exploration des données et l'expérimentation.

- **Intégration des Mathématiques et des Équations**

Jupyter Notebook prend en charge la syntaxe LaTeX pour l'intégration des formules mathématiques et des équations. Cela en fait un outil populaire pour les domaines tels que les mathématiques, la physique et l'ingénierie.

Utilisation de Jupyter Notebook

- **Installation**

Jupyter Notebook peut être installé via la distribution Anaconda ou via la commande pip pour les utilisateurs de Python. Une fois installé, vous pouvez lancer le notebook en exécutant la commande `jupyter notebook` dans votre terminal.

- **Création de Notebooks**

Lorsque vous lancez Jupyter Notebook, vous êtes dirigé vers une interface web où vous pouvez créer de nouveaux notebooks ou ouvrir des notebooks existants. Chaque notebook est constitué d'une série de cellules qui peuvent contenir du code, du texte ou des visualisations.

- **Types de Cellules**

Les deux types principaux de cellules dans un notebook sont les cellules de code et les cellules Markdown. Les cellules de code sont destinées à l'exécution de code, tandis que les cellules Markdown permettent d'écrire du texte formaté, y compris des titres, des listes et des liens.

- **Exécution de Cellules**

Pour exécuter une cellule de code, vous pouvez cliquer sur le bouton "Exécuter" ou utiliser le raccourci clavier Shift + Enter. Les cellules sont exécutées dans l'ordre où elles sont exécutées.

- **Visualisation des Résultats**

Les résultats d'exécution, tels que les sorties de code ou les visualisations, sont affichés sous la cellule respective. Les cellules Markdown affichent le texte formaté directement dans le notebook.

- **Partage et Exportation**

Les notebooks Jupyter peuvent être partagés en exportant le document au format `.ipynb`. Ils peuvent également être convertis en d'autres formats tels que HTML, PDF ou diaporama pour une présentation plus formelle.

Conclusion

Jupyter Notebook est un outil puissant pour l'analyse interactive de données, le développement de code et la communication de résultats. Sa capacité à combiner code, texte et visualisations en fait un choix populaire parmi les professionnels de divers domaines. Que ce soit pour la recherche, l'enseignement, l'analyse de données ou la présentation, Jupyter Notebook offre un environnement souple et collaboratif pour travailler avec du code et des données.

Si vous voulez vous passer d'installation et que vous avez un compte gmail, vous pouvez aussi utiliser [google colab](#).

5.2 Github & IDE: Pycharm, VS code, Spyder...

GitHub

- **Gestion de Versions** : GitHub est une plateforme de contrôle de versions qui facilite le suivi et la gestion des modifications apportées au code.
- **Collaboration** : Il offre des fonctionnalités avancées de collaboration, notamment les demandes de tirage, les problèmes et les discussions.
- **Hébergement de Code** : Vous pouvez héberger des dépôts de code en ligne, ce qui facilite le partage et la collaboration.
- **Intégration CI/CD** : GitHub propose des intégrations avec des services d'intégration continue (CI) et de déploiement continu (CD).

- **Communauté Active** : La communauté GitHub est vaste et active, offrant un accès à de nombreuses ressources et projets open source.

PyCharm

- **Environnement de Développement Intégré (IDE)** : PyCharm est un IDE puissant qui offre des fonctionnalités avancées pour le développement Python.
- **Autocomplétion et Vérification Syntaxique** : Il propose une autocomplétion intelligente et une vérification en temps réel de la syntaxe.
- **Débogage Intégré** : PyCharm dispose d'outils de débogage avancés pour faciliter le processus de débogage.
- **Gestion de Projet** : Il offre une gestion de projet avancée avec des fonctionnalités de suivi de dépendances et de packages. Edition Web et Scientifique : PyCharm prend en charge le développement Web et scientifique grâce à ses fonctionnalités étendues.

Visual Studio Code (VS Code)

- **Légèreté et Polyvalence** : VS Code est un éditeur de code léger mais puissant, adapté à une variété de langages de programmation. Et ne nécessite pas d'accès root pour l'installer.
- **Extensions** : Il offre une multitude d'extensions qui permettent aux utilisateurs de personnaliser leur environnement de développement.
- **Intégration Git** : VS Code propose une intégration native avec Git pour la gestion de versions.
- **Débogage Intégré** : Il dispose d'un débogueur intégré et de fonctionnalités de profilage pour faciliter le développement.

Spyder

- **Scientifique et Analytique** : Spyder est conçu pour le développement scientifique et analytique avec une intégration étroite de bibliothèques telles que NumPy et Matplotlib.
- **IDE pour Data Science** : Il propose des fonctionnalités spécifiques à la data science, comme la visualisation des données et l'exécution pas à pas.
- **Environnement IPython** : Spyder intègre un terminal IPython interactif pour faciliter les expérimentations et l'analyse.

5.3 Power BI

Qu'est-ce que Power BI ?

Power BI est une plateforme de business intelligence développée par Microsoft qui permet aux utilisateurs de visualiser et d'analyser leurs données de manière interactive et efficace. Elle offre des outils de création de rapports, de tableaux de bord et d'analyses avancées, aidant ainsi les entreprises à prendre des décisions éclairées basées sur les données.

Fonctionnalités clés

- **Connectivité étendue** : Power BI peut se connecter à une variété de sources de données, allant des bases de données aux services cloud, permettant ainsi aux utilisateurs d'explorer des ensembles de données divers.
- **Visualisations interactives** : La plateforme propose une large gamme de visualisations, telles que des graphiques, des cartes, des tableaux croisés dynamiques, et plus encore, pour donner vie à vos données et les rendre compréhensibles en un coup d'œil.
- **Analyse avancée** : Power BI permet l'application de fonctions et de calculs complexes pour l'analyse statistique, la modélisation prédictive et d'autres analyses avancées.
- **Tableaux de bord interactifs** : Créez des tableaux de bord interactifs en regroupant des visualisations, des filtres et des éléments interactifs pour permettre aux utilisateurs d'explorer les données selon leurs besoins.

- **Partage et collaboration** : Power BI permet de partager facilement des rapports et des tableaux de bord avec d'autres utilisateurs, qu'ils soient dans l'entreprise ou à l'extérieur.
- **Intégration avec Microsoft** : En tant que produit Microsoft, Power BI s'intègre parfaitement avec d'autres applications Microsoft telles que Excel, SharePoint et Azure.

Le langage DAX, Power Query et le langage M

Dans Power BI, vous avez trois langages différents qui jouent des rôles spécifiques dans le traitement et la manipulation des données : le langage DAX (Data Analysis Expressions), le langage M et Power Query. Voici les différences entre ces trois langages :

- **Langage DAX (Data Analysis Expressions)** :
 - Fonction : Le langage DAX est principalement utilisé pour créer des formules et des expressions qui effectuent des calculs sur les données, comme des agrégations, des mesures et des calculs personnalisés.
 - Utilisation : Il est utilisé pour créer des mesures, des colonnes calculées et des tableaux calculés, ainsi que pour créer des calculs dans les graphiques, les tableaux croisés dynamiques et autres visualisations.
 - Contexte : Les calculs DAX tiennent compte du contexte de ligne et du contexte de sommation dans les visualisations.
 - Exemple : `SUM(Ventes[Montant])` pour calculer la somme des montants de ventes.
- **Langage M** :
 - Fonction : Le langage M est utilisé principalement pour la transformation des données, y compris l'importation, la fusion, le nettoyage et la manipulation des données brutes.
 - Utilisation : Il est utilisé dans les étapes de transformation des données dans Power Query Editor, où vous pouvez appliquer des opérations de nettoyage, de filtrage, de regroupement et d'ajout de colonnes.
 - Exemple : `Table.SelectColumns(Table1, "Colonne1", "Colonne2")` pour sélectionner uniquement certaines colonnes de la table.
- **Power Query** :
 - Fonction : Power Query est la technologie sous-jacente pour la gestion des données dans Power BI, utilisant le langage M pour la transformation des données.
 - Utilisation : Il est utilisé pour charger et transformer les données provenant de sources diverses, comme des bases de données, des fichiers CSV, des feuilles Excel, etc. Étapes de Transformation : Power Query Editor permet de définir des étapes de transformation sur les données, y compris l'application de filtres, la fusion de tables, la suppression de lignes, etc.
 - Exemple : Utilisation d'opérations visuelles et d'expressions M pour nettoyer et préparer les données.

En résumé, le langage DAX est utilisé pour les calculs et les mesures dans les visualisations, le langage M est utilisé pour la transformation des données dans Power Query, et Power Query est l'outil qui vous permet de charger et de transformer vos données depuis différentes sources. Chacun de ces langages a un rôle spécifique dans le processus de création d'analyses dans Power BI.

Versions de Power BI

Power BI propose différentes versions pour répondre aux besoins des utilisateurs :

- **Power BI Desktop** : L'outil de création et de développement où vous concevez des rapports et des visualisations.
- **Power BI Service** : Une plateforme basée sur le cloud pour publier, partager et collaborer sur des rapports et des tableaux de bord.
- **Power BI Mobile** : Une application mobile pour accéder à vos rapports et visualisations depuis n'importe où.

Connexion aux données

Lancez Power BI Desktop et connectez-vous à vos sources de données, telles que des bases de données ou des fichiers Excel. Utilisez les connecteurs disponibles pour importer vos données dans Power BI.

Création de rapports

- **Exploration des données** : Parcourez vos données pour en comprendre la structure et les relations entre les différentes tables.
- **Création de visualisations** : Choisissez parmi les visualisations disponibles (graphiques, cartes, etc.) et glissez-déposez les champs de données pour les créer.
- **Ajout de filtres** : Appliquez des filtres pour focaliser vos visualisations sur des segments spécifiques de données.

Création de tableaux de bord

- **Construction de tableaux de bord** : Combinez vos visualisations, filtres et éléments interactifs pour créer un tableau de bord complet.
- **Interactivité** : Utilisez les options de filtrage croisé pour permettre aux utilisateurs de sélectionner des éléments et de voir les effets sur l'ensemble du tableau de bord.

Transformation de données

- **Étapes de transformation** : Utilisez l'éditeur de requêtes pour nettoyer, transformer et modeler vos données avant de les visualiser.
- **Formules DAX** : Apprenez à utiliser les formules DAX (Data Analysis Expressions) pour créer des calculs et des mesures personnalisés.

Modélisation de données

- **Relations entre tables** : Établissez des relations entre les tables pour créer un modèle de données cohérent et permettre des analyses croisées.
- **Tableaux de faits et de dimensions** : Comprenez la distinction entre les tableaux de faits (mesures quantitatives) et les tables de dimensions (attributs des données).

Analyse avancée

- **Calculs complexes** : Utilisez DAX pour réaliser des calculs complexes, tels que les agrégations, les pourcentages, les ratios, etc.
- **Visualisations personnalisées** : Intégrez des visualisations personnalisées pour répondre à des besoins spécifiques de l'entreprise.

Partage et collaboration

- **Publication sur le service Power BI** : Publiez vos rapports et tableaux de bord sur Power BI Service pour permettre à d'autres utilisateurs d'y accéder.
- **Partage et collaboration** : Partagez vos rapports avec des collègues et configurez les autorisations d'accès.

Qu'est-ce qu'une mesure dans Power BI ?

Dans Power BI, une mesure (ou "measure" en anglais) est une valeur calculée basée sur des données existantes dans votre modèle de données. Les mesures sont utilisées pour effectuer des calculs agrégés, des opérations mathématiques ou des analyses plus avancées sur vos données. Contrairement aux colonnes calculées, les mesures ne stockent pas physiquement les valeurs dans la base de données, mais elles sont calculées à la volée en fonction des filtres et des

contextes d'analyse.

Les mesures sont particulièrement utiles lorsque vous avez besoin de réaliser des calculs dynamiques et interactifs dans vos visualisations et vos tableaux de bord. Elles vous permettent de créer des agrégations, des totaux, des pourcentages, des ratios et d'autres indicateurs clés de performance (KPIs) en fonction des besoins spécifiques de votre analyse.

Pour créer une mesure dans Power BI, vous utilisez le langage DAX (Data Analysis Expressions), qui est spécifique à Power BI et à d'autres produits Microsoft tels que Excel. Voici un exemple simple de mesure DAX pour illustrer :

Supposons que vous ayez une table de ventes avec une colonne "Montant" et une colonne "Quantité". Vous pouvez créer une mesure pour calculer le chiffre d'affaires total en multipliant la quantité par le montant :

Chiffre d'affaires = SUM(Ventes[Montant] * Ventes[Quantité])

Une fois la mesure créée, vous pouvez l'utiliser dans vos visualisations, tels que des graphiques, des tableaux croisés dynamiques et des cartes. Les mesures sont mises à jour automatiquement lorsque vous appliquez des filtres ou modifiez les dimensions de vos visualisations, garantissant ainsi que vos analyses sont toujours pertinentes et précises.

En résumé, une mesure dans Power BI est une valeur calculée en utilisant le langage DAX, qui permet d'effectuer des calculs dynamiques et interactifs sur vos données pour obtenir des insights et des indicateurs pertinents pour votre analyse.

Contexte et agrégation

Dans Power BI, le terme "contexte" fait référence à l'ensemble des filtres et des conditions qui définissent les conditions dans lesquelles une expression DAX (Data Analysis Expressions) est évaluée. Le contexte joue un rôle crucial dans la façon dont les mesures et les calculs sont effectués dans Power BI.

Il y a deux principaux types de contexte dans Power BI : le contexte de ligne et le contexte de sommation.

Contexte de Ligne

Le contexte de ligne se réfère au filtre ou à la ligne spécifique dans une visualisation pour laquelle une mesure est calculée. Lorsque vous ajoutez une mesure à une visualisation, chaque ligne de cette visualisation a son propre contexte de ligne. Les calculs de la mesure sont effectués en tenant compte de ce contexte.

Par exemple, si vous avez une mesure qui calcule la somme des ventes, le contexte de ligne détermine pour quelle ligne spécifique de la visualisation la somme est calculée. Cela signifie que la mesure est évaluée individuellement pour chaque ligne de la visualisation, en tenant compte des filtres appliqués à cette ligne.

Contexte de Sommation

Le contexte de sommation se réfère aux filtres et aux regroupements appliqués à une visualisation qui agrège les données, comme un tableau croisé dynamique ou un graphique. Lorsque vous effectuez une agrégation, comme la somme ou la moyenne, le contexte de sommation définit quelles données sont incluses dans l'agrégation.

Par exemple, si vous regroupez les données par catégorie de produits dans un tableau croisé dynamique, le contexte de sommation définit quelles lignes de données sont regroupées et incluses dans le calcul de la somme des ventes pour chaque catégorie.

Power BI ne stock pas de calcul intermédiaire, mais stock les transformations à opérer sur les données d'entrée pour générer les différentes visualisations. Le "debuggeage" y est moins direct comme pour un code Fortran ou Python. C'est pourquoi il faut comprendre comment le contexte et les fonction d'agrégation s'articule.

- **Contexte de Filtrage :** Lorsque vous utilisez une fonction d'agrégation, comme SUM ou AVERAGE, dans une mesure DAX, la fonction est appliquée dans le contexte de filtrage actuel. Cela signifie que seules les lignes qui correspondent aux filtres appliqués contribuent au calcul de l'agrégation.

- **Contexte de Ligne** : Lorsque vous utilisez une mesure dans une visualisation, le contexte de ligne de la visualisation affecte le calcul de la mesure. Chaque ligne de la visualisation a son propre contexte, ce qui signifie que la mesure est calculée séparément pour chaque ligne, en tenant compte des filtres appliqués à cette ligne.
- **Contexte de Sommation** : Lorsque vous placez une mesure d'agrégation dans une visualisation qui agrège les données (par exemple, un tableau croisé dynamique), les valeurs agrégées sont elles-mêmes agrégées dans le contexte de sommation de la visualisation. Par exemple, si vous avez une mesure pour calculer la somme des ventes, et que vous regroupez les données par région, la somme des ventes sera calculée pour chaque région.
- **Agrégation des Agrégations** : Dans certaines situations, des agrégations peuvent être imbriquées. Par exemple, si vous avez une mesure pour calculer la somme des ventes par région et une autre mesure pour calculer la moyenne de ces sommes par catégorie de produits, la seconde mesure agrège les agrégations de la première mesure.
- **Tables de Date** : Lorsque vous travaillez avec des tables de dates, vous pouvez utiliser des fonctions telles que DATESYTD (year-to-date) ou TOTALYTD pour effectuer des calculs agrégés spécifiques aux périodes temporelles, comme les totaux cumulatifs depuis le début de l'année.
- **Contexte de Filtres de Page** : Les filtres appliqués aux pages elles-mêmes peuvent également affecter le contexte de calcul des mesures. Les mesures tiennent compte des filtres de page lors du calcul

Conclusion

Power BI offre une solution complète pour la visualisation et l'analyse de données, aidant les entreprises à prendre des décisions basées sur des informations pertinentes. En explorant ses fonctionnalités, de la connexion de données à la création de visualisations avancées, vous pouvez tirer parti de cette plateforme pour transformer des données brutes en insights exploitables.

5.4 Tableau

Qu'est-ce que Tableau ?

Tableau est un puissant outil de visualisation de données et de business intelligence qui permet aux utilisateurs d'explorer, d'analyser et de présenter des données de manière visuellement captivante. Il vous permet de créer des tableaux de bord, des rapports et des graphiques interactifs et partageables qui vous aident à prendre des décisions éclairées en fonction des insights tirés des données.

Fonctionnalités clés

- **Interface de glisser-déposer** : Tableau propose une interface intuitive qui permet aux utilisateurs de créer des visualisations en faisant simplement glisser et déposer des champs de données sur la toile.
- **Compatibilité avec diverses sources de données** : Il peut se connecter à différentes sources de données, notamment des bases de données, des feuilles de calcul, des services cloud, etc., ce qui facilite le travail avec des ensembles de données diversifiés.
- **Tableaux de bord interactifs** : Avec Tableau, vous pouvez créer des tableaux de bord interactifs qui permettent aux utilisateurs de filtrer, d'approfondir et d'explorer les données par eux-mêmes, favorisant ainsi un environnement d'analyse en libre-service.
- **Collaboration en temps réel** : Tableau prend en charge la collaboration en temps réel, permettant à plusieurs utilisateurs de travailler sur le même projet simultanément, de partager des insights et de prendre des décisions éclairées ensemble.
- **Analyse avancée** : Il offre des fonctionnalités pour l'analyse avancée, notamment l'analyse statistique, les prévisions, l'identification des tendances et la cartographie géographique.
- **Mises à jour automatisées** : Tableau vous permet de configurer des rafraîchissements automatiques des données, garantissant que vos visualisations reflètent toujours les informations les plus récentes.

Versions de Tableau

Tableau propose différentes versions adaptées aux besoins de différents utilisateurs :

- **Tableau Desktop** : L'environnement de création et de développement où vous créez des visualisations.
- **Tableau Server** : Une plateforme pour le partage, la collaboration et la gestion du contenu Tableau au sein d'une organisation.
- **Tableau Online** : Une version basée sur le cloud de Tableau Server qui permet un partage et un accès faciles aux visualisations de n'importe où.

Connexion aux données

Lancez Tableau Desktop et connectez-vous à votre source de données. Choisissez parmi divers connecteurs de données, tels que les bases de données, les feuilles de calcul, les services cloud, etc. Importez les données en sélectionnant les tables, les feuilles ou les fichiers.

Création de visualisations

Dimensions et mesures : Comprenez la différence entre les dimensions (données catégoriques) et les mesures (données quantitatives).

Marques et filtres : Apprenez à utiliser les marques (points de données) pour créer des éléments visuels et à appliquer des filtres pour vous concentrer sur des sous-ensembles de données spécifiques.

Visualisations par glisser-déposer : Créez votre première visualisation en faisant glisser des dimensions et des mesures sur la toile. Par exemple, créez un diagramme à barres pour comparer les ventes dans différentes régions.

Création de tableaux de bord

Espace de travail des tableaux de bord : Explorez l'espace de travail des tableaux de bord où vous combinerez plusieurs visualisations dans une mise en page cohérente.

Interactivité : Ajoutez des filtres, des paramètres et des actions pour créer une expérience interactive pour les utilisateurs.

Formatage et conception : Personnalisez l'apparence de votre tableau de bord en ajustant les couleurs, les polices et la mise en page pour améliorer la clarté et l'esthétique.

Cartographie et visualisation géographique

Visualisations géographiques : Apprenez à créer des cartes qui affichent des données géographiquement, comme la représentation des ventes par région sur une carte.

Géocodage personnalisé : Importez des données géographiques personnalisées pour visualiser des emplacements non couverts par la cartographie par défaut.

Champs et expressions calculés

Champs calculés : Créez des calculs personnalisés à l'aide de formules pour obtenir de nouveaux insights à partir de données existantes.

Calculs de tableaux : Effectuez des calculs qui tiennent compte des données de votre visualisation, comme les totaux cumulés ou les moyennes mobiles.

Fusion de données

Fusion de données : Combine les données de plusieurs sources en une seule visualisation pour obtenir des insights complets.

Relations : Établissez des relations entre les sources de données pour effectuer des analyses sur différents ensembles de données.

Publication et partage

Publication sur Tableau Server/Online : Apprenez à publier vos visualisations sur Tableau Server ou Tableau Online pour les partager avec vos collègues.

Intégration de visualisations : Intégrez les visualisations Tableau dans des sites web, des blogs ou des applications pour toucher un public plus large.

les fonctions d'agrégation

Dans Tableau Software, les fonctions d'agrégation jouent un rôle crucial dans le calcul des agrégations de données et la création de mesures. Voici quelques règles importantes à connaître lors de l'utilisation des fonctions d'agrégation dans Tableau :

- **Contexte de VizQL :** Tableau utilise le langage VizQL pour générer des visualisations. Lorsque vous utilisez une fonction d'agrégation dans une visualisation, la fonction est appliquée dans le contexte de la visualisation, en fonction des dimensions et des marques visuelles présentes.
- **Fonctions d'Aggrégation Standard :** Tableau offre un ensemble standard de fonctions d'agrégation telles que SUM, AVG, MIN, MAX, COUNT, etc. Ces fonctions calculent des agrégations basées sur les valeurs des lignes dans le contexte de la visualisation.
- **Grand-Total :** Lorsque vous ajoutez une mesure à la partie "Texte" d'une visualisation (par exemple, une étiquette de grand total), Tableau applique automatiquement une agrégation à cette mesure pour calculer le grand total. L'agrégation par défaut dépend du type de mesure (par exemple, SUM pour les mesures numériques).
- **Groupes et Sets :** Si vous créez des groupes, des ensembles ou des hiérarchies personnalisées dans Tableau, les fonctions d'agrégation agissent en fonction de la définition de ces groupes. Par exemple, un groupe personnalisé pourrait avoir une agrégation différente de celle des mesures brutes.
- **Calculs de Tableau Calculé :** Vous pouvez créer des calculs de tableau calculé (calculated fields) dans Tableau pour effectuer des opérations de calcul personnalisées. Les fonctions d'agrégation peuvent être utilisées dans ces calculs pour créer des mesures personnalisées.
- **Utilisation de la Clause "Compute Using" :** Vous pouvez spécifier comment une fonction d'agrégation doit être calculée en utilisant la clause "Compute Using". Cela vous permet de contrôler comment l'agrégation est déterminée en fonction des dimensions de la visualisation.
- **Relations entre les Sources de Données :** Si vous travaillez avec plusieurs sources de données liées, Tableau peut gérer automatiquement les agrégations à travers les relations. Les fonctions d'agrégation sont alors calculées en fonction des dimensions de la source de données principale.

En résumé, les fonctions d'agrégation dans Tableau sont appliquées dans le contexte de la visualisation et sont influencées par les dimensions, les groupes, les ensembles et les relations de données. Comprendre comment ces règles fonctionnent vous permettra de créer des mesures précises et cohérentes dans vos visualisations et vos tableaux de bord.

Il existe 2 types d'agrégation dans Tableau :

- **SUM :**

La fonction SUM est utilisée pour calculer la somme d'une colonne numérique. Elle effectue l'agrégation dans le contexte de la visualisation (par exemple, un graphique ou un tableau croisé dynamique). La somme est calculée en additionnant toutes les valeurs de la colonne spécifiée. Elle ne prend pas en compte la notion de fenêtre ou d'ordre particulier dans les données.

SUM([Ventes])

- **WINDOW_SUM :**

La fonction WINDOW_SUM est utilisée pour calculer des sommes cumulatives sur une fenêtre spécifiée de données. Elle nécessite une table visuelle pour définir le contexte de la fenêtre, et elle effectue les calculs dans ce contexte. La somme est calculée en additionnant les valeurs dans la fenêtre spécifiée (souvent basée sur des dimensions telles que le temps). Elle est utilisée pour obtenir des agrégations cumulatives, par exemple, la somme cumulée des ventes jusqu'à un certain point. Résultat à noter, la fonction ne s'applique que sur un résultat d'agrégation.

WINDOW_SUM(SUM([Ventes]), -1, 0)

Conclusion

Tableau permet aux utilisateurs de transformer des données complexes en insights exploitables en fournissant une interface conviviale pour créer des visualisations interactives et des tableaux de bord. Avec ses nombreuses fonctionnalités, de la connexion de données à l'analyse avancée, Tableau est un outil essentiel pour les individus et les organisations souhaitant prendre des décisions basées sur les données. À mesure que vous plongerez plus en profondeur dans Tableau, vous découvrirez son plein potentiel pour transformer les données en connaissances.

Chapter 6

PiML

6.1 Physics informed Machine Learning (PiML)

Voici un tableau regroupant des stratégies d'intégration pour les modèles basés sur la physique et l'apprentissage automatique.

- Il existe deux grandes catégories de catégories de prévision, à savoir : A intégration séquentielle et B : modèles hybrides. Les diagrammes d'architecture sont dessinés pour décrire principalement les flux de travail de prédiction.
- Les points d'entrée des données d'entraînement expérimentales et informatiques sont explicitement indiqués.
- Les échelles de couleur indiquent, de manière relative, le rapport attendu entre les informations capturées par la partie d'apprentissage automatique et la partie physique d'un modèle intégré.
- Dans tous les schémas, les caractéristiques d'entrée dépendantes du temps sont contenues dans le vecteur d'apprentissage automatique.
- Dans tous les schémas, les caractéristiques d'entrée dépendantes du temps sont contenues dans le vecteur $X(t)$ et les mesures de santé cibles dépendantes du temps sont $y(t)$. $X(t)$ peut être une combinaison de données brutes de séries temporelles (par exemple, tension, température, etc.). (par exemple, la tension, le courant, la température) et/ou toute autre caractéristique dérivée.
- Dans A2, X désigne des ensembles de paramètres d'entrée pour le modèle PB définissant des conditions d'utilisation (par exemple, temps vs courant) analogues à celles des données expérimentales pour produire un ensemble de données de valeurs y PBM simulées utiles pour l'apprentissage par transfert.

L'apprentissage automatique guidé par la physique, ou physics-informed machine learning, intègre généralement des a priori physiques dans le processus d'apprentissage en minimisant une fonction de perte qui inclut à la fois un terme basé sur les données et une régularisation par équation différentielle partielle (EDP). En s'appuyant sur la formulation du problème comme une tâche de régression par noyau, des méthodes de Fourier sont utilisées pour approximer le noyau associé et proposer un estimateur exploitable qui minimise la fonction de risque guidée par la physique. Cette approche est connue sous le nom d'apprentissage par noyau guidé par la physique, ou physics-informed kernel learning (PIKL). Ce cadre offre des garanties théoriques, permettant de quantifier l'impact du a priori physique sur la vitesse de convergence. Nous démontrons la performance numérique de l'estimateur PIKL à travers des simulations, à la fois dans le contexte de la modélisation hybride et dans la résolution d'EDP.

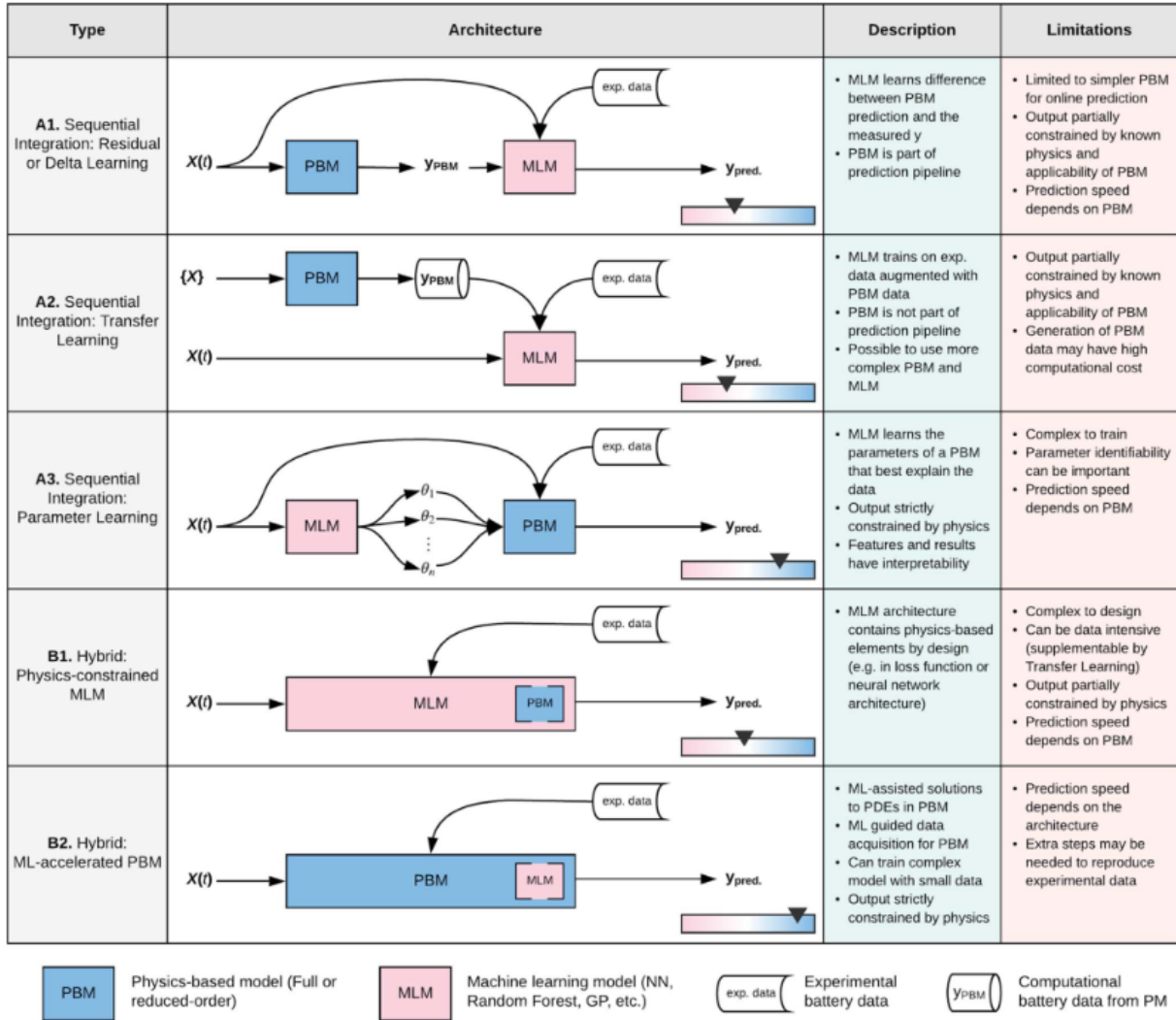


Figure 6.1: Physics informed Machine Learning - Source article

Chapter 7

Environnement ML

7.1 Les tests logicielles

Pour vérifier le bon fonctionnement d'un système, comme les clignotants d'une voiture, il faut tester chaque composant (ampoules, batterie, commutateur) individuellement. En développement logiciel, cette approche est structurée sous la forme d'une **pyramide des tests**, allant des tests les plus rapides et isolés (unitaires) aux tests les plus lents et intégrés (end-to-end). Chaque niveau de la pyramide répond à un besoin spécifique : les **tests unitaires** valident des fonctions isolées, les **tests d'intégration** vérifient les interactions entre composants, et les **tests end-to-end (E2E)** simulent un parcours utilisateur complet. L'objectif est de détecter les erreurs dès leur apparition, en privilégiant les tests rapides et automatisables à la base de la pyramide, tout en limitant les tests coûteux et complexes au sommet.

7.1.1 Les tests unitaires : fondations et outils

Les **tests unitaires** forment la base de la pyramide. Ils vérifient le comportement d'une fonction ou d'une méthode **sans dépendre d'éléments externes**, ce qui les rend rapides et faciles à automatiser. Par exemple, une fonction qui extrait les nombres pairs d'une liste doit être testée avec différents types de données (listes vides, valeurs non entières, etc.). Deux outils populaires en Python permettent de les implémenter :

- **Doctest** : Intégré directement dans les docstrings des fonctions, il permet de définir des exemples de code et leurs résultats attendus. Simple à utiliser, il signale automatiquement les fonctions non testées.
- **Pytest** : Plus flexible, ce framework utilise des assertions et des décorateurs comme `@pytest.mark.parametrize` pour tester plusieurs jeux de données en une seule fonction. Il offre aussi des **mocks** (simulations de dépendances externes) via `pytest-mock`, essentiels pour isoler le code testé. Par exemple, on peut simuler le comportement d'une fonction externe ou d'une classe (comme un arbre de décision dans `scikit-learn`) pour éviter les effets de bord.

7.1.2 Tests d'intégration et fonctionnels : valider les interactions

Les **tests d'intégration** vérifient que les composants développés séparément (par exemple, par différents développeurs) fonctionnent correctement ensemble. Dans une application bancaire, un test d'intégration pourrait valider qu'un retrait modifie bien le solde du compte. Les **tests fonctionnels**, quant à eux, se concentrent sur une fonctionnalité spécifique (comme un processus d'authentification) et s'appuient sur les spécifications métiers plutôt que sur le code. Par exemple, dans une API FastAPI, un test fonctionnel pourrait créer un utilisateur, puis vérifier sa présence dans la base de données. Ces tests utilisent des outils comme `TestClient` pour simuler des requêtes HTTP et valider les réponses, sans nécessiter de connaître l'implémentation interne.

7.1.3 Les tests end-to-end et l'équilibre de la stratégie de test

Au sommet de la pyramide, les **tests E2E** reproduisent un scénario utilisateur complet, de l'interface graphique à la base de données. Bien qu'indispensables pour valider l'expérience utilisateur (comme un processus d'authentification), ils sont lents, coûteux à maintenir et difficiles à déboguer en cas d'échec. Leur rôle est complémentaire : ils ne remplacent

pas les tests unitaires ou d'intégration, mais **garantissent la cohérence globale** du système. Une stratégie de test équilibrée combine tous ces niveaux pour détecter les bugs tôt (via les tests unitaires), valider les interactions (tests d'intégration), et confirmer le bon fonctionnement global (E2E). Des outils comme `pytest-cov` mesurent la **couverture de code**, tandis que des plugins (ex: `pytest-xdist`) accélèrent l'exécution des tests. En structurant les tests par module et en utilisant des *fixtures* (contextes réutilisables), on optimise la maintenabilité et la fiabilité du code, tout en facilitant l'ajout de nouvelles fonctionnalités sans régression.

7.2 IA générative

Une Intelligence Artificielle Générative (IA générative) est un type de modèle d'intelligence artificielle conçu pour générer de nouvelles données qui ressemblent à celles existantes dans un ensemble d'apprentissage. Contrairement aux modèles d'apprentissage supervisé qui apprennent à faire des prédictions basées sur des données existantes, les IA génératives créent de nouvelles données qui ne font pas nécessairement partie de l'ensemble d'apprentissage initial. Ces modèles sont utilisés dans divers domaines tels que la génération d'images, de texte, de musique et plus encore.

7.2.1 Variational Autoencoder (VAE)

Les Variational Autoencoders (VAE) sont des modèles d'apprentissage automatique qui combinent des techniques d'autoencodage et de modélisation probabiliste pour apprendre une représentation continue et structurée des données. Les VAE sont conçus pour encoder des données complexes en compressant l'information importante dans un espace latent de dimensions réduites. Ils introduisent un processus stochastique d'échantillonnage dans la phase d'encodage, permettant ainsi de modéliser la distribution latente des données. Ce processus probabiliste permet aux VAE de générer de nouvelles données en effectuant des échantillonnages dans l'espace latent. Les VAE sont largement utilisés dans la génération d'images, la réduction de dimension et la création de données synthétiques tout en favorisant la régularisation et la découverte de caractéristiques pertinentes dans les données.

- **Fonctionnement** : Un VAE est un type d'autoencodeur qui apprend une représentation latente des données d'entrée et peut générer de nouvelles données en effectuant des échantillonnages dans l'espace latent.
- **Caractéristiques** : Les VAE sont basés sur la modélisation probabiliste et utilisent des techniques d'échantillonnage stochastique pour générer des données nouvelles et diverses.
- **Avantages** : Ils sont adaptés à la génération d'images et de données continuellement variables.
- **Inconvénients** : La qualité des échantillons générés peut être moins constante que dans les GANs.
- **Applications** : Génération d'images, interpolation de données manquantes.

7.2.2 Generative Adversarial Network (GAN)

Les Generative Adversarial Networks (GAN) sont une classe de modèles d'apprentissage automatique qui reposent sur un cadre de jeu entre deux réseaux neuronaux : un générateur et un discriminateur. Le générateur crée des données synthétiques à partir d'un bruit aléatoire, tandis que le discriminateur évalue la probabilité que les données proviennent soit du générateur soit du véritable ensemble de données. Les deux réseaux s'améliorent mutuellement par un processus d'entraînement itératif, où le générateur cherche à tromper le discriminateur en produisant des données de plus en plus réalistes, tandis que le discriminateur s'ajuste pour mieux distinguer les données réelles des données générées. Les GAN sont largement utilisés pour la génération d'images, de vidéos, de sons et d'autres types de données synthétiques de haute qualité, tout en stimulant l'innovation dans le domaine de l'intelligence artificielle générative.

- **Fonctionnement** : Un GAN consiste en deux réseaux neuronaux, le générateur et le discriminateur, qui s'affrontent dans un jeu du chat et de la souris. Le générateur tente de créer des données réalistes, tandis que le discriminateur tente de les différencier des données réelles. **Caractéristiques** : Les GANs génèrent des données en utilisant un processus itératif et sont célèbres pour produire des échantillons de haute qualité.
- **Avantages** : Ils produisent des résultats visuellement convaincants et sont utilisés pour la génération d'images, vidéos, sons, etc.

- **Inconvénients** : Le processus d'entraînement peut être délicat et nécessite un équilibre entre le générateur et le discriminateur.
- **Applications** : Génération d'images, synthèse de données réalistes.

7.2.3 Diffusion models

Les modèles de diffusion sont des modèles probabilistes avancés utilisés pour générer des données réalistes en modélisant la transformation progressive d'une distribution de probabilité initiale en une distribution finale. Ces modèles captent les changements subtils et les détails fins des données en appliquant un processus itératif de diffusion. Pendant chaque étape de diffusion, les données sont transformées tout en maintenant une structure cohérente et réaliste. Les modèles de diffusion sont particulièrement efficaces pour générer des images haute résolution et réalistes, en gérant des détails complexes et des textures riches. Ils ont trouvé des applications dans la génération d'images, la compression d'images et la synthèse de données de haute qualité, tout en contribuant à l'avancement de l'intelligence artificielle générative.

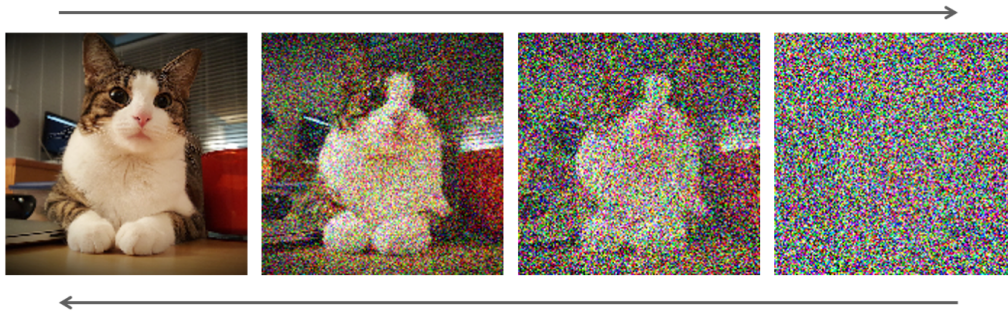


Figure 7.1: Principe de diffusion.

Un modèle de diffusion en intelligence artificielle s'inspire du concept de diffusion observé en physique. En physique, la diffusion est le processus par lequel les particules se déplacent aléatoirement dans un milieu, se dispersant du plus concentré au moins concentré, jusqu'à ce qu'un équilibre soit atteint. Ce processus est régi par la loi de Fick, une équation différentielle qui décrit comment une substance se diffuse à travers un milieu.

Dans le contexte des modèles de diffusion en intelligence artificielle, le parallèle avec la loi de Fick réside dans le processus itératif de transformation progressive d'une distribution initiale vers une distribution finale. Tout comme les particules se déplacent pour équilibrer la concentration dans la diffusion physique, les modèles de diffusion en IA itèrent pour transformer les données en conservant une structure cohérente et réaliste. Cela peut être analogue à la manière dont une substance se propage dans un milieu en se déplaçant des régions les plus concentrées vers les moins concentrées. Dans ce contexte, les modèles de diffusion visent à capturer des caractéristiques complexes des données, tout en créant une transition fluide et naturelle entre différentes parties des distributions.

En somme, le concept de diffusion en physique et son analogue dans les modèles de diffusion en IA soulignent la capacité de ces modèles à capturer des variations subtiles tout en maintenant une certaine cohérence globale, ce qui en fait des outils puissants pour la génération de données réalistes et complexes.

- **Fonctionnement** : Un modèle de diffusion est utilisé pour générer des échantillons à partir d'une distribution initiale en appliquant un processus de diffusion itératif.
- **Caractéristiques** : Ces modèles sont souvent utilisés pour la génération d'images haute résolution et la synthèse d'échantillons réalistes.
- **Avantages** : Ils sont capables de générer des images de haute qualité et réalistes.
- **Inconvénients** : La complexité du modèle et du processus d'entraînement peut être élevée.
- **Applications** : Génération d'images haute résolution, synthèse de données complexes.

Pour aller plus loin : [LoRA Keras](#)

7.2.4 LLM

Les Language Models (LLM), ou modèles de langage, sont des modèles d'intelligence artificielle conçus pour comprendre et générer du texte de manière cohérente et contextuellement appropriée. Ils utilisent des réseaux neuronaux pour apprendre les probabilités des séquences de mots dans un langage donné, en capturant les structures grammaticales et sémantiques. Les LLM sont souvent pré-entraînés sur de vastes quantités de texte, leur permettant d'acquérir une connaissance du langage. Ils sont utilisés dans diverses applications telles que la traduction automatique, la génération de texte, la rédaction assistée par ordinateur et l'analyse de sentiments. Les modèles LLM de pointe, tels que GPT (Generative Pre-trained Transformer), ont démontré une capacité impressionnante à produire du texte fluide et pertinent, tout en suscitant des discussions sur les implications éthiques et la véracité du contenu généré. Notamment ils n'ont pas de "regard critique" sur leur réponse. Si vous faite une erreur dans votre question ou "prompt", le LLM va considérer que c'est la vérité et concevoir son univers de réponse en fonction. Sa réponse sera une réflexion de votre propre confusion (ex: confusion entre les fonctionnalités de Tableau et Power BI). En somme, la réponse sera le reflet des biais contenus dans votre question et des biais implémentés par les organisations qui l'ont développé. Les LLM sont de puissant outis, mais il vaut mieux avoir une idée de la réponse quand on pause la question.

- **Fonctionnement** : Un Language Model apprend les probabilités de séquences de mots et est capable de générer du texte cohérent en fonction du contexte.
- **Caractéristiques** : Les LLMs sont utilisés pour générer du texte naturel, et leur architecture peut être basée sur des réseaux récurrents ou des réseaux transformer.
- **Avantages** : Ils sont largement utilisés pour la génération de texte, comme la création d'articles, de dialogues, etc.
- **Inconvénients** : Les LLMs peuvent générer du texte plausible mais parfois peu cohérent ou inapproprié.
- **Applications** : Génération de texte, rédaction automatique, traduction automatique.

De plus en plus, les utilisateurs développent un regard critiques sur les résultats des LLM et demande les sources. par construction les LLM ne connaissaient pas leur source et renvoyaient vers une source "probable". Mais de nouveaux modèle apparaissent.

Pleias-RAG : Des Modèles Légers qui Citent leurs Sources L'avenir de l'IA digne de confiance pourrait ne pas dépendre de la taille des modèles, mais plutôt des choix de conception d'entraînement qui modifient fondamentalement la manière dont les modèles interagissent avec les connaissances. Aujourd'hui, Pleias-RAG-350m et Pleias-RAG-1B sont introduits : des modèles spécialisés dans le raisonnement qui réalisent ce que leurs homologues plus grands ne peuvent pas faire : citer nativement leurs sources avec une exactitude intrinsèque, tout en ayant une empreinte computationnelle réduite.

Ces modèles ultra-légers ne se contentent pas de rivaliser avec des modèles plus grands sur des tâches de raisonnement (RAG) ; ils les surpassent. Par exemple, le modèle de 350 millions de paramètres surpasse des modèles établis de 8 milliards de paramètres sur des benchmarks critiques de récupération. De plus, ils maintiennent des performances constantes dans les principales langues européennes, mettant fin à l'ère des petits modèles linguistiques centrés sur l'anglais.

Les défis fondamentaux des modèles linguistiques étaient initialement épistémologiques : ils ne "savaient" pas vraiment les choses, ils prédisaient du texte. L'ancrage des sources a souvent été traité comme un ajout, plutôt que comme une intégration dans le raisonnement. Les modèles Pleias-RAG remettent en question ce paradigme en faisant du raisonnement basé sur les sources le fondement même de leur conception, prouvant que des modèles linguistiques spécialisés et petits peuvent surpasser des modèles généralistes beaucoup plus grands sur des tâches cruciales pour le déploiement en entreprise.

Ces modèles représentent un changement de paradigme dans trois dimensions critiques :

1. **Citation au Cœur de la Génération** : Contrairement aux techniques d'attribution a posteriori, ces modèles génèrent des références pendant l'inférence, permettant un contrôle précis des sources et un raccourcissement automatique des citations.
2. **Redéfinition des Modèles Linguistiques** : Ces petits modèles surpassent des modèles 8 fois plus grands sur des benchmarks RAG standardisés, tout en maintenant des performances constantes dans les principales langues européennes.

3. **Démocratisation du Déploiement** : Avec l'exactitude intégrée dans l'architecture, ces petits modèles deviennent des alternatives viables pour des cas d'utilisation où la précision et la traçabilité sont indispensables. Certains déploiements précoces fonctionnent déjà sur des Raspberry Pi 4, soutenant des professionnels du droit dans des contextes humanitaires complexes.

Les implications industrielles vont au-delà des arguments traditionnels d'efficacité. Lorsque les organisations peuvent déployer ces modèles à grande échelle sans sacrifier la précision ou les capacités de vérification, l'équation du déploiement de l'IA change fondamentalement. L'avenir de l'IA en entreprise ne sera pas défini par des modèles qui en savent plus, mais par des modèles qui peuvent prouver ce qu'ils savent.

Pour aller plus loin :

- [Large Language Models are Zero-Shot Reasoners](#)
- [ChemCrow: Augmenting large-language models with chemistry tools](#)
- [Les usages de l'IA générative](#)

7.2.5 Les transformeurs :

- **L'architecture du transformateur :**

Un transformateur est composé de plusieurs couches, chacune contenant deux sous-couches principales : une couche d'auto-attention et une couche de feed-forward. L'auto-attention permet au modèle de pondérer l'importance de différentes parties d'une séquence d'entrée lorsqu'il traite chaque élément. Par exemple, dans une phrase, l'auto-attention permet au modèle de comprendre que le mot "il" se réfère au sujet de la phrase. La couche de feed-forward, quant à elle, applique une transformation non-linéaire aux sorties de la couche d'auto-attention.

- **Le mécanisme d'auto-attention :**

L'auto-attention est le cœur du transformateur. Elle permet au modèle de capturer les relations entre les différents éléments d'une séquence. Pour chaque élément, le modèle calcule trois vecteurs : un vecteur de requête, un vecteur de clé et un vecteur de valeur. Les vecteurs de requête et de clé sont utilisés pour calculer des scores d'attention qui indiquent l'importance de chaque élément par rapport à l'élément en cours de traitement. Les vecteurs de valeur sont ensuite pondérés par les scores d'attention pour obtenir une représentation contextuelle de l'élément.

- **Les avantages des transformateurs :**

Les transformateurs présentent plusieurs avantages par rapport aux modèles précédents :

- **Traitement parallèle** : L'auto-attention permet de traiter en parallèle toutes les positions d'une séquence, ce qui accélère considérablement l'entraînement. **Capture de dépendances à longue distance** : Les transformateurs sont capables de capturer des dépendances entre des mots éloignés dans une séquence, ce qui est particulièrement utile pour des tâches comme la traduction automatique ou la génération de texte.
- **Flexibilité** : Les transformateurs peuvent être appliqués à une grande variété de tâches de NLP, telles que la classification de texte, la traduction automatique, la génération de texte, et bien d'autres.

- **Les applications des transformateurs :**

Les transformateurs ont connu un succès fulgurant dans le domaine du NLP, mais leurs applications s'étendent bien au-delà. Ils sont utilisés dans des domaines tels que :

- **La vision par ordinateur** : Les transformateurs sont de plus en plus utilisés pour des tâches de vision par ordinateur, comme la reconnaissance d'images et la segmentation.
- **La génération de musique** : Les transformateurs peuvent être utilisés pour générer de la musique en imitant différents styles.
- **Les jeux vidéo** : Les transformateurs peuvent être utilisés pour générer des environnements de jeu plus réalistes et dynamiques.

7.2.6 Les hallucinations :

Les grands modèles de langage (LLM) sont devenus omniprésents dans notre quotidien, mais leur capacité à générer des réponses factuellement incorrectes, appelées hallucinations, reste un problème majeur. Ici on explore différentes méthodes pour détecter et mesurer ces hallucinations dans le contexte de la réponse à des questions.

- Les hallucinations : un problème complexe
Les hallucinations des LLM peuvent être causées par plusieurs facteurs, notamment l'exposition à des informations contradictoires sur Internet, l'incapacité à distinguer les sources fiables des sources non fiables, et le besoin de compléter des informations manquantes de manière convaincante. Bien que les humains puissent facilement repérer les erreurs factuelles produites par les LLM, l'automatisation de la détection des hallucinations est essentielle pour améliorer la fiabilité et la sécurité de ces modèles.
- Métriques pour détecter les hallucinations
 - **Consistency scoring:** Cette méthode évalue la cohérence des réponses générées en comparant plusieurs réponses à la même question.
 - **NLI contradiction scoring:** Cette méthode utilise un modèle de classification de langage naturel pour identifier les contradictions entre la réponse attendue et la réponse générée.
 - **HHEM scoring:** Ce modèle spécifique à la détection des hallucinations évalue la probabilité de la présence d'une hallucination dans une réponse.
 - **CoT flagging:** Cette méthode consiste à demander à un LLM de juger lui-même si une réponse contient une hallucination.
 - **Self-consistency CoT scoring:** Cette méthode combine la méthode CoT flagging avec le scoring de cohérence pour obtenir un score global de confiance.

7.2.7 L'Intelligence Artificielle Générative au Service de l'Éducation

L'intelligence artificielle (IA) a révolutionné de nombreux domaines, et l'éducation n'y fait pas exception. L'IA générative, en particulier, offre des possibilités inédites pour l'enseignement et l'apprentissage. Ce cours vise à explorer comment les enseignants peuvent utiliser les modèles de langage de grande taille (LLM - Large Language Models) pour enrichir leur pédagogie et personnaliser l'expérience d'apprentissage.

1. Introduction aux LLM en Éducation

Les LLM, tels que GPT-3, sont des systèmes d'IA capables de générer du texte cohérent et pertinent. En éducation, ces modèles peuvent être utilisés pour créer des plans de cours, des quiz, des exemples et des explications adaptés aux besoins spécifiques des étudiants. L'enseignant, en tant qu'expert, rédige une consigne précise et le modèle génère le contenu correspondant, que l'enseignant évalue et affine ensuite.

2. Création de Plans de Cours avec les LLM

Pour créer un plan de cours, l'enseignant fournit au LLM des informations sur le niveau des élèves, les objectifs d'apprentissage et les sujets à couvrir. Le modèle peut alors proposer un plan structuré incluant des accroches pédagogiques, des schémas explicatifs, une liste de vocabulaire clé et des ressources de lecture. Cela permet de gagner du temps tout en produisant un contenu personnalisé et pertinent.

3. Génération d'Exemples et d'Explications

Les LLM peuvent générer de nombreux exemples pour illustrer des concepts abstraits, rendant l'apprentissage plus concret et mémorable. Ils peuvent aussi fournir des explications claires et détaillées, adaptées au niveau de compréhension des apprenants. Cela aide les étudiants à maîtriser des concepts complexes et favorise une meilleure compréhension.

4. Évaluation et Personnalisation de l'Apprentissage

Les LLM peuvent être utilisés pour créer des quiz diagnostiques qui évaluent les connaissances et les compétences des étudiants sur un sujet donné. Les enseignants peuvent ainsi identifier les lacunes et adapter leur enseignement en conséquence. De plus, l'IA permet de personnaliser les exemples et les exercices pour chaque apprenant, offrant une expérience d'apprentissage sur mesure.

5. Défis et Limites

Bien que les LLM offrent de nombreux avantages, il est essentiel que l'enseignant évalue le contenu généré pour s'assurer de son exactitude et de sa pertinence. Il faut être vigilant face aux "hallucinations" de l'IA, où le modèle peut générer des informations incorrectes. L'enseignant doit donc toujours vérifier et valider le contenu produit par l'IA.

Par ailleurs, du fait du coût et de la consommation énergétique de ses systèmes, les administrateurs décident souvent de "brider" ses IA afin qu'elles donnent la réponse la plus courte possible. ce qui peu donner une impression de "fainéantise de l'IA" à l'utilisateur. L'utilisateur doit souvent enrichir la proposition de l'IA.

Conclusion

L'intégration des LLM dans l'éducation ouvre des horizons pédagogiques novateurs. Elle permet aux enseignants de créer du matériel didactique personnalisé et engageant, tout en économisant du temps précieux. Cependant, l'expertise humaine reste indispensable pour guider l'utilisation de ces outils et assurer la qualité de l'enseignement

7.2.8 L'Intelligence Artificielle Générative au Service du développement informatique. Cas de Github Copilot

Depuis le 30/11/2022, date de sortie grand public de ChatGPT, des outils sont de plus en plus proposé pour assister le développement informatique, le rendre accessible au plus grand nombre. Voici quelques observations qui ont été fait à ce sur sujet sur l'utilisation de l'outil que Microsoft intgre de plus en plus dans ses produits.

- Les développeurs utilisant Copilot consacrent plus d'efforts au codage et moins à la gestion de projet. Les chercheurs suggèrent que Copilot réduit le besoin pour les développeurs de demander de l'aide aux autres, leur permettant de se concentrer sur l'écriture de code. Ce changement vers le travail principal correspond aux préférences des développeurs et améliore l'efficacité du développement de logiciels open source.
- Les changements observés sont attribués à une augmentation du travail autonome et à une diminution des efforts collaboratifs. Ce changement est visible dans les types d'activités auxquelles les développeurs s'engagent. Par exemple, les utilisateurs de Copilot sont plus susceptibles de pousser des commits directement et de créer des dépôts, deux activités qui peuvent être réalisées de manière indépendante. En revanche, les activités nécessitant une coordination, telles que le fork de dépôts et la création de pull requests, diminuent légèrement. Ce schéma suggère que Copilot aide les développeurs à travailler plus efficacement en réduisant le besoin de collaboration et de communication étendues.
- Copilot encourage l'exploration et réduit la dépendance aux pratiques établies. Les développeurs ayant accès à Copilot sont plus susceptibles d'expérimenter avec de nouveaux dépôts et langages de programmation. Cette conclusion suggère que Copilot non seulement améliore l'efficacité, mais favorise également l'innovation en facilitant l'essai de nouvelles approches par les développeurs.
- L'impact de Copilot est plus prononcé pour les développeurs ayant des compétences moindres. Cette observation implique que l'IA peut aider à niveler le terrain de jeu dans le développement de logiciels open source en permettant aux développeurs moins expérimentés de contribuer plus efficacement. Les auteurs suggèrent que l'IA peut aider à combler l'écart entre les "généralistes" de haut niveau et les "spécialistes" de niveau inférieur.

Ces observations ont des implications plus larges pour l'avenir du travail dans l'économie du savoir. A savoir que :

1. L'IA générative pourrait aplatir les hiérarchies organisationnelles en permettant aux individus à tous les niveaux d'effectuer des tâches qui nécessitaient auparavant une expertise spécialisée ou une coordination étendue. Cela pourrait conduire à des organisations plus efficaces et agiles.
2. Les managers pourraient s'engager plus directement dans les tâches principales de travail avec l'aide de l'IA. Cela pourrait améliorer leur compréhension des processus de travail et leur connexion avec leurs équipes.

7.3 Des menaces nouvelles.

- Cybersécurité
 - Niveaux Fonctionnels :

* Détection des Menaces :

- Défis : Les attaquants peuvent utiliser le ML pour créer des attaques plus sophistiquées, rendant difficile la détection des nouvelles menaces.
- Solutions : Utilisation de modèles ML pour détecter des anomalies et des comportements malveillants dans les flux de données.

* **Authentification et Contrôle d'Accès :**

- Défis : Les attaquants peuvent exploiter des vulnérabilités dans les modèles d'authentification basés sur le ML, contournant les systèmes de sécurité.
- Solutions : Intégration de techniques de renforcement de la sécurité, telles que l'authentification multi-facteur.

- Niveaux Techniques :

* Attaques Adversaires :

- Défis : Les modèles ML sont vulnérables aux attaques adversaires, où des données malveillantes sont injectées pour perturber les prédictions.
- Solutions : Utilisation de techniques de renforcement de la robustesse, tels que l'entraînement adversaire.

*** Confidentialité des Données :**

- Défis : Les modèles ML peuvent mémoriser des données sensibles lors de l'entraînement, exposant la confidentialité des données.
- Solutions : Application de techniques de protection de la vie privée, comme la suppression de certaines informations des données d'entraînement.

- Santé

- Niveaux Fonctionnels :

* **Diagnostic Médical :**

- Défis : Les modèles ML de diagnostic peuvent être trompés par des données altérées, conduisant à des erreurs de diagnostic.
- Solutions : Utilisation de techniques de validation croisée et de modèles robustes.

*** Systèmes de Santé Connectés :**

- Défis : Les dispositifs médicaux connectés peuvent être ciblés par des attaques pour perturber les services de santé.
- Solutions : Renforcement de la sécurité des dispositifs IoT et mise en place de mécanismes de détection d'anomalies.

- Niveaux Techniques :

*** Vulnérabilités des Données de Santé :**

- Défis : Les données médicales sensibles peuvent être exposées à des risques si les modèles ML ne sont pas correctement sécurisés.
- Solutions : Chiffrement des données médicales, accès restreint aux informations sensibles.

- IA Générative

- Niveaux Fonctionnels :

* **Génération de Contenu :**

- Défis : Les modèles d'IA générative peuvent générer du contenu malveillant ou trompeur, comme des faux médias.
- Solutions : Mise en place de mécanismes de vérification de l'authenticité et de l'intégrité du contenu généré.

*** Protection des Données :**

- Défis : Les modèles génératifs peuvent être utilisés pour contourner les mécanismes de protection des données et la confidentialité.

- Exemple ChatGPT :
Vous exposez l'organisation à une responsabilité inconnue (et inconnaissable) en matière de brevets et de droits d'auteur.
Il n'y a pas de mécanisme de retour d'information. Si ChatGPT a tendance à générer le même type de bogues de sécurité subtils, les développeurs plus expérimentés que vous doivent perdre du temps à examiner votre code pour voir s'ils ont été introduits à nouveau. Si vous ajoutez des bogues de sécurité subtils, un développeur plus expérimenté peut vous le signaler et s'attendre à ce que vous cessiez de commettre cette erreur.
En tant que développeur junior, vous ne savez pas ce que vous ne savez pas. Si vous suivez des exemples tirés de la documentation du fournisseur, vous avez des raisons de croire que l'approche générale est saine, mais vous n'êtes pas en mesure d'en juger par vous-même. Vous n'êtes pas non plus en mesure de faire cette détermination pour le code ChatGPT.
- Solutions : Intégration de techniques de filtrage et de contrôle du contenu généré.
- Niveaux Techniques :
 - * **Attaques Adversaires dans l'IA Générative :**
 - Défis : Les modèles génératifs sont particulièrement vulnérables aux attaques adversaires, compromettant la qualité et l'intégrité des générations.
 - Solutions : Développement de techniques adversariales pour renforcer la résistance des modèles génératifs.
 - * **Protection des Modèles Génératifs :**
 - Défis : Les modèles génératifs peuvent être ciblés par des attaques de vol de modèle, compromettant leur propriété intellectuelle.
 - Solutions : Utilisation de techniques de sécurité pour détecter et prévenir les tentatives de vol de modèle.

Pour aller plus loin : [HUB France IA - Youtube](#)

7.4 L'IA et l'éthique dans le contexte de l'Europe

L'Union européenne (UE) adopte une position robuste en matière d'éthique de l'intelligence artificielle (IA) pour promouvoir un développement responsable de cette technologie émergente. Cette position repose sur plusieurs principes fondamentaux et objectifs clés visant à équilibrer l'innovation technologique avec les valeurs éthiques et les droits de l'homme.

- Respect des Droits de l'Homme : L'UE considère la protection des droits fondamentaux comme une priorité. Dans ce contexte, elle exige que les systèmes d'IA respectent les droits à la vie privée, à la non-discrimination, à la dignité humaine et à la liberté d'expression. Tout déploiement d'IA doit se conformer aux normes internationales des droits de l'homme.
- Transparence et Traçabilité : L'UE encourage la transparence dans les systèmes d'IA. Les entreprises et les développeurs d'IA sont tenus de fournir des informations claires sur la manière dont leurs systèmes fonctionnent, y compris leurs algorithmes et leur prise de décision. La traçabilité est également essentielle pour permettre une vérification indépendante des processus décisionnels.
- Gouvernance et Responsabilité : L'Union européenne préconise une gouvernance claire et une responsabilité adéquate pour les systèmes d'IA. Les développeurs doivent être tenus responsables des conséquences de leurs systèmes, et des mécanismes de contrôle doivent être en place pour surveiller les décisions prises par les systèmes d'IA.
- Qualité des Données : L'UE souligne l'importance de la qualité des données pour garantir l'éthique de l'IA. Les données utilisées pour entraîner les systèmes doivent être fiables, représentatives et exemptes de biais. L'accent est mis sur la collecte et l'utilisation responsables des données.
- Sécurité et Robustesse : L'Union européenne insiste sur la nécessité de garantir la sécurité et la robustesse des systèmes d'IA. Les systèmes doivent être conçus pour résister à la manipulation malveillante et aux attaques, garantissant ainsi leur fiabilité et leur sécurité.

- Participation Publique : L'UE promeut la participation publique et le dialogue ouvert dans l'élaboration des politiques et des réglementations liées à l'IA. Il est essentiel d'inclure les opinions des citoyens, des experts et des parties prenantes pour garantir des décisions équilibrées et démocratiques.
- Coopération Internationale : L'Union européenne cherche à jouer un rôle de leader mondial en matière d'éthique de l'IA. Elle coopère avec d'autres nations et organisations pour établir des normes internationales communes en matière d'IA éthique, favorisant ainsi une approche cohérente à l'échelle mondiale.
- Investissement dans la Recherche Éthique : L'UE encourage la recherche et le développement éthiques de l'IA. Elle soutient financièrement des projets qui visent à résoudre des problèmes éthiques spécifiques liés à l'IA, tout en favorisant l'innovation responsable.
- Éducation et Formation : L'Union européenne met l'accent sur l'éducation et la formation des professionnels de l'IA, ainsi que sur la sensibilisation du public aux enjeux éthiques. Elle cherche à garantir que toutes les parties prenantes comprennent les implications éthiques de l'IA.

En résumé, l'Union européenne adopte une approche holistique en matière d'éthique de l'IA, mettant l'accent sur les droits de l'homme, la transparence, la responsabilité, la qualité des données, la sécurité et la coopération internationale. Cette position vise à guider le développement de l'IA dans le respect des valeurs européennes et des normes internationales tout en favorisant une innovation éthique et durable.

L'association Hub France IA propose une grille d'évaluation pour évaluer la prise en compte d'aspect éthique dans des processus d'IA.

**HUB
FRANCE
IA**

Grille de maturité IA et Éthique - mai 2023

	Niveau 1 - Non maîtrisé	Niveau 2 - Initial	Niveau 3 - Défini	Niveau 4 - Managé et largement diffusé	Niveau 5 - Optimisé & Intégré
Sûreté	Vérification des vulnérabilités inexistante ou sous-évaluée	Identification des vulnérabilités limitées aux cas d'application	Identification des vulnérabilités indépendantes des cas d'application et mise en place de standard sur la robustesse, fiabilité des résultats et protection physique contre les imprévus	Approche structurée de la sécurité IA déployée à l'échelle de l'entreprise pour gérer l'ensemble des risques selon le cycle de vie	Au-delà du management des risques de sécurité, l'anticipation des modifications substantielles de l'environnement est une composante essentielle de la stratégie de sécurité et est intégrée à tous les niveaux et processus IA et data de l'entreprise
Impact durable	Les enjeux de l'impact durable ne sont pas pris en considération par l'entreprise	Identification des métriques d'évaluation et sensibilisation par rapport à l'impact durable	Application au niveau des algorithmes des méthodes de prise en compte des impacts pour les optimiser	Prendre en compte les impacts dès la première étape de conception du SIA sur les 3 impacts environnemental, social et santé	Avoir ce pipeline généralisable sur tous les cas d'application
Autonomie	L'autonomie de l'humain n'est pas pensée dans le SIA	Le niveau de délégation n'est regardé que dans certains cas d'application, au regard du contexte de déploiement et des tâches remplies	Il existe des protocoles pour qualifier le contexte de l'IA, le niveau de délégation et le rôle de l'humain. Mais ils ne sont pas appliqués systématiquement à toutes les phases du cycle de vie	Les protocoles pour qualifier le contexte de l'IA, le niveau de délégation, le recueil du consentement de l'utilisateur et le rôle de l'humain sont déployés et intégrés dans tous les projets IA. Ces protocoles sont effectifs et prévoient que l'humain reprenne le contrôle de l'IA en cas de nécessité.	L'humain est au cœur de la stratégie de l'entreprise : au-delà des contrôles nécessaires au contexte, la place de l'homme est intégrée dans le développement de la stratégie IA de l'entreprise et est en ligne avec ses valeurs et propres missions
Responsabilité humaine	La responsabilité humaine n'est pas évaluée et aucune traçabilité n'a été mise en œuvre	Il existe une procédure d'enregistrement et de traitement des événements indésirables	Au-delà des responsabilités légales, il existe une procédure qui clarifie les rôles et responsabilités éthiques, pour chaque acteur de la chaîne de valeur	Il existe un dispositif de management des risques potentiels et avérés qui couvre l'ensemble des acteurs de la chaîne de valeur, le cycle de vie de l'IA ainsi que les données utilisées. Ce processus est maintenu et documenté pour réaliser des audits réguliers.	L'anticipation des responsabilités humaines est intégrée dans l'ADN de l'entreprise et se fait au-delà de l'entreprise dans l'ensemble de l'écosystème. Au-delà des procédures d'audit, de la gouvernance et de la gestion proactive des risques potentiels, il y a une culture des responsabilités humaines (e.g programmes d'éducation permanent, forum de communication, etc.)
Explicabilité	Les principes de transparence, d'explicabilité et d'interprétation sont sous-évalués et peu pris en compte	Certains cas d'application existent où des modèles sont plus transparents et des méthodes d'interprétabilité sont appliquées	Des méthodes sont développées et existent pour expliquer la conception du modèle, sa performance, les résultats obtenus ainsi que les limites techniques et choix techniques. Mais la documentation et l'utilisation de ces méthodes restent hétérogènes.	De solides standards sont systématiquement appliqués pour expliquer la conception du modèle, sa performance, les résultats obtenus ainsi que les limites techniques et choix techniques.	Les principes de transparence, d'explicabilité et d'interprétation font partie intégrante des cycles de conception de l'IA et ils sont revus régulièrement pour anticiper tout risque notamment éthique.
Équité	L'interaction humain machine n'est pas appréhendée, ni aucune appréhension des biais n'est appliquée	Des approches de détection et correction des biais sont déployées pour certains modèles	Lorsque l'IA aboutit à des conséquences négatives pour l'utilisateur, un humain a la possibilité de vérifier les biais et doit valider la décision	Les méthodes de détection de biais sont disponibles et systématiquement utilisées pour tout modèle IA	L'utilisation de méthodes de détection de biais, l'implication d'équipes diverses et la révision régulière des modèles sont systématiques
Respect de la vie privée	Peu d'attention aux aspects de vie privée et/ou confidentialité des données	Seulement des contrôles légalement obligatoires sont appliqués pour chaque cas traité	Toutes les étapes du cycle de vie de la donnée et du SIA respectent les obligations réglementaires dont celles autres que le respect de la vie privée	Mise en place d'une gouvernance afin d'intégrer dans la méthodologie projet des vérifications, des procédures et des comités opérationnels	Mettre en place un écosystème actif lié à la protection des utilisateurs par les données et le SIA. L'entreprise va au-delà de ses obligations réglementaires, de gouvernance et de révision. Et participation externe à la reconnaissance de leur ligne de conduite

Figure 7.2: Hub France IA - source

Des normes ISO sont sorties sur le sujet, voici une catégorisation de quelques normes et de leurs applications.

Objectif Éthique	Normes ISO de Référence	Exemple de KPI	Comment Cela Influence l'Éthique de l'IA
Équité	- ISO12791 : Fournit des méthodes pour identifier et atténuer les biais dans les ensembles de données et les modèles. - ISO5339 : Soutient les évaluations de l'équité par l'engagement des parties prenantes.	- Pourcentage d'incidents de biais identifiés résolus. - Variance des performances du modèle entre les groupes démographiques.	Assure que les ensembles de données et les modèles évitent les résultats discriminatoires et renforcent la confiance.
Transparence	- ISO5339 : Encourage le développement de systèmes d'IA explicables. - ISO38507 : Recommande une gouvernance pour garantir la traçabilité des décisions de l'IA.	- Satisfaction des parties prenantes avec l'explicabilité du système. - Pourcentage de systèmes d'IA avec des processus de décision traçables.	Fournit de la clarté sur la prise de décision de l'IA, favorisant la confiance des utilisateurs et la conformité réglementaire.
Responsabilité	- ISO38507 : Met l'accent sur les structures de gouvernance pour maintenir la responsabilité organisationnelle des résultats de l'IA. - ISO37301 : Soutient la surveillance de la conformité et les mécanismes de rétroaction.	- Nombre de décisions de l'IA annulées. - Pourcentage de systèmes d'IA audités pour la conformité.	Maintient la supervision humaine et assure que les organisations restent responsables des résultats du système.
Atténuation des Risques	- ISO37301 : Recommande des évaluations des risques axées sur la conformité. - ISO12791 : Met en avant l'importance de la détection des biais pour éviter les violations éthiques ou légales.	- Pourcentage d'évaluations des risques complétées avant le déploiement. - Économies de coûts grâce à la prévention des violations éthiques ou légales.	Aborde proactivement les risques potentiels, réduisant les préjudices financiers et réputationnels.
Impact Sociétal	- ISO38507 : Encourage les évaluations des implications sociétales de l'IA. - ISO5339 : Met en avant l'engagement des parties prenantes pour aligner l'IA avec les valeurs sociétales.	- Analyse des sentiments de la perception publique concernant les systèmes d'IA. - Nombre de partenariats abordant les défis éthiques de l'IA.	Assure que les systèmes d'IA contribuent positivement à la société, renforçant la confiance publique et la durabilité organisationnelle.

Figure 7.3: afnor

L'AFNOR présente, par ailleurs, un référentiel pour l'évaluation et la réduction de l'impact environnemental des systèmes d'intelligence artificielle (IA) [AFNOR SPEC 2314](#). Il vise à fournir des méthodes et des bonnes pratiques pour mesurer et minimiser les impacts environnementaux des services d'IA tout au long de leur cycle de vie, en tenant compte des aspects matériels, énergétiques et des données. Le référentiel met l'accent sur la nécessité d'intégrer des pratiques éthiques et durables dès la conception des projets d'IA, en questionnant les besoins et en explorant des solutions alter-

natives, y compris celles qui ne reposent pas sur l'IA.

1. **Indicateurs Environnementaux** : Le référentiel met en avant plusieurs indicateurs environnementaux clés pour évaluer les impacts des systèmes d'IA. Parmi les plus importants, on trouve le changement climatique, l'épuisement des ressources naturelles (minéraux et métaux), la consommation d'énergie primaire et finale, ainsi que l'utilisation de l'eau. Ces indicateurs permettent de quantifier les impacts directs et indirects des systèmes d'IA sur l'environnement. Le document insiste sur l'importance de la transparence et de la précision dans la collecte et l'utilisation des données pour ces évaluations, en utilisant des méthodes telles que l'Analyse du Cycle de Vie (ACV) et des outils de mesure de la consommation énergétique.
2. **Bonnes Pratiques** : Le référentiel propose une série de bonnes pratiques opérationnelles pour aider les organisations à adopter des approches plus durables dans le développement et le déploiement de l'IA. Ces pratiques couvrent divers aspects, notamment l'optimisation de l'utilisation des équipements existants, la gestion efficace des données, et la promotion de la transparence et de la responsabilité dans la gouvernance des projets d'IA. Par exemple, il est recommandé d'utiliser des méthodes de compression pour réduire l'empreinte des algorithmes d'IA, de définir des critères justifiant le ré-entraînement des modèles, et de favoriser les terminaux utilisateurs existants pour l'entraînement ou l'inférence des services d'IA.
3. **Gouvernance et Formation** : Le document souligne l'importance de la gouvernance et de la formation dans l'adoption de pratiques durables pour l'IA. Il recommande de mettre en place une gouvernance qui permet de piloter et de maîtriser les impacts environnementaux de l'IA, en intégrant ces considérations dans les processus décisionnels et les modèles de pilotage des projets. De plus, il insiste sur la nécessité de former et d'acculturer les parties prenantes aux enjeux environnementaux liés à l'IA, en commençant par les acteurs clés tels que les chefs de projet IA, les architectes d'entreprises, et les data scientists. La formation doit permettre aux acteurs de comprendre et de proposer des solutions d'IA en tenant compte de leur frugalité.

7.5 Des outils commerciaux.

- Texte : ChatGPT, Bard, Bing

- **ChatGPT** : ChatGPT est un modèle d'IA de traitement du langage naturel développé par OpenAI, basé sur l'architecture GPT-3.5. Il est conçu pour comprendre et générer du texte de manière contextuelle. Les utilisations de ChatGPT incluent la génération de contenu texte, la réponse aux questions, la création de chatbots, l'assistance à la rédaction, la traduction automatique, et bien plus encore. ChatGPT est un outil polyvalent pour traiter diverses tâches liées au langage naturel.
- **Bard** : Bard est également un modèle d'IA de traitement du langage naturel développé par OpenAI. Bien qu'il ne soit pas aussi connu que ChatGPT, Bard partage des caractéristiques similaires pour comprendre et générer du texte de manière contextuelle. Il peut être utilisé pour des tâches similaires à celles de ChatGPT, telles que la création de contenu, la réponse aux questions, et la génération de texte de qualité.
- **Bing** : Bing est le moteur de recherche de Microsoft. Bien que Bing utilise l'IA dans plusieurs aspects de son fonctionnement, l'une des utilisations les plus visibles est l'amélioration des résultats de recherche. L'IA est utilisée pour comprendre la signification des requêtes de recherche, classer les pages web en fonction de leur pertinence, et fournir des résultats personnalisés en fonction de l'historique de recherche de l'utilisateur. De plus, Bing utilise l'IA pour des fonctionnalités telles que la traduction automatique, la recherche visuelle, et la prédiction de recherche pour anticiper les requêtes de l'utilisateur.

- Design : Midjourney, Adobe Firefly, Microsoft Designer

- **Midjourney** est un outil d'IA générative développé par OpenAI. Il utilise un modèle de langage génératif appelé DALL-E 2 pour créer des images à partir de descriptions textuelles. DALL-E 2 a été entraîné sur un ensemble de données massif d'images et de texte, et il est capable de générer des images réalistes et créatives.

- **Adobe Firefly** est un outil d'IA générative développé par Adobe. Il utilise un modèle de langage génératif appelé CLIP pour créer des images, du texte et des effets visuels. CLIP a été entraîné sur un ensemble de données massif d'images, de texte et de code, et il est capable de générer des résultats créatifs et pertinents.
 - **Microsoft Designer** est un outil d'IA générative développé par Microsoft. Il utilise un modèle de langage génératif appelé Turing NLG pour créer des images, du texte et des vidéos. Turing NLG a été entraîné sur un ensemble de données massif de texte et de code, et il est capable de générer des résultats réalistes et informatifs.
- Vidéo : Runway, Pictory, Descript
 - **Runway ML** : Runway ML est une plateforme qui permet aux artistes, aux designers, aux créateurs de contenu et aux développeurs d'intégrer facilement des modèles d'IA dans leurs projets. Runway offre une interface conviviale pour l'entraînement, le déploiement et l'utilisation de modèles d'IA, qu'il s'agisse de modèles de traitement d'image, de traitement du langage naturel, de génération de contenu ou d'autres domaines de l'IA. Les utilisateurs peuvent exploiter des modèles pré-entraînés ou créer les leurs pour ajouter des capacités d'IA à leurs créations.
 - **Pictory** : Pictory est une application qui utilise l'IA pour aider les utilisateurs à créer des vidéos narratives à partir de photos et de vidéos. Il peut automatiquement générer des vidéos en ajoutant des effets de transition, des animations et des fonds sonores pour raconter des histoires à partir du contenu visuel fourni. L'IA est utilisée pour automatiser le processus de montage vidéo et faciliter la création de vidéos engageantes.
 - **Descript** : Descript est un outil de traitement automatique de la parole (ASR) qui utilise l'IA pour transcrire automatiquement l'audio en texte. Il permet également de modifier facilement la transcription en modifiant simplement le texte, ce qui met à jour simultanément la piste audio correspondante. En plus de la transcription, Descript offre des fonctionnalités d'édition audio avancées et des outils de création de contenu vidéo qui utilisent l'IA pour simplifier le processus de montage.
 - Contenu : Opus Clip, t. Tweetmonk, Synthesia
 - **Opus Clip** : Opus Clip est une application de montage vidéo en ligne qui utilise l'IA pour simplifier et accélérer le processus de création de vidéos. Il offre des fonctionnalités telles que la reconnaissance vocale, la génération automatique de sous-titres, la traduction automatique, la suppression de l'arrière-plan, et d'autres outils d'édition vidéo automatisés. L'IA est utilisée pour effectuer ces tâches rapidement et efficacement, ce qui permet aux utilisateurs de créer des vidéos de haute qualité plus facilement.
 - **t.Tweetmonk** : Je ne dispose pas d'informations précises sur un produit ou un service appelé "t.Tweetmonk" dans ma base de données jusqu'en septembre 2021. Il est possible que ce soit un projet ou un service qui a été lancé après cette date ou qui n'a pas encore gagné en notoriété.
 - **Synthesia** : Synthesia est une plateforme qui utilise l'IA pour générer des vidéos avec des présentateurs virtuels. Les utilisateurs peuvent créer des vidéos en fournissant un script et en choisissant un personnage virtuel, puis l'IA anime le personnage pour qu'il parle et bouge de manière réaliste tout en prononçant le script. Cela permet de créer des vidéos de présentation rapidement, sans avoir besoin d'un acteur ou d'une actrice réel(le). Synthesia est souvent utilisé pour la création de contenu vidéo automatisé, notamment dans le domaine de la formation en ligne et de la publicité.

- Présentation : Tome, Decktopus, beautiful.ai

- **Tome** : Tome est un outil de génération de contenu qui utilise l'IA pour aider les écrivains et les créateurs de contenu à rédiger plus efficacement. Il peut suggérer des idées de sujets, générer des titres, des introductions et des conclusions, et même créer du contenu texte complet. L'IA dans Tome analyse les données et les tendances pour fournir des suggestions de contenu pertinentes, ce qui peut aider à accélérer le processus de création de contenu.
- **Decktopus** : Decktopus est un outil de création de présentations qui utilise l'IA pour simplifier et améliorer la conception de diapositives. Il propose des modèles de conception automatisée, suggère des mises en page, des couleurs et des polices appropriées, et permet aux utilisateurs de créer des présentations professionnelles rapidement. L'IA est utilisée pour aider les utilisateurs à concevoir des diapositives visuellement attrayantes sans nécessiter de compétences en conception graphique.
- **Beautiful.ai** : Beautiful.ai est également un outil de création de présentations qui exploite l'IA pour améliorer la conception de diapositives. Il propose des modèles intelligents qui ajustent automatiquement la mise en page et le design en fonction du contenu que l'utilisateur ajoute. L'IA optimise la disposition, la hiérarchie visuelle et l'alignement pour créer des présentations esthétiquement agréables et professionnelles.

- Productivité : Notion AI, Perplexity, Grammarly

- **Notion AI** : Notion est une plateforme de gestion de tâches et de collaboration en ligne qui peut intégrer des plug-ins et des extensions développés par des tiers. "Notion AI" ne désigne pas un produit spécifique que je connais jusqu'en septembre 2021, mais il est possible que des extensions tierces utilisent l'IA pour ajouter des fonctionnalités avancées à Notion. Par exemple, certaines extensions peuvent utiliser l'IA pour automatiser des tâches de gestion de projet, de recherche d'informations ou d'analyse de données à l'intérieur de l'environnement Notion.
- **Perplexity** : Perplexity est un terme qui est couramment utilisé dans le domaine du traitement du langage naturel (NLP) pour évaluer la qualité des modèles de langage. En NLP, la perplexité mesure à quel point un modèle de langage est capable de prédire correctement la probabilité des séquences de mots dans un texte. Il ne s'agit pas d'un produit ou d'un service spécifique, mais plutôt d'une métrique utilisée pour évaluer les performances des modèles de NLP, qui peuvent être alimentés par l'IA.
- **Grammarly** : Grammarly est un outil de vérification de la grammaire, de la syntaxe, de l'orthographe et du style de l'écriture qui utilise l'IA pour fournir des suggestions d'amélioration du texte. Il peut détecter et corriger les erreurs grammaticales, mais aussi offrir des conseils sur la clarté, la cohérence et la lisibilité du texte. L'IA est utilisée pour analyser le texte et suggérer des améliorations en temps réel, que ce soit dans le cadre de la rédaction de courriels, de documents professionnels ou de contenus en ligne.

- Code : Copilot X, Codex, CodeWhisper

Ces outils d'aide au développement informatique utilisent l'IA de la manière suivante :

- **Modèles de Langage Pré-entraînés** : ils sont basés sur des modèles de langage profond pré-entraînés. Ces modèles ont été formés sur une grande quantité de code source open source. Cela signifie qu'ils ont une compréhension approfondie de la syntaxe, des structures de données et des modèles de codage courants dans de nombreux langages de programmation.
- **Analyse Contextuelle** : L'IA analyse en permanence le code que le développeur écrit en temps réel. Elle tient compte du contexte, y compris des variables précédemment déclarées, des fonctions appelées, des commentaires du développeur et des schémas de codage courants pour générer des suggestions contextuelles.
- **Suggestion de Code** : L'outil génère automatiquement des suggestions de code lors de la saisie, suggérant des lignes de code complètes ou des morceaux de code qui sont pertinents pour le contexte actuel. Ces suggestions peuvent inclure des déclarations de variables, des boucles, des conditions, des fonctions, etc.
- **Compréhension du Langage Naturel** : Ils sont capables de comprendre les commentaires en langage naturel que les développeurs ajoutent à leur code. Si un développeur écrit un commentaire expliquant ce

qu'une fonction doit accomplir, ils peuvent générer automatiquement le code pour réaliser cette tâche en se basant sur la description en langage naturel.

- **Détection d'Erreurs** : L'IA peut aider à détecter des erreurs potentielles dans le code en offrant des suggestions pour corriger des problèmes courants, tels que des incompatibilités de types ou des erreurs de syntaxe.
- **Support pour Plusieurs Langages** : Ils prennent en charge de nombreux langages de programmation populaires, ce qui le rend polyvalent pour une variété de projets de développement.

7.6 Des voraces énergétiques.

L'intelligence artificielle (IA) et le traitement automatique du langage naturel (TALN) ont révolutionné de nombreux domaines, offrant des capacités d'analyse et de compréhension des données textuelles à une échelle sans précédent. Cependant, ces technologies ne sont pas sans conséquences sur l'environnement, notamment en termes de consommation énergétique et d'émissions de gaz à effet de serre.

1. La consommation énergétique des modèles d'IA et de TALN :

Les modèles d'apprentissage automatique, en particulier les grands modèles de langage (LLMs), nécessitent une quantité significative d'énergie pour leur entraînement et leur inférence. L'inférence est le processus par lequel un modèle déjà entraîné est utilisé pour faire des prédictions ou des analyses sur de nouvelles données. Bien que l'entraînement soit une phase énergivore, l'inférence peut également accumuler une consommation énergétique importante, surtout lorsqu'elle est effectuée à grande échelle, comme dans les applications de traduction automatique ou de recherche sur le web.

2. Les émissions de carbone :

Les émissions de carbone associées à l'utilisation des modèles d'IA et de TALN varient considérablement en fonction de la taille du modèle, de l'architecture, et de la tâche à accomplir. Par exemple, les modèles générant du texte ou des images sont généralement plus énergivores et donc plus polluants que ceux qui classifient des images ou du texte en catégories.

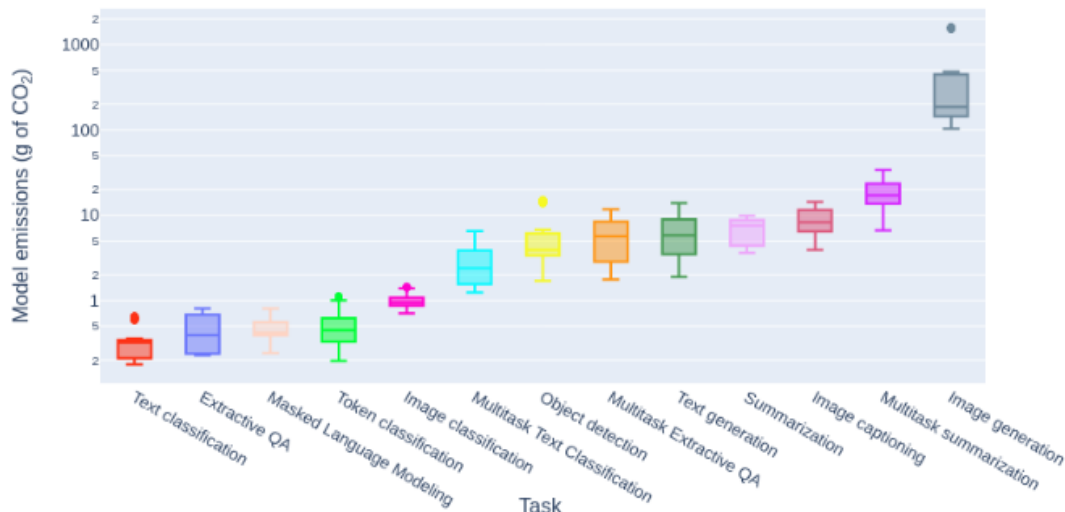


Figure 7.4: CO₂ VS type d'IA
(<https://arxiv.org/pdf/2311.16863>).

3. L'importance de la recherche sur l'impact environnemental :

La recherche sur l'impact environnemental de l'IA et du TALN est cruciale pour comprendre et réduire l'empreinte carbone de ces technologies. Les auteurs de l'étude ont analysé l'impact de différents modèles et architectures

sur l'environnement, en se concentrant sur la phase d'inférence. Ils ont mesuré la consommation d'énergie et les émissions de carbone de 88 modèles à travers 10 tâches et 30 ensembles de données.

4. Les défis et les opportunités :

Son analyse peut être perçue comme une critique de l'IA en général, mais son intention est d'éclairer les impacts environnementaux pour aider les développeurs et les chercheurs à faire des choix plus éclairés. Il existe des défis à relever, tels que la nécessité de modèles plus efficaces sur le plan énergétique et de pratiques de déploiement durables. Cependant, il y a aussi des opportunités, comme l'innovation dans la conception de modèles plus verts et l'utilisation de sources d'énergie renouvelables.

5. Vers une IA durable :

Pour une IA durable, il est essentiel de prendre en compte l'impact environnemental dès la conception des modèles. Cela inclut le choix de modèles plus petits et plus efficaces, l'optimisation des algorithmes pour réduire la consommation d'énergie, et l'utilisation de centres de données alimentés par des énergies renouvelables. La transparence sur l'empreinte carbone des modèles et la sensibilisation des utilisateurs aux impacts environnementaux sont également importantes.

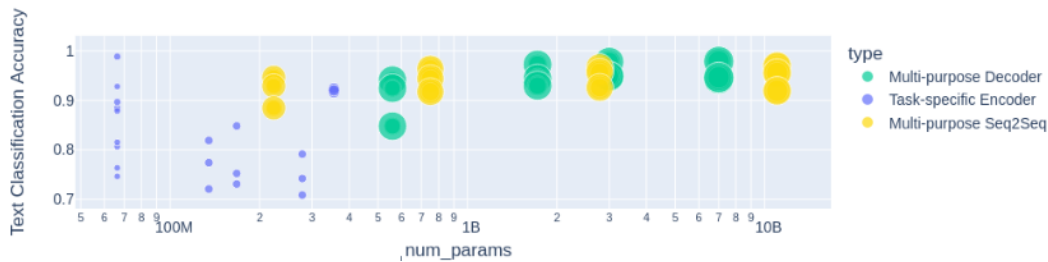


Figure 7.5: Des modèles de plus en plus gros.
(<https://arxiv.org/pdf/2311.16863>).

En conclusion, alors que l'IA et le TALN continuent de progresser, il est impératif de considérer leur impact environnemental. Les chercheurs, les développeurs et les utilisateurs doivent collaborer pour promouvoir des pratiques durables et réduire l'empreinte écologique de ces technologies puissantes mais énergivores.

Chapter 8

Conclusions

En conclusion, ce cours d'introduction aux techniques de Machine Learning appliquées au domaine de la thermique du bâtiment a permis de jeter un regard fascinant sur la convergence entre l'ingénierie du bâtiment et l'intelligence artificielle. Nous avons exploré comment les méthodes de Machine Learning peuvent révolutionner la manière dont nous abordons la gestion thermique des bâtiments, en offrant des outils pour prédire, optimiser et contrôler de manière plus efficace les systèmes énergétiques. De la prédiction des charges thermiques à la modélisation de la consommation d'énergie, en passant par l'optimisation des systèmes HVAC, chaque concept a été éclairé par les techniques de ML. Tout au long de ce cours, nous avons compris comment exploiter les données massives disponibles et comment développer des modèles prédictifs pour une prise de décision éclairée dans la conception, la gestion et l'efficacité énergétique des bâtiments. Alors que le domaine de la thermique du bâtiment continue d'évoluer, l'intégration du Machine Learning ouvre des perspectives prometteuses pour des solutions plus durables, intelligentes et économes en énergie, créant ainsi une synergie stimulante entre l'ingénierie traditionnelle et l'innovation technologique.

Appendix A

Notions de Machine Learning

- **Bagging** : Formation d'un ensemble de modèles en **parallèle**. Chaque modèle apprend à partir d'un sous-ensemble aléatoire de données, où l'ensemble de données est de la même taille que l'original, mais est échantillonné aléatoirement avec remplacement (bootstrap).
- **Boosting** : Formation d'un groupe de modèles de manière **séquentielle**. Chaque modèle apprend des erreurs du modèle précédent. En d'autres termes, le modèle suivant tente d'expliquer et de prédire l'erreur laissée par le modèle précédent.
- **Gradient Boosting Decision Trees** : Tous les arbres sont connectés en série et chaque arbre tente de minimiser l'erreur de l'arbre précédent. Ce processus en série fait que les algorithmes de boosting sont généralement lents à apprendre (contrôlables par le développeur à l'aide du paramètre de taux d'apprentissage), mais également très précis.
- **Out of Time** : Pour les time series. L'idée est de tester l'algorithme sur une plage temporelle qui n'est pas dans l'apprentissage.
- **A/B testing** : Une manière statistique de comparer deux techniques (ou plus) - A et B. Généralement, A est une technique existante et B est une nouvelle technique. Les tests A/B permettent non seulement de déterminer quelle technique est la plus performante, mais aussi de savoir si la différence est statistiquement significative. Les tests A/B comparent généralement une seule mesure pour deux techniques ; par exemple, comment se compare la précision du modèle pour deux techniques ? Cependant, les tests A/B peuvent également comparer un nombre fini de paramètres.
- **accelerator chip** : Une catégorie de composants matériels spécialisés conçus pour effectuer les calculs clés nécessaires aux algorithmes d'apprentissage profond.

Les puces accélératrices (ou simplement les accélérateurs, en abrégé) peuvent augmenter de manière significative la vitesse et l'efficacité des tâches d'apprentissage et d'inférence par rapport à un processeur général. Elles sont idéales pour l'entraînement des réseaux neuronaux et d'autres tâches similaires à forte intensité de calcul.

Voici quelques exemples de puces accélératrices:

- **Tensor Processing Units (TPU)** de Google, matériel dédié à l'apprentissage en profondeur.
- Les **GPU de NVIDIA** qui, bien qu'initialement conçus pour le traitement graphique, sont conçus pour permettre le traitement parallèle, ce qui peut augmenter de manière significative la vitesse de traitement.
- **action** : Dans l'apprentissage par renforcement, le mécanisme par lequel l'agent passe d'un état à l'autre de l'environnement. L'agent choisit l'action en utilisant une politique.
- **anomaly detection** : Processus d'identification des valeurs aberrantes. Par exemple, si la moyenne d'une caractéristique donnée est de 100 avec un écart type de 10, la détection des anomalies doit signaler une valeur de 200 comme suspecte.

- **backpropagation** : L'algorithme qui met en œuvre la descente de gradient dans les réseaux neuronaux.

La formation d'un réseau neuronal implique de nombreuses itérations du cycle à deux passages suivant:

Au cours de la passe avant (propagation), le système traite un lot d'exemples afin d'obtenir une ou plusieurs prédiction(s). Le système compare chaque prédiction à chaque valeur d'étiquette. La différence entre la prédiction et la valeur de l'étiquette est la perte pour cet exemple. Le système agrège les pertes pour tous les exemples afin de calculer la perte totale pour le lot en cours.

Lors du passage en arrière (rétropropagation), le système réduit les pertes en ajustant les poids de tous les neurones dans toutes les couches cachées. Les réseaux neuronaux contiennent souvent de nombreux neurones répartis sur plusieurs couches cachées. Chacun de ces neurones contribue à la perte globale de différentes manières. La rétropropagation détermine s'il convient d'augmenter ou de diminuer les poids appliqués à certains neurones.

- **bag of words** : Représentation des mots d'une phrase ou d'un passage, quel que soit leur ordre. Par exemple, le sac de mots représente à l'identique les trois phrases suivantes:

- le chien saute
- saute le chien
- le chien saute le

Chaque mot est associé à un index dans un vecteur clairsemé, où le vecteur a un index pour chaque mot du vocabulaire. Par exemple, la phrase le chien saute est mise en correspondance avec un vecteur de caractéristiques comportant des valeurs non nulles aux trois indices correspondant aux mots le, chien et saute. La valeur non nulle peut être l'une des suivantes :

- Un 1 pour indiquer la présence d'un mot.
- Nombre de fois qu'un mot apparaît dans le sac. Par exemple, si la phrase était le chien marron est un chien au pelage marron, les mots marron et chien seraient représentés par 2, tandis que les autres mots seraient représentés par 1.
- Une autre valeur, telle que le logarithme du nombre de fois qu'un mot apparaît dans le sac.

- **batch size** : Le nombre d'exemples dans un lot. Par exemple, si la taille du lot est de 100, le modèle traite 100 exemples par itération.

Voici les stratégies les plus courantes en matière de taille des lots :

- Stochastic Gradient Descent (SGD), dans laquelle la taille du lot est de 1.
- full batch, dans lequel la taille du batch est le nombre d'exemples dans l'ensemble de l'ensemble de formation. Par exemple, si l'ensemble de formation contient un million d'exemples, la taille du batch sera d'un million d'exemples. La méthode du batch complet est généralement une stratégie inefficace.
- mini-batch dans laquelle la taille du batch est généralement comprise entre 10 et 1000. Le mini-batch est généralement la stratégie la plus efficace.

- **Bayesian neural network** : Réseau neuronal probabiliste qui tient compte de l'incertitude des poids et des sorties. Un modèle de régression de réseau neuronal standard prédit généralement une valeur scalaire ; par exemple, un modèle standard prédit un prix de maison de 853 000. En revanche, un réseau neuronal bayésien prédit une distribution de valeurs ; par exemple, un modèle bayésien prédit un prix de l'immobilier de 853 000 avec un écart type de 67 200.

Un réseau neuronal bayésien s'appuie sur le théorème de Bayes pour calculer les incertitudes dans les poids et les prédictions. Un réseau neuronal bayésien peut être utile lorsqu'il est important de quantifier l'incertitude, comme dans les modèles liés aux produits pharmaceutiques. Les réseaux neuronaux bayésiens peuvent également contribuer à éviter l'ajustement excessif.

- **Bayesian optimization** : Technique de modèle de régression probabiliste permettant d'optimiser des fonctions objectives coûteuses en calcul en optimisant à la place un substitut qui quantifie l'incertitude par le biais d'une technique d'apprentissage bayésien. L'optimisation bayésienne étant elle-même très coûteuse, elle est généralement utilisée pour optimiser des tâches coûteuses à évaluer qui comportent un petit nombre de paramètres, comme la sélection d'hyperparamètres.
- **bucketing** : Conversion d'une caractéristique unique en plusieurs caractéristiques binaires appelées "buckets" ou "bins", généralement sur la base d'une plage de valeurs. L'élément découpé est généralement un élément continu.

Par exemple, au lieu de représenter la température sous la forme d'une seule caractéristique continue en virgule flottante, vous pouvez découper les plages de températures en tranches distinctes :

- ≤ 10 degrés Celsius serait le bucket "froid".
- $11 - 24$ degrés Celsius serait le bucket "tempéré".
- ≥ 25 degrés Celsius serait le bucket "chaud".

Le modèle traitera de manière identique toutes les valeurs d'un même bucket. Par exemple, les valeurs 13 et 22 se trouvent toutes deux dans le bucket tempéré, le modèle traite donc les deux valeurs de manière identique.

- **calibration layer** : Ajustement postérieur à la prédiction, généralement pour tenir compte du biais de prédiction. Les prédictions et probabilités ajustées doivent correspondre à la distribution d'un ensemble d'étiquettes observées.
- **candidate generation** : L'ensemble initial de recommandations choisies par un système de recommandation. Prenons l'exemple d'une librairie qui propose 100 000 titres. La phase de génération de candidats crée une liste beaucoup plus réduite de livres convenant à un utilisateur particulier, disons 500. Mais même 500 livres sont beaucoup trop nombreux pour être recommandés à un utilisateur. Les phases suivantes, plus coûteuses, d'un système de recommandation (telles que la notation et le reclassement) réduisent ces 500 livres à un ensemble de recommandations beaucoup plus petit et plus utile.
- **candidate sampling** : Optimisation du temps de formation qui calcule une probabilité pour toutes les étiquettes positives, en utilisant, par exemple, softmax, mais seulement pour un échantillon aléatoire d'étiquettes négatives. Par exemple, étant donné un exemple étiqueté beagle et dog, l'échantillonnage candidat calcule les probabilités prédites et les termes de perte correspondants pour :
 - chat
 - chien
 - un sous-ensemble aléatoire des classes négatives restantes (par exemple, chat, sucette, clôture).

L'idée est que les classes négatives peuvent apprendre grâce à un renforcement négatif moins fréquent tant que les classes positives bénéficient toujours d'un renforcement positif adéquat, ce qui est effectivement observé de manière empirique.

L'échantillonnage des candidats est plus efficace en termes de calcul que les algorithmes de formation qui calculent les prédictions pour toutes les classes négatives, en particulier lorsque le nombre de classes négatives est très élevé.

- **centroid** : Le centre d'un cluster déterminé par un algorithme k-means ou k-medians. Par exemple, si k est égal à 3, l'algorithme k-means ou k-medians trouve 3 centroïdes.
- **centroid-based clustering** : Catégorie d'algorithmes de regroupement qui organise les données en clusters ou grappes non hiérarchiques. k-means est l'algorithme de regroupement basé sur les centroïdes le plus largement utilisé.
- **checkpoint** : Données qui capturent l'état des paramètres d'un modèle à une itération de formation particulière. Les points de contrôle permettent d'exporter les poids du modèle ou d'effectuer la formation sur plusieurs sessions. Les points de contrôle permettent également à la formation de se poursuivre après des erreurs (par exemple,

la préemption d'un travail).

Lors du réglage fin, le point de départ de l'entraînement du nouveau modèle sera un point de contrôle spécifique du modèle pré-entraîné.

- **class** : Catégorie à laquelle une étiquette peut appartenir. Par exemple :

Dans un modèle de classification binaire qui détecte le spam, les deux classes peuvent être le spam et le non-spam.

- Dans un modèle de classification multi-classes qui identifie les races de chiens, les classes peuvent être caniche, beagle, carlin, etc.
- Un modèle de classification prédit une classe. En revanche, un modèle de régression prédit un nombre plutôt qu'une classe.

- **classification threshold** : Dans une classification binaire, un nombre entre 0 et 1 qui convertit la sortie brute d'un modèle de régression logistique en une prédiction de la classe positive ou de la classe négative. Il convient de noter que le seuil de classification est une valeur choisie par un être humain, et non une valeur choisie par l'apprentissage du modèle.

Un modèle de régression logistique produit une valeur brute comprise entre 0 et 1 :

- Si cette valeur brute est supérieure au seuil de classification, la classe positive est prédite.
- Si cette valeur brute est inférieure au seuil de classification, la classe négative est prédite.

Par exemple, supposons que le seuil de classification soit de 0,8. Si la valeur brute est de 0,9, le modèle prédit la classe positive. Si la valeur brute est de 0,7, le modèle prédit la classe négative.

Le choix du seuil de classification influence fortement le nombre de faux positifs et de faux négatifs.

- **class-imbalanced dataset** : Un ensemble de données pour un problème de classification dans lequel le nombre total d'étiquettes de chaque classe diffère de manière significative. Par exemple, considérons un ensemble de données de classification binaire dont les deux étiquettes sont divisées comme suit :

- 1,000,000 labels négatifs
- 10 labels positifs

Le rapport entre les étiquettes négatives et positives est de 100 000 pour 1. Il s'agit donc d'un ensemble de données déséquilibré en termes de classes.

En revanche, l'ensemble de données suivant n'est pas déséquilibré en termes de classes, car le rapport entre les étiquettes négatives et les étiquettes positives est relativement proche de 1 :

- 517 labels négatifs
- 483 labels positifs

Les ensembles de données multi-classes peuvent également être déséquilibrés. Par exemple, l'ensemble de données de classification multi-classes suivant est également déséquilibré en termes de classes, car une étiquette a beaucoup plus d'exemples que les deux autres :

- 1 000 000 labels avec la classe "vert"
- 200 200 labels avec la classe "violet"
- 350 labels avec la classe "orange"

- **clipping** : Technique permettant de traiter les valeurs aberrantes en procédant à l'une ou l'autre des opérations suivantes, ou aux deux :

- Réduire les valeurs des caractéristiques qui sont supérieures à un seuil maximal jusqu'à ce seuil maximal.
- Augmentation des valeurs des caractéristiques inférieures à un seuil minimal jusqu'à ce seuil minimal.

Par exemple, supposons que 0.5% des valeurs d'une caractéristique particulière se situent en dehors de l'intervalle 40-60. Dans ce cas, vous pouvez procéder comme suit :

- écrêter toutes les valeurs supérieures à 60 (le seuil maximum) pour qu'elles soient exactement 60.
- Réduire toutes les valeurs inférieures à 40 (le seuil minimum) pour qu'elles soient exactement 40.

Les valeurs aberrantes, ou outliers, peuvent endommager les modèles, entraînant parfois un dépassement des poids pendant la formation. Certaines valeurs aberrantes peuvent également altérer considérablement des paramètres tels que la précision. L'écrêtage est une technique courante pour limiter les dégâts.

Gradient clipping force les valeurs de gradient dans une fourchette donnée au cours de l'entraînement.

- **co-adaptation** : Lorsque les neurones prédisent des modèles dans les données d'apprentissage en s'appuyant presque exclusivement sur les sorties d'autres neurones spécifiques au lieu de s'appuyer sur le comportement du réseau dans son ensemble. Lorsque les modèles à l'origine de la coadaptation ne sont pas présents dans les données de validation, la coadaptation entraîne un surajustement. La régularisation de l'abandon réduit la coadaptation car l'abandon garantit que les neurones ne peuvent pas s'appuyer uniquement sur d'autres neurones spécifiques.
- **confirmation bias** : La tendance à rechercher, interpréter, privilégier et rappeler des informations d'une manière qui confirme les croyances ou les hypothèses préexistantes. Les développeurs de l'apprentissage automatique peuvent, par inadvertance, collecter ou étiqueter des données de manière à influencer un résultat soutenant leurs croyances existantes. Le biais de confirmation est une forme de biais implicite.

Le biais de l'expérimentateur est une forme de biais de confirmation dans lequel un expérimentateur continue à former des modèles jusqu'à ce qu'une hypothèse préexistante soit confirmée.

- **convenience sampling** : Utilisation d'un ensemble de données non recueillies scientifiquement afin de réaliser des expériences rapides. Plus tard, il est essentiel de passer à un ensemble de données recueillies de manière scientifique.
- **convolution** : En mathématiques, il s'agit d'un mélange de deux fonctions. En apprentissage automatique, une convolution mélange le filtre convolutif et la matrice d'entrée afin d'entraîner les poids.

Le terme "convolution" dans le domaine de l'apprentissage automatique est souvent une manière abrégée de se référer à l'opération convolutive ou à la couche convolutive.

Sans les convolutions, un algorithme d'apprentissage automatique devrait apprendre un poids distinct pour chaque cellule d'un grand tenseur. Par exemple, un algorithme d'apprentissage automatique s'entraînant sur 2K x 2K images serait obligé de trouver 4M poids distincts. Grâce aux convolutions, un algorithme d'apprentissage automatique ne doit trouver des poids que pour chaque cellule du filtre convolutionnel, ce qui réduit considérablement la mémoire nécessaire à l'apprentissage du modèle. Lorsque le filtre convolutif est appliqué, il est simplement répliqué dans les cellules de manière à ce que chacune d'entre elles soit multipliée par le filtre.

- **convolutional filter** : L'un des deux acteurs d'une opération convolutive (l'autre acteur est une tranche d'une matrice d'entrée). (L'autre acteur est une tranche d'une matrice d'entrée.) Un filtre convolutionnel est une matrice ayant le même rang que la matrice d'entrée, mais une forme plus petite. Par exemple, étant donné une matrice d'entrée de 28x28, le filtre pourrait être n'importe quelle matrice 2D plus petite que 28x28.

Dans la manipulation photographique, toutes les cellules d'un filtre convolutionnel sont généralement réglées sur un modèle constant de uns et de zéros. Dans l'apprentissage automatique, les filtres convolutifs sont généralement ensemencés avec des nombres aléatoires, puis le réseau apprend les valeurs idéales.

- **crash blossom** : Phrase ou expression dont le sens est ambigu. Les "crash blossoms" posent un problème important en matière de compréhension du langage naturel. Par exemple, le titre Red Tape Holds Up Skyscraper (La paperasse retient le gratte-ciel) est une fleur d'écrasement, car un modèle d'ULA pourrait l'interpréter au sens propre ou au sens figuré.

- **cross-entropy** : Une généralisation de la perte logarithmique, ou Log Loss, aux problèmes de classification multi-classes. L'entropie croisée quantifie la différence entre deux distributions de probabilité.
- **cross-validation** : Mécanisme permettant d'estimer le degré de généralisation d'un modèle à de nouvelles données en testant le modèle par rapport à un ou plusieurs sous-ensembles de données ne se recoupant pas et ne faisant pas partie de l'ensemble d'apprentissage.
- **data analysis** : Obtenir une compréhension des données en tenant compte des échantillons, des mesures et de la visualisation. L'analyse des données peut être particulièrement utile lorsqu'un ensemble de données est reçu pour la première fois, avant de construire le premier modèle. Elle est également cruciale pour comprendre les expériences et déboguer les problèmes du système.
- **data augmentation** : Augmentation artificielle de la gamme et du nombre d'exemples de formation en transformant les exemples existants pour créer des exemples supplémentaires. Par exemple, supposons que les images soient l'une de vos caractéristiques, mais que votre ensemble de données ne contienne pas suffisamment d'exemples d'images pour que le modèle puisse apprendre des associations utiles. Idéalement, vous devriez ajouter suffisamment d'images étiquetées à votre ensemble de données pour permettre à votre modèle de s'entraîner correctement. Si ce n'est pas possible, l'augmentation des données peut faire pivoter, étirer et refléter chaque image pour produire de nombreuses variantes de l'image originale, ce qui peut permettre d'obtenir suffisamment de données étiquetées pour permettre un excellent entraînement.
- **data parallelism** : Une façon de mettre à l'échelle la formation ou l'inférence qui réplique un modèle entier sur plusieurs appareils et transmet ensuite un sous-ensemble des données d'entrée à chaque appareil. Le parallélisme des données peut permettre la formation et l'inférence sur des lots de très grande taille ; cependant, le parallélisme des données exige que le modèle soit suffisamment petit pour tenir sur tous les appareils.
- **dimension reduction** : Diminution du nombre de dimensions utilisées pour représenter une caractéristique particulière dans un vecteur de caractéristiques, généralement par conversion en un vecteur d'intégration ou embedding vector.
- **downsampling** : Terme surchargé qui peut signifier l'une ou l'autre des choses suivantes :
 - Réduction de la quantité d'informations dans une caractéristique afin d'entraîner un modèle plus efficacement. Par exemple, avant d'entraîner un modèle de reconnaissance d'images, il est possible de réduire l'échantillonnage d'images à haute résolution à un format à plus faible résolution.
 - Formation sur un pourcentage disproportionnellement faible d'exemples de classes surreprésentées afin d'améliorer la formation du modèle sur les classes sous-représentées. Par exemple, dans un ensemble de données déséquilibré, les modèles ont tendance à apprendre beaucoup de choses sur la classe majoritaire et pas assez sur la classe minoritaire. Le déséchantillonnage permet d'équilibrer la quantité de formation sur les classes majoritaires et minoritaires.
- **dropout regularization** : Une forme de régularisation utile dans la formation des réseaux neuronaux. La régularisation par élimination supprime une sélection aléatoire d'un nombre fixe d'unités dans une couche du réseau pour un seul pas de gradient. Plus le nombre d'unités supprimées est important, plus la régularisation est forte. Cela revient à entraîner le réseau à émuler un ensemble exponentiellement grand de réseaux plus petits.
- **early stopping** : Une méthode de régularisation qui implique l'arrêt de la formation avant que la perte de formation ne finisse de diminuer. Dans le cas de l'arrêt anticipé, vous arrêtez intentionnellement la formation du modèle lorsque la perte sur un ensemble de données de validation commence à augmenter, c'est-à-dire lorsque les performances de généralisation se détériorent.
- **embedding layer** : Couche cachée spéciale qui s'entraîne sur une caractéristique catégorielle de haute dimension pour apprendre progressivement un vecteur d'intégration de dimension inférieure. Une couche d'intégration permet à un réseau neuronal de s'entraîner beaucoup plus efficacement que s'il s'entraînait uniquement sur la caractéristique catégorielle de haute dimension.
- **embedding space** : L'espace vectoriel d-dimensionnel dans lequel les caractéristiques d'un espace vectoriel de dimension supérieure sont mises en correspondance. Idéalement, l'espace d'intégration contient une structure qui produit des résultats mathématiques significatifs ; par exemple, dans un espace d'intégration idéal, l'addition et la soustraction d'intégrations permettent de résoudre des tâches d'analogie de mots.

- **ensemble** : Ensemble de modèles formés indépendamment dont les prédictions sont moyennées ou agrégées. Dans de nombreux cas, un ensemble produit de meilleures prédictions qu'un modèle unique. Par exemple, une forêt aléatoire est un ensemble construit à partir de plusieurs arbres de décision. Notez que les forêts de décision ne sont pas toutes des ensembles.
- **environnement** : Dans l'apprentissage par renforcement, le monde qui contient l'agent et lui permet d'observer l'état de ce monde. Par exemple, le monde représenté peut être un jeu comme les échecs ou un monde physique comme un labyrinthe. Lorsque l'agent applique une action à l'environnement, celui-ci passe d'un état à l'autre.
- **episode** : Dans l'apprentissage par renforcement, chacune des tentatives répétées de l'agent pour apprendre un environnement.
- **epoch** : Un passage complet sur l'ensemble de l'apprentissage, de sorte que chaque exemple a été traité une fois.

Une epoch représente des itérations d'entraînement de taille N/batch , où N est le nombre total d'exemples.

- **epsilon greedy policy** : Dans l'apprentissage par renforcement, une politique qui suit soit une politique aléatoire avec une probabilité epsilon, soit une politique avide dans le cas contraire. Par exemple, si epsilon est égal à 0,9, la politique suit une politique aléatoire 90 % du temps et une politique avide 10 % du temps.

Au cours d'épisodes successifs, l'algorithme réduit la valeur d'epsilon afin de passer d'une politique aléatoire à une politique gourmande. En changeant de politique, l'agent explore d'abord l'environnement de manière aléatoire, puis exploite avec avidité les résultats de l'exploration aléatoire.

- **exploding gradient problem** : La tendance des gradients dans les réseaux neuronaux profonds (en particulier les réseaux neuronaux récurrents) à devenir étonnamment raides (élevés). Les gradients abrupts entraînent souvent des mises à jour très importantes des poids de chaque nœud d'un réseau neuronal profond.

Les modèles souffrant du problème de l'explosion du gradient deviennent difficiles, voire impossibles à entraîner. Le gradient clipping ou, écrêtage du gradient, peut atténuer ce problème.

- **federated learning** : Une approche distribuée de l'apprentissage automatique qui forme des modèles d'apprentissage automatique en utilisant des exemples décentralisés résidant sur des appareils tels que les smartphones. Dans l'apprentissage fédéré, un sous-ensemble d'appareils télécharge le modèle actuel à partir d'un serveur central de coordination. Les appareils utilisent les exemples stockés sur eux pour améliorer le modèle. Les appareils téléchargent ensuite les améliorations du modèle (mais pas les exemples d'apprentissage) vers le serveur de coordination, où elles sont regroupées avec d'autres mises à jour pour produire un modèle global amélioré. Après l'agrégation, les mises à jour du modèle calculées par les appareils ne sont plus nécessaires et peuvent être supprimées.
- **feedback loop** : En apprentissage automatique, situation dans laquelle les prédictions d'un modèle influencent les données d'apprentissage du même modèle ou d'un autre modèle. Par exemple, un modèle qui recommande des films influencera les films que les gens voient, ce qui influencera ensuite les modèles de recommandation de films suivants.
- **few-shot learning** : Une approche d'apprentissage automatique, souvent utilisée pour la classification d'objets, conçue pour former des classificateurs efficaces à partir d'un petit nombre d'exemples d'apprentissage.
- **forget gate** : La partie d'une cellule de mémoire à long terme qui régule le flux d'informations à travers la cellule. Les portes d'oubli maintiennent le contexte en décidant quelles informations doivent être éliminées de l'état de la cellule.
- **greedy policy** : Dans l'apprentissage par renforcement, une politique qui choisit toujours l'action dont le rendement attendu est le plus élevé.
- **hallucination** : La production de résultats apparemment plausibles mais factuellement incorrects par un modèle génératif qui prétend faire une affirmation sur le monde réel. Par exemple, si un agent de dialogue affirme que Barack Obama est mort en 1865, il est en train d'halluciner.

- **hashing** : Dans le domaine de l'apprentissage automatique, mécanisme permettant de regrouper des données catégorielles, en particulier lorsque le nombre de catégories est important, mais que le nombre de catégories apparaissant réellement dans l'ensemble de données est relativement faible.

Par exemple, la Terre abrite environ 73 000 espèces d'arbres. Vous pourriez représenter chacune de ces 73 000 espèces d'arbres dans 73 000 catégories distinctes. Par ailleurs, si seulement 200 de ces espèces d'arbres apparaissent réellement dans un ensemble de données, vous pouvez utiliser le hachage pour diviser les espèces d'arbres en 500 catégories ou buckets.

Un seul bucket peut contenir plusieurs espèces d'arbres. Par exemple, le hachage pourrait placer le baobab et l'érable rouge - deux espèces génétiquement différentes - dans le même bucket . Quoi qu'il en soit, le hachage reste un bon moyen de faire correspondre de grands ensembles catégoriels au nombre souhaité de buckets. Le hachage transforme une caractéristique catégorielle ayant un grand nombre de valeurs possibles en un nombre de valeurs beaucoup plus petit en regroupant les valeurs de manière déterministe.

- **hidden layer** : Couche d'un réseau neuronal située entre la couche d'entrée (les caractéristiques) et la couche de sortie (la prédiction). Chaque couche cachée est constituée d'un ou plusieurs neurones.
- **holdout data** : Exemples intentionnellement non utilisés ("retenus") pendant la formation. L'ensemble de données de validation et l'ensemble de données de test sont des exemples de données non utilisées. Les données d'exclusion permettent d'évaluer la capacité de votre modèle à se généraliser à des données autres que celles sur lesquelles il a été formé. La perte sur l'ensemble d'exclusion fournit une meilleure estimation de la perte sur un ensemble de données non vu que la perte sur l'ensemble de formation.
- **hyperparameter** : Les variables que vous ou un service de réglage des hyperparamètres ajustez au cours des exécutions successives de la formation d'un modèle. Par exemple, le taux d'apprentissage est un hyperparamètre. Vous pouvez fixer le taux d'apprentissage à 0,01 avant une session de formation. Si vous déterminez que 0,01 est trop élevé, vous pouvez peut-être fixer le taux d'apprentissage à 0,003 pour la session de formation suivante.

En revanche, les paramètres sont les différents poids et biais que le modèle apprend au cours de l'entraînement.

- **hyperplane** : Une frontière qui sépare un espace en deux sous-espaces. Par exemple, une ligne est un hyperplan en deux dimensions et un plan est un hyperplan en trois dimensions. Plus généralement, dans le domaine de l'apprentissage automatique, un hyperplan est la frontière qui sépare un espace à haute dimension. Les machines à vecteurs de support à noyau utilisent des hyperplans pour séparer les classes positives des classes négatives, souvent dans un espace à très haute dimension.
- **keypoints** : Les coordonnées de caractéristiques particulières dans une image. Par exemple, pour un modèle de reconnaissance d'images qui distingue les espèces de fleurs, les points clés peuvent être le centre de chaque pétale, la tige, l'étamine, etc.
- **k-fold cross validation** : Un algorithme permettant de prédire la capacité d'un modèle à se généraliser à de nouvelles données. Le k de k-fold fait référence au nombre de groupes égaux en lesquels vous divisez les exemples d'un ensemble de données ; en d'autres termes, vous formez et testez votre modèle k fois. Pour chaque cycle de formation et de test, un groupe différent constitue l'ensemble de test, et tous les groupes restants deviennent l'ensemble de formation. Après k cycles de formation et de test, vous calculez la moyenne et l'écart type de la ou des métriques de test souhaitées.
- **L0 regularization** : Un type de régularisation qui pénalise le nombre total de poids non nuls dans un modèle. Par exemple, un modèle ayant 11 poids non nuls sera plus pénalisé qu'un modèle similaire ayant 10 poids non nuls.
- **language model** : Modèle qui estime la probabilité qu'un jeton ou une séquence de token se produise dans une séquence plus longue de tokens.
- **logits** : Le vecteur de prédictions brutes (non normalisées) qu'un modèle de classification génère et qui est généralement transmis à une fonction de normalisation. Si le modèle résout un problème de classification multi-classes, les logits deviennent généralement une entrée de la fonction softmax. La fonction softmax génère alors un vecteur de probabilités (normalisées) avec une valeur pour chaque classe possible.

- **Markov property** : Propriété de certains environnements, où les transitions d'état sont entièrement déterminées par des informations implicites dans l'état actuel et l'action de l'agent.
- **mini-batch** : Un petit sous-ensemble d'un lot sélectionné de manière aléatoire et traité en une seule itération. La taille d'un mini-batch est généralement comprise entre 10 et 1 000 exemples.

Par exemple, supposons que l'ensemble de l'apprentissage (le lot complet) se compose de 1 000 exemples. Supposons en outre que vous fixiez la taille de chaque mini-lot à 20. Par conséquent, chaque itération détermine la perte sur un échantillon aléatoire de 20 des 1 000 exemples, puis ajuste les poids et les biais en conséquence.

Il est beaucoup plus efficace de calculer la perte sur un mini-batch que la perte sur tous les exemples du batch complet.

- **Momentum** : Un algorithme sophistiqué de descente de gradient dans lequel une étape d'apprentissage dépend non seulement de la dérivée de l'étape actuelle, mais aussi des dérivées de l'étape ou des étapes qui l'ont immédiatement précédée. Le momentum implique le calcul d'une moyenne mobile à pondération exponentielle des gradients dans le temps, analogue au momentum en physique. Le momentum empêche parfois l'apprentissage de rester bloqué dans des minima locaux.
- **one-hot encoding** : Représentation des données catégorielles sous la forme d'un vecteur dans lequel :
 - Un élément est mis à 1.
 - Tous les autres éléments sont mis à 0.
 - One-hot encoding est couramment utilisé pour représenter des chaînes de caractères ou des identificateurs qui ont un ensemble fini de valeurs possibles.
- **one-shot learning** : Une approche d'apprentissage automatique, souvent utilisée pour la classification d'objets, conçue pour apprendre des classificateurs efficaces à partir d'un seul exemple d'apprentissage.
- **one-vs.-all** : Pour un problème de classification comportant N classes, la solution consiste en N classificateurs binaires distincts - un classificateur binaire pour chaque résultat possible. Par exemple, étant donné un modèle qui classe les exemples comme animal, végétal ou minéral, une solution "un contre tous", ou one-vs.-all, fournirait les trois classificateurs binaires distincts suivants :
 - animal vs. pas animal
 - végétal vs. pas végétal
 - minéral vs. pas minéral
- **policy** : Dans l'apprentissage par renforcement, la correspondance probabiliste d'un agent entre les états et les actions.
- **pooling** : Réduction d'une matrice (ou de matrices) créée par une couche convolutionnelle antérieure en une matrice plus petite. La mise en commun consiste généralement à prendre la valeur maximale ou moyenne de la zone mise en commun.
- **Q-function** : Dans l'apprentissage par renforcement, la fonction qui prédit le rendement attendu d'une action dans un état et de la poursuite d'une politique donnée.

La fonction Q est également connue sous le nom de fonction de valeur d'action d'état.

- **Q-learning** : Dans l'apprentissage par renforcement, un algorithme qui permet à un agent d'apprendre la fonction, dite Q-fonction, optimale d'un processus de décision de Markov en appliquant l'équation de Bellman. Le processus de décision de Markov modélise un environnement.
- **regularization rate** : Un nombre qui spécifie l'importance relative de la régularisation pendant la formation. L'augmentation du taux de régularisation réduit le surajustement mais peut réduire le pouvoir prédictif du modèle. Inversement, la réduction ou l'absence de taux de régularisation augmente l'overfitting.

- **reinforcement learning** : Une famille d'algorithmes qui apprennent une politique optimale, dont l'objectif est de maximiser le rendement lors de l'interaction avec un environnement. Par exemple, la récompense ultime de la plupart des jeux est la victoire. Les systèmes d'apprentissage par renforcement peuvent devenir experts en matière de jeux complexes en évaluant les séquences de mouvements précédents qui ont conduit à des victoires et les séquences qui ont conduit à des défaites.
- **sampling with replacement** : Méthode de sélection d'éléments à partir d'un ensemble d'éléments candidats dans laquelle le même élément peut être sélectionné plusieurs fois. L'expression "avec remplacement" signifie qu'après chaque sélection, l'élément sélectionné est réintroduit dans l'ensemble des éléments candidats. La méthode inverse, l'échantillonnage sans remplacement, signifie qu'un élément candidat ne peut être choisi qu'une seule fois.
- **shrinkage** : Un hyperparamètre dans le gradient boosting qui contrôle l'overfitting. Le rétrécissement ou shrinkage dans le gradient boosting est analogue au taux d'apprentissage dans la descente de gradient. La réduction est une valeur décimale comprise entre 0,0 et 1,0. Une valeur de rétrécissement plus faible réduit davantage l'overfitting qu'une valeur de rétrécissement plus élevée.
- **undersampling** : Suppression des exemples de la classe majoritaire dans un ensemble de données déséquilibré afin de créer un ensemble de formation plus équilibré.

Prenons l'exemple d'un ensemble de données dans lequel le rapport entre la classe majoritaire et la classe minoritaire est de 20:1. Pour remédier à ce déséquilibre entre les classes, vous pouvez créer un ensemble de formation composé de tous les exemples de la classe minoritaire, mais seulement d'un dixième des exemples de la classe majoritaire, ce qui créerait un rapport entre les classes de l'ensemble de formation de 2:1. Grâce au sous-échantillonnage, cet ensemble de formation plus équilibré pourrait produire un meilleur modèle. En revanche, cet ensemble de formation plus équilibré pourrait ne pas contenir suffisamment d'exemples pour former un modèle efficace.

- **Wasserstein loss** : L'une des fonctions de loss couramment utilisées dans les réseaux adversaires génératifs, basée sur la distance entre la distribution des données générées et les données réelles.
- **zero-shot learning** : Un type de formation à l'apprentissage automatique dans lequel le modèle déduit une prédiction pour une tâche sur laquelle il n'a pas été spécifiquement formé. En d'autres termes, le modèle ne reçoit aucun exemple de formation spécifique à une tâche, mais il lui est demandé de faire une inférence pour cette tâche.

